

AgriDabaw-3D

Comment Archive

Every comment removed from the Unity game code and the backend server code, kept file by file so none of the explanations are lost.

Side	Files scanned	Files with comments	Comments	Comment lines
Unity	205	149	6468	6473
Backend	70	40	343	826

Each entry gives the line numbers in the file **as it was before** the comments were removed, the kind of comment, and the code it sat above ("before") or beside ("on the line"). The full original files also remain in each project's git history.

Archived 25 September 2026

Contents

Part A — Unity game	6
Assets/Editor	
Assets/Editor/DistrictMapArtDrawer.cs	6
Assets/Editor/DistrictMapArtPostprocessor.cs	6
Assets/Editor/MeshDecimationPostprocessor.cs	6
Assets/Editor/MeshStatsReporter.cs	8
Assets/Editor/PlantingAssetsSetup.cs	9
Assets/Editor/TutorialPictureImporter.cs	9
Assets/Resources/Shaders	
Assets/Resources/Shaders/StormSky.shader	10
Assets/Scripts/AreaSelection/Districts	
Assets/Scripts/AreaSelection/Districts/DavaoGeoReference.cs	10
Assets/Scripts/AreaSelection/Districts/DistrictAreaMap.cs	10
Assets/Scripts/AreaSelection/Districts/DistrictDescriptions.cs	11
Assets/Scripts/AreaSelection/Selection	
Assets/Scripts/AreaSelection/Selection/AreaSelectionBuilder.cs	11
Assets/Scripts/AreaSelection/Selection/DraggableSelectionBox.cs	16
Assets/Scripts/AreaSelection/Selection/LoopingVideoBackground.cs	16
Assets/Scripts/AreaSelection/Selection/MapZoomPanController.cs	17
Assets/Scripts/AreaSelection/Selection/TutorialSlideBoard.cs	18
Assets/Scripts/MainMenu/Account	
Assets/Scripts/MainMenu/Account/AccountRules.cs	19
Assets/Scripts/MainMenu/Account/AccountUIBuilder.cs	20
Assets/Scripts/MainMenu/Account/AuthUIBuilder.cs	23
Assets/Scripts/MainMenu/Menu	
Assets/Scripts/MainMenu/Menu/CreditsUIBuilder.cs	26
Assets/Scripts/MainMenu/Menu/MainMenuBuilder.cs	27
Assets/Scripts/Shared/Accessibility	
Assets/Scripts/Shared/Accessibility/GameAccessibility.cs	29
Assets/Scripts/Shared/Audio	
Assets/Scripts/Shared/Audio/GameAudioManager.cs	30
Assets/Scripts/Shared/Audio/SceneMusicPlayer.cs	31
Assets/Scripts/Shared/Backend	
Assets/Scripts/Shared/Backend/ApiClient.cs	31
Assets/Scripts/Shared/Backend/ApiConfiguration.cs	31
Assets/Scripts/Shared/Backend/AuthSession.cs	31
Assets/Scripts/Shared/Backend/BackendModels.cs	33
Assets/Scripts/Shared/Backend/DeviceIdentity.cs	35
Assets/Scripts/Shared/Backend/SavedSession.cs	35
Assets/Scripts/Shared/Backend/SessionTakeover.cs	35
Assets/Scripts/Shared/Districts	
Assets/Scripts/Shared/Districts/DavaoDistrictService.cs	36
Assets/Scripts/Shared/Districts/DistrictCropPools.cs	36
Assets/Scripts/Shared/Settings	
Assets/Scripts/Shared/Settings/GameSettings.cs	38
Assets/Scripts/Shared/Settings/MobilePerformanceBootstrap.cs	39
Assets/Scripts/Shared/Settings/PauseMenuBuilder.cs	39
Assets/Scripts/Shared/Settings/SettingsUIBuilder.cs	40
Assets/Scripts/Shared/Settings/UIScaleService.cs	42
Assets/Scripts/Shared/UI	
Assets/Scripts/Shared/UI/DescriptionBoard.cs	44
Assets/Scripts/Shared/UI/GameFonts.cs	45
Assets/Scripts/Shared/UI/HudIconButton.cs	45
Assets/Scripts/Shared/UI/HudRegistry.cs	46
Assets/Scripts/Shared/UI/MobileTextEntryOverlay.cs	47
Assets/Scripts/Shared/UI/UIPlank.cs	48
Assets/Scripts/Shared/UI/UIThemeSprites.cs	48
Assets/Scripts/TerrainPreview/AIAdviser	
Assets/Scripts/TerrainPreview/AIAdviser/AiBackendClient.cs	49
Assets/Scripts/TerrainPreview/AIAdviser/AiResponseStyle.cs	49
Assets/Scripts/TerrainPreview/AIAdviser/AiText.cs	50
Assets/Scripts/TerrainPreview/AIAdviser/FarmAdvisorChatSystem.cs	50
Assets/Scripts/TerrainPreview/AIAdviser/FarmAdvisorContextBuilder.cs	50
Assets/Scripts/TerrainPreview/AIAdviser/FarmAdvisorGeminiclient.cs	51
Assets/Scripts/TerrainPreview/AIAdviser/FarmAdvisorToolReference.cs	51
Assets/Scripts/TerrainPreview/AIAdviser/FarmAdvisorUIBuilder.cs	51
Assets/Scripts/TerrainPreview/Audio	
Assets/Scripts/TerrainPreview/Audio/FarmAudioController.cs	53
Assets/Scripts/TerrainPreview/ClimateEvaluation	
Assets/Scripts/TerrainPreview/ClimateEvaluation/ClimateEvaluationModels.cs	53
Assets/Scripts/TerrainPreview/ClimateEvaluation/ClimateEvaluationPopupBuilder.cs	53
Assets/Scripts/TerrainPreview/ClimateEvaluation/ClimateEventTracker.cs	54
Assets/Scripts/TerrainPreview/ClimateEvaluation/ClimateResilienceGeminiclient.cs	54

Assets/Scripts/TerrainPreview/ClimateMaintenance

Assets/Scripts/TerrainPreview/ClimateMaintenance/ClimateActionAutoObserver.cs	55
Assets/Scripts/TerrainPreview/ClimateMaintenance/ClimateMaintenanceInteractionSystem.cs	55
Assets/Scripts/TerrainPreview/ClimateMaintenance/ClimateMaintenanceToastUI.cs	55
Assets/Scripts/TerrainPreview/ClimateMaintenance/ClimateMitigationWorldObject.cs	55
Assets/Scripts/TerrainPreview/ClimateMaintenance/CropClimateMaintenanceState.cs	56
Assets/Scripts/TerrainPreview/ClimateMaintenance/CropRuntimeAdapter.cs	57
Assets/Scripts/TerrainPreview/ClimateMaintenance/TerrainModificationService.cs	57

Assets/Scripts/TerrainPreview/CropInstances

Assets/Scripts/TerrainPreview/CropInstances/BananaPlantInstance.cs	57
Assets/Scripts/TerrainPreview/CropInstances/CoconutTreeInstance.cs	58
Assets/Scripts/TerrainPreview/CropInstances/TropicalCropPlantInstance.cs	59
Assets/Scripts/TerrainPreview/CropInstances/TropicalCropTypes.cs	61

Assets/Scripts/TerrainPreview/DevTools

Assets/Scripts/TerrainPreview/DevTools/DevCropVitals.cs	61
Assets/Scripts/TerrainPreview/DevTools/DevToolsUIBuilder.cs	61
Assets/Scripts/TerrainPreview/DevTools/FpsCounterHud.cs	66
Assets/Scripts/TerrainPreview/DevTools/UiResponseTimerHud.cs	66

Assets/Scripts/TerrainPreview/Economy

Assets/Scripts/TerrainPreview/Economy/PesoPrice.cs	67
Assets/Scripts/TerrainPreview/Economy/ShippingBinSpawner.cs	68
Assets/Scripts/TerrainPreview/Economy/ShopItemDescriptions.cs	68
Assets/Scripts/TerrainPreview/Economy/ShopUIBuilder.cs	69

Assets/Scripts/TerrainPreview/Farming

Assets/Scripts/TerrainPreview/Farming/CropClimateRules.cs	72
Assets/Scripts/TerrainPreview/Farming/CropFieldDescriptions.cs	72
Assets/Scripts/TerrainPreview/Farming/CropNaming.cs	72
Assets/Scripts/TerrainPreview/Farming/CropProtectionState.cs	72
Assets/Scripts/TerrainPreview/Farming/DigSpot.cs	73
Assets/Scripts/TerrainPreview/Farming/FarmingInteractionSystem.cs	73
Assets/Scripts/TerrainPreview/Farming/FarmingInteractionSystem.Planting.cs	75
Assets/Scripts/TerrainPreview/Farming/GrowthStageVisualController.cs	77

Assets/Scripts/TerrainPreview/FarmTasks

Assets/Scripts/TerrainPreview/FarmTasks/AIAdvisorTaskGeminiClient.cs	78
Assets/Scripts/TerrainPreview/FarmTasks/AIAdvisorTaskSystem.cs	78
Assets/Scripts/TerrainPreview/FarmTasks/DailyTaskCatalog.cs	80
Assets/Scripts/TerrainPreview/FarmTasks/DailyTaskSystem.cs	80
Assets/Scripts/TerrainPreview/FarmTasks/FarmTaskActionHub.cs	81
Assets/Scripts/TerrainPreview/FarmTasks/FarmTaskActionObserver.cs	81
Assets/Scripts/TerrainPreview/FarmTasks/FarmTaskContextBuilder.cs	82
Assets/Scripts/TerrainPreview/FarmTasks/FarmTaskModels.cs	82
Assets/Scripts/TerrainPreview/FarmTasks/FarmTaskPopupUI.cs	82
Assets/Scripts/TerrainPreview/FarmTasks/FarmTaskRuntimeBootstrap.cs	82
Assets/Scripts/TerrainPreview/FarmTasks/FarmTaskUIBuilder.cs	82

Assets/Scripts/TerrainPreview/Harvest

Assets/Scripts/TerrainPreview/Harvest/CollectibleBanana.cs	83
--	----

Assets/Scripts/TerrainPreview/Inventory

Assets/Scripts/TerrainPreview/Inventory/InventoryItemType.cs	83
Assets/Scripts/TerrainPreview/Inventory/InventorySlotUI.cs	84
Assets/Scripts/TerrainPreview/Inventory/InventoryTrashUI.cs	84
Assets/Scripts/TerrainPreview/Inventory/InventoryUIBuilder.cs	85
Assets/Scripts/TerrainPreview/Inventory/PlayerInventory.cs	86

Assets/Scripts/TerrainPreview/Map

Assets/Scripts/TerrainPreview/Map/FarmMapFieldInput.cs	87
Assets/Scripts/TerrainPreview/Map/FarmMapIcons.cs	88
Assets/Scripts/TerrainPreview/Map/FarmMapUIBuilder.cs	88
Assets/Scripts/TerrainPreview/Map/MapBoundarySystem.cs	92

Assets/Scripts/TerrainPreview/Nursery

Assets/Scripts/TerrainPreview/Nursery/NurseryRuntimeBootstrap.cs	93
Assets/Scripts/TerrainPreview/Nursery/NurserySystem.cs	93
Assets/Scripts/TerrainPreview/Nursery/PlantingAvailability.cs	94
Assets/Scripts/TerrainPreview/Nursery/PlantingMaterialCatalog.cs	95
Assets/Scripts/TerrainPreview/Nursery/SeedlingTentInstance.cs	97
Assets/Scripts/TerrainPreview/Nursery/SeedlingTentUIBuilder.cs	97

Assets/Scripts/TerrainPreview/Persistence

Assets/Scripts/TerrainPreview/Persistence/CropPersistenceMapper.cs	98
Assets/Scripts/TerrainPreview/Persistence/FarmPersistenceManager.cs	98
Assets/Scripts/TerrainPreview/Persistence/FarmSnapshotModels.cs	99
Assets/Scripts/TerrainPreview/Persistence/SaveFarmButtonBuilder.cs	99

Assets/Scripts/TerrainPreview/PestDisease

Assets/Scripts/TerrainPreview/PestDisease/AreaMitigationTrapInstance.cs	99
Assets/Scripts/TerrainPreview/PestDisease/PestDiseaseAffectedCrop.cs	99
Assets/Scripts/TerrainPreview/PestDisease/PestDiseasePopupUI.cs	100
Assets/Scripts/TerrainPreview/PestDisease/PestDiseaseSystem.cs	100

Assets/Scripts/TerrainPreview/Player

Assets/Scripts/TerrainPreview/Player/MobileHudBuilder.cs	101
--	-----

Assets/Scripts/TerrainPreview/SocialMarketplace

Assets/Scripts/TerrainPreview/SocialMarketplace/EconomyRevisionMerge.cs	101
Assets/Scripts/TerrainPreview/SocialMarketplace/SocialMarketplaceApi.cs	101

Assets/Scripts/TerrainPreview/SocialMarketplace/SocialMarketplaceCatalog.cs	102
Assets/Scripts/TerrainPreview/SocialMarketplace/SocialMarketplaceController.cs	102
Assets/Scripts/TerrainPreview/SocialMarketplace/SocialMarketplaceDtos.cs	104
Assets/Scripts/TerrainPreview/SocialMarketplace/SocialMarketplaceUIBuilder.Core.cs	104
Assets/Scripts/TerrainPreview/SocialMarketplace/SocialMarketplaceUIBuilder.Marketplace.cs	104
Assets/Scripts/TerrainPreview/SocialMarketplace/SocialMarketplaceUIBuilder.PlayersChat.cs	104
Assets/Scripts/TerrainPreview/SocialMarketplace/SocialMarketplaceUIBuilder.Trade.cs	104
Assets/Scripts/TerrainPreview/SocialMarketplace/TradeDragSource.cs	111
Assets/Scripts/TerrainPreview/Soil	
Assets/Scripts/TerrainPreview/Soil/SoilGridsJsonParser.cs	111
Assets/Scripts/TerrainPreview/Soil/SoilGridsService.cs	112
Assets/Scripts/TerrainPreview/Soil/SoilInfoPanelBuilder.cs	112
Assets/Scripts/TerrainPreview/Terrain	
Assets/Scripts/TerrainPreview/Terrain/DistanceCullable.cs	113
Assets/Scripts/TerrainPreview/Terrain/DistanceCullingManager.cs	113
Assets/Scripts/TerrainPreview/Terrain/SoilAwareTerrainGenerator.cs	113
Assets/Scripts/TerrainPreview/Terrain/TemporaryTerrainGenerator.cs	114
Assets/Scripts/TerrainPreview/Tutorial	
Assets/Scripts/TerrainPreview/Tutorial/TutorialDialogueUI.cs	116
Assets/Scripts/TerrainPreview/Tutorial/TutorialDirector.cs	118
Assets/Scripts/TerrainPreview/Tutorial/TutorialGates.cs	121
Assets/Scripts/TerrainPreview/Tutorial/TutorialState.cs	121
Assets/Scripts/TerrainPreview/UI	
Assets/Scripts/TerrainPreview/UI/FarmChoicePopup.cs	122
Assets/Scripts/TerrainPreview/UI/FarmConfirmPopup.cs	123
Assets/Scripts/TerrainPreview/UI/WorldLoadingScreen.cs	123
Assets/Scripts/TerrainPreview/Weather	
Assets/Scripts/TerrainPreview/Weather/FarmGraceperiod.cs	123
Assets/Scripts/TerrainPreview/Weather/GameTimeSystem.cs	124
Assets/Scripts/TerrainPreview/Weather/WeatherSystem.cs	124
Assets/Scripts/TerrainPreview/Weather/WeatherVisualController.cs	124
Assets/TutorialInfo/Scripts/Editor	
Assets/TutorialInfo/Scripts/Editor/ReadmeEditor.cs	127
Part B — Backend server	128
(project root)	
Dockerfile	128
src/main/java/com/agridabao/api/admin	
src/main/java/com/agridabao/api/admin/AdminCommand.java	128
src/main/java/com/agridabao/api/admin/AdminController.java	128
src/main/java/com/agridabao/api/ai	
src/main/java/com/agridabao/api/ai/AiController.java	130
src/main/java/com/agridabao/api/ai/AiGuardrails.java	131
src/main/java/com/agridabao/api/ai/GeminiClient.java	132
src/main/java/com/agridabao/api/auth	
src/main/java/com/agridabao/api/auth/AuthController.java	132
src/main/java/com/agridabao/api/auth/AuthService.java	132
src/main/java/com/agridabao/api/auth/CodeRequestResponse.java	133
src/main/java/com/agridabao/api/auth/EmailVerification.java	133
src/main/java/com/agridabao/api/auth/EmailVerificationRepository.java	133
src/main/java/com/agridabao/api/auth/ForgotPasswordRequest.java	134
src/main/java/com/agridabao/api/auth/ForgotPasswordVerifyRequest.java	134
src/main/java/com/agridabao/api/auth/LoginRequest.java	134
src/main/java/com/agridabao/api/auth/PasswordResetTicketResponse.java	134
src/main/java/com/agridabao/api/auth/ResetPasswordRequest.java	134
src/main/java/com/agridabao/api/auth/VerificationPurpose.java	134
src/main/java/com/agridabao/api/auth/VerificationService.java	134
src/main/java/com/agridabao/api/config	
src/main/java/com/agridabao/api/config/SecurityConfig.java	135
src/main/java/com/agridabao/api/farm	
src/main/java/com/agridabao/api/farm/DistrictSeedPools.java	135
src/main/java/com/agridabao/api/farm/EconomyJsonService.java	136
src/main/java/com/agridabao/api/farm/PesoRounding.java	138
src/main/java/com/agridabao/api/farm/WeatherMitigationTradableItems.java	138
src/main/java/com/agridabao/api/mail	
src/main/java/com/agridabao/api/mail/MailService.java	138
src/main/java/com/agridabao/api/mail/ResendMailClient.java	139
src/main/java/com/agridabao/api/marketplace	
src/main/java/com/agridabao/api/marketplace/MarketplaceController.java	139
src/main/java/com/agridabao/api/security	
src/main/java/com/agridabao/api/security/JwtService.java	140
src/main/java/com/agridabao/api/settings	
src/main/java/com/agridabao/api/settings/PlayerSettings.java	140
src/main/java/com/agridabao/api/settings/PlayerSettingsRequest.java	140
src/main/java/com/agridabao/api/settings/PlayerSettingsService.java	140
src/main/java/com/agridabao/api/social	
src/main/java/com/agridabao/api/social/PresenceController.java	141

src/main/java/com/agridabao/api/trade	
src/main/java/com/agridabao/api/trade/TradeController.java	141
src/main/java/com/agridabao/api/user	
src/main/java/com/agridabao/api/user/AccountFields.java	141
src/main/java/com/agridabao/api/user/AccountService.java	142
src/main/java/com/agridabao/api/user/AppUser.java	142
src/main/java/com/agridabao/api/user/AppUserRepository.java	143
src/main/java/com/agridabao/api/user/ChangeEmailRequest.java	143
src/main/java/com/agridabao/api/user/ChangePasswordRequest.java	143
src/main/java/com/agridabao/api/user/ConfirmEmailChangeRequest.java	143
src/main/resources	
src/main/resources/application.yml	143
Files that had no comments	145

Part A — Unity game

Unity project AgriDabao2ndVer: C# scripts under Assets and the StormSky shader. Line numbers are from commit 22dcdfc.

Assets/Editor/DistrictMapArtDrawer.cs

10 comments · 10 comment lines removed · file had 97 lines

Lines 6–14 · doc comment · before [CustomPropertyDrawer(typeof(DistrictMapArt))]

```
/// <summary>
/// Draws each row of the Area Selection Builder district list with the
/// district's own name on the header, and a count of how many of its three
/// pictures are still empty.
///
/// Without this the rows read "Element 0" to "Element 5", which makes it easy
/// to drop one district's art into another district's row - a mistake that
/// does not fail, it just shows the wrong map.
/// </summary>
```

Line 92 · comment · before return (line + pad) * (SlotNames.Length + 2);

```
// Header plus the name field plus the three sprite slots.
```

Assets/Editor/DistrictMapArtPostprocessor.cs

41 comments · 41 comment lines removed · file had 167 lines

Lines 7–36 · doc comment · before public class DistrictMapArtPostprocessor : AssetPostprocessor

```
/// <summary>
/// Import settings for every picture under Sprites/DavaoCityMap, applied
/// automatically so none of them can arrive in a state the game cannot use.
///
/// Three jobs, three rules:
///
/// DavaoCityMap/ the TERRAIN SOURCES - satellite,
/// elevation and slope. The generator reads
/// all three pixel by pixel, so they have to
/// be readable. Left uncompressed at 2048,
/// which is what the existing maps already
/// use, because the elevation and slope
/// readings are colour matches against a
/// legend and compression moves colours.
///
/// DavaoCityMap/Districts/ the area selection pictures that get
/// DRAWN - district highlight and barangay
/// highlight. Ordinary compressed sprites;
/// nothing reads them.
///
/// DavaoCityMap/Districts/AreaSpots/ the picture that gets READ to decide
/// whether a plot is allowed. Readable,
/// uncompressed and point filtered: block
/// compression smears green into red along
/// every boundary, and a bilinear filter
/// invents colours that are neither.
///
/// Dropping a file into any of them is all that is needed. Existing files can
/// be brought in line with AgriDabao -> Reimport Davao Map Art.
/// </summary>
```

Lines 43–49 · doc comment · before private const int SpotMapResolution = 512;

```
/// <summary>
/// Sampling resolution of the area spot maps. The full map is roughly 63 km
/// across, so 512 works out near 120 m per pixel - far finer than a barangay
/// boundary needs, and small enough that the six of them together cost a
/// couple of megabytes rather than a couple of hundred. Raise it if a thin
/// sliver of a barangay is being misjudged.
/// </summary>
```

Lines 81–82 · comment · before if (StartsWith(path, MapFolder + "/") &&

```
// Only files sitting directly in the map folder, so an unrelated
// subfolder added later is not silently forced uncompressed.
```

Lines 133–134 · comment · before importer.ClearPlatformTextureSettings("Android");

```
// Per-platform overrides win over everything set above, and the mobile
// ones compress by default, so they are removed rather than fought.
```

Assets/Editor/MeshDecimationPostprocessor.cs

107 comments · 107 comment lines removed · file had 307 lines

Lines 1–3 · comment · before using UnityMeshSimplifier;

```
// Requires the UnityMeshSimplifier package. Until it is installed AND the
// UNITY_MESH_SIMPLIFIER scripting define is added, everything below compiles away
// to nothing, so dropping this file into the project changes no behaviour.
```

Lines 13–25 · doc comment · before public class MeshDecimationPostprocessor : AssetPostprocessor

```
/// <summary>
/// Reduces the triangle count of imported models on the way in.
```

```

///
/// The source FBX files are never modified. Unity re-imports them from disk, and
/// this runs on the result, so the simplified mesh is what every prefab, scene
/// and material sees while the original file stays intact. Turning
/// <see cref="Enabled"/> off and reimporting restores full detail - which is why
/// this is preferable to decimating in Blender, where the reduction is baked into
/// the file and getting the detail back means re-exporting all over again.
///
/// Measured before writing this: 20 mature mangosteens rendered 316k triangles
/// and 517k vertices, roughly three times a mid-range phone's comfortable budget.
/// </summary>

```

Lines 28–37 · doc comment · before `private static readonly bool Enabled = true;`

```

/// <summary>
/// Master switch. Left OFF deliberately: importing this script should change
/// nothing until the decision to decimate has actually been made, and that
/// decision belongs after a real device build, not before one.
/// </summary>
/// <remarks>
/// Deliberately not a <c>const</c>: as a compile-time constant the guard below
/// makes the rest of the method provably unreachable and the compiler warns
/// about it every time this is switched off.
/// </remarks>

```

Lines 40–42 · comment · before `private const float FoliageQuality = 0.4f;`

```

// Share of the original triangles to keep, grouped by what the geometry can
// survive. Solid shapes lose triangles gracefully; thin, gappy, alpha-cut
// geometry does not, so nets and foliage are treated far more gently.

```

Line 44 · doc comment · before `private const float FoliageQuality = 0.4f;`

```

/// <summary>Tree canopies. Verified on the mangosteen: 10,365 -> ~4,100 tris.</summary>

```

Lines 47–51 · doc comment · before `private const float WispyQuality = 0.6f;`

```

/// <summary>
/// Grass, netting and trellis. These are thin strips with holes in them, and
/// aggressive decimation leaves visible gaps and jagged edges rather than a
/// slightly rounder shape, so they keep far more of their geometry.
/// </summary>

```

Line 54 · doc comment · before `private const float SolidQuality = 0.4f;`

```

/// <summary>Hard-surface props - tanks, greenhouses, bins, traps.</summary>

```

Lines 57–61 · doc comment · before `private const float GroundQuality = 0.3f;`

```

/// <summary>
/// The dug planting spot. Essentially a flat patch of ground, so it tolerates
/// heavy reduction, and one is spawned per planting - twenty crops means twenty
/// of these, which at 6,357 tris each is a bigger cost than it looks.
/// </summary>

```

Lines 64–77 · doc comment · before `private static readonly Target[] Targets =`

```

/// <summary>
/// Only models under these folders are touched.
///
/// Listed rather than reorganised on disk: every 2nd-stage prefab already shares
/// its source model with the adult of the same crop - there is no separate
/// 2nd-stage FBX - so decimating each "<crop>Tree" folder covers both stages at
/// once. Sprouts and fruit live in their own folders and are left at full
/// detail, since they are small on screen and cheap already.
///
/// Deliberately excluded:
///   Assets/Models/PineappleSprout/ - Pineapple2ndStage is the one prefab that
///   reuses a SPROUT model rather than its tree, so decimating it would also
///   decimate the pineapple seedling. Add it only if that is acceptable.
/// </summary>

```

Lines 80–81 · comment · before `new Target("Assets/Models/BananaTree/", FoliageQuality),`

```

// Crop trees - each covers both the 2nd-stage and adult prefab, because
// every 2nd-stage prefab reuses its adult's model at a smaller scale.

```

Lines 96–97 · comment · before `new Target("Assets/Models/DrainageCanal/", SolidQuality),`

```

// Climate mitigation structures. The shade net and windbreak are netting
// rather than solid shapes, so they are treated like foliage.

```

Line 105 · comment · before `new Target("Assets/Models/MulchPatch/", SolidQuality),`

```

// Crop maintenance placeables.

```

Line 110 · comment · before `new Target("Assets/Models/AphidTrap/", SolidQuality),`

```

// Pest and disease traps.

```

Lines 115–116 · comment · before `new Target("Assets/Models/DirtSpot/", GroundQuality),`

```

// Ground and world props. DirtSpot is spawned once per dug planting spot,
// so its cost multiplies with the size of the farm.

```

Lines 120–121 · comment · before `new Target("Assets/Models/GrasspatchLarge/", WispyQuality),`

```

// Grass. Only a handful are visible at once, but there are 450 of them and
// their vertex-to-triangle ratio is the worst in the project.

```

Lines 125–127 · comment · before `new Target("Assets/Models/SeedlingTent/", SolidQuality),`

```

// The planting system. The tent and its racks stand once per farm, the
// bags up to eight times inside it; the seedlings and planting

```

```
// materials appear in the bags and for a crop's first days in the field.
```

Line 148 · doc comment · before `private readonly struct Target`

```
/// <summary>One folder of models and how much of their geometry to keep.</summary>
```

Lines 161–164 · doc comment · before `private static bool TryGetQuality(string path, out float quality)`

```
/// <summary>
/// Finds the reduction configured for an asset, or reports that it is not a
/// target and should be imported untouched.
/// </summary>
```

Lines 180–183 · doc comment · before `private const int SkipBelowTriangles = 3000;`

```
/// <summary>
/// Meshes at or below this are already cheap, and simplifying them costs
/// silhouette quality for a saving too small to measure.
/// </summary>
```

Lines 186–198 · doc comment · before `private void OnPreprocessModel()`

```
/// <summary>
/// Turns Read/Write on for the models this processor rewrites.
///
/// Unity blocks writes to a mesh whose importer has Read/Write disabled, which
/// is the default and is normally the setting you want – it lets the engine drop
/// the CPU-side copy once the mesh reaches the GPU. Simplifying the mesh means
/// writing to it, so the flag has to be on for these specific models.
///
/// The cost is that their vertex data also stays in system memory at runtime.
/// That is a real trade: the triangles saved are worth more than the RAM spent
/// here, but it applies only to models under <see cref="TargetFolder"/>, which
/// is why the scope is kept narrow.
/// </summary>
```

Lines 207–210 · comment · before `if (assetImporter is ModelImporter importer && !importer.isReadable)`

```
// Only write when the value actually changes. Assigning it unconditionally
// dirties the .meta on every single import, and Unity's determinism check –
// which imports twice and compares – can then report the containing folder
// as having produced an inconsistent result.
```

Lines 242–243 · comment · before `if (simplified == null ||`

```
// Refuse to apply a result that came back empty. Without this a failed
// simplification silently wipes the model instead of leaving it alone.
```

Lines 258–262 · comment · before `CopyInto(simplified, mesh);`

```
// Write the result INTO the mesh Unity already owns rather than swapping
// the reference. A Mesh constructed in here is not part of the imported
// asset, so assigning it to the filter leaves the model referencing an
// object that never gets serialised – which imports as a prefab with no
// mesh at all rather than a lighter one.
```

Lines 275–279 · doc comment · before `private static void CopyInto(Mesh source, Mesh destination)`

```
/// <summary>
/// Replaces every channel of <paramref name="destination"/> with the contents of
/// <paramref name="source"/>, keeping the destination object identity so the
/// importer still serialises it as part of the model.
/// </summary>
```

Line 294 · comment · before `destination.boneWeights = source.boneWeights;`

```
// Only meaningful for skinned meshes; static props carry empty arrays.
```

Assets/Editor/MeshStatsReporter.cs

18 comments · 18 comment lines removed · file had 128 lines

Lines 9–16 · doc comment · before `public static class MeshStatsReporter`

```
/// <summary>
/// Lists the triangle count of every imported model in one pass.
///
/// The importer shows this per file in its preview pane, but the project has
/// 54 of them, and the number that matters is how they compare to each other
/// and to a mobile budget – which is impossible to judge one file at a time.
/// This prints them sorted heaviest first so the worst offenders are obvious.
/// </summary>
```

Lines 19–23 · doc comment · before `private const int HeavyModelTriangles = 20000;`

```
/// <summary>
/// Rough per-model ceiling for a phone. A mid-range Android device can push
/// somewhere around 100k triangles per frame in total, so any single crop
/// costing more than this leaves almost nothing for the rest of the farm.
/// </summary>
```

Line 26 · doc comment · before `private const int SevereModelTriangles = 100000;`

```
/// <summary>Above this a single model can blow the whole frame budget on its own.</summary>
```

Lines 60–61 · comment · before `foreach (Object sub in AssetDatabase.LoadAllAssetsAtPath(path))`

```
// A single model file can contain several meshes; the cost of placing
// it in the scene is the sum of all of them.
```

Lines 112–113 · comment · before `string outPath = System.IO.Path.Combine(`
// The console truncates long messages, so the full table also goes to a
// file that can be opened and sorted outside Unity.

Assets/Editor/PlantingAssetsSetup.cs

34 comments · 34 comment lines removed · file had 382 lines

Lines 8–19 · doc comment · before `public static class PlantingAssetsSetup`

```
/// <summary>
/// Builds the planting system's prefabs from the Meshy models, sets the new
/// UI images up as sprites, and wires both into the farm scene and the UI
/// theme. Kept as menu items so any step can be run again after a model or
/// sprite is replaced.
///
/// The Meshy models all import with a 100x scale on their root and are about
/// 1.9 units across at that scale, pivot at their centre. Each prefab here
/// is an empty root standing at the model's base, with the model inside it
/// scaled to its real size, so every one of them can be placed on the ground
/// by its root.
/// </summary>
```

Line 55 · comment · before `importer.textureType = TextureImporterType.Sprite;`
// Matches the existing HUD and item icons.

Lines 74–75 · comment · before `BuildModelPrefab("Seedling_Cacao", "SproutCacao", 0.18f, true);`
// Seedlings: shown in the tent's bags, and for a transplanted crop's
// first days in the field.

Line 87 · comment · before `BuildModelPrefab("Seednut_Coconut", "Seednut_Coconut", 0.3f, true);`
// Planting materials that go straight into the ground.

Line 93 · comment · before `BuildModelPrefab("SeedlingBag_Empty", "SeedlingBag_Empty", 0.1f, false);`
// Seedling bags, about 20 cm tall like a real polybag.

Line 169 · comment · before `private static void BuildModelPrefab(string prefabName, string modelFolder, float scale, bool collider)`
// ----- building

Lines 182–186 · doc comment · before `private static void BuildGroundPrefabs(Material dugSoil)`

```
/// <summary>
/// Prepared ground, drawn from the dug-soil model: flattened for tilled
/// ground and the top of a raised bed, with a dark opening for a hole and
/// a trench between two ridges for a furrow.
/// </summary>
```

Lines 220–224 · doc comment · before `private static void BuildTentPrefab()`

```
/// <summary>
/// The Seedling Tent: the tent, two racks along its side walls, and eight
/// bag positions on the racks' middle shelves. The tent is a trigger so the
/// player can walk in among the racks; the racks are solid.
/// </summary>
```

Lines 274–278 · doc comment · before `private static GameObject AddModel(Transform parent, string modelFolder, float scale, Vector3 shape,`
`string na`

```
/// <summary>
/// Adds a model under a holder whose scale reshapes it, then seats it so its
/// base is on the holder's origin and it is centred over it. Returns the
/// model instance; its parent is the holder.
/// </summary>
```

Line 293 · comment · before `Bounds bounds = WorldBounds(instance);`
// Built at the world origin with no rotation, so world and local agree.

Assets/Editor/TutorialPictureImporter.cs

14 comments · 14 comment lines removed · file had 65 lines

Lines 7–18 · doc comment · before `public class TutorialPictureImporter : AssetPostprocessor`

```
/// <summary>
/// Imports the area selection tutorial's screenshots as sprites.
///
/// Each tutorial page takes a Sprite, but this project was set up in 3D mode,
/// so a freshly dropped PNG arrives as a plain texture and the page's Picture
/// slot refuses it until its Texture Type is changed by hand. Anything placed
/// in this folder skips that step.
///
/// The settings only apply as a picture is imported, so drop new screenshots
/// straight into the folder. A picture moved in from somewhere else keeps its
/// old settings until AgriDabao -> Reimport Tutorial Pictures is run.
/// </summary>
```

Lines 40–41 · comment · before `importer.isReadable = false;`
// Screenshots are only ever shown, never read, so ordinary compression
// is fine; 2048 keeps a full-HD capture sharp at the size it is drawn.

Assets/Resources/Shaders/StormSky.shader

9 comments · 9 comment lines removed · file had 77 lines

Lines 1–3 · comment · before Shader "AgriDabao/StormSky"

```
// The overcast sky WeatherVisualController fades in over the normal sky during
// rain and typhoons. It lives in Resources and is loaded by name, so it ships in
// builds without being assigned to anything.
```

Lines 15–17 · comment · before Tags

```
// URP draws the skybox between the opaque and transparent queues. One
// step into the transparent range puts this after the sky and before
// everything else transparent, so rain still falls in front of it.
```

Lines 26–27 · comment · before Cull Front

```
// Seen from inside, so the inner faces are the ones drawn. It never writes
// depth, and anything nearer - terrain, crops - hides it as usual.
```

Line 63 · comment · before o.height = normalize(v.vertex.xyz).y;

```
// The dome is never rotated, so its own up is the world's up.
```

Assets/Scripts/AreaSelection/Districts/DavaoGeoReference.cs

6 comments · 6 comment lines removed · file had 54 lines

Lines 7–10 · comment · before private const double TopLeftLat = 7.6089;

```
// Corners of the area selection satellite map. These belong to the map
// picture the player drags the box on, so they have to be replaced
// together with it - a new map with the old corners would hand the terrain
// generator coordinates for somewhere else entirely.
```

Lines 23–24 · comment · before public static Vector2 LatLonFromNormalized(float u, float v)

```
// u = 0 left, 1 right
// v = 0 bottom, 1 top
```

Assets/Scripts/AreaSelection/Districts/DistrictAreaMap.cs

71 comments · 71 comment lines removed · file had 366 lines

Lines 7–14 · doc comment · before [Serializable]

```
/// <summary>
/// The three pictures one district needs on the area selection screen.
///
/// All three are drawn on the same canvas as the base satellite map and must
/// be exported at the same pixel size, because the game lines them up by
/// stretching each one over the same rectangle. If one of them is a different
/// shape, its barangays will not sit where the satellite says they are.
/// </summary>
```

Line 44 · doc comment · before Valid,

```
/// <summary>Every sampled point of the box is on green land.</summary>
```

Line 47 · doc comment · before OutsideDistrict,

```
/// <summary>Some part of the box is off the district altogether.</summary>
```

Line 50 · doc comment · before NonAgricultural,

```
/// <summary>The box is inside the district but touches red land.</summary>
```

Line 53 · doc comment · before SpotMapMissing,

```
/// <summary>No agricultural area spot picture has been assigned yet.</summary>
```

Line 56 · doc comment · before SpotMapUnreadable

```
/// <summary>The picture is assigned but the game is not allowed to read it.</summary>
```

Lines 60–78 · doc comment · before public static class DistrictAreaSpot

```
/// <summary>
/// Reads a district's "barangay agricultural area spot" picture.
///
/// The picture is a flat map: green where the barangay is an agricultural
/// area, red where it is not, and white everywhere outside the district. That
/// is the whole rule - the game asks this picture what colour is under the
/// player's box and answers accordingly, the same way the old screen asked the
/// district border map what colour a point was.
///
/// Colours are classified by which channel leads rather than by matching an
/// exact swatch, so a slightly different green, a resized import or a little
/// JPEG-ish softening around the shapes all still read correctly. Pixels that
/// lead with neither - the blended ring one pixel wide where green meets red -
/// are reported as undecided and simply ignored.
///
/// Each picture is decoded once and kept, because the player steps back and
/// forth between districts and re-reading a few million pixels on every
/// Generate would stutter.
/// </summary>
```

Lines 102–107 · doc comment · before public static void Prewarm(Sprite spot)

```
/// <summary>
/// Decodes a picture ahead of time. Reading one costs a few hundred
/// thousand pixels, which is nothing spread over the frames after the
```

```

/// screen opens but is a visible hitch if it lands on the frame the player
/// presses >> on.
/// </summary>

```

Lines 113-116 · doc comment · before `public static void ClearCache()`

```

/// <summary>
/// Drops every decoded picture. Called when the area selection screen goes
/// away, so the farm does not carry a few megabytes of map around with it.
/// </summary>

```

Lines 123-128 · doc comment · before `public static bool TryGetFootprint(Sprite spot, out Rect bounds, out Vector2 center)`

```

/// <summary>
/// The district's own extent in the picture, as a normalized rect and
/// centroid (u = 0 left to 1 right, v = 0 bottom to 1 top). Green and red
/// together make up the district, so both count toward the footprint.
/// Used to aim the camera at the district and to park the selection box.
/// </summary>

```

Lines 143-148 · doc comment · before `public static AreaSelectionVerdict Validate(Sprite spot, Rect normalizedRect, int resolution)`

```

/// <summary>
/// Checks the box the player dragged against the district's spot picture.
/// Every sampled point has to be on green land; one point off the district
/// or on red land is enough to refuse, which is what keeps a plot from
/// straddling a boundary it should not cross.
/// </summary>

```

Lines 187-190 · comment · before `if (anyOutside || (!anyAgricultural && !anyNonAgricultural))`

```

// Leaving the district is the more basic mistake, so it is reported
// first even when the box also clips a red barangay. A box that landed
// entirely on blended edge pixels counts as off the district too -
// there is no evidence it is on farmland.

```

Lines 247-248 · comment · before `Rect region = spot.textureRect;`

```

// textureRect rather than the whole texture, so a sprite packed into an
// atlas or trimmed on import still reads its own region.

```

Lines 294-297 · comment · before `if (count > width * (long)height * 6L / 10L)`

```

// One district on a whole-city canvas is a small part of it. Much more
// than this and the wrong picture is almost certainly in the slot - a
// satellite photo reads as green almost everywhere, and would quietly
// approve every area the player picked.

```

Line 341 · comment · before `if (min >= 200 && max - min <= 30)`

```

// Paper white is the land that does not belong to this district.

```

Lines 345-350 · comment · before `if (max <= 60)`

```

// Dark ink is undecided rather than outside, which matters if the spot
// map is ever exported with barangay outlines on it: a box sitting on
// the line between two green barangays would otherwise be refused as
// off the district. Undecided points are ignored, so the green either
// side of the line still decides it. A box that finds nothing but ink
// is refused anyway, so a black background behaves like a white one.

```

Assets/Scripts/AreaSelection/Districts/DistrictDescriptions.cs

18 comments · 18 comment lines removed · file had 167 lines

Lines 5-22 · doc comment · before `public static class DistrictDescriptions`

```

/// <summary>
/// What each Davao City district is like, shown on the area selection map
/// before the player commits to one.
///
/// Two different kinds of fact are mixed here, and it is worth knowing which
/// is which:
///
/// The STARTER CROPS section is read off DistrictCropPools, so it is exactly
/// what the game will hand the player. If a pool changes, the matching lines
/// here have to change with it.
///
/// The opening paragraph and the TERRAIN section are real-world background
/// about Davao City, written from general knowledge rather than taken from a
/// cited elevation and slope map. They are accurate in outline - which
/// districts are highland, which are coastal, what each is known for growing
/// - but the specific figures should be checked against a proper source
/// before any of this is quoted in the paper.
/// </summary>

```

Assets/Scripts/AreaSelection/Selection/AreaSelectionBuilder.cs

263 comments · 263 comment lines removed · file had 1729 lines

Lines 13-28 · doc comment · before `public class AreaSelectionBuilder : MonoBehaviour`

```

/// <summary>
/// Picking where the farm will be, in two steps.
///
/// STEP ONE - the district. Only the district being looked at is tinted on the
/// map, and the camera travels to it as the player steps through with the
/// arrows. There is no selection box yet: choosing a district and choosing a
/// plot inside it are different questions, and asking both at once was what

```

```

/// left beta testers unsure which district they were even looking at.
///
/// STEP TWO - the area. Pressing Next locks the district in and swaps the map
/// to that district's barangays, drawn green where farming is allowed and red
/// where it is not. The box appears, Generate replaces Next, and Generate
/// checks the box against the district's agricultural area spot picture. Being
/// inside a production district is no longer enough on its own - a barangay
/// inside one can still be off limits, and the red areas are where.
/// </summary>

Lines 75-76 · comment · before [Tooltip("Let the player drag the map inside the frame. The drag is held to the " +
// Replaces lockMapView under a new name on purpose: the scene saved that one
// as on, and carrying it across would keep dragging switched off.

Line 153 · doc comment · before private enum SelectionPhase
/// <summary>Which of the two questions the screen is asking right now.</summary>

Line 199 · comment · before private float appliedGrabZoom = -1f;
// The map zoom the box's grab area was last sized for; -1 forces a resize.

Lines 236-238 · comment · before var oldTutorial = GameObject.Find(TutorialRootName);
// Built under the canvas rather than under the screen's own root, so it
// sits above everything on it; cleared here for the same reason the root
// is, so building the screen twice does not stack two tutorials.

Lines 276-277 · comment · before districtBoard = DescriptionBoard.Create(viewportRect, theme);
// Built last so it is the newest sibling and already draws over the
// map and the selector; Show also re-raises it each time.

Lines 288-291 · comment · before Canvas.ForceUpdateCanvases();
// Sized and framed once here and again from the coroutine below. The
// map's height is worked out from the canvas width, and if that is
// still zero on this frame the map would be a zero-height strip until
// the coroutine catches it - a blank screen for the first frame or two.

Lines 308-313 · doc comment · before private IEnumerator LayoutThenFrameFirstDistrict()
/// <summary>
/// The map's height depends on the canvas width, which is not known until
/// the layout has run, so the very first framing waits for it. Later
/// district switches do not - by then the sizes are settled, and waiting
/// would put a two frame stutter at the start of every glide.
/// </summary>

Lines 326-328 · comment · before for (int i = 0; i < DistrictCount(); i++)
// The remaining districts are decoded one per frame while the player is
// still reading the first screen, so pressing >> never waits on a
// picture being read for the first time.

Lines 343-344 · comment · before DistrictAreaSpot.ClearCache();
// The decoded area spot maps are only of use on this screen, and the
// cache is static, so it would otherwise follow the player into the farm.

Lines 361-363 · comment · before private static DistrictMapArt[] BuildPlaceholderArt()
// -----
// District art
// -----

Lines 376-385 · doc comment · before private void EnsureDistrictArt()
/// <summary>
/// Keeps the art list usable no matter what state the Inspector is in: a
/// list that has never been filled in gets every production district as
/// empty rows, a row added by hand gets a name so it is not a blank entry in
/// the selector, and a district the game knows but this list does not is
/// added at the end. Without that last step a district added in code would
/// be missing from the selector while the shop and the seed pools already
/// name it. Its row starts empty, and the screen refuses to generate on it
/// until the three pictures are dropped in.
/// </summary>

Lines 454-458 · doc comment · before private void WarnAboutMismatchedArt()
/// <summary>
/// Every picture is stretched over the same rectangle, so one exported at a
/// different shape would put its barangays somewhere the satellite does not
/// agree with. Cheaper to say so here than to hunt it as a visual bug.
/// </summary>

Lines 526-528 · comment · before private Canvas EnsureCanvas()
// -----
// Scene plumbing
// -----

Line 540 · trailing comment · on the line scaler.matchWidthOrHeight = 1f; // landscape: scale by height
// landscape: scale by height

Line 555 · trailing comment · on the line canvasScaler.matchWidthOrHeight = 1f; // landscape: scale by height
// landscape: scale by height

Lines 566-568 · comment · before private void CreateBackdrop(RectTransform parent)
// -----
// The map

```

```
// -----

Lines 570-578 · doc comment · before private void CreateBackdrop(RectTransform parent)
/// <summary>
/// The same picture as the map, drawn behind it at the size it would have if
/// it were not zoomed at all, and dimmed.
///
/// It only shows where the framed map does not reach, which happens when a
/// district has been pulled back to get clear of the UI. A flat panel there
/// reads as a hole in the screen; the same view dimmed reads as the map
/// carrying on past the edge of what is in focus, which is what it is.
/// </summary>

Lines 616-624 · doc comment · before private void CreateFrameGroup(RectTransform parent)
/// <summary>
/// The wooden frame and the window it holds.
///
/// The window is what the map is now seen through, so it - not the whole
/// screen - is what the district framing measures against, and it is masked
/// so a zoomed map is cut off at the frame rather than spilling across the
/// screen. The frame itself is drawn after the window so its inner edge
/// laps over the map, the way a real frame sits on a photograph.
/// </summary>

Lines 653-655 · comment · before mapFrameImage.preserveAspect = false;
// Stretched rather than kept to its own proportions - the art is a
// landscape frame and the map is upright, which is the warp the frame
// was asked to take.

Line 660 · doc comment · before private Vector2 FrameInset()
/// <summary>How far the frame art insets its opening, as a fraction of its size.</summary>

Lines 677-681 · comment · before rect.anchorMin = new Vector2(0f, 0.5f);
// Pinned to the left and right edges of the screen and centred
// vertically, with the height worked out from the picture's own shape
// in LayoutMap. The map used to be stretched to the screen on both
// axes, which squashed it; now it keeps its proportions and is simply
// taller than the screen, which is what the view scrolls through.

Lines 692-695 · comment · before var overlay = CreateImage("DistrictHighlight", rect, null);
// The per-district picture rides on the map as a child, so zooming and
// panning move the two together and the barangays stay where the
// satellite puts them. It never takes clicks - the box and the map's own
// zoom handler are underneath it.

Lines 711-715 · comment · before zoomPan.allowUserPan = allowMapDragging;
// Both are held to the district. FocusCurrentDistrict raises the zoom floor
// to the district's own framing and records that view as the anchor, and
// the controller keeps every later view - zoomed or dragged - inside it.
// So zooming out stops where the district opened, and a drag can explore
// a zoomed-in district without ever reaching the one next door.

Lines 722-730 · doc comment · before private void LayoutMap()
/// <summary>
/// Gives the map its true shape at the current screen width. Called once the
/// canvas has laid out, and again whenever the window changes size.
/// </summary>
/// <summary>
/// Hangs the framed picture in the gap the UI leaves and sizes the map to
/// the opening, so that at zoom 1 the whole of Davao sits in the frame and
/// zooming in frames one district of it.
/// </summary>

Lines 745-747 · comment · before float clearHeight = Mathf.Max(160f, screenHeight - uiTopMargin - uiBottomMargin);
// The picture hangs in the strip between the hint plank and the button
// row, centred on that strip rather than on the screen - the two bands
// are not the same height.

Lines 754-756 · comment · before Vector2 inset = FrameInset();
// The frame is bigger than its opening by however much of the art is
// rail, so the opening lands exactly on the strip and the woodwork
// reaches out past it, behind the plank and the sign.

Lines 769-770 · comment · before mapRect.sizeDelta = new Vector2(0f, openingWidth * aspect);
// Width comes from the window through the stretched anchors; the height
// is the map's own shape, which makes it exactly fill the opening.

Lines 773-774 · comment · before if (backdropRect != null)
// The dimmed copy is the screen background now, so it covers the screen
// rather than the opening.

Lines 803-804 · comment · before return 2950f / 2500f;
// Nothing assigned yet: fall back to the shape the Davao maps are drawn
// at, so the screen still looks right while the art is being made.

Lines 808-810 · comment · before private RectTransform CreateSelectionBox(RectTransform parent)
// -----
// The selection box
// -----
```

Lines 830-831 · comment · before `image.sprite = frame;`

```
// The frame art already reads as a border, so the plain outline
// used by the flat-colour fallback is switched off.
```

Lines 859-864 · doc comment · before `private void LayoutSelectionBox()`

```
/// <summary>
/// Keeps the box the same slice of the map at any screen size. The size is
/// a fraction of the picture, not of the screen, so the patch of Davao it
/// covers - and therefore the coordinates handed to the terrain generator -
/// does not change when the window does.
/// </summary>
```

Lines 880-893 · doc comment · before `private void UpdateSelectionGrabArea()`

```
/// <summary>
/// Sizes the area the selection box can be grabbed by.
///
/// It used to be padded out to at least 140 units with 56 more on every side,
/// which suited the old full-screen map where the box could not be zoomed.
/// But the padding lives on the box, and the box lives on the zoomed map, so
/// zooming in multiplied it - at four times zoom the grab area was around 600
/// units across, and a drag meant for the map kept picking up the box.
///
/// Now the grab area is the box as drawn. The optional minimum is worked out
/// in screen units and converted back into the box's own units at the current
/// zoom, so it stays the same size on screen however far in the player goes,
/// and stops mattering entirely once the box itself is bigger.
/// </summary>
```

Line 909 · comment · before `selectionImage.raycastPadding = new Vector4(-padX, -padY, -padX, -padY);`

```
// Negative padding grows the raycast area; zero leaves it exactly the box.
```

Lines 932-934 · comment · before `private void CreateGenerateButton(RectTransform parent)`

```
// -----
// Buttons
// -----
```

Lines 948-950 · comment · before `var button = CreateThemedButton()`

```
// Deliberately the same corner as Generate. The two are never on screen
// together - one asks for a district, the other for a plot inside it -
// so the player presses the same place twice to go forward.
```

Lines 1001-1004 · comment · before `if (theme != null && HasAnyDistrictSign())`

```
// A painted sign per district when the art is present, otherwise the
// original plain text label. Both are built once any sign exists, so a
// district whose own sign has not been painted yet still shows its name
// instead of an empty slot. RefreshDistrictChrome picks which one shows.
```

Lines 1059-1063 · comment · before `RectTransform rect = button.GetComponent<RectTransform>();`

```
// Under the district sign. The sign is drawn 130 tall centred at
// y=150, so its painted edge stops around y=85; Back and Generate sit
// out at the screen corners, which leaves the middle of this strip
// free. It stays available after the district is locked in, because
// what it describes - the district - has not changed.
```

Lines 1083-1085 · comment · before `private void ApplyPhase(SelectionPhase newPhase)`

```
// -----
// Phases
// -----
```

Lines 1114-1115 · comment · before `districtOverlayImage.enabled = sprite != null;`

```
// Hidden rather than left as a blank white rectangle over the map when
// a district's picture has not been drawn yet.
```

Lines 1118-1119 · comment · before `if (backdropImage != null)`

```
// The dimmed copy follows whatever the map is showing, so the two never
// disagree about which district is highlighted.
```

Lines 1134-1136 · comment · before `infoText.text = phase == SelectionPhase.District`

```
// The button's wording is painted into its art, so this names the same
// word the player is looking at. Change both together if that art is
// ever replaced.
```

Lines 1150-1152 · comment · before `districtSignImage.enabled = sign != null;`

```
// Hide the slot entirely if this one district's sign is missing,
// rather than showing a blank white box. The text label takes its
// place.
```

Lines 1198-1201 · comment · before `if (zoomPan != null && zoomPan.IsAwayFromFocus)`

```
// Picking an area starts from the district's own framing. Had the player
// zoomed in to look around first, the barangay map would open partly
// hidden, and the box - placed at the middle of the district - could land
// outside the part they are looking at.
```

Lines 1205-1206 · comment · before `if (TryGetDistrictFrame(CurrentDistrictName(), out _, out Vector2 center))`

```
// The box starts in the middle of the district rather than wherever it
// was left, so it is always somewhere the player can see it.
```

```

Lines 1217-1219 · comment · before if (phase == SelectionPhase.Area)
// Back steps out of the area, then out of the screen. Without this the
// only way to change a district after pressing Next would be to leave
// for the main menu and come in again.

Lines 1229-1231 · comment · before private bool TryGetDistrictFrame(string districtName, out Rect bounds, out Vector2 center)
// -----
// Framing
// -----

Lines 1245-1247 · comment · before return DistrictAreaSpot.TryGetFootprint(art.agriculturalAreaSpot, out bounds, out center);
// The district's extent is read off the spot picture: green and red
// together are exactly the barangays that belong to it, which is a
// tighter and more honest frame than any hand-entered rectangle.

Line 1260 · comment · before bounds = new Rect(0f, 0f, 1f, 1f);
// No art to aim at: show the whole map rather than guessing.

Lines 1267-1269 · comment · before zoomPan.clampInsetTop = 0f;
// Nothing covers the opening, so the map simply has to fill it and the
// district simply has to sit in the middle of it. The offsets that used
// to dodge the plank and the sign belong to the frame's placement now.

Lines 1274-1277 · comment · before zoomPan.minZoom = fitZoom;
// The district's own framing becomes the zoom-out limit, so the player
// can move in closer but never back out past the view this district
// opened at. Set before focusing, since both focus calls clamp the zoom
// they are given to this same range.

Lines 1280-1286 · comment · before Vector2 framingPoint = bounds.center;
// The camera aims at the middle of the district's extent rather than at
// its centre of mass. A district shaped like Paquibato - wide at one end,
// tapering at the other - has a centroid well off the middle of its
// outline, and aiming there pushes the far end back under the UI even
// though the whole thing would have fitted. The selection box still
// starts at the centroid, which is somewhere solidly inside the district
// rather than possibly in a notch of its bounding box.

Lines 1297-1301 · doc comment · before private float ComputeFitZoom(Rect bounds)
/// <summary>
/// The zoom to frame a district at, measured against the frame's opening -
/// the map is only seen through that, so the screen and the UI on it no
/// longer come into this at all.
/// </summary>

Lines 1319-1320 · comment · before float needed = Mathf.Min(
// The zoom this district just fits the opening at, with a little to
// spare so it does not sit flush against the frame.

Lines 1325-1328 · comment · before float floor = needed >= 1f ? 1f : Mathf.Max(minimumDistrictZoom, needed);
// The map fills the opening at zoom 1, so that is normally the floor and
// no gap can open inside the frame. A district larger than the opening -
// only possible if the frame is scaled down hard - is allowed to pull
// back just far enough to be seen whole.

Lines 1334-1336 · comment · before private void BuildTutorial(Transform canvasTransform)
// -----
// Tutorial
// -----

Lines 1346-1348 · comment · before tutorialPromptBlocker = new GameObject("PromptBlocker", typeof(RectTransform), typeof(Image));
// Only needed behind the question. The question's plank takes clicks on
// itself, but ENTER, << and >> are still showing around it and would
// otherwise work straight through it.

Lines 1361-1366 · doc comment · before private void OfferTutorial()
/// <summary>
/// Asks whether to run the tutorial, with the very same prompt the farm uses
/// when it asks about its own. The farm builds that popup in its scene and
/// this screen has none, so one is made here - the same class, which is what
/// keeps the two questions looking identical.
/// </summary>

Line 1376 · comment · before tutorialLayer.SetAsLastSibling();
// Layer first, then the popup, so the popup lands above the blocker.

Lines 1409-1413 · doc comment · before private void ShowAreaTutorialIfDue()
/// <summary>
/// The second half of the tutorial, the first time a district is locked in.
/// Once only per visit, so stepping back out and pressing ENTER again does not
/// make the player read it twice.
/// </summary>

Lines 1424-1427 · doc comment · before private static TutorialSlide[] SlidesOrDefault(TutorialSlide[] slides, System.Func<TutorialSlide[]>
fallback)
/// <summary>
/// The Inspector list, or the built-in pages if someone has emptied it, so
/// a Yes never opens onto nothing.
/// </summary>

```

Lines 1468-1470 · comment · before `public void OnGeneratePressed()`

```
// -----  
// Generate  
// -----
```

Lines 1518-1520 · comment · before `private void CreateInfoText(RectTransform parent)`

```
// -----  
// Text, popup and small helpers  
// -----
```

Lines 1526-1527 · comment · before `Transform textParent = parent;`

```
// With art, the hint sits on a wooden plank; without it, the text  
// floats on its own exactly as before.
```

Lines 1543-1544 · comment · before `plankImage.type = Image.Type.Sliced;`

```
// Sliced so the notched ends keep their shape while the middle  
// stretches to fit the longest district name.
```

Line 1558 · comment · before `Stretch(rect);`

```
// Fill the plank, inset so the words stay off the wooden edges.
```

Line 1608 · comment · before `bg.color = new Color(0f, 0f, 0f, 0.80f);`

```
// Unchanged fallback, so the warning still reads before the art exists.
```

Lines 1618-1619 · comment · before `float padX = board != null ? size.x * 0.12f : 20f;`

```
// Inset off the plank's painted edges when themed; the flat box needs  
// only a small margin.
```

Line 1630 · comment · before `popupText.color = board != null ? Color.black : Color.white;`

```
// Dark ink on wood, white on the flat fallback.
```

Lines 1647-1649 · comment · before `float seconds = Mathf.Clamp(1.6f + message.Length * 0.035f, 2.6f, 6f);`

```
// The non-agricultural warning is twice the length of the old one, and  
// 2.6 seconds is not enough to read it, so longer messages stay up  
// longer instead of every player having to trigger it twice.
```

Lines 1669-1673 · doc comment · before `private Button CreateThemedButton()`

```
/// <summary>  
/// A button that uses its painted sprite when one is set (the wording is  
/// in the art, so no Text child is added), and otherwise falls back to  
/// the original dark box with a text label.  
/// </summary>
```

Assets/Scripts/AreaSelection/Selection/DraggableSelectionBox.cs

20 comments · 20 comment lines removed · file had 167 lines

Line 45 · comment · before `if (dragging && eventData.pointerId != dragPointerId)`

```
// A second finger landing mid-drag must not take the box over.
```

Lines 57-60 · comment · before `grabOffset = boxRect.anchoredPosition - pointer;`

```
// Kept for the whole drag, so the box travels with the pointer from  
// wherever it was grabbed. The version before this measured the grab  
// from the box's centre while positioning it by its corner, which made  
// the box jump half its own size down and left the moment a drag began.
```

Lines 82-85 · comment · before `if (visibleArea != null && TryGetVisibleBounds(out Rect visible))`

```
// Held to what can actually be seen. The map is larger than the frame's  
// window once it is zoomed, and a box pushed past the window's edge  
// carries on across the hidden part of the map where the player has no  
// way to get it back.
```

Lines 94-95 · comment · before `if (maxX < minX) minX = maxX = (minX + maxX) * 0.5f;`

```
// A window narrower than the box leaves no valid range at all; centre it  
// rather than letting the clamp pick one edge.
```

Lines 120-123 · doc comment · before `private bool TryPointerInArea(PointerEventData eventData, out Vector2 point)`

```
/// <summary>  
/// The pointer measured from the drag area's bottom-left corner - the same  
/// space the box's anchoredPosition is in, since the box is anchored there.  
/// </summary>
```

Lines 141-145 · doc comment · before `private bool TryGetVisibleBounds(out Rect bounds)`

```
/// <summary>  
/// The part of the drag area currently showing through  
/// <see cref="visibleArea"/>, in the same bottom-left space. Worked out from  
/// world corners, so the map's zoom and pan are already accounted for.  
/// </summary>
```

Assets/Scripts/AreaSelection/Selection/LoopingVideoBackground.cs

34 comments · 34 comment lines removed · file had 267 lines

Lines 7-25 · doc comment · before `public class LoopingVideoBackground : MonoBehaviour`

```
/// <summary>  
/// A video that fills the screen behind a menu and keeps looping.  
///  
/// Unity's VideoPlayer does not reliably keep running on its own. It stops at
```

```

/// the loop point on some devices, loses its render texture when the app is
/// backgrounded or the on-screen keyboard opens and closes, and can sit
/// "playing" while the picture is frozen. Setting isLooping and walking away
/// was the cause of the main menu background stopping after a while.
///
/// So the looping is watched rather than trusted. The player is re-asserted at
/// every loop point, restarted on an error, restarted when the app comes back
/// to the foreground or the keyboard closes, and checked a few times a second
/// for a frame counter that has stopped moving. The render texture is rebuilt
/// if it was lost. This is the same treatment the main menu background needed,
/// kept in one place so a second screen does not have to rediscover it.
///
/// Audio is switched off at the source - a background loop is decoration, and
/// whatever music is baked into the clip would fight the game's own.
/// </summary>

```

Lines 43-47 · doc comment · before public static LoopingVideoBackground Create(

```

/// <summary>
/// Builds the picture and its player under <paramref name="parent"/>, filling
/// it. Returns null when there is no clip to play, so the caller can fall
/// back to a still background.
/// </summary>

```

Lines 67-68 · comment · before raw.enabled = false;

```

// Hidden until the first frame lands, so the fallback behind it shows
// instead of one frame of whatever the texture happened to contain.

```

Lines 95-96 · comment · before player.audioOutputMode = VideoAudioOutputMode.None;

```

// The clip's own soundtrack is discarded here rather than turned down,
// so it can never be heard however the game's volume is set.

```

Lines 182-185 · doc comment · before private bool IsStalled()

```

/// <summary>
/// True when the player claims to be playing but the picture has not moved
/// for a while - the failure that looks like the video simply stopping.
/// </summary>

```

Lines 223-224 · comment · before if (!source.isPlaying)

```

// Some devices stop at the loop point instead of wrapping, so the wrap
// is done by hand when that happens.

```

Assets/Scripts/AreaSelection/Selection/MapZoomPanController.cs

81 comments · 81 comment lines removed · file had 515 lines

Lines 49-53 · comment · before private bool hasFocusAnchor;

```

// The view a district opened at, captured by FocusNormalizedPoint. Zooming
// toward a point moves the content as well as scaling it, so without an
// anchor to measure against, zooming in on one corner and back out on
// another leaves the map somewhere it never started - the floor returns the
// right scale but not the right place.

```

Lines 58-61 · comment · before private bool gliding;

```

// A district switch travels to the next framing instead of cutting to it.
// While that is running the focus anchor is dropped, because the anchor
// holds the view at the district it was set for and would drag the glide
// backwards; it is re-armed on the last frame at the district arrived at.

```

Line 70 · doc comment · before public bool IsGliding => gliding;

```

/// <summary>True while the view is travelling to a new district.</summary>

```

Lines 89-90 · comment · before if (gliding || !allowUserZoom || viewport == null || content == null)

```

// Input is ignored mid-glide rather than cancelling it: a wheel notch
// during the travel would fight the tween for the same two values.

```

Lines 104-105 · comment · before if (draggingMap && eventData.pointerId != dragPointerId)

```

// A second finger joining a drag is the start of a pinch, which Update
// handles. It must not take the drag over.

```

Lines 114-122 · comment · before if (!RectTransformUtility.ScreenPointToLocalPointInRectangle(

```

// Every button pans, including the left one. It used to be kept for the
// selection box, but that was never needed: the event system hands a
// drag to whichever object it started on, and the box is its own drag
// handler drawn over the map. A drag that starts on the box moves the
// box; one that starts on bare map moves the map.
//
// Measured from where the press began rather than where the drag
// threshold was crossed, so the map stays under the finger instead of
// lagging behind it by the threshold distance.

```

Lines 153-154 · comment · before if (gliding || pinching)

```

// Tracked but not applied while a pinch or a glide owns the view, so the
// map does not leap by the whole distance moved when either one ends.

```

Lines 160-162 · comment · before ClampContentToViewport();

```

// This is what keeps the drag inside the district: the same clamp that
// stops zooming out past the district's framing also stops a drag from
// carrying the view past it.

```

Lines 285–287 · comment · before `float requiredHeight = Mathf.Max(120f, viewportHeight - clampInsetTop - clampInsetBottom);`

```
// The map has to cover the strip the player can actually see the map
// through, not the whole screen. With both insets at zero that is the
// whole screen and this behaves exactly as it did before.
```

Lines 301–315 · comment · before `if (hasFocusAnchor && focusZoom > 0.0001f)`

```
// Hold the view inside the region the district opened showing.
//
// At the focus zoom the viewport framed a rectangle of the map; at any
// higher zoom it frames a smaller one, and the rule here is simply that
// the smaller rectangle must stay inside the original. Writing the
// viewport centre in content-local units as c = -pos / scale, that reads
//
// |c - c0| <= (V / 2) * (1 / focusZoom - 1 / scale)
//
// which rearranges into the position clamp below with k = scale/focusZoom.
// The allowance is exactly zero at k = 1, so the map is not merely
// nudged back toward the district framing as it zooms out - it arrives
// at precisely the position it started from, from any path in or out.
// Above the floor the allowance opens up smoothly, so zooming in still
// travels toward whatever the player pointed at.
```

Lines 346–347 · comment · before `hasFocusAnchor = false;`

```
// No district is framed any more, so nothing to hold the view to, and
// a glide still in flight would immediately undo this.
```

Lines 352–356 · doc comment · before `public void FocusNormalizedPoint(Vector2 uv, float zoom)`

```
/// <summary>
/// Zooms to <paramref name="zoom"/> and pans so the given normalized content
/// point (0..1, origin bottom-left) sits at the centre of the viewport.
/// The result is clamped to the viewport just like interactive panning.
/// </summary>
```

Lines 362–363 · comment · before `hasFocusAnchor = false;`

```
// Dropped before positioning: the previous district's anchor would
// otherwise clamp this one's framing back toward the old view.
```

Lines 381–385 · comment · before `focusZoom = z;`

```
// Recorded after the clamp, not before. A district near the edge of the
// map cannot actually be centred - the clamp slides it back so the map
// still covers the viewport - and the anchor has to be the view the
// player really sees, or zooming out would return them to a framing that
// was never on screen.
```

Lines 391–395 · doc comment · before `public void GlideToNormalizedPoint(Vector2 uv, float zoom, float seconds)`

```
/// <summary>
/// The same framing as <see cref="FocusNormalizedPoint"/>, but travelled to
/// over <paramref name="seconds"/> instead of jumped to, so stepping through
/// districts reads as the map moving rather than as a cut.
/// </summary>
```

Lines 410–413 · comment · before `FocusNormalizedPoint(uv, zoom);`

```
// The destination is worked out by actually going there and reading the
// result back, because where the map ends up is decided by the viewport
// clamp - a district near an edge cannot be centred - and that clamp
// only ever runs against the live transform.
```

Lines 429–432 · doc comment · before `public bool IsAwayFromFocus`

```
/// <summary>
/// True when the player has zoomed in or dragged away from the view the
/// current district opened at.
/// </summary>
```

Lines 445–448 · doc comment · before `public void GlideBackToFocus(float seconds)`

```
/// <summary>
/// Travels back to the view the current district opened at, from wherever
/// the player has zoomed or dragged to.
/// </summary>
```

Line 469 · comment · before `hasFocusAnchor = false;`

```
// Re-armed on the glide's last frame, at these same values.
```

Line 473 · doc comment · before `public void CancelGlide()`

```
/// <summary>Stops a glide where it stands. Nothing happens if none is running.</summary>
```

Lines 493–494 · comment · before `float eased = t * t * (3f - 2f * t);`

```
// Smoothstep, so the map pulls away and settles instead of starting and
// stopping at full speed.
```

Assets/Scripts/AreaSelection/Selection/TutorialSlideBoard.cs

38 comments · 38 comment lines removed · file had 311 lines

Line 7 · doc comment · before `[Serializable]`

```
/// <summary>One page of a picture-and-caption tutorial.</summary>
```

Lines 29–41 · doc comment · before `public class TutorialSlideBoard`

```
/// <summary>
/// A short picture-book tutorial: one page at a time on the log-ended board,
```

```

/// turned with the >> plank, closing itself after the last page.
///
/// It sits over a dim full-screen layer that swallows every click, so the
/// screen behind cannot be used while a page is up - otherwise a player could
/// press ENTER through the page explaining what ENTER does.
///
/// A plain class rather than a MonoBehaviour, the same as DescriptionBoard: it
/// owns its objects and needs no update loop. It also uses DescriptionBoard's
/// board art and lettering, so the tutorial reads as part of the same screen
/// rather than as something bolted on.
/// </summary>

```

Line 47 · comment · before `private const float ButtonBottom = 42f;`
`// Heights measured up from the board's bottom edge.`

Line 55 · doc comment · before `private const float ButtonWidth = 250f;`
`/// <summary>RightButton.png is 400x180, so preserveAspect keeps this true.</summary>`

Line 70 · doc comment · before `public static TutorialSlideBoard Create(Transform parent, UIThemeSprites theme)`
`/// <summary>Builds the board hidden. Call <see cref="Show"/> to open it.</summary>`

Line 117 · comment · before `float inset = DescriptionBoard.ContentInset(theme, BoardWidth);`
`// How far in from each side the planks start, clear of the painted logs.`

Lines 143–144 · comment · before `pictureImage.preserveAspect = true;`
`// Screenshots come in whatever shape they were captured at; they are`
`// fitted inside the band rather than stretched to it.`

Lines 159–160 · comment · before `bodyText.resizeTextForBestFit = true;`
`// The longest page is about three times the length of the shortest, so`
`// the size gives way before any line can be cut off.`

Line 171 · comment · before `RectTransform rect = go.GetComponent<RectTransform>();`
`// Bottom right, its right edge against the log, where the mockups put it.`

Lines 205–208 · doc comment · before `private static void PlaceBand(RectTransform rect, float inset, float bottom, float top)`
`/// <summary>`
`/// A horizontal band of the board, inset from the logs on both sides and`
`/// running between two heights measured up from the board's bottom edge.`
`/// </summary>`

Lines 218–222 · doc comment · before `public void Show(TutorialSlide[] slides, Action finished)`
`/// <summary>`
`/// Opens on the first page. <paramref name="finished"/> runs once the last`
`/// page is turned. With no pages to show it runs straight away, so a`
`/// caller waiting on it is never left stuck.`
`/// </summary>`

Lines 252–255 · doc comment · before `private void FitToScreen()`
`/// <summary>`
`/// Shrinks the board on a screen too narrow or short for it, so the >> plank`
`/// can never end up off the edge where the player could not press it.`
`/// </summary>`

Lines 265–266 · comment · before `if (width < 1f || height < 1f)`
`// Before the canvas has laid out its size reads as zero; full size is`
`// the right answer then, not the smallest allowed.`

Assets/Scripts/MainMenu/Account/AccountRules.cs

63 comments · 63 comment lines removed · file had 178 lines

Lines 3–29 · doc comment · before `public static class AccountRules`

```

/// <summary>
/// The account rules the server enforces, checked on the phone first so a
/// player sees the problem the moment they press the button instead of after
/// a round trip. There is one method per form, each named after the request
/// it guards: sign-up, the new password at the end of a reset, and the three
/// changes on the Account panel.
///
/// The server stays the authority. These copy RegisterRequest,
/// ResetPasswordRequest, ChangeEmailRequest, ChangePasswordRequest,
/// ChangeDisplayNameRequest and AccountFields.validateDisplayName, and they
/// must not turn away a player the server would accept - a client rule
/// tighter than the server's is worse than the round trip it saves, because
/// it locks someone out with no way round it. Where the server is loose, this
/// is loose too. If either side changes, change both.
///
/// Anything that needs the database stays with the server alone: whether an
/// address or display name is taken or already yours, and whether a current
/// password is the right one.
///
/// Each method returns the first problem it finds, checking fields in the
/// order they appear from top to bottom, or null when everything here passes.
/// Callers pass emails and display names already trimmed and passwords exactly
/// as typed, which is how each is sent.
///
/// Deliberately free of any Unity type, so it can be compiled and exercised

```

```

/// on its own.
/// </summary>

Line 38 · doc comment · before public static string SignupProblem(string email, string password,
/// <summary>Create Player Account (RegisterRequest).</summary>

Lines 44–46 · comment · before return EmailProblem(email, "Enter your email address.")
// Blank is fine for the display name: it is optional at sign-up, and
// the server stores none. Only a name that was actually typed has to
// meet the rules.

Line 52 · doc comment · before public static string ResetPasswordProblem(string newPassword, string confirmPassword)
/// <summary>The new password at the end of a reset (ResetPasswordRequest).</summary>

Line 58 · doc comment · before public static string ChangeEmailProblem(string newEmail)
/// <summary>Change Email on the Account panel (ChangeEmailRequest).</summary>

Lines 64–68 · doc comment · before public static string ChangePasswordProblem(string currentPassword,
/// <summary>
/// Change Password on the Account panel (ChangePasswordRequest). The
/// current password only has to be filled in: its length is not checked,
/// because the server does not check it either.
/// </summary>

Lines 78–82 · doc comment · before public static string ChangeDisplayNameProblem(string displayName)
/// <summary>
/// Change Display Name on the Account panel (ChangeDisplayNameRequest).
/// Unlike at sign-up, a blank is refused: this request exists to set a
/// name, and the server will not take an empty one.
/// </summary>

Lines 107–111 · doc comment · before private static string PasswordProblem(string password, string confirmPassword, bool isNew)
/// <summary>
/// A password and its confirmation. On the forms that replace a password
/// the messages say "new password", so a player looking at a current and a
/// new password box is never left guessing which one a message means.
/// </summary>

Lines 141–142 · comment · before if (displayName.IndexOf('@') >= 0)
// Sign-in tries typed text as an address before a display name, so
// a name containing @ could never be signed in with.

Lines 149–161 · doc comment · before public static bool LooksLikeEmail(string email)
/// <summary>
/// Something before an @, something after it, and no spaces.
///
/// That catches the mistakes people actually make - no @ at all, a stray
/// space, or a display name typed into the email box - without judging the
/// domain, which the server does not do either: "name@localhost" is valid
/// to it, so it is valid here.
///
/// The one place this is stricter is a quoted local part containing a
/// space, such as "john doe@example.com, which the server's validator
/// technically allows. No real mailbox a player would sign up with looks
/// like that, and rejecting spaces catches a far more common typo.
/// </summary>

```

Assets/Scripts/MainMenu/Account/AccountUIBuilder.cs

137 comments · 137 comment lines removed · file had 830 lines

```

Lines 8–19 · doc comment · before public class AccountUIBuilder : MonoBehaviour
/// <summary>
/// The Account board, opened from the Main Menu.
///
/// Shows the account's display name and email, and offers the three things a
/// signed-in player can change about it. Each change is its own small board,
/// and every one of them ends back here with the new value already on the
/// planks - the panel reads AuthSession.CurrentUser, which the session
/// updates as soon as the server confirms a change.
///
/// Built at runtime like every other panel in the game, out of the planks the
/// tutorial and trade screens already ship: there are no UI prefabs to edit.
/// </summary>

Lines 24–32 · doc comment · before private const float PanelWidth = 1200f;
/// <summary>
/// Wider than the login board, for two reasons. The rows have to be tall
/// enough to seat a caption plank and a value under it, and these planks
/// are drawn at their own proportions rather than squashed - ObjectivePlank
/// is a wide 1400x300 and TradeRequestBoard a squat 1520x600, neither
/// carrying 9-slice borders - so a row only gets taller by getting wider.
/// And the board's rolled logs eat a fixed amount off each side, so a
/// wider board is the only way to buy usable width between them.
/// </summary>

Lines 35–43 · comment · before private const float BoardArtWidth = 1760f;
// Where TradeConfirmBoard.png actually stops being a rolled log and
// starts being the plank wall, measured off the 1760x1320 art: the left
// log runs to x=280 and the right one begins again at x=1458.

```

```
//
// These are NOT the sprite's 9-slice border, which is only 130 either
// side. That border is set to the least that can be repeated without the
// art distorting, not to the edge of the painted log, so laying content
// out against it puts the first 80-odd units of every row on top of the
// log. Which is exactly what it did.
```

Lines 48-57 · doc comment · before `private float BoardContentInset(float panelWidth)`

```
/// <summary>
/// How far in from each edge the plank wall begins, for a board of this
/// width.
///
/// It depends on the width because of how 9-slicing draws: the corners
/// keep their true size and everything between them is stretched, so the
/// part of the log that falls outside the border region moves inward as
/// the board grows. Computed rather than fixed so that changing a panel's
/// width cannot silently push its contents back onto the wood.
/// </summary>
```

Line 73 · comment · before `return Mathf.Max(left, right);`

```
// The wider of the two, so the layout stays centred and clears both.
```

Line 77 · doc comment · before `private float BoardContentWidth(float panelWidth)`

```
/// <summary>Usable width between the two logs on a board this wide.</summary>
```

Lines 83-89 · doc comment · before `private const float DetailsHeight = 800f;`

```
/// <summary>
/// Every board is kept well under 900 tall on purpose. The interface-size
/// setting shrinks each canvas's reference resolution, so at its 1.20
/// ceiling only 900 units of height are left: a taller board would push
/// its own heading off the top of the screen, and one that reached the
/// full 900 would sit under the status line along the bottom.
/// </summary>
```

Lines 120-124 · doc comment · before `private string pendingNewEmail;`

```
/// <summary>
/// The address a pending email change is moving to. Kept only to word the
/// code board; the server reads the real target from its own pending
/// record, so nothing here can redirect a change.
/// </summary>
```

Lines 127-131 · doc comment · before `private bool busy;`

```
/// <summary>
/// One request at a time. Without it a double tap on Enter sends two
/// code requests, and the second is refused by the resend cooldown -
/// which the player reads as the button being broken.
/// </summary>
```

Line 162 · comment · before `public void Show()`

```
// ----- showing
```

Lines 198-202 · doc comment · before `private void RefreshDetails()`

```
/// <summary>
/// Repaints the two planks from the live session. Also runs on every
/// session change, so a name or address edited here lands on the board
/// without this panel having to be told which change happened.
/// </summary>
```

Lines 233-237 · doc comment · before `private void Fail(string message)`

```
/// <summary>
/// Ends a request: clears the busy latch and reports what went wrong.
/// Every failure path goes through here so none of them can leave the
/// panel stuck refusing to send anything again.
/// </summary>
```

Line 244 · comment · before `private void OnChangeEmailPressed()`

```
// ----- change email
```

Lines 260-263 · comment · before `string problem = AccountRules.ChangeEmailProblem(newEmail);`

```
// Only the form of the address is checked here, by the rules the
// server applies to any address. Whether it is free is the server's
// call, and whether it can receive mail is settled by the code
// arriving at it.
```

Line 346 · comment · before `private void OnChangePasswordPressed()`

```
// ----- change password
```

Lines 366-369 · comment · before `string problem = AccountRules.ChangePasswordProblem(current, next, confirm);`

```
// Checked here as well as on the server so a mismatch, or a new
// password the server would refuse, is caught before the current
// password is sent anywhere. Whether the current one is right is
// only the server's to say.
```

Line 392 · comment · before `private void OnChangeDisplayNamePressed()`

```
// ----- change display name
```

Lines 411-413 · comment · before `string problem = AccountRules.ChangeDisplayNameProblem(displayName);`

```
// AccountRules copies the server's display-name rules, including why
// an @ sign is refused. Whether the name is taken, or already yours,
```

```

// is the server's call.

Lines 449-450 · comment · before if (response != null && !string.IsNullOrEmpty(response.devCode))
// Only ever filled in when the server was deliberately started with
// APP_VERIFICATION_EXPOSE_CODE=true; a normal deployment never sends it.

Line 457 · comment · before private void Build()
// ----- building

Lines 469-470 · comment · before blocker.raycastTarget = true;
// Swallows every press behind the board, so the menu underneath
// cannot be driven while an account change is in progress.

Lines 488-490 · comment · before BuildStatusLine();
// Built last so it draws over the boards rather than under them.
// Sibling order is what decides that here, and a server message that
// ends up behind a plank is a message the player never reads.

Lines 496-500 · doc comment · before private void BuildStatusLine()
/// <summary>
/// The one line that carries every server message, stretched to the
/// screen rather than given a fixed width: these are the longest strings
/// on the panel and they must not lose their tail at any text size.
/// </summary>

Lines 511-513 · comment · before rect.sizeDelta = new Vector2(-120f, 100f);
// Pinned low and kept shallow so that at the 1.20 interface ceiling,
// where only 900 units of height exist, it still clears the bottom of
// the tallest board rather than sitting across its Enter button.

Lines 525-529 · comment · before Vector2 halfButton = new Vector2(300f, 100f);
// 96 tall against a 290 width is a 3.0 ratio to the plank's own 2.53,
// near enough that the wood does not read as stretched. Sliced rather
// than aspect-preserved because a button has to be the size the row
// gives it, and with no border regions on this sprite the two modes
// differ only in whether that size is honoured.

Lines 539-540 · comment · before CreateButton(parent, "AccountClose", new Vector2(0f, -668f),
// Back rather than Enter: this board only reports and dispatches, so
// there is nothing on it to confirm - leaving is all the button does.

Lines 545-548 · doc comment · before private void CreateDetailRow(Transform parent, string name, string caption, float y, out Text value)
/// <summary>
/// One labelled value: the wide tutorial plank with a small trade plank
/// resting on it for the caption and the value written underneath.
/// </summary>

Lines 575-578 · comment · before UIPlank.Create(row.transform, "Caption", theme?.tradeRequestBoard, true,
// Caption plank on top, value written on the bare wood below it - the
// arrangement the mock-up uses. Fonts are kept small deliberately:
// this is the tightest pair of text boxes on the panel, and at the
// 1.15 text ceiling a line of Playpen Sans is about 45 units tall.

Lines 596-600 · comment · before newEmailInput = CreateInput(parent, "New email (e.g. name@gmail.com)", false, -170f);
// Each hint on these boards states the rule the server enforces, as the
// sign-up board does; AccountRules holds the rules themselves. These
// fields leave about 663 units for text, and every hint was measured to
// stay on one line at the largest text size - a hint that wraps spills
// its second line out of the field.

Lines 609-613 · doc comment · before private void BuildVerify(Transform parent)
/// <summary>
/// The code board for an email change. No hanging sign: the painted one
/// reads "Enter Login Code", and the line below says where the code went
/// anyway.
/// </summary>

Lines 639-641 · comment · before currentPasswordInput = CreateInput(parent, "Current password", true, -160f);
// The current password has no rule to show - the server only needs it
// filled in - so its hint just says which password it wants. The new
// one reads the same as on the forgotten-password reset board.

Line 670 · comment · before private float PaddingX => theme != null ? theme.panelPaddingX : 170f;
// ----- primitives

Lines 688-689 · comment · before image.sprite = board;
// Sliced so the rolled log ends keep their true size at every
// board height; only the middle planks stretch.

Lines 711-712 · comment · before rect.sizeDelta = new Vector2(BoardContentWidth(PanelWidth) * 0.96f, 70f);
// Measured off the board rather than assumed, and pulled in a little
// further so the white field never touches the wood beside it.

Lines 717-719 · comment · before Text text = CreateText(go.transform, "Text", 25, TextAnchor.MiddleLeft);
// Pinned to Overflow, never Truncate: Unity discards a line that does
// not fit rather than clipping it, so at the largest text size the
// field would draw nothing while still taking keystrokes.

```

```
Line 757 · comment · before image.sprite = sprite;
// The word is painted into the art, so no Text child is added.
```

Assets/Scripts/MainMenu/Account/AuthUIBuilder.cs

169 comments · 169 comment lines removed · file had 1026 lines

```
Lines 37-39 · comment · before private InputField signupBirthYear;
// Date of birth is three number boxes rather than one text box: typing
// "2004-03-12" by hand on a phone keyboard is slow and easy to get wrong,
// and the dashes were a common cause of the format error.
```

```
Line 57 · comment · before private AuthMode pendingMode;
// Pending flow the code-entry panel is confirming.
```

```
Lines 63-65 · comment · before private string pendingResetIdentifier;
// Recovery keeps the typed identifier rather than an address: a player
// who started from their display name is never told the email, so that
// is the only thing both the confirm and resend calls can be keyed on.
```

```
Lines 69-74 · doc comment · before private bool recovering;
/// <summary>
/// One recovery request in flight at a time. Sign-up and login lean on
/// AuthSession.IsBusy for this; recovery keeps its own flag so a request
/// that never comes back cannot leave that shared one raised, which would
/// stop the player opening their farm afterwards.
/// </summary>
```

```
Lines 111-114 · doc comment · before private void ShowForgot()
/// <summary>
/// The "enter your email or display name" board, reached from Forgot
/// Password on the login board.
/// </summary>
```

```
Lines 122-123 · comment · before resetTicket = null;
// Whatever is in the box is left alone, so stepping back here from
// the code board does not wipe the identifier that got you there.
```

```
Lines 129-133 · doc comment · before private void ShowVerify(AuthMode mode, string email, string info)
/// <summary>
/// The code board. <paramref name="info"/> is what the server said about
/// where the code went - the full address for sign-up and login, a masked
/// one for recovery, which is why it is quoted rather than rebuilt here.
/// </summary>
```

```
Lines 144-148 · comment · before bool isRecovery = mode == AuthMode.ForgotPassword;
// The painted sign says "Enter Login Code", which is wrong for a
// password reset, so recovery takes it down and lets the line below
// carry the wording on its own.
// Named apart from the "recovering" field, which is the in-flight
// latch and a different thing entirely.
```

```
Lines 152-153 · comment · before if (verifyHeadingSign != null)
// The sign is wrong for sign-up as well, so sign-up swaps it for a
// plank that says "Enter Sign-up Code" (ISS-16). Login keeps the sign.
```

```
Lines 159-160 · comment · before if (verifyHeading != null)
// Null once the painted sign is in use - the sign carries the
// wording, and the info line below still names the flow.
```

```
Line 176 · doc comment · before private void OnlyShow(GameObject panel)
/// <summary>Brings one board to the front and puts every other one away.</summary>
```

```
Lines 201-202 · comment · before resetTicket = null;
// The reset ticket only ever authorises the one password change it
// was issued for, so it is dropped the moment this board closes.
```

```
Lines 227-231 · comment · before string email = response != null && !string.IsNullOrEmpty(response.email)
// The account's real address, which the server returns once
// the password has checked out. A player who signed in with
// their display name has never seen it, and the confirm call
// is keyed on it, so falling back to what they typed would
// leave them unable to finish.
```

```
Line 242 · comment · before private void OnForgotPasswordPressed()
// ----- password recovery
```

```
Lines 246-247 · comment · before string typed = loginEmail.text.Trim();
// Carried over so a player who typed their email and then realised
// they have forgotten the password does not type it again.
```

```
Lines 273-276 · comment · before string info = response != null && !string.IsNullOrEmpty(response.message)
// The server's own wording, because it names the account in
// masked form and only it knows the address behind a display
// name. Rebuilding the sentence here would either print the
// display name back at the player or leak the address.
```

```

Line 287 · doc comment · before private void FailRecovery(string message)
/// <summary>Clears the recovery latch, then reports why it stopped.</summary>

Lines 299-303 · doc comment · before private void ShowResetPassword()
/// <summary>
/// Opened once the reset code is accepted. The ticket earned there is
/// what authorises the change, so this board is never reachable without
/// having passed the code first.
/// </summary>

Line 322 · comment · before string problem = AccountRules.ResetPasswordProblem(password, confirm);
// The same password rules as sign-up, checked before the round trip.

Lines 346-347 · comment · before resetTicket = null;
// Spent. Keeping it would leave a usable ticket lying in
// memory for the rest of the session.

Lines 368-371 · comment · before string problem = AccountRules.SignupProblem(
// Checked here first so a bad value is caught the moment Create is
// pressed, rather than after a trip to the server. The server still
// checks everything again; AccountRules explains why the two have to
// stay in step.

Lines 380-382 · comment · before if (!TryBuildDateOfBirth(out string dateOfBirth, out string dateError))
// The date row sits below the other fields, so it is checked after
// them: the first message a player sees always belongs to the highest
// field that needs fixing.

Lines 515-518 · comment · before if (response != null && !string.IsNullOrEmpty(response.devCode))
// devCode is only ever non-empty when the server was deliberately started
// with APP_VERIFICATION_EXPOSE_CODE=true (demo/dev mode) - the server is
// the real gate here, so showing it on every platform (not just the
// Editor) is safe: a normally-configured deployment never sends this field.

Lines 534-535 · comment · before blocker.color = new Color(0f, 0f, 0f, 0.45f);
// Lighter than before so the painted farm background still reads
// behind the wooden boards.

Lines 540-550 · comment · before statusText.horizontalOverflow = HorizontalWrapMode.Wrap;
// This line carries the longest strings in the game - a sent-code
// confirmation with the address in it, the dev code appended after it,
// and every backend error message - so it is the one label that must
// never lose its tail.
//
// It used to be a fixed 900x80 box on Unity's default Truncate, which
// silently ate whatever did not fit. Stretching it to the screen width
// instead of pinning a width keeps it correct on a 4:3 tablet and at
// any interface scale, where a fixed 900 could be wider than the canvas
// itself. Overflow then guarantees a long message grows upward into
// empty background rather than being cut off.

Lines 557-558 · comment · before statusRect.sizeDelta = new Vector2(-120f, 120f);
// Negative width = inset from both screen edges, so this is a 60-unit
// margin each side rather than a fixed 900.

Lines 584-594 · doc comment · before private const float PlankHeadingWidth = 540f;
/// <summary>
/// Width of the blank planks the recovery screens write on. They are the
/// tutorial objective plank, which has no wording painted in, so the text
/// is drawn live on top - see <see cref="UIPlank"/>.
///
/// Held to the same width as the input boxes below it, which is what
/// keeps it off the board's rolled logs. BackgroundUI.png is 1024 wide
/// and its painted log runs to x=157; drawn 9-sliced at 900 that puts the
/// plank wall's left edge 153 units in, so panelPaddingX of 170 leaves
/// room to spare and anything wider than 560 climbs onto the wood.
/// </summary>

Lines 602-605 · comment · before if (verifyHeadingSign != null)
// Only needed when the painted "Enter Login Code" sign is in use; the
// plain text heading already says "Enter Sign-up Code" by itself. The
// same blank plank and position as the recovery boards' headings, and
// hidden until ShowVerify opens the board for sign-up.

Lines 635-637 · comment · before float contentLeft = -(LoginWidth * 0.5f - PaddingX);
// Tucked under the password box on the left, the way it is written on
// a real sign-in form. Left-aligned rather than centred so it reads as
// a footnote to the field above it and not as a third button.

Lines 648-655 · doc comment · before private void BuildForgot(Transform parent)
/// <summary>
/// "Enter Email or Display Name" - the first step of recovery.
///
/// A player who has forgotten the address as well as the password is
/// pointed at the support mailbox rather than left with nowhere to go:
/// there is no automatic way to prove an account is yours once you can
/// name neither of the two things that identify it.
/// </summary>

```

Line 680 · doc comment · before `private void BuildReset(Transform parent)`
`/// <summary>The new-password board, reached only after the code is accepted.</summary>`

Lines 687-690 · comment · before `resetPassword = CreateInput(parent, "New password (min. 8 characters)", true, -200f, ResetWidth);`
`// Hints state the server's rules, as on the sign-up board, and are`
`// measured the same way against the same 528 units of text width.`
`// "New password (at least 8 characters)" ran past that at the largest`
`// text size, which is why this one says "min." instead.`

Lines 703-715 · comment · before `signupEmail = CreateInput(parent, "Email (e.g. name@gmail.com)", false, -175f, SignupWidth);`
`// Each hint states the rule the server will actually enforce, so a`
`// player learns it before pressing Create rather than from a rejection`
`// afterwards. The rules come from RegisterRequest and`
`// AccountFields.validateDisplayName: email must be a valid address,`
`// password 8 to 72 characters, display name 3 to 80 with no @ sign.`
`//`
`// Wording is measured, not guessed. These fields leave 528 units for`
`// text, and the placeholder wraps rather than clipping, with vertical`
`// overflow on - so a hint too long for one line spills its second line`
`// onto the field below. Every string here stays under that width at the`
`// largest text setting. The fuller "Display name (optional, 3-80, no @)"`
`// did not, which is why "optional" is left to the fact that a blank`
`// name is simply accepted.`

Line 737 · trailing comment · on the line `scaler.matchWidthOrHeight = 1f; // landscape: scale by height`
`// landscape: scale by height`

Lines 758-759 · comment · before `image.sprite = board;`
`// Sliced so the rolled log ends keep their true size at every`
`// panel height - only the middle planks stretch.`

Lines 772-776 · doc comment · before `private Text CreateHeading(Transform parent, string value, Sprite sprite, float panelWidth)`
`/// <summary>`
`/// The hanging sign above a panel. With a sprite it is a picture (the`
`/// wording is painted in) and null is returned; without one it falls`
`/// back to a live Text, which is returned so callers can retile it.`
`/// </summary>`

Lines 782-786 · doc comment · before `private Text CreateHeading(Transform parent, string value, Sprite sprite, float panelWidth,`
`/// <summary>`
`/// As above, and hands back the sign itself so a caller can take it down.`
`/// The code board needs that: its sign says "Enter Login Code", which is`
`/// the wrong words when the same board is confirming a password reset.`
`/// </summary>`

Lines 800-801 · comment · before `signRect.sizeDelta = new Vector2(panelWidth - 60f, LabelHeight);`
`// preserveAspect fits the sign inside this box, so each sign`
`// keeps its own width and they all share one height.`

Lines 824-829 · doc comment · before `private bool TryBuildDateOfBirth(out string isoDate, out string error)`
`/// <summary>`
`/// Joins the three birth boxes into the "yyyy-MM-dd" the backend expects.`
`///`
`/// The field stays optional, exactly as it was when it was one text box:`
`/// all three blank returns null and no error.`
`/// </summary>`

Lines 853-854 · comment · before `if (!System.DateTime.TryParseExact(`
`// TryParseExact also rejects dates that do not exist, such as`
`// 2004-02-31, which the old single text box would have accepted.`

Lines 870-874 · doc comment · before `private void CreateDateOfBirthRow(Transform parent, float y, float panelWidth)`
`/// <summary>`
`/// Year / month / day side by side on one row, in place of the single`
`/// text box. The row spans the same width the old field did, so nothing`
`/// above or below it has to move.`
`/// </summary>`

Lines 881-883 · comment · before `float yearWidth = Mathf.Round(usable * 0.44f);`
`// Month and day take whatever the year leaves, so the three boxes and`
`// the two gaps add up to exactly the old field's width - rounding both`
`// independently left the day box hanging a pixel past the edge.`

Lines 902-903 · comment · before `input.contentType = InputField.ContentType.IntegerNumber;`
`// IntegerNumber opens the phone's number pad instead of the full`
`// keyboard, and refuses anything that is not a digit.`

Line 916 · comment · before `InputField input = CreateInputAt(`
`// Inset so the field sits between the board's two log ends.`

Lines 936-942 · comment · before `Text text = CreateText(go.transform, "Text", 25, TextAnchor.MiddleLeft);`
`// Font 25 in the 54 units left after padding has room to spare even at`
`// the largest text setting, so unlike the shop's amount box these do not`
`// currently run out. They are pinned to Overflow anyway: Unity's default`
`// Truncate discards a line that does not fit rather than clipping it, so`
`// the failure mode here is a field that silently draws nothing while`
`// still accepting keystrokes - and on the login screen that is a player`
`// who cannot sign in at all.`

Line 978 · comment · before `image.sprite = sprite;`
// The word is painted into the art, so no Text child is added.

Assets/Scripts/MainMenu/Menu/CreditsUIBuilder.cs

50 comments · 50 comment lines removed · file had 510 lines

Lines 8–18 · doc comment · before `public class CreditsUIBuilder : MonoBehaviour`

```
/// <summary>
/// The rolling credits.
///
/// The roll scrolls upward on its own, but the player can grab and drag it at
/// any time. Auto-scroll pauses while they hold it, waits a moment after they
/// let go, then resumes from wherever they left it – and stops for good once
/// the end is reached rather than looping.
///
/// Built entirely in code like the rest of the game's UI, so there is no
/// prefab to wire up; only the sprites in UITheme are needed.
/// </summary>
```

Line 23 · doc comment · before `private const float ScrollSpeed = 45f;`
/// <summary>Roll speed in reference pixels per second.</summary>

Line 26 · doc comment · before `private const float ResumeDelay = 1.2f;`
/// <summary>Idle time after the player lets go before the roll resumes.</summary>

Lines 32–35 · doc comment · before `private const float BoardInset = 160f;`
/// <summary>
/// BackgroundUI.png is 9-sliced with 130px log caps, so content is inset
/// past them exactly as the settings and social boards do.
/// </summary>

Line 63 · comment · before `Canvas.ForceUpdateCanvases();`
// Always start from the top so the game logo leads every time.

Lines 83–84 · comment · before `if (dragging || Time.unscaledTime < resumeAt)`
// Hold off while the player is dragging, and for a beat afterwards so
// the roll does not fight them the instant they release.

Line 93 · comment · before `float step = ScrollSpeed / scrollable * Time.unscaledDeltaTime;`
// verticalNormalizedPosition runs 1 (top) to 0 (bottom).

Line 107 · comment · before `private void Build()`
// ----- build

Lines 140–142 · comment · before `BuildScrollArea(panelRect);`
// No title above the roll: the game logo is the first thing the roll
// shows, so a "CREDITS" heading would only repeat what is already there
// and eat vertical space the roll can use instead.

Lines 156–157 · comment · before `scrollRect.offsetMin = new Vector2(BoardInset, 120f);`
// Reaches nearer the top now that no title sits above it; the inset
// still clears the board's top log cap.

Line 161 · comment · before `GameObject viewportGo = new GameObject("Viewport",`
// RectMask2D needs no Image of its own, so the board shows through.

Line 167 · comment · before `Image viewportImage = viewportGo.GetComponent<Image>();`
// Transparent, but still a raycast target so drags anywhere register.

Line 205 · comment · before `scroll.horizontalScrollbar = null;`
// No scrollbar: nothing should be drawn over the board art.

Line 213 · doc comment · before `internal void NotifyDragging(bool isDragging)`
/// <summary>Called by the drag watcher while the player is holding the roll.</summary>

Lines 220–221 · comment · before `reachedEnd = false;`
// Touching the roll again re-arms it, so a player who scrolls back
// up is not left with a frozen screen.

Line 257 · comment · before `private void BuildCreditsContent(Transform parent)`
// ----- content

Line 261 · comment · before `AddLogo(parent, theme?.gameLogo, 520f, 200f);`
// The roll opens on the game logo and closes on the team logo.

Lines 292–293 · comment · before `AddSubtle(parent, "Copyright (c) 2007 James Newton-King");`
// MIT requires the copyright notice to travel with the software, so it
// is reproduced here rather than only linked.

Line 316 · comment · before `AddSubtle(parent, "Licensed under CC BY 4.0");`
// CC BY 4.0 requires visible attribution wherever the work is used.

Lines 351–354 · comment · before `AddSubtle(parent, "(c) 2023 The Playpen Sans Project Authors");`
// The OFL requires the copyright notice and the licence to travel with
// the font wherever it is redistributed, and shipping it inside the APK
// is redistribution. The full licence text is bundled alongside the
// typeface at Resources/Fonts/PlaypenSans-OFL.txt.

Line 385 · doc comment · before private void AddSubtle(Transform parent, string value)
/// <summary>Licence notices and disclaimers, set smaller and dimmer.</summary>

Line 428 · comment · before private static Text CreateText(RectTransform parent, string name, int size,
// ----- helpers

Lines 487–492 · doc comment · before public class CreditsDragWatcher : MonoBehaviour, IBeginDragHandler, IEndDragHandler
/// <summary>
/// Reports drag start and end to the credits roll.
///
/// ScrollRect handles the dragging itself; this only tells the builder when to
/// hold the auto-scroll so the two do not fight for the same position.
/// </summary>

Assets/Scripts/MainMenu/Menu/MainMenuBuilder.cs

108 comments · 108 comment lines removed · file had 1273 lines

Lines 133–135 · comment · before backgroundRoot = new GameObject("BackgroundRoot", typeof(RectTransform));
// The background art and the logo/buttons live on separate layers:
// the white fade reveals the background alone, and the logo and
// buttons only join it once the player has logged in.

Lines 159–170 · comment · before if (!shouldPlayIntro &&
// Deliberately gated on the intro rather than on being signed in.
//
// This shortcut exists for coming back from the farm mid-session, where
// replaying the studio logo every time would be tiresome. Being signed
// in used to be a good enough stand-in for that, because the only way to
// reach the menu already authenticated was to have just come from the
// farm. Remembering the login broke that assumption: a cold launch now
// arrives here authenticated too, and took this branch, which is why the
// intro stopped playing for anyone with a saved account.
//
// introAlreadyPlayed is static, so it is false exactly once per app
// launch – which is precisely when the logo should be shown.

Lines 180–181 · comment · before if (SessionTakeover.Consume(out string takeoverMessage))
// A player thrown out of the farm because another device took the
// account lands here. Say so, or the farm just vanishes on them.

Line 195 · comment · before CreateIntroOverlay(rootRect);
// Created last, so the intro renders above the white cover.

Line 237 · comment · before whiteCoverImage.raycastTarget = alpha > 0.01f;
// Stop swallowing clicks once it is essentially invisible.

Lines 241–244 · doc comment · before private IEnumerator RevealBackgroundThenLogin()
/// <summary>
/// Fades the white cover away to reveal the background art on its own,
/// then – and only then – opens the login board.
/// </summary>

Lines 284–288 · comment · before if (!authenticated)
// Signed out while the menu was already up – the account's password
// was changed on another device, so this login was retired mid-visit.
// The menu has to be put away as well as the login form brought up:
// that form only dims what is behind it, so Start, Settings, Account
// and Logout would otherwise stay visible and pressable through it.

Line 339 · trailing comment · on the line scaler.matchWidthOrHeight = 1f; // landscape: scale by height
// landscape: scale by height

Line 360 · trailing comment · on the line canvasScaler.matchWidthOrHeight = 1f; // landscape: scale by height
// landscape: scale by height

Lines 676–677 · comment · before StartCoroutine(RevealBackgroundThenLogin());
// The screen is solid white underneath the intro; fade it away to
// reveal the background, then open the login board.

Lines 681–689 · doc comment · before private void ShowAuthenticationIfNeeded()
/// <summary>
/// Runs once the white cover has faded, after the intro or in its place.
///
/// A player who is not signed in gets the login form, and the menu fades in
/// later when SessionChanged fires. A player restored from the phone's saved
/// login has nothing to sign in to, and no session change is coming – so the
/// menu has to be brought in here, or it would sit at zero alpha behind a
/// cleared cover and the game would look like it had frozen on the artwork.
/// </summary>

Lines 702–706 · comment · before if (SessionTakeover.Consume(out string takeoverMessage))
// Read in both cases, not just when signed in. A device thrown out of
// the farm because the account's password changed elsewhere arrives
// here signed OUT, and used to return above without ever showing the
// explanation – so the farm simply vanished and the login form
// appeared, with nothing to say why.

```

Lines 764-770 · doc comment · before private void CreateCreditsButton(RectTransform parent)
    /// <summary>
    /// The Credits button, pinned to the bottom-right corner.
    ///
    /// It is parented to the same menu content root as the other buttons, which
    /// is only revealed once the player is authenticated - so the button
    /// inherits that gating rather than needing a check of its own.
    /// </summary>

Line 797 · comment · before image.color = new Color(0.20f, 0.45f, 0.22f, 0.95f);
    // Plain fallback so the button still works before the art is dropped in.

Line 822 · comment · before private GameObject logoutPrompt;
    // ----- logout

Lines 829-836 · doc comment · before private void CreateLogoutButton(RectTransform parent)
    /// <summary>
    /// The Logout button, in the bottom-left corner opposite Credits.
    ///
    /// Parented to the same menu content root as the other buttons, which is
    /// only revealed once the player is signed in, so it cannot be pressed on
    /// the login screen. It is the only way to change accounts on a phone:
    /// the saved login is otherwise kept until it expires.
    /// </summary>

Lines 842-846 · doc comment · before private void CreateAccountButton(RectTransform parent)
    /// <summary>
    /// The Account button, on the same row as Logout and using the same
    /// wooden disc. It opens the board where a player reads their display
    /// name and email and can change either, or their password.
    /// </summary>

Line 849 · comment · before CreateCornerDiscButton(parent, "AccountButton", "Account", 210f, OnAccountPressed);
    // 40 for the left margin plus the 150 disc, then 20 of gap.

Lines 853-859 · doc comment · before private void CreateCornerDiscButton(RectTransform parent, string name, string text,
    /// <summary>
    /// One of the small wooden discs in the bottom-left corner.
    ///
    /// Parented to the same menu content root as the other buttons, which is
    /// only revealed once the player is signed in, so neither disc can be
    /// pressed on the login screen.
    /// </summary>

Lines 871-872 · comment · before rect.sizeDelta = art != null ? new Vector2(150f, 150f) : new Vector2(200f, 70f);
    // Square, because the knob art is a disc and preserveAspect would letterbox
    // anything else. Large enough that the word inside stays readable.

Line 901 · comment · before label.color = art != null ? new Color(0.20f, 0.12f, 0.04f, 1f) : Color.white;
    // Dark on the wooden disc, white on the plain fallback.

Lines 924-925 · comment · before string email = AuthSession.Instance?.CurrentUser?.email;
    // Naming the account matters on a shared phone, where the whole reason
    // to open this is not being sure who is signed in.

Lines 942-943 · comment · before AuthSession.Instance?.Logout();
    // Clears the phone's saved login and releases this device's claim on
    // the account, so the next launch asks who is playing.

Lines 946-949 · comment · before menuRevealed = false;
    // Put the menu back to its signed-out state. The login form only dims
    // what is behind it, so without this the Start, Settings and Logout
    // buttons stay visible through it - and the menu would still count as
    // revealed, so the next sign-in would snap in without its fade.

Line 978 · doc comment · before private void ShowAccountBusyPrompt(string message)
    /// <summary>Shown when this account is already being played on another phone.</summary>

Lines 993-994 · comment · before accountBusyMessage.text = string.IsNullOrEmpty(message)
    // The server owns this wording, so a change there does not need a
    // matching edit here; the fallback only covers it going missing.

Lines 1005-1009 · doc comment · before private GameObject CreatePromptBoard(string name, out Text message)
    /// <summary>
    /// A dimmed backdrop with the trade-request plank on it, shared by both
    /// prompts. Parented to the canvas rather than the menu content so it sits
    /// over everything, including the login form.
    /// </summary>

Lines 1067-1068 · comment · before Transform board = parent.Find("Board");
    // Parented to the board, not the backdrop, so the positions below are
    // measured from the plank rather than the screen.

Lines 1186-1196 · doc comment · before private IEnumerator ClaimThenStart()
    /// <summary>
    /// Takes the account's play session before opening the farm.
    ///
    /// Checked here rather than at login because a player may well be signed
    /// in on two phones - that is the point of remembering the login - and only

```

```

/// one of them may be playing. Pressing Start is the moment that matters.
///
/// A network failure is deliberately not treated as a refusal: being unable
/// to reach the server should not stop someone playing their own farm, and
/// the in-farm heartbeat re-checks the claim continuously anyway.
/// </summary>

```

Assets/Scripts/Shared/Accessibility/GameAccessibility.cs

74 comments · 74 comment lines removed · file had 451 lines

Lines 10–29 · doc comment · before `public class GameAccessibility : MonoBehaviour`

```

/// <summary>
/// Makes the game readable by Android accessibility services (TalkBack,
/// Accessibility Scanner).
///
/// Unity draws the whole game – the 3D farm and every wooden UI panel – into
/// one Android SurfaceView. Android's accessibility framework therefore sees a
/// single opaque view and nothing inside it, which is why Accessibility Scanner
/// reports the same two findings on every screen:
///
/// * "Item label" – the SurfaceView carries no contentDescription.
/// * "Unexposed Text" – the SurfaceView publishes none of its contents.
///
/// This component answers both. It sets a contentDescription on the Unity view,
/// and it publishes an <see cref="AccessibilityHierarchy"/> describing each
/// on-screen control, which is Unity's supported route for exposing in-game UI
/// to a screen reader.
///
/// Nothing here moves, resizes or restyles any UI. It only reads the existing
/// canvases and reports what is already on screen.
/// </summary>

```

Line 32 · doc comment · before `public const string SurfaceLabel =`

```

/// <summary>Announced for the Unity surface itself.</summary>

```

Lines 38–42 · doc comment · before `private const float RefreshInterval = 0.75f;`

```

/// <summary>
/// How often the published hierarchy is rebuilt. The UI is constructed at
/// runtime and panels open and close constantly, so a poll is simpler and
/// more reliable than trying to hook every builder.
/// </summary>

```

Lines 50–52 · comment · before `private AndroidJavaObject androidActivity;`

```

// Held in a field, not a local: runOnUiThread runs the callback after
// ApplySurfaceLabel has already returned, so a local would be eligible for
// collection before the callback touches it.

```

Line 84 · comment · before `lastSignature = 0;`

```

// A scene swap replaces every canvas, so never trust the cached signature.

```

Lines 96–97 · comment · before `int signature = BuildSignature();`

```

// Only pay for a rebuild when the set of visible controls actually
// changed – opening a panel, closing one, switching scene.

```

Line 106 · comment · before `private struct Entry`

```

// ----- hierarchy

```

Line 108 · doc comment · before `private struct Entry`

```

/// <summary>One announceable control, gathered before the reading-order sort.</summary>

```

Lines 137–138 · comment · before `entries.Sort((a, b) =>`

```

// Screen readers walk nodes in order, so present them the way a person
// reads the screen: top to bottom, then left to right.

```

Lines 154–155 · comment · before `RectTransform rect = entry.rect;`

```

// frameGetter keeps the reported rectangle correct as panels move,
// without having to rebuild the whole hierarchy.

```

Lines 189–190 · comment · before `continue;`

```

// A described control's own children (its icon, its caption) are
// part of that control, not separate things to announce.

```

Lines 198–200 · doc comment · before `private static bool TryDescribe(GameObject go, out string label,`

```

/// <summary>
/// Decides whether a GameObject is worth announcing, and what to call it.
/// </summary>

```

Lines 258–262 · doc comment · before `private static string LabelForControl(GameObject go)`

```

/// <summary>
/// A control's caption if it has one. Most buttons in this game carry their
/// word inside the painted art and have no Text child at all, so the object
/// name is the fallback – which is why the builders' names matter here.
/// </summary>

```

Line 278 · comment · before `private static string Humanize(string raw)`

```

// ----- utilities

```

Line 280 · doc comment · before `private static string Humanize(string raw)`
`/// <summary>Turns "Button_SellItems" / "SaveFarmButton" into "Sell Items".</summary>`

Line 288 · comment · before `if (value.StartsWith("Button_"))`
`// Drop the decoration the builders add to object names.`

Line 302 · comment · before `if (i > 0 && char.IsUpper(c) && !char.IsUpper(value[i - 1]) && value[i - 1] != ' ')`
`// Split camelCase so "SellItems" reads as two words.`

Lines 318-321 · doc comment · before `private static Rect ScreenFrameOf(RectTransform rect, Canvas canvas)`
`/// <summary>`
`/// The node rectangle in the top-left-origin screen space that accessibility`
`/// services expect; Unity's own screen coordinates start bottom-left.`
`/// </summary>`

Lines 345-348 · doc comment · before `private int BuildSignature()`
`/// <summary>`
`/// A cheap fingerprint of what is currently visible, so Update can skip the`
`/// walk when nothing has opened or closed.`
`/// </summary>`

Line 370 · comment · before `private void ApplySurfaceLabel()`
`// ----- Android`

Lines 372-375 · doc comment · before `private void ApplySurfaceLabel()`
`/// <summary>`
`/// Gives the Unity SurfaceView a contentDescription. This is the "Item label"`
`/// finding: without it a screen reader announces the whole app as unlabelled.`
`/// </summary>`

Lines 408-412 · doc comment · before `private static void LabelViews(AndroidJavaObject view, int depth)`
`/// <summary>`
`/// Walks the activity's view tree and describes the Unity surface. The tree is`
`/// only a few levels deep - decor, content frame, the Unity view - so the`
`/// depth cap is generous rather than restrictive.`
`/// </summary>`

Lines 425-426 · comment · before `view.Call("setImportantForAccessibility", 1);`
`// IMPORTANT_FOR_ACCESSIBILITY_YES - without this the surface can be`
`// skipped entirely by accessibility services.`

Line 431 · comment · before `int count;`
`// Recurse only into ViewGroups.`

Line 439 · trailing comment · on the line `return;` `// not a ViewGroup`
`// not a ViewGroup`

Assets/Scripts/Shared/Audio/GameAudioManager.cs

29 comments · 29 comment lines removed · file had 273 lines

Lines 8-23 · doc comment · before `public class GameAudioManager : MonoBehaviour`
`/// <summary>`
`/// Central, additive audio engine for the game: looping background music,`
`/// looping weather/scene ambience (both with crossfade), and one-shot SFX.`
`/// SFX clips are Inspector fields the developer drags their own files into.`
`///`
`/// Persistent singleton (survives scene loads); also lazily self-creates so`
`/// any Play* call is safe even in a scene that forgot to add the component.`
`/// It touches no gameplay logic.`
`///`
`/// This component does NOT know which music belongs to which scene - because`
`/// it persists, only the very first instance loaded would ever run its own`
`/// Start(). Instead, each scene that wants its own track adds a small,`
`/// non-persistent <see cref="SceneMusicPlayer"/> (or, for the Farm scene,`
`/// <see cref="FarmAudioController"/>), which reruns every time that scene`
`/// loads and simply calls Instance.PlayMusic(...) with its own clip.`
`/// </summary>`

Lines 65-66 · comment · before `[Header("Transition")]`
`// Volumes now live in GameSettings (user-adjustable + persisted) and are`
`// read live at playback / applied via the GameSettings.Changed event.`

Lines 100-103 · comment · before `Destroy(this);`
`// Destroy only this duplicate component, not the whole GameObject:`
`// a scene may also carry a sibling SceneMusicPlayer/FarmAudioController`
`// on the same object, and that sibling must survive to run its own`
`// Start() and tell the surviving (persistent) instance what to play.`

Line 143 · trailing comment · on the line `src.spatialBlend = 0f;` `// 2D, non-positional`
`// 2D, non-positional`

Line 177 · comment · before `public void PlaySfx(AudioClip clip)`
`// ----- SFX -----`

Lines 194-195 · comment · before `public void PlayTill() => PlaySfx(tillSfx != null ? tillSfx : digSfx);`
`// The persistent instance is the Main Menu's, so these clips are assigned`
`// there. Each falls back to the nearest existing sound until one is.`

Line 201 · comment · before `public void PlayMusic(AudioClip clip)`
// ----- Music / ambience (crossfaded) -----

Line 223 · trailing comment · on the line `return;` // a null clip is allowed - fades to silence
// a null clip is allowed - fades to silence

Line 256 · comment · before `float targetVolume = isMusic ? GameSettings.MusicVolume : GameSettings.AmbienceVolume;`
// Read the target volume live so a settings change mid-fade is respected.

Assets/Scripts/Shared/Audio/SceneMusicPlayer.cs

12 comments · 12 comment lines removed · file had 29 lines

Lines 5-16 · doc comment · before `public class SceneMusicPlayer : MonoBehaviour`
/// <summary>
/// Plays this scene's background music through the persistent
/// <see cref="GameAudioManager"/>. Add one of these (not GameAudioManager
/// itself) to any scene that needs its own distinct track - Main Menu,
/// Area Selection, etc.
///
/// Unlike GameAudioManager, this component is deliberately NOT a singleton
/// and NOT DontDestroyOnLoad, so it is destroyed and freshly recreated with
/// every scene load. That means its Start() reliably fires each time this
/// scene becomes active, correctly switching the music even though the
/// audio engine itself persists across scenes.
/// </summary>

Assets/Scripts/Shared/Backend/ApiClient.cs

1 comment · 1 comment line removed · file had 165 lines

Line 136 · comment · before `}`
// Fall back to the raw response below.

Assets/Scripts/Shared/Backend/ApiConfiguration.cs

7 comments · 7 comment lines removed · file had 92 lines

Lines 45-46 · comment · before `if (hasPublic)`
// A deployed public server is always preferred on a real device:
// LAN and emulator addresses are unreachable over mobile data.

Lines 58-62 · doc comment · before `public bool IsCleartext =>`
/// <summary>
/// True when the resolved address is plain HTTP. Android 9+ blocks cleartext
/// traffic by default, so an http:// address silently fails on device unless
/// the manifest opts in.
/// </summary>

Assets/Scripts/Shared/Backend/AuthSession.cs

141 comments · 141 comment lines removed · file had 733 lines

Lines 35-44 · doc comment · before `private void RestoreSavedSession()`
/// <summary>
/// Brings back the login stored on this phone, so a returning player goes
/// straight to the menu instead of signing in again.
///
/// Restored synchronously from PlayerPrefs and then checked against the
/// server in the background. Waiting for that check first would leave the
/// menu showing a login form for the length of a round trip and then
/// snatch it away, so the token is trusted up front and revoked later if
/// the server disagrees - the only cost of being wrong is one failed call.
/// </summary>

Lines 53-56 · comment · before `HasFarm = false;`
// Unknown rather than false: hasFarm only ever comes back on a login
// response, and this session did not go through one. StartGame asks the
// server instead of reading this, so the value below is a placeholder
// until the first farm load fills it in.

Lines 63-68 · doc comment · before `private IEnumerator VerifyRestoredSession()`
/// <summary>
/// Confirms the restored token is still good. A rejected token is cleared
/// and the player is asked to sign in; any other failure - no signal, the
/// server down - is left alone, since throwing the session away over a
/// dropped connection would be worse than keeping it.
/// </summary>

Lines 92-97 · doc comment · before `public IEnumerator ClaimPlaySession()`
/// <summary>
/// Claims this account's play session for this device.
///
/// <paramref name="onBusy"/> carries the server's wording for the case
/// where another phone is already playing on the account.
/// </summary>

```

Line 123 · comment · before public IEnumerator RequestRegister(
// Step 1 of sign-up: the backend validates the form and emails a code.

Line 149 · comment · before public IEnumerator VerifyRegister(
// Step 2 of sign-up: confirm the code, then the account is created and a token issued.

Line 177 · comment · before public IEnumerator RequestLogin(
// Step 1 of login: the backend validates credentials and emails a 2FA code.

Line 203 · comment · before public IEnumerator VerifyLogin(
// Step 2 of login: confirm the 2FA code, then a token is issued.

Line 231 · comment · before public IEnumerator RequestPasswordReset(
// ----- password recovery

Lines 233-237 · doc comment · before public IEnumerator RequestPasswordReset(
/// <summary>
/// Step 1 of recovery: email a code to the account behind an address or
/// a display name. Nothing is signed in and nothing is written to the
/// phone - the account is only remembered once the reset finishes.
/// </summary>

Lines 251-255 · doc comment · before public IEnumerator VerifyPasswordReset(
/// <summary>
/// Step 2: confirm the emailed code. What comes back is a reset ticket,
/// not a login - the API refuses it as a bearer token - so it is handed
/// to the caller to spend on the next call rather than stored here.
/// </summary>

Lines 270-280 · doc comment · before public IEnumerator CompletePasswordReset(
/// <summary>
/// Step 3: set the new password. The server signs the player in as part
/// of the same call, so this is the first moment the account is written
/// to the phone.
///
/// Deliberately does not raise IsBusy. These coroutines are started by
/// the login board, which is an ordinary scene object, so one that is
/// cut short by a scene change would leave the flag raised forever - and
/// StartGame refuses to open the farm while it is. Double submits are
/// held off by the board itself, which is the thing that gets destroyed.
/// </summary>

Line 305 · comment · before public IEnumerator RequestEmailChange(
// ----- account panel

Line 307 · doc comment · before public IEnumerator RequestEmailChange(
/// <summary>Step 1 of moving the account: email a code to the new address.</summary>

Lines 321-325 · doc comment · before public IEnumerator ConfirmEmailChange(
/// <summary>
/// Step 2: confirm the code. A fresh token comes back with the moved
/// account, and both replace what the phone had - otherwise the saved
/// login would keep showing the old address after the change.
/// </summary>

Lines 343-351 · doc comment · before public IEnumerator ChangePassword(
/// <summary>
/// Changes the password, which also ends every other session on the
/// account - any phone still signed in is refused from that moment on.
///
/// That includes the one making the change, so the server hands back a
/// replacement token with the response and it is adopted here. Without
/// adopting it the player would be signed out by their own password
/// change on the very next request.
/// </summary>

Line 395 · comment · before public IEnumerator LookupAccount(
// ----- developer tools only

Lines 397-400 · doc comment · before public IEnumerator LookupAccount(
/// <summary>
/// Asks whether an address belongs to an account, so the dev tools can
/// name what they are about to delete before deleting it.
/// </summary>

Lines 414-418 · doc comment · before public IEnumerator DeleteAccount(
/// <summary>
/// Deletes an account and everything belonging to it, freeing the email
/// for a new sign-up. Irreversible, and refused by the server unless
/// this account is on its admin list.
/// </summary>

Lines 435-439 · doc comment · before public IEnumerator GetAdminCapability(
/// <summary>
/// Asks the server what this account may do with the developer tools.
/// Answers honestly for a non-admin rather than refusing, so the game can
/// simply not draw the button.
/// </summary>

```

Line 452 · doc comment · before public IEnumerator SendAdminCommand()

```
/// <summary>Queues an event or a payment for one online player. Admin only.</summary>
```

Lines 469–472 · doc comment · before public IEnumerator CollectAdminCommands()

```
/// <summary>
/// Collects anything a developer has queued for this player. Scoped to
/// the caller on the server, so it can only ever return their own.
/// </summary>
```

Lines 486–492 · doc comment · before private void ApplyUserUpdate(UserResponseDto user)

```
/// <summary>
/// Takes an edited profile everywhere it is held: in memory, on the
/// phone, and anywhere listening for a session change. Every screen that
/// shows a name or an address reads CurrentUser live, so this is all it
/// takes for the pause menu, the tutorial and the logout prompt to catch
/// up at once.
/// </summary>
```

Lines 508–520 · comment · before StartCoroutine(LoadFarmAndOpenWorld());

```
// Always ask the server whether this account has a farm rather than
// trusting HasFarm.
//
// That flag arrives on the login response and nowhere else, so a
// session restored from the phone has never been told either way. It
// used to short-circuit to area selection whenever the flag was false,
// which meant every returning player was sent off to pick land they
// already owned. Their save survived – the revision check refused the
// overwrite – but they could not reach it.
//
// LoadFarmAndOpenWorld already treats a 404 as "no farm yet" and routes
// to AreaSelection itself, so one request answers both cases and there
// is no cached answer left to go stale.
```

Lines 556–564 · doc comment · before public void EndSessionLocally()

```
/// <summary>
/// Ends the session without telling the server, for when the server is
/// the one that ended it.
///
/// Used when a request comes back refused because the account's password
/// was changed elsewhere. Logout would be wrong here: it posts to the
/// server first, and that post carries the very token that was just
/// rejected, so it can only fail.
/// </summary>
```

Line 570 · doc comment · before private void ClearSession()

```
/// <summary>Drops the session from memory and from this phone.</summary>
```

Lines 607–611 · comment · before if (errorCode == 401 || errorCode == 403)

```
// A refused token means this login is finished – the account's
// password was changed somewhere else, which retires every token
// issued before it. Without this the menu would look signed in and
// Start would quietly do nothing, because the message below is
// written onto a login board that is not currently on screen.
```

Lines 661–663 · comment · before yield return ApiClient.Instance.PostJson<PresenceHeartbeatRequestDto, object>()

```
// Hand the play session back, or this account stays claimed by this
// phone for the length of the online window and cannot be played
// anywhere else in the meantime.
```

Lines 679–680 · comment · before SavedSession.Save(AccessToken, CurrentUser);

```
// Kept on the phone so the next launch skips the login screen. Cleared
// only by Logout, which is the one way to change accounts on a device.
```

Line 685 · comment · before StartCoroutine(LoadPlayerSettings());

```
// Pull this account's saved settings and apply audio/render distance.
```

Lines 689–698 · doc comment · before public void SaveSettingsToAccount()

```
/// <summary>
/// Writes the player's settings to their account.
///
/// Runs here rather than on the Settings panel because that panel is an
/// ordinary scene object: pressing Save and then starting the game
/// destroyed it mid-request, so the write was abandoned and the account
/// kept its old values. ApplyFromDto then restored those old values on the
/// next launch, quietly undoing the change. AuthSession is
/// DontDestroyOnLoad, so the request finishes wherever the player goes.
/// </summary>
```

Lines 714–715 · comment · before (message, _) => Debug.LogWarning()

```
// Logged rather than swallowed. A silent failure here looks like the
// setting simply not working, which is exactly how it was found.
```

Assets/Scripts/Shared/Backend/BackendModels.cs

68 comments · 68 comment lines removed · file had 307 lines

Lines 15–19 · doc comment · before [Serializable]

```
/// <summary>
/// Sign-in credentials. <see cref="identifier"/> is whatever the player typed
```

```

/// into the single login box - an email address or their display name - and
/// the server works out which it is.
/// </summary>

```

Lines 41–46 · doc comment · before `public string email;`

```

/// <summary>
/// The address the code was sent to, for the confirmation step to quote
/// back. Empty on password recovery: the player has proven nothing at
/// that point, so the server only ever names the account in masked form
/// inside <see cref="message"/>.
/// </summary>

```

Line 50 · comment · before `[Serializable]`

```

// ----- recovery

```

Lines 65–69 · doc comment · before `[Serializable]`

```

/// <summary>
/// Proof that the emailed reset code was accepted. Held in memory for the
/// length of one reset and never written to the phone - it is not a login,
/// and the account is only remembered once the reset actually completes.
/// </summary>

```

Line 85 · comment · before `[Serializable]`

```

// ----- account panel

```

Line 131 · comment · before `[Serializable]`

```

// ----- developer tools only

```

Lines 133–137 · doc comment · before `[Serializable]`

```

/// <summary>
/// Names an account for the admin tools. The server refuses these calls
/// unless the signed-in account is on its own allow list, so holding this
/// type is not itself permission to use it.
/// </summary>

```

Lines 144–148 · doc comment · before `[Serializable]`

```

/// <summary>
/// What the server says this account may do with the developer tools. The
/// client never decides this for itself - the admin list and the mobile
/// switch both live on the server, so the phone has to ask.
/// </summary>

```

Lines 173–177 · doc comment · before `[Serializable]`

```

/// <summary>
/// An event or payment a developer is sending to one player mid-session, so
/// an alpha tester can meet a typhoon or an outbreak inside a short sitting
/// rather than waiting for the random roll to produce one.
/// </summary>

```

Line 182 · doc comment · before `public string commandType;`

```

/// <summary>GRANT_MONEY, FORCE_WEATHER or FORCE_PEST_DISEASE.</summary>

```

Line 184 · doc comment · before `public string payload;`

```

/// <summary>The weather or pest enum name; unused for money.</summary>

```

Line 197 · doc comment · before `[Serializable]`

```

/// <summary>One instruction the server had waiting for this player.</summary>

```

Lines 205–212 · doc comment · before `public int? amount;`

```

/// <summary>
/// Nullable, because the server sends null for whichever of these the
/// command does not use - no duration on a money grant, no amount on a
/// weather event. Declared as plain ints these threw on every fetch
/// ("cannot convert null to System.Int32") and the whole batch was
/// dropped, so a command arrived and was thrown away without ever being
/// applied.
/// </summary>

```

Lines 226–229 · doc comment · before `[Serializable]`

```

/// <summary>
/// Claims this account's play session for one device. The server answers
/// 409 when a different device is already playing on the account.
/// </summary>

```

Lines 244–248 · doc comment · before `public float uiScale;`

```

/// <summary>
/// Interface and text size multipliers. A server that predates these
/// columns leaves them at 0; GameSettings.ApplyFromDto reads a
/// non-positive value as "not supplied" rather than clamping it.
/// </summary>

```

Lines 252–256 · doc comment · before `public bool aiSummarization;`

```

/// <summary>
/// When true the adviser and the climate evaluation answer briefly. A bool
/// has no "absent" value, so an older server reports false, which is the
/// default anyway.
/// </summary>

```

Lines 260–264 · doc comment · before [Serializable]

```
/// <summary>
/// One AI request. The prompt is still assembled in the game so it stays
/// tunable in the Inspector, but the Gemini key and the generation settings
/// now live on the server – the phone never holds the secret.
/// </summary>
```

Line 268 · doc comment · before public string feature;

```
/// <summary>ADVISOR, CLIMATE_EVALUATION, TASK_GENERATE or TASK_CHECK.</summary>
```

Line 278 · doc comment · before public bool available;

```
/// <summary>False when the adviser is switched off server-side.</summary>
```

Line 281 · doc comment · before public string finishReason;

```
/// <summary>"STOP" means the model finished; anything else was cut off.</summary>
```

Line 283 · doc comment · before public string message;

```
/// <summary>In-character text to show when <see cref="available"/> is false.</summary>
```

Assets/Scripts/Shared/Backend/DeviceIdentity.cs

12 comments · 12 comment lines removed · file had 45 lines

Lines 6–16 · doc comment · before public static class DeviceIdentity

```
/// <summary>
/// A stable name for this installation, used to tell one phone from another
/// when the same account signs in on both.
///
/// Generated once and kept in PlayerPrefs rather than read from
/// SystemInfo.deviceUniqueIdentifier, which on Android can change between OS
/// versions, is not guaranteed on every device, and identifies the hardware
/// rather than the install. A value of our own is stable for as long as the
/// game is installed, which is exactly the lifetime of a saved login, and it
/// says nothing about the device or its owner.
/// </summary>
```

Line 23 · doc comment · before public static string Current

```
/// <summary>This installation's id. Created on first use.</summary>
```

Assets/Scripts/Shared/Backend/SavedSession.cs

25 comments · 25 comment lines removed · file had 87 lines

Lines 5–19 · doc comment · before public static class SavedSession

```
/// <summary>
/// The login this phone remembers between launches.
///
/// Holding the access token means a returning player lands on the menu
/// instead of the login form. It is written when a login succeeds and erased
/// by Logout, which is deliberately the only way to change accounts on a
/// device – there is no "switch account" path that leaves the old one behind.
///
/// PlayerPrefs is not a secure store: on Android it is an XML file in the
/// app's private directory, readable on a rooted phone or through a backup.
/// The token is therefore treated as something that can be lost rather than
/// something that cannot – it expires on its own, and the server checks it on
/// every call. Anything genuinely secret, the password included, is never
/// written here.
/// </summary>
```

Line 25 · doc comment · before public static void Save(string accessToken, UserResponseDto user)

```
/// <summary>Remembers this login on this phone, replacing any earlier one.</summary>
```

Lines 39–43 · doc comment · before public static bool TryLoad(out string accessToken, out UserResponseDto user)

```
/// <summary>
/// The remembered login, if there is one. The token is returned unchecked –
/// only the server can say whether it is still good, and AuthSession asks
/// it in the background.
/// </summary>
```

Lines 55–56 · comment · before try

```
// A half-written or older payload should cost the player the cached
// name, not the whole session – the server sends it again anyway.
```

Line 70 · doc comment · before public static void Clear()

```
/// <summary>Forgets the login, so the next launch asks for one.</summary>
```

Line 78 · doc comment · before public static string RememberedEmail()

```
/// <summary>The email of the remembered account, for the logout prompt.</summary>
```

Assets/Scripts/Shared/Backend/SessionTakeover.cs

39 comments · 39 comment lines removed · file had 94 lines

Lines 6–20 · doc comment · before public static class SessionTakeover

```
/// <summary>
/// Ends this device's play session when another device takes the account.
///
```

```

/// The farm heartbeat is what notices: it claims the account every fifteen
/// seconds, and the server refuses with 409 once a different device holds it.
/// That only happens if this phone went quiet for longer than the online
/// window - backgrounded, locked, or out of signal - long enough for the
/// other phone to claim it legitimately.
///
/// The farm is deliberately NOT saved on the way out. The other device owns
/// the account now, so a save here would be rejected by the revision check
/// anyway, and trying would only replace one clear explanation with two
/// confusing ones. Anything since the last save is lost, which is the honest
/// outcome: this device stopped being the one playing some time ago.
/// </summary>

```

Lines 28-31 · doc comment · before `public static void Raise(string reason)`

```

/// <summary>
/// Leaves the farm at once and arranges for the menu to explain why.
/// Safe to call more than once; only the first call does anything.
/// </summary>

```

Lines 42-44 · comment · before `FarmLoadContext.Clear();`

```

// Dropped so the farm being abandoned cannot be restored into the next
// scene load, and so a later Start fetches the account fresh from the
// server rather than trusting what this device had in memory.

```

Lines 50-61 · doc comment · before `public static void RaiseSignedOut(string reason)`

```

/// <summary>
/// Leaves the farm because this device's login is no longer accepted -
/// which today means the account's password was changed somewhere else.
///
/// Unlike a takeover, the saved login is thrown away as well. The token
/// on this phone can never work again, so keeping it would only produce a
/// menu that looks signed in and refuses every button; clearing it sends
/// the player to the login form, which is where they can actually fix it.
///
/// The farm is not saved on the way out, for the same reason as a
/// takeover: the server would refuse the write anyway.
/// </summary>

```

Lines 76-80 · doc comment · before `public static bool Consume(out string reason)`

```

/// <summary>
/// True once, for the menu to show its account-in-use prompt. Clearing on
/// read stops the prompt reappearing every time the player returns to the
/// menu afterwards.
/// </summary>

```

Assets/Scripts/Shared/Districts/DavaoDistrictService.cs

19 comments · 19 comment lines removed · file had 508 lines

Lines 65-73 · doc comment · before `public static readonly string[] ProductiveDistrictOrder =`

```

/// <summary>
/// The production districts in selector order. Names match the
/// entries returned by <see cref="TryGetDistrictAtNormalized"/>.
///
/// Adding a district here is not enough on its own. It also needs a row of
/// map art on the Area Selection Builder, a painted sign in the UI theme, a
/// seed pool in DistrictCropPools and in the backend's DistrictSeedPools,
/// and a description in DistrictDescriptions.
/// </summary>

```

Lines 311-315 · doc comment · before `public AreaValidationResult ValidateSelectionForDistrict(`

```

/// <summary>
/// Validates that every sampled point of the box falls inside one specific
/// district. Used to keep the player's plot within the district they picked
/// from the selector.
/// </summary>

```

Lines 364-367 · doc comment · before `public bool TryGetDistrictNormalizedBounds(string districtName, out Rect bounds, out Vector2 center)`

```

/// <summary>
/// Normalized bounding rect and centroid (u=0 left..1 right, v=0 bottom..1 top)
/// of a district's coloured region, used to aim the camera and place the box.
/// </summary>

```

Line 425 · comment · before `int stride = Mathf.Max(1, Mathf.RoundToInt(Mathf.Max(width, height) / 512f));`

```

// Sample roughly 512 points per axis regardless of source resolution.

```

Assets/Scripts/Shared/Districts/DistrictCropPools.cs

58 comments · 58 comment lines removed · file had 323 lines

Lines 6-26 · doc comment · before `public static class DistrictCropPools`

```

/// <summary>
/// Which crops each Davao district actually grows, and the starting planting
/// materials a new farm is given because of it.
///
/// The game used to hand every new player all thirteen seed types at once,
/// which made the district choice cosmetic and left nothing to discover in the
/// shop. A farm now starts with three kinds of planting material drawn from its
/// own district's crops.

```

```

///
/// The list is also the limit. A farm can only get the planting materials of
/// crops its district grows - every material of that crop, so a Calinan farm
/// can have both the banana plantlet and the banana sucker. The shop greys the
/// rest out and will not sell them, the marketplace does not show them, and a
/// trade cannot hand one over. The server holds the same table
/// (DistrictSeedPools in the backend) and refuses marketplace purchases and
/// trades on its side too, so the two must be kept identical.
///
/// The five old seed items (banana, coconut, mango, pineapple and strawberry
/// seed) are no longer sold, but they are still limited by their crop, so an
/// older save or listing cannot move them to a district that does not grow it.
/// </summary>

Line 29 · doc comment · before public const int StartingSeedKinds = 3;
/// <summary>How many different planting materials a new farm starts with.</summary>

Line 32 · doc comment · before public const int SeedsPerKind = 3;
/// <summary>How many of each.</summary>

Lines 102-106 · doc comment · before private static readonly FarmCropType[] FallbackPool =
/// <summary>
/// Fallback for a farm whose district is missing or unrecognised - an
/// older save, or a district renamed later. Four hardy crops that appear in
/// most of the districts, so such a farm is still playable.
/// </summary>

Line 115 · doc comment · before public static bool TryGetCrop(InventoryItemType item, out FarmCropType crop)
/// <summary>The crop an item is the planting material of, old seed items included.</summary>

Lines 129-132 · doc comment · before public static bool IsDistrictRestricted(InventoryItemType item)
/// <summary>
/// Whether the item is planting material, and so limited to districts at
/// all. Produce, tools and kits are not - anyone can buy and trade those.
/// </summary>

Line 138 · doc comment · before public static bool IsAvailableIn(InventoryItemType item, string districtName)
/// <summary>Whether a farm in <paramref name="districtName"/> may obtain the item.</summary>

Line 147 · doc comment · before public static bool IsAvailableToPlayer(InventoryItemType item)
/// <summary>Whether the farm being played may obtain the item.</summary>

Line 153 · doc comment · before public static List<string> DistrictsGrowing(InventoryItemType item)
/// <summary>The districts that grow an item's crop, in the order the game lists districts.</summary>

Line 172 · doc comment · before public static string JoinNames(IReadOnlyList<string> names)
/// <summary>"Calinan, Toril and Baguio".</summary>

Line 193 · doc comment · before public static FarmCropType[] CropsFor(string districtName)
/// <summary>Every crop the named district grows, or the fallback list.</summary>

Line 205 · doc comment · before public static IReadOnlyList<InventoryItemType> PoolFor(string districtName)
/// <summary>Every planting material the named district's farms can get, in shelf order.</summary>

Lines 220-230 · doc comment · before public static List<InventoryItemType> RollStartingSeeds(string districtName)
/// <summary>
/// Picks the planting materials a new farm in this district starts with:
/// three kinds from three different crops.
///
/// The draw always includes one material that goes straight into the ground
/// and one raised in the Seedling Tent, so a new player has something to
/// plant on day one and something to learn the tent with. Every district
/// grows both kinds. The first pick avoids strawberry where it can, since a
/// runner needs a mulched bed before it can go in. Ready-grown squash
/// seedlings are never dealt - they are the expensive shortcut.
/// </summary>

Lines 235-236 · comment · before for (int i = crops.Count - 1; i > 0; i--)
// Copy before shuffling: the pool arrays are shared and must not be
// reordered, or every later roll would inherit this one's shuffle.

Line 246 · comment · before if (!TryPick(crops, usedCrops, picked, direct: true, allowMulch: false))
// 1. Something to plant today, preferring a crop that needs no mulch.

Line 250 · comment · before TryPick(crops, usedCrops, picked, direct: false, allowMulch: true);
// 2. Something for the Seedling Tent.

Line 253 · comment · before while (picked.Count < StartingSeedKinds &&
// 3. Anything else the district grows, either way.

Lines 294-295 · comment · before picked.Add(options[Random.Range(0, options.Count)]);
// Squash seed can go either way; picked as the tent material it
// teaches the tent, picked first it is planted straight away.

Line 304 · doc comment · before public static string Describe(IReadOnlyList<InventoryItemType> seeds)
/// <summary>"Durian Seed, Banana Sucker and Corn Seed" - for Antonio's dialogue and the log.</summary>

```

Assets/Scripts/Shared/Settings/GameSettings.cs

58 comments · 58 comment lines removed · file had 222 lines

Lines 6–17 · doc comment · before `public static class GameSettings`

```
/// <summary>
/// Central, additive store for user-adjustable settings: the three audio
/// volumes, the render (culling) distance, and the two accessibility scales
/// for interface and text size. It is the single source of truth that
/// <see cref="GameAudioManager"/>, <see cref="DistanceCullingManager"/>,
/// the grass spawner and <see cref="UIScaleService"/> read from and react to
/// via <see cref="Changed"/>.
///
/// Locally cached in PlayerPrefs (so it survives app restarts and is available
/// before login); the per-account copy from the backend is applied via
/// <see cref="ApplyFromDto"/> right after login and overrides the local cache.
/// </summary>
```

Lines 23–33 · doc comment · before `public const float UiScaleMin = 0.80f;`

```
/// <summary>
/// Bounds for the interface scale.
///
/// The ceiling is not arbitrary. Scaling the UI up works by shrinking every
/// canvas's reference resolution, so a larger scale means fewer reference
/// units fit on screen. At 1.20 the reference height falls to 900, which
/// still clears the tallest fixed panel in the game (the marketplace board
/// at 800) and leaves the Settings board's hanging sign on screen. Raising
/// this further would start pushing panel headings off the top edge on a
/// 4:3 tablet, so it is a real limit rather than a preference.
/// </summary>
```

Lines 37–47 · doc comment · before `public const float TextScaleMin = 0.80f;`

```
/// <summary>
/// Bounds for the extra text scale, applied on top of the interface scale.
///
/// Tightened at the top and loosened at the bottom when the game moved to
/// Playpen Sans. Many labels sit in rects sized as a multiple of their own
/// font size, and text scale grows the font without growing the rect, so a
/// high ceiling eats that headroom – with a handwriting face, whose line
/// box and advance widths are both larger than the old font's, it runs out
/// sooner. The floor drops in return, because a wider face is the case
/// where a player is most likely to want text smaller rather than larger.
/// </summary>
```

Line 56 · doc comment · before `private const float DefaultUiScale = 1f;`

```
/// <summary>Today's look is 1.0 for both, so an existing player sees no change.</summary>
```

Lines 60–63 · doc comment · before `private const bool DefaultAiSummarization = false;`

```
/// <summary>
/// Off by default, which is the adviser's existing full-length answer. A
/// player who never opens Settings sees exactly what they see today.
/// </summary>
```

Line 79 · doc comment · before `public static float UiScale { get; private set; }`

```
/// <summary>Multiplier on every canvas, so panels and buttons grow together.</summary>
```

Line 82 · doc comment · before `public static float TextScale { get; private set; }`

```
/// <summary>Multiplier on font sizes, applied on top of <see cref="UiScale"/>.</summary>
```

Lines 85–91 · doc comment · before `public static bool AiSummarization { get; private set; }`

```
/// <summary>
/// When on, the farm adviser and the climate evaluation answer in a couple
/// of sentences instead of a full write-up. It carries the same findings,
/// just without the bullet lists. The two task features are deliberately
/// unaffected – they must return strict JSON, so shortening them would
/// break parsing rather than reading nicer.
/// </summary>
```

Line 94 · doc comment · before `public static event Action Changed;`

```
/// <summary>Fired whenever any value changes, so live consumers re-apply.</summary>
```

Line 178 · doc comment · before `public static void ApplyFromDto(SettingsDto dto)`

```
/// <summary>Applies the account's saved settings (from the backend). Authoritative.</summary>
```

Lines 189–193 · comment · before `if (dto.uiScale > 0f)`

```
// A server that predates the scale columns omits these, and Unity's
// deserializer leaves an absent float at 0. Clamping that would silently
// shrink the player's interface to the minimum on their next login, so a
// non-positive value is read as "the account has no opinion" and the
// value already loaded from PlayerPrefs is kept.
```

Lines 199–200 · comment · before `AiSummarization = dto.aiSummarization;`

```
// A plain bool has no "absent" value the way the scales do, so an older
// server simply reports false – which is the default anyway.
```

Line 203 · trailing comment · on the line `SaveLocal();` // keep the local cache in sync with the account

```
// keep the local cache in sync with the account
```

Assets/Scripts/Shared/Settings/MobilePerformanceBootstrap.cs

40 comments · 40 comment lines removed · file had 68 lines

Lines 5–16 · doc comment · before `public static class MobilePerformanceBootstrap`

```
/// <summary>
/// Applies the device-side performance settings once, before the first scene loads.
///
/// These are deliberately set in code rather than left to the Quality Settings
/// asset. The values there apply per quality tier, and a phone can silently land
/// on a different tier than the one that was tuned, so the frame cap - the setting
/// that most affects how the game feels over a long session - could quietly go
/// missing on the exact devices that need it most.
///
/// NOTE: because this runs at start-up it overrides the shadow values in
/// Project Settings > Quality. Change them here, not there.
/// </summary>
```

Lines 19–31 · doc comment · before `private const int TargetFrameRate = 60;`

```
/// <summary>
/// Frames per second to aim for.
///
/// Without a cap the GPU renders as fast as it physically can. The phone then
/// heats up, the chip throttles itself to cool down, and the frame rate drops
/// for the rest of the session - which reads to a player as "the game got
/// laggy after a while". Capping lets the device finish its work and idle
/// until the next frame instead.
///
/// 60 suits a farming game on a mid-range phone. If a device still throttles
/// after a long session, 30 gives it far more headroom and stays perfectly
/// smooth for this kind of game.
/// </summary>
```

Lines 34–41 · doc comment · before `private const float ShadowDistance = 25f;`

```
/// <summary>
/// How far from the camera objects still cast real-time shadows, in metres.
///
/// Everything inside this radius has its geometry submitted a second time to
/// draw the shadow map, so the cost scales with how much of the farm it
/// covers. 40 reached well past the crops the player is actually working on;
/// 25 keeps shadows where they read clearly and stops paying for the rest.
/// </summary>
```

Lines 44–48 · doc comment · before `private const int ShadowCascades = 1;`

```
/// <summary>
/// Cascades split the shadow distance into bands of differing quality. Each
/// band is another map to render and sample, which is an expensive trade on
/// mobile for a view this close to the ground. One band is enough at 25m.
/// </summary>
```

Lines 54–55 · comment · before `QualitySettings.vSyncCount = 0;`

```
// targetFrameRate is ignored while VSync is driving the frame rate, so the
// cap below only takes effect with VSync off.
```

Assets/Scripts/Shared/Settings/PauseMenuBuilder.cs

39 comments · 39 comment lines removed · file had 412 lines

Lines 8–20 · doc comment · before `public class PauseMenuBuilder : MonoBehaviour`

```
/// <summary>
/// In-farm pause menu. A round pause button in the top-left opens an overlay
/// showing the player's display name and chosen district, plus Settings and
/// Main Menu buttons. The farm keeps simulating in the background (no
/// timeScale change).
/// - Settings reuses the same <see cref="SettingsUIBuilder"/> as the Main Menu,
///   so audio/render changes apply live to the running farm.
/// - Main Menu asks whether to save the farm first (Yes = Save Farm then leave,
///   No = leave without saving, Cancel = stay).
/// Added at runtime in the TerrainPreview scene by BackendRuntimeBootstrap.
/// Art comes from the shared <see cref="UIThemeSprites"/> asset; every slot is
/// optional and falls back to the old flat-colour style.
/// </summary>
```

Line 25 · doc comment · before `public const float CornerButtonSize = 90f;`

```
/// <summary>Size of the round corner buttons; shared with the chat toggle.</summary>
```

Lines 28–33 · doc comment · before `private const float PauseContentX = 75f;`

```
/// <summary>
/// The board art's leaves hang off its left edge, so the usable plank area
/// sits right of the sprite's centre. The board is shifted left by this much
/// and its content shifted right by the same amount, which lands the buttons
/// on the true screen centre while keeping them centred on the planks.
/// </summary>
```

Lines 75–76 · comment · before `HudRegistry.RegisterIconButton(HudIconButton.SlotPause, go);`

```
// Built here rather than through HudIconButton.Create, so it has to
// register itself or the beginner guide cannot hide it.
```

```

Lines 122-124 · comment · before Vector2 panelSize = board != null ? new Vector2(900f, 618f) : new Vector2(640f, 480f);
// The decorated board carries corner art (leaves, fruit), so the panel
// is sized to the sprite's own 1440x988 aspect and drawn Simple, not
// 9-sliced.

Lines 128-129 · comment · before if (board != null)
// Shift the whole board left so its plank column lines up with the
// screen centre, where the buttons sit.

Lines 158-160 · comment · before CreateButton(panel.transform, "Settings", new Vector2(PauseContentX, -230f), PauseButtonSize,
// Rows sit inside the plank stack. The board's leaves overhang the
// left edge, so the whole column is nudged right to stay centred
// on the planks rather than on the sprite.

Line 185 · comment · before Sprite board = theme != null && theme.leavePromptBoard != null
// Falls back to the shared login/settings board when no dedicated one is set.

Lines 194-195 · comment · before rect.sizeDelta = board != null ? new Vector2(820f, 635f) : new Vector2(600f, 280f);
// Sized to the board art's 1240x960 aspect, tall enough for the wrapped
// question plus all three buttons without crowding.

Line 216 · comment · before msgRect.sizeDelta = new Vector2(board != null ? -320f : -40f, 150f);
// Narrow enough that the wrapped question stays between the two logs.

Line 304 · comment · before internal static void ApplySpriteTint(Button button)
// ----- UI helpers -----

Lines 306-308 · doc comment · before internal static void ApplySpriteTint(Button button)
/// <summary>
/// The standard hover/press tint shared by every sprite-backed button.
/// </summary>

Line 337 · comment · before image.sprite = sprite;
// The word is painted into the art, so no Text child is added.

```

Assets/Scripts/Shared/Settings/SettingsUIBuilder.cs

129 comments · 129 comment lines removed · file had 815 lines

```

Lines 9-15 · doc comment · before public class SettingsUIBuilder : MonoBehaviour
/// <summary>
/// Runtime Settings overlay opened from the Main Menu and the in-farm pause
/// menu. Six horizontal sliders (BG music, SFX, weather ambience, render
/// distance, interface size, text size) drive <see cref="GameSettings"/> live.
/// Closing after a change prompts to save; Save persists locally and (if
/// logged in) to the backend account, No reverts to the last saved values.
/// </summary>

Lines 48-51 · comment · before private const float PanelWidth = 1200f;
// Widened from 1000 to pay for the larger knob. The knob has to stop half
// its own width before each end of the groove, so a bigger knob costs travel
// unless the bar grows with it. BackgroundUI.png's caps are 130, and content
// is inset by panelPaddingX (170), so this still leaves 40 of clearance.

Lines 54-63 · doc comment · before private const float PanelHeight = 740f;
/// <summary>
/// Panel height is unchanged at 740, and the rows were tightened from 90 to
/// 72 apart to seat six of them instead of four rather than growing the
/// board. That is a hard constraint, not a preference: the interface-size
/// slider shrinks every canvas's reference resolution, so at the 1.20
/// ceiling the usable height falls to 900. A 740-tall board centred there
/// puts the top of its hanging "Settings" sign at 433 of the 450 available,
/// and a taller board would push that sign off the screen - on the one
/// panel the player needs in order to undo the setting.
/// </summary>

Lines 67-72 · doc comment · before private const float RowSpacing = 64f;
/// <summary>
/// Tightened again from 72 to seat a seventh row. The board deliberately
/// does not grow to make space: the height above is what keeps its hanging
/// sign on screen at the 1.20 interface ceiling, so rows have to fit within
/// it rather than the other way round.
/// </summary>

Line 76 · doc comment · before private const int AiRowIndex = 6;
/// <summary>Row index of the AI Summarization toggle - always the last one.</summary>

Lines 84-87 · comment · before private const float SliderGrooveLeftInset = 40f;
// Measured from SliderBar.png (840x72, border {40,0,40,0}): the grooved
// channel runs from x=40 to x=796, i.e. exactly between the two rounded
// caps. Because 9-slice caps render at native size, these insets hold at
// any bar width.

Lines 91-95 · doc comment · before private const float SliderKnobSize = 48f;
/// <summary>
/// Knob diameter. Sized as a phone touch target rather than to the art:
/// 48 clears the ~44dp minimum most mobile guidelines use. SliderBarHeight
/// must stay above this or the knob pokes out above and below the wood.
/// </summary>

```

Lines 98–102 · doc comment · before `private const float ToggleButtonSize = 60f;`

```
/// <summary>
/// The AI Summarization knob. Larger than the slider knob because it
/// carries a word, and held under RowSpacing so it cannot reach the row
/// above it.
/// </summary>
```

Lines 106–116 · doc comment · before `private const float LabelColumnWidth = 280f;`

```
/// <summary>
/// Width of the label column on every row, and of the value column at the
/// right end of the slider rows.
///
/// Sized for the largest Text Size (115%), where these labels are drawn at
/// 28 instead of 24. "BG SFX (Weather)" and "AI Summarization" measure about
/// 250 there, and "100%" and "60 m" about 80, so at the old 240 and 70 they
/// wrapped onto a second line the 60-tall row cannot show and the board
/// read "BG SFX", "AI" and "100". The slider gives up the difference and
/// still has over 300 of knob travel.
/// </summary>
```

Line 135 · comment · before `baseMusic = GameSettings.MusicVolume;`

```
// Snapshot the current values so we can detect changes on close.
```

Lines 157–159 · comment · before `UIScaleService.ApplyNow();`

```
// The board is about to be looked at closely, so make sure it is already
// at the player's chosen sizes rather than being corrected by the next
// poll a fraction of a second later.
```

Lines 196–199 · comment · before `uiScaleSlider = CreateSliderRow(panel.transform, "UI Size",`

```
// Both scale rows apply live. UIScaleService listens to GameSettings
// and re-applies on the same frame, so the board under the player's
// thumb resizes as the knob moves and they can judge the result on the
// real interface rather than on a preview.
```

Lines 210–211 · comment · before `CreateToggleRow(panel.transform, "AI Summarization", RowY(AiRowIndex),`

```
// A toggle rather than a slider, so it borrows the slider row's geometry
// but puts a single round knob button where the track would be.
```

Line 222 · doc comment · before `private void CreateHeading(Transform parent, string value, Sprite sprite)`

```
/// <summary>The hanging "Settings" sign, or a plain Text when no sprite is set.</summary>
```

Line 275 · comment · before `if (theme?.saveSettingsLabel != null)`

```
// Hanging "Save Settings?" sign above the plank.
```

Lines 315–316 · comment · before `if (aiConfirmPopup != null && aiConfirmPopup.activeSelf)`

```
// The confirm board sits over the panel; closing underneath it would
// strand it on screen with nothing behind it.
```

Lines 330–332 · comment · before `AuthSession.Instance?.SaveSettingsToAccount();`

```
// Handed to AuthSession, which outlives this panel. Starting the request
// here meant it died the moment the player left the scene, leaving the
// account on its old values.
```

Lines 340–347 · comment · before `RestoreBaseline();`

```
// Discard the edits rather than merely skipping the write.
//
// The sliders apply their value live as they are dragged, so declining to
// save previously left the new values sitting in GameSettings for the rest
// of the session. Show() reads its slider positions from GameSettings, so
// re-opening the panel showed the rejected values as though they had been
// saved. Restoring the snapshot taken in Show() puts the audio, render
// distance and slider positions back to the last saved state.
```

Lines 352–357 · doc comment · before `private void RestoreBaseline()`

```
/// <summary>
/// Returns every setting to the values captured when the panel was opened,
/// which is the last saved state. Assigning through the GameSettings setters
/// raises its Changed event, so the audio mixer and culling distance follow
/// the revert immediately instead of waiting for a restart.
/// </summary>
```

Line 399 · comment · before `private void CreateToggleRow(Transform parent, string label, float y,`

```
// ----- UI helpers -----
```

Lines 401–407 · doc comment · before `private void CreateToggleRow(Transform parent, string label, float y,`

```
/// <summary>
/// A label with a single round button where a slider's track would be.
///
/// Laid out from the same measurements as <see cref="CreateSliderRow"/> so
/// the label column lines up with the six rows above it, and the button
/// lands on the centre of the control column rather than floating.
/// </summary>
```

Lines 436–437 · comment · before `rect.sizeDelta = new Vector2(ToggleButtonSize, ToggleButtonSize);`

```
// Kept inside the row spacing so the knob never overlaps the slider on
// the row above it.
```

Line 465 · comment · before `stateText = CreateText(go.transform, "State", 22, TextAnchor.MiddleCenter);`

```
// The word sits on the knob, so it is dark to read against the wood.
```

Lines 473–476 · doc comment · before `private void BuildAiConfirmPopup(RectTransform parent)`

```
/// <summary>
/// The "Do you want to turn ... on/off?" board, built from the same trade
/// request art the social panel uses and the shared Yes/No buttons.
/// </summary>
```

Line 524 · doc comment · before `private void OnAiSummarizationPressed()`

```
/// <summary>Asks before flipping, so a mis-tap never silently changes it.</summary>
```

Lines 537–538 · comment · before `GameSettings.SetAiSummarization(!GameSettings.AiSummarization);`

```
// Applied straight away like the sliders. Closing the board still asks
// whether to keep it, and answering No puts it back.
```

Lines 553–554 · comment · before `const float labelWidth = LabelColumnWidth;`

```
// Lay the row out between the board's two log ends: label on the
// left, then the grooved bar, then the live value on the right.
```

Lines 586–587 · comment · before `sliderRect.anchoredPosition = new Vector2(sliderLeft + sliderWidth * 0.5f, y - 2f);`

```
// -2 rather than -10 so the taller bar's centre still lines up with the
// 60-tall label beside it.
```

Lines 630–631 · comment · before `fillImage.color = barSprite != null`

```
// The wooden groove art already reads as the track, so the green
// progress fill is hidden once the bar sprite is in use.
```

Lines 641–649 · comment · before `float knobHalf = (knobSprite != null ? SliderKnobSize : 28f) * 0.5f;`

```
// This rect is the knob's travel range, and Unity places the knob's
// CENTRE on its edges - so at the old inset of 10 a 44-wide knob hung
// 12 units past each end of the bar.
//
// SliderBar.png is 840x72 with a {40,0,40,0} border. Those caps are the
// rounded wooden ends and are drawn at native size whatever the bar's
// width, so the grooved channel between them always begins 40 in from
// the left and 44 from the right. Insetting by that plus half the knob
// keeps the knob inside the groove at both extremes.
```

Lines 660–661 · comment · before `handleRect.sizeDelta = new Vector2(SliderKnobSize, SliderKnobSize);`

```
// Kept below SliderBarHeight so the knob never pokes out above or
// below the wood, as it did when 44 sat in a 40-tall bar.
```

Line 704 · comment · before `image.sprite = sprite;`

```
// The word is painted into the art, so no Text child is added.
```

Line 744 · comment · before `image.sprite = board;`

```
// Sliced so the rolled log ends keep their true size.
```

Lines 757–763 · doc comment · before `private static void FitToRow(Text text)`

```
/// <summary>
/// Lets a row's label or value shrink to fit its column instead of losing
/// its second line, the same fix as the shop's status row and the tutorial's
/// objective plank. The columns are sized so every current label fits at the
/// largest text size, so this is only a safety net - best fit never draws
/// above the font size, and the text size setting scales both limits.
/// </summary>
```

Line 797 · trailing comment · on the line `scaler.matchWidthOrHeight = 1f; // landscape: scale by height`

```
// landscape: scale by height
```

Assets/Scripts/Shared/Settings/UIScaleService.cs

100 comments · 100 comment lines removed · file had 351 lines

Lines 8–34 · doc comment · before `public class UIScaleService : MonoBehaviour`

```
/// <summary>
/// Applies the player's interface and text size settings to every canvas and
/// every label in the game, in every scene.
///
/// All of AgriDabaw's UI is constructed in C# at runtime by a couple of dozen
/// <c>*UIBuilder</c> classes, and panels are created, shown and destroyed
/// continuously. There is no single place to hook, so this follows the same
/// approach <see cref="GameAccessibility"/> already uses for the same reason:
/// re-scan on a short interval rather than trying to intercept every builder.
/// That keeps the feature entirely additive - no builder is modified, no
/// RectTransform is repositioned, and nothing breaks if a new panel is added
/// later, because it is picked up by the next sweep.
///
/// Two different mechanisms, because the two sliders mean different things:
///
/// * <b>Interface size</b> divides each canvas's reference resolution. Every
///   canvas in the game is ScaleWithScreenSize at 1920x1080 matched to
///   height, so halving the reference height doubles everything drawn on it -
///   panels, buttons, icons and the text inside them - with no layout maths
///   of our own. Nothing in the project reads referenceResolution back, so
///   rewriting it has no other effect.
///
/// * <b>Text size</b> multiplies each label's font size on top of that, for
///   players who want readable text without giving up screen space.
```

```

/// Both default to 1.0, which reproduces today's appearance exactly.
/// </summary>

Lines 39-51 · doc comment · before private const float PollInterval = 0.1f;
/// <summary>
/// How often newly built UI is picked up.
///
/// Panels that are built and shown in the same instant - a marketplace
/// listing row, a chat line, a task entry - would otherwise be drawn at
/// their authored size for one interval before being corrected, which
/// reads as a flicker. At 10 Hz that window is short enough not to be
/// seen, and the scan itself is two native type queries over a few hundred
/// components, which costs far less than the panel it is correcting.
///
/// Neither settings changes nor opening the Settings board wait for this;
/// both apply immediately.
/// </summary>

Lines 56-60 · doc comment · before private sealed class CanvasRecord
/// <summary>
/// A canvas's authored reference resolution, plus what we last wrote, so a
/// builder re-setting it on an existing canvas is recognised as a new
/// baseline instead of being treated as our own scaled value.
/// </summary>

Line 68 · doc comment · before private sealed class TextRecord
/// <summary>The same idea for a label's authored font size.</summary>

Lines 98-101 · doc comment · before public static void ApplyNow()
/// <summary>
/// Re-applies both scales right now. Called after a scene loads and after
/// any settings change; safe to call at any time.
/// </summary>

Lines 138-139 · comment · before canvases.Clear();
// The previous scene's canvases and labels are gone; their records would
// only be pruned lazily, and their instance ids are never reused.

Line 160 · comment · before private void ApplyCanvasScale(float uiScale)
// ----- interface

Lines 188-189 · comment · before record.baseResolution = scaler.referenceResolution;
// A builder wrote its own reference resolution after we scaled
// this canvas. That value is the new authored baseline.

Line 203 · comment · before private void ApplyTextScale(float textScale)
// ----- text

Lines 235-236 · comment · before if (text.fontSize != record.appliedFontSize)
// Some panels rewrite a label's size after it is built - the
// shop's item name is one. Whatever they set is the new baseline.

Lines 252-253 · comment · before if (text.resizeTextForBestFit)
// Best-fit labels ignore fontSize and clamp themselves between these
// two, so they need the same treatment or they would not scale.

Lines 272-292 · doc comment · before private static void RescueFromTruncation(Text text, int fontSize)
/// <summary>
/// Stops a one-line label from vanishing when it outgrows its own box.
///
/// Unity's legacy Text does not clip a line that is too tall for its rect -
/// under the default Truncate it discards the line outright, so the label
/// renders as nothing at all. Playpen Sans has a line box around 1.45x its
/// font size where the font the UI was laid out against sat near 1.15x, and
/// a good number of labels live in rects sized as a fixed multiple of their
/// font. Those flipped from comfortable to invisible on the font change,
/// and the text setting pushes more of them over the line, because it grows
/// the font without growing the rect.
///
/// The test below is deliberately narrow. A rect shorter than
/// <see cref="MinLineBoxRatio"/> times its font size cannot fit even one
/// line, so Truncate there can only ever produce an empty label - switching
/// it to Overflow strictly improves matters. Anything with a taller rect is
/// left alone, which is what keeps the deliberate Truncate on genuinely
/// multi-line text intact: the climate evaluation body and the social and
/// marketplace rows all sit in tall containers and use truncation to stay
/// inside them, and they never match this condition.
/// </summary>

Lines 304-305 · comment · before if (available <= 0f)
// A rect that has not been laid out yet reports zero; leave it for a
// later sweep rather than acting on a height that is not real.

Lines 313-329 · doc comment · before private const float MinLineBoxRatio = 1.85f;
/// <summary>
/// Height, as a multiple of font size, below which a rect cannot hold one
/// line of text.
///
/// Read out of PlaypenSans.ttf rather than estimated. With unitsPerEm 1000
/// the font declares hhea ascender 1170 and descender -340 with no line gap

```

```

/// (1.510 em), and OS/2 winAscent 1294 with winDescent 502 (1.796 em).
/// FreeType, which is what Unity's dynamic font rendering sits on, normally
/// takes the larger OS/2 pair, so the threshold is set just above 1.796
/// rather than between the two - being too high only converts a few labels
/// that would have been fine to Overflow, which costs nothing on a
/// single-line label, while being too low leaves them invisible.
///
/// For contrast, the built-in font this UI was originally laid out against
/// sits near 1.15 em. That 56% jump is why so many boxes that were
/// comfortable became too short the moment the typeface changed.
/// </summary>

```

Assets/Scripts/Shared/UI/DescriptionBoard.cs

68 comments · 68 comment lines removed · file had 345 lines

Lines 6-19 · doc comment · before `public class DescriptionBoard`

```

/// <summary>
/// The wooden board that shows a long description, centred over whatever
/// opened it, with a CLOSE plank along the bottom.
///
/// Shared rather than copied because two screens use it - the shop's item
/// descriptions and the area selection's district descriptions - and they are
/// meant to be the same board. Kept as a plain class rather than a
/// MonoBehaviour: it owns its GameObjects and needs no update loop, so the
/// caller holds one of these as a field and forgets about it.
///
/// It parents itself to whatever transform it is given, which is what makes it
/// safe to leave lying around: hiding the parent panel hides this too, without
/// every close path having to know it exists.
/// </summary>

```

Lines 22-29 · doc comment · before `private const float BoardWidth = 1200f;`

```

/// <summary>
/// TradeConfirmBoard.png is 1760x1320 with fixed 130px 9-slice corners, and
/// its painted logs run to x=280 and resume at x=1458. Because slicing
/// holds the corners at true size, those logs eat a near-constant amount
/// off the board however wide it is drawn - at 380 wide only about 96 units
/// of plank would remain to write on. At 1200 the text column is about 730,
/// which is what makes this readable rather than a ribbon.
/// </summary>

```

Line 36 · doc comment · before `private const float CloseWidth = 230f;`

```

/// <summary>CloseButton.png is 640x190, so preserveAspect keeps this true.</summary>

```

Line 40 · doc comment · before `private const float Footer = 104f;`

```

/// <summary>Room left under the text for the close plank.</summary>

```

Line 50 · doc comment · before `public static DescriptionBoard Create(Transform parent, UIThemeSprites theme)`

```

/// <summary>Builds the board hidden. Call <see cref="Show"/> to open it.</summary>

```

Line 82 · comment · before `boardImage.raycastTarget = true;`

```

// Left on so the board swallows clicks meant for whatever is behind it.

```

Lines 102-114 · doc comment · before `private void BuildScroll(float inset)`

```

/// <summary>
/// A scrolling text column.
///
/// The label IS the scroll content rather than a child of a layout group.
/// Wrapping it in one left the label sized by the group instead of by the
/// viewport, and the mask then cut characters off both edges. Stretched
/// horizontal anchors with a zero sizeDelta give it exactly the viewport's
/// width, so nothing can fall underneath the mask.
///
/// The text also overflows vertically rather than truncating: a truncating
/// label silently drops whole lines once the player raises the text size,
/// so the words would vanish rather than scroll.
/// </summary>

```

Lines 203-210 · doc comment · before `public static Button CreateOpenButton(Transform parent, UIThemeSprites theme,`

```

/// <summary>
/// The plank button that opens a board.
///
/// Shared for the same reason the board itself is: two screens put one of
/// these on screen and they are meant to be the same button. The caller
/// positions the returned button; only its size is passed in, because the
/// two screens have very different space to give it.
/// </summary>

```

Lines 229-231 · comment · before `image.preserveAspect = false;`

```

// TradeRequestBoard.png has no 9-slice border, so it can only be
// stretched, never sliced. Its grain runs lengthwise, which is why
// a wider-than-native plank still reads as a plank.

```

Line 254 · doc comment · before `public void Show(string title, string body)`

```

/// <summary>Fills the board and brings it to the front.</summary>

```

Line 267 · comment · before `root.transform.SetAsLastSibling();`

```

// Drawn last so it covers everything its parent panel holds.

```

Lines 270–271 · comment · before `if (scroll != null)`

```
// A new subject starts at the top rather than wherever the last one was
// left scrolled to.
```

Lines 282–290 · doc comment · before `internal static float ContentInset(UIThemeSprites theme, float width)`

```
/// <summary>
/// How far in from each edge the plank wall begins.
///
/// Computed from the width rather than fixed, because 9-slicing keeps the
/// corners at true size and stretches only the middle, so the painted log
/// moves inward as the board narrows. Laying text out against the sprite's
/// 9-slice border instead would put the first eighty-odd units of every
/// line on top of the log.
/// </summary>
```

Lines 310–314 · doc comment · before `internal static Text CreateLabel(Transform parent, int fontSize,`

```
/// <summary>
/// A label styled for the board's planks. The dark browns used elsewhere
/// are near-invisible against this art, so it uses a light fill over a hard
/// outline, which stays readable over both plank and log.
/// </summary>
```

Assets/Scripts/Shared/UI/GameFonts.cs

25 comments · 25 comment lines removed · file had 79 lines

Lines 5–18 · doc comment · before `public static class GameFonts`

```
/// <summary>
/// The one place the game decides what its text is drawn in.
///
/// Every label in AgriDabaw is built in code, and each builder used to ask
/// Unity for the built-in LegacyRuntime.ttf directly – fifty-three separate
/// calls across twenty-five files. Routing them all through here means the
/// typeface is chosen once, and a future change is one edit rather than a
/// sweep that is easy to do incompletely.
///
/// The font is loaded from Resources rather than referenced from a scene,
/// because none of the UI exists as a prefab to hold a reference. If it is
/// missing the game falls back to the built-in font and keeps running – a
/// missing font asset should look wrong, not crash a farm.
/// </summary>
```

Lines 21–30 · doc comment · before `private const string FontFolder = "Fonts";`

```
/// <summary>
/// Where the typeface lives. The shipped file is Google's variable build,
/// renamed from PlaypenSans[wght].ttf – the square brackets in the upstream
/// name are awkward in a Resources path, and Unity renders the font's
/// default instance (weight 400) either way, since the legacy Text
/// component cannot address variable axes.
///
/// An exact miss falls through to a scan of the same folder, so swapping in
/// a static build later needs no code change.
/// </summary>
```

Line 40 · doc comment · before `public static Font Primary`

```
/// <summary>The typeface every label in the game is drawn in.</summary>
```

Assets/Scripts/Shared/UI/HudIconButton.cs

16 comments · 16 comment lines removed · file had 103 lines

Lines 7–13 · doc comment · before `public static class HudIconButton`

```
/// <summary>
/// The row of round wooden buttons along the farm HUD's top-left edge.
///
/// Every builder that owns one of these buttons asks for a slot number rather
/// than hard-coding a position, so the row stays evenly spaced and nothing
/// overlaps even though five different scripts create the buttons.
/// </summary>
```

Line 21 · comment · before `public const int SlotPause = 0;`

```
// Left-to-right order of the row.
```

Lines 36–40 · doc comment · before `public static Button CreateC`

```
/// <summary>
/// Creates a round icon button in the given slot. With no sprite it falls
/// back to a dark box carrying <paramref name="fallbackLabel"/>, so the HUD
/// still works before the art is dropped in.
/// </summary>
```

Lines 60–62 · comment · before `HudRegistry.RegisterIconButton(slot, go);`

```
// Every one of these buttons is created through here, so registering
// once covers the whole row. The beginner guide reveals them one at a
// time as Antonio introduces each feature.
```

Assets/Scripts/Shared/UI/HudRegistry.cs

71 comments · 71 comment lines removed · file had 289 lines

Line 6 · doc comment · before `public enum HudPiece`

```
/// <summary>The individually hideable pieces of the farm HUD.</summary>
```

Lines 17–19 · comment · before `CropInfoPanel,`

```
// Panels below are registered so the beginner guide can tell when the  
// player has actually opened and closed one. They are never hidden by the  
// guide – their own buttons already control that.
```

Lines 31–43 · doc comment · before `public static class HudRegistry`

```
/// <summary>  
/// A lookup of the farm HUD's parts, so the beginner guide can reveal them one  
/// at a time instead of dropping the whole interface on a new player at once.  
///  
/// The alternative was a public Show/Hide method on each of the eight builders  
/// that create HUD elements. This is one line per element instead, added where  
/// the element is already being built, and it changes no existing behaviour –  
/// an unregistered or unrevealed piece simply stays as the builder left it.  
///  
/// Everything is looked up defensively. Unity objects become "fake null" when  
/// their scene unloads, so a stale entry from a previous farm is treated as  
/// absent rather than throwing.  
/// </summary>
```

Lines 52–61 · doc comment · before `private static bool tutorialControlsVisibility;`

```
/// <summary>  
/// While true, the beginner guide owns HUD visibility: anything not yet  
/// revealed is hidden, including pieces that register later.  
///  
/// This matters because the builders do not all finish at the same time.  
/// Hiding once, at one moment, only ever hid whatever happened to exist by  
/// then – the joystick, the jump button and any button built a frame later  
/// simply stayed on screen. Recording the intent instead, and applying it  
/// at registration, makes the order irrelevant.  
/// </summary>
```

Line 78 · doc comment · before `public static void RegisterIconButton(int slot, GameObject go)`

```
/// <summary>Registers one of the round buttons, keyed by its HudIconButton slot.</summary>
```

Lines 112–116 · doc comment · before `public static void BeginTutorialControl()`

```
/// <summary>  
/// Hands visibility to the beginner guide. Everything currently registered  
/// is hidden, and everything registered from now on is hidden as it arrives,  
/// until the guide reveals it by name.  
/// </summary>
```

Line 136 · doc comment · before `public static void EndTutorialControl()`

```
/// <summary>Gives visibility back to the game and shows the full HUD.</summary>
```

Lines 156–166 · doc comment · before `public static void CloseOtherPanels(HudPiece opening)`

```
/// <summary>  
/// Closes every other full-screen panel, so only one is ever open.  
///  
/// Each panel used to be responsible only for itself, which meant tapping a  
/// second HUD button stacked its panel on top of the first instead of  
/// replacing it. Every panel now calls this as it opens and the previous  
/// one steps aside.  
///  
/// The map is a special case: it is a permanent part of the HUD in its  
/// small form, so it is folded back down rather than switched off.  
/// </summary>
```

Lines 189–192 · doc comment · before `private static bool IsPanel(HudPiece piece)`

```
/// <summary>  
/// Panels open and close on their own buttons, so the guide only watches  
/// them – it must never force one open by "revealing" it.  
/// </summary>
```

Line 221 · doc comment · before `public static void SetAllVisible(bool visible)`

```
/// <summary>Shows or hides every registered piece and every round button.</summary>
```

Lines 237–257 · doc comment · before `public static void ReleaseControl()`

```
/// <summary>  
/// Drops references to a farm that has been unloaded. Called when a farm  
/// scene is left, so the next farm's registrations start clean.  
/// </summary>  
/// <summary>  
/// Drops the beginner guide's ownership of the HUD when a farm is left, and  
/// forgets objects that went away with it.  
///  
/// It deliberately does NOT wipe the whole registry. Several builders –  
/// the crop info panel, the objectives book, the save confirmation – build  
/// and register during Awake, and Unity raises sceneLoaded after every  
/// Awake but before any Start. Emptying the dictionaries there threw those  
/// registrations away a moment after they were made, which is why the  
/// objectives button was never hidden and why tapping a crop could not  
/// satisfy the inspection step: the guide was looking up a panel that was
```

```

/// no longer in the registry.
///
/// Entries belonging to the old scene are destroyed Unity objects, so they
/// compare equal to null and are pruned here; anything still alive is
/// simply overwritten when the new scene's builder registers.
/// </summary>

```

Assets/Scripts/Shared/UI/MobileTextEntryOverlay.cs

63 comments · 63 comment lines removed · file had 268 lines

Lines 7–28 · doc comment · before `public class MobileTextEntryOverlay : MonoBehaviour`

```

/// <summary>
/// Shows what the player is typing in a bar across the top of the screen,
/// because on a phone in landscape the soft keyboard covers most of the game
/// and usually the text box itself with it.
///
/// Android has its own version of this – the fullscreen "extract" editor –
/// but it cannot be relied on. uGUI already asks for it: InputField sets
/// TouchScreenKeyboard.hideInput from shouldHideMobileInput, which defaults to
/// false on Android, so the native editor is enabled and still does not
/// appear. Whether it shows is up to the device's keyboard app and the
/// activity's window flags, neither of which the game controls. Drawing it
/// ourselves gives every phone the same behaviour.
///
/// This watches the EventSystem instead of being wired into each panel, so it
/// covers every text box in the game – login, sign-up, the verification code,
/// the AI adviser, player chat, marketplace search, the shop – without any of
/// those builders knowing it exists.
///
/// It is deliberately inert: its canvas has no GraphicRaycaster and nothing on
/// it is a raycast target, so it cannot take focus from the field being typed
/// into, swallow a tap, or change any existing behaviour. It only reads.
/// </summary>

```

Line 49 · doc comment · before `private const int MaxVisibleCharacters = 240;`

```

/// <summary>Longest tail of text kept on screen; older characters scroll off the front.</summary>

```

Lines 54–64 · doc comment · before `private GameObject canvasRoot;`

```

/// <summary>
/// The overlay's own canvas, kept switched OFF whenever nobody is typing.
///
/// This is load-bearing, not tidiness. Every UI builder in the game finds
/// its canvas with FindObjectOfType<Canvas>() and assumes the scene has
/// exactly one, so a second permanent canvas makes two builders pick
/// different ones and whichever has the lower sorting order disappears
/// behind the other. FindObjectOfType skips inactive objects, so while
/// this is off it cannot be found, and nothing is built while the player
/// is mid-keystroke.
/// </summary>

```

Line 102 · doc comment · before `private InputField ResolveFocusedField()`

```

/// <summary>The InputField the keyboard is currently typing into, if any.</summary>

```

Lines 135–151 · doc comment · before `private string BuildDisplayText(InputField field)`

```

/// <summary>
/// The typed text, trimmed to the most recent characters, with a blinking
/// caret on the end.
///
/// Passwords are shown here in the clear, deliberately. The field itself
/// still masks them – that is Unity drawing its own Password contentType,
/// and it is untouched. This bar exists only because the soft keyboard
/// covers the box being typed into, so repeating the mask up here made it
/// useless for exactly the fields where a typo is hardest to notice and
/// most annoying to recover from: the password, its confirmation, and the
/// sign-up password.
///
/// The trade is that a password is briefly readable over the player's
/// shoulder while they type it. That is the same exposure as any "show
/// password" eye toggle, it lasts only while the field has focus, and the
/// bar is already showing every other credential they type.
/// </summary>

```

Lines 163–167 · doc comment · before `private static string PlaceholderOf(InputField field)`

```

/// <summary>
/// Names the box being typed into, so the bar is not just a floating line
/// of text. Falls back to the object's name when a field has no
/// placeholder of its own.
/// </summary>

```

Lines 188–189 · comment · before `canvas.sortingOrder = 32000;`

```

// Above every other canvas in the game. No GraphicRaycaster is added,
// which is what keeps this bar from ever intercepting a touch.

```

Line 195 · trailing comment · on the line `scaler.matchWidthOrHeight = 1f; // landscape: scale by height`

```

// landscape: scale by height

```

Lines 244–245 · comment · before `return text;`

```

// Geometry is left to the caller; both of them position their label
// differently and would only overwrite anything set here.

```

Line 249 · doc comment · before `private void CreateUnderline()`

```
/// <summary>The green rule along the bottom edge, echoing a text field's underline.</summary>
```

Assets/Scripts/Shared/UI/UIPlank.cs

45 comments · 45 comment lines removed · file had 168 lines

Lines 6–18 · doc comment · before `public static class UIPlank`

```
/// <summary>
/// A wooden plank with words written on it.
///
/// Most signs in the game are pictures with their wording painted in, so the
/// builders just drop a sprite. The account and recovery screens cannot work
/// that way – "Change Display Name", a player's own email address, the
/// heading that has to say a different thing on four different panels – so
/// they reuse the blank planks the tutorial and trade screens already ship
/// and print live text on top.
///
/// Shared between the login board and the account panel because both need
/// exactly this and neither should own it.
/// </summary>
```

Line 21 · doc comment · before `private static readonly Color TextShadow = new Color(0.12f, 0.07f, 0.02f, 0.95f);`

```
/// <summary>Dark brown, the colour the wood art is outlined in.</summary>
```

Lines 24–33 · doc comment · before `public static Vector2 SizeFor(Sprite sprite, float width, float fallbackHeight)`

```
/// <summary>
/// A box the plank fills exactly, rather than one it is letterboxed
/// inside.
///
/// The non-sliced planks are drawn with preserveAspect, which fits the
/// art inside its box and leaves empty space on two sides when the box is
/// the wrong shape. Text stretched across that box would then sit off the
/// wood. Deriving the height from the sprite means the box is always the
/// plank's own shape and there is no empty space to sit in.
/// </summary>
```

Lines 42–45 · doc comment · before `public static GameObject Create()`

```
/// <summary>
/// A plank carrying one line of text, anchored to the top of its parent
/// like everything else on these boards.
/// </summary>
```

Line 72 · doc comment · before `public static Button CreateButton()`

```
/// <summary>The same plank, pressable.</summary>
```

Lines 114–115 · comment · before `image.type = Image.Type.Sliced;`

```
// 9-sliced planks keep their rope ends at any size, so the box
// can be whatever shape the layout wants.
```

Lines 132–140 · comment · before `rect.offsetMin = new Vector2(plankSize.x * 0.10f, plankSize.y * 0.12f);`

```
// Inset from the painted rope ends and the plank's rounded edges,
// proportionally so it holds at every plank size.
//
// The vertical inset is the smaller of the two on purpose. Playpen
// Sans draws a line box about 1.8x its font size – well over half
// again what the old built-in font needed – so at the largest text
// setting a generous top-and-bottom margin is what pushes a line out
// of its plank. There is nothing painted along those edges to avoid
// anyway; the rope ends are on the left and right.
```

Lines 152–154 · comment · before `label.verticalOverflow = VerticalWrapMode.Overflow;`

```
// Never Truncate: Unity throws away a line that does not fit rather
// than clipping it, and this font's line box is tall enough that a
// label at the largest text setting would simply vanish.
```

Lines 158–159 · comment · before `Outline outline = go.AddComponent<Outline>();`

```
// White reads on the dark tutorial plank but not on the lighter trade
// plank, and both are used here. The outline carries it on either.
```

Assets/Scripts/Shared/UI/UIThemeSprites.cs

25 comments · 25 comment lines removed · file had 522 lines

Lines 6–22 · doc comment · before `[System.Serializable]`

```
/// <summary>
/// Drag-and-drop home for every Main Menu / Auth / Settings UI sprite.
///
/// The Login, Sign-up, Verify and Settings panels are all built at runtime,
/// so those components have no Inspector of their own to drop sprites into.
/// This asset solves that: it lives in Assets/Resources as a single
/// "UITheme" asset, the builders load it by name, and every slot below shows
/// up in one Inspector.
///
/// Every slot is optional. A slot left empty simply falls back to the old
/// flat-colour look for that one element, so the UI never breaks while the
/// art is still being made.
///
/// SETUP: right-click in Assets/Resources -> Create -> AgriDabao -> UI Theme,
```

```

/// and name it exactly "UITheme".
/// </summary>
/// <summary>One district's painted name sign, paired with the district it belongs to.</summary>

```

Line 38 · doc comment · before public static UIThemeSprites Instance

```

/// <summary>The one theme asset, or null if it has not been created yet.</summary>

```

Lines 49–50 · comment · before if (cached == null)

```

// Normalise Unity's "fake null" to a real null so callers can
// safely use the ?. operator against this reference.

```

Line 139 · doc comment · before public Sprite GetDistrictSign(string districtName)

```

/// <summary>The painted sign for a district, or null to fall back to text.</summary>

```

Lines 269–272 · comment · before [Tooltip("Waving. Greetings, farewells, praise.")]

```

// These four are VideoClips, not Sprites, so Antonio's portrait moves while
// he talks. Drop the .mp4 files straight in. Note that MP4 carries no alpha
// channel: whatever is behind him in the video is what will show inside the
// frame, so keep all four on the same background.

```

Assets/Scripts/TerrainPreview/AIAdviser/AiBackendClient.cs

26 comments · 26 comment lines removed · file had 122 lines

Lines 8–17 · doc comment · before public static class AiBackendClient

```

/// <summary>
/// Routes every AI request through the game's own backend instead of calling
/// Google directly.
///
/// A Unity build serialises its Inspector values into the shipped package, so
/// an API key held on a MonoBehaviour can be recovered by unpacking the APK.
/// The phone now sends only the prompt and its player JWT; the backend holds
/// the Gemini key, applies the generation settings, and can be switched off
/// centrally without shipping a new build.
/// </summary>

```

Line 25 · doc comment · before public const string OfflineMessage =

```

/// <summary>Shown if the server is unreachable, matching its own wording.</summary>

```

Lines 29–36 · doc comment · before public static IEnumerator Generate()

```

/// <summary>
/// Sends one generation request.
///
/// <paramref name="onUnavailable"/> covers the adviser being switched off
/// server-side, or the server failing to reach Gemini. It carries text
/// written in the adviser's voice, so callers can show it as a normal
/// reply rather than surfacing an error to the player.
/// </summary>

```

Lines 76–77 · comment · before Debug.LogWarning("[AI] Backend request failed: " + error);

```

// The server itself is unreachable. Answer in character rather than
// showing the player a transport error mid-game.

```

Lines 98–101 · comment · before if (string.IsNullOrEmpty(response.text))

```

// An empty body with available=true should not happen - the server turns
// blank replies into an unavailable response - but if it ever does, the
// caller would render nothing and the player would see a blank panel
// with no clue why. Fail loudly instead.

```

Line 115 · doc comment · before public static bool Completed(string finishReason)

```

/// <summary>True when the model finished its own sentence rather than being cut off.</summary>

```

Assets/Scripts/TerrainPreview/AIAdviser/AiResponseStyle.cs

23 comments · 23 comment lines removed · file had 47 lines

Lines 3–16 · doc comment · before public static class AiResponseStyle

```

/// <summary>
/// Turns the player's "AI Summarization" setting into a line of prompt.
///
/// Both talking AI features - the adviser chat and the post-event climate
/// evaluation - carry their own system guidance, written as a tunable field on
/// their MonoBehaviour, and each of those describes a full-length answer with
/// bullet lists. Rather than keep two shortened copies of that guidance in
/// sync, the brief instruction is appended last, where it overrides the format
/// the guidance above it asked for.
///
/// The two task features deliberately do not use this. They must return strict
/// JSON for the game to parse, so trimming their wording would break them
/// rather than make them easier to read.
/// </summary>

```

Lines 19–23 · doc comment · before private const string BriefDirective =

```

/// <summary>
/// Deliberately about length and shape only. It does not tell the model to
/// leave anything out, because a shorter answer that quietly drops the
/// warning the player needed is worse than a long one.
/// </summary>

```

Lines 32–35 · doc comment · before `public static string Apply(string guidance)`

```
/// <summary>
/// Returns <paramref name="guidance"/> with the brevity instruction appended
/// when the player has summarization on, and unchanged when they do not.
/// </summary>
```

Assets/Scripts/TerrainPreview/AIAdviser/AiText.cs

24 comments · 24 comment lines removed · file had 75 lines

Lines 5–16 · doc comment · before `public static class AiText`

```
/// <summary>
/// Takes the Markdown out of a Gemini reply before a text box shows it.
///
/// The AI Adviser and the Climate Resilience Evaluation ask for bullets and
/// short sections, and Gemini marks those up in Markdown – bold, "*"
/// bullets, "## " headings. The game's text boxes print text exactly as they
/// get it, so players saw asterisks around words (ISS-15). Every word stays;
/// only the marks go, and bullets become a plain "-", which any font has.
///
/// Underscores are left alone on purpose. Crop names such as mangosteen_1 are
/// written with them, and reading them as Markdown would eat them.
/// </summary>
```

Line 19 · doc comment · before `private static readonly Regex CodeFenceLine =`

```
/// <summary>A ``` line opening or closing a code block, with its line break.</summary>
```

Line 23 · doc comment · before `private static readonly Regex DividerLine =`

```
/// <summary>A line of three or more -, * or _ with nothing else on it: a divider.</summary>
```

Line 27 · doc comment · before `private static readonly Regex HeadingMarks =`

```
/// <summary>The #'s in front of a heading.</summary>
```

Line 31 · doc comment · before `private static readonly Regex StarBullet =`

```
/// <summary>A "*" or "+" bullet; "-" bullets are already plain.</summary>
```

Lines 35–38 · doc comment · before `private static readonly Regex Italic =`

```
/// <summary>
/// *words* in italics. The stars must hug the words, so "2 * 3" and
/// "x*y" are not touched.
/// </summary>
```

Line 42 · doc comment · before `private static readonly Regex InlineCode =`

```
/// <summary>`words` in a code span.</summary>
```

Lines 60–61 · comment · before `result = result.Replace("**", string.Empty);`

```
// Bold first, so "* **Water early:** ..." is an ordinary bullet by the
// time bullets are looked for, and "***word***" is left as an italic.
```

Line 68 · comment · before `result = ExtraBlankLines.Replace(result, "\n\n");`

```
// Removed dividers and code fences leave empty lines behind.
```

Assets/Scripts/TerrainPreview/AIAdviser/FarmAdvisorChatSystem.cs

5 comments · 5 comment lines removed · file had 137 lines

Line 88 · comment · before `uiBuilder.SetMood(AdvisorMood.Thinking);`

```
// Antonio thinks while the request is in flight, then teaches once it lands.
```

Line 110 · comment · before `uiBuilder.SetMood(AdvisorMood.Teaching);`

```
// Stop the thinking loop even when the request failed.
```

Lines 116–118 · comment · before `Debug.LogWarning("[FarmAdvisor] Reply was incomplete; asking the player to repeat.");`

```
// The reply was cut off partway. Half an answer about crop
// health is worse than none, so ask the player to try again
// rather than showing the fragment.
```

Assets/Scripts/TerrainPreview/AIAdviser/FarmAdvisorContextBuilder.cs

7 comments · 7 comment lines removed · file had 306 lines

Line 54 · doc comment · before `[System.Serializable]`

```
/// <summary>One bag in the Seedling Tent. The tent is not near any crop, so it is listed on its own.</summary>
```

Lines 109–114 · doc comment · before `private static List<AdvisorSeedlingBagData> BuildSeedlingTent()`

```
/// <summary>
/// The Seedling Tent's bags, wherever the player stands. Crops are only
/// described within 25 m of the player, and the tent is usually further
/// away than that, so without this the adviser could not see a single
/// seedling the player is raising.
/// </summary>
```

Assets/Scripts/TerrainPreview/AIAdviser/FarmAdvisorGeminiClient.cs

33 comments · 33 comment lines removed · file had 90 lines

Lines 8-15 · doc comment · before `public class FarmAdvisorGeminiClient : MonoBehaviour`

```
/// <summary>
/// Asks the farm adviser a question.
///
/// The Gemini key and model used to live on this component, which meant they
/// shipped inside the APK and could be recovered by unpacking it. The request
/// now goes to the game's own backend, which holds the key. The prompt itself
/// stays here so it remains tunable in the Inspector.
/// </summary>
```

Lines 32-36 · doc comment · before `public IEnumerator AskAdvisor(string question, string contextJson,`

```
/// <param name="onIncomplete">
/// Raised when the reply was cut off before the model finished - almost
/// always the token ceiling. The partial text is discarded rather than
/// shown, since half a piece of farming advice can mislead.
/// </param>
```

Lines 45-53 · comment · before `question`

```
// The typed question goes last and goes bare, with none of our own
// wording wrapped around it.
//
// The server quotes this part and writes the framing itself. Doing
// it there rather than here is the whole point: this file ships
// inside the APK, so anything written here can be rewritten by
// someone who unpacks a build, and a guard that can be edited out
// is not a guard. Adding wording here would also push the real
// question further from the rules the server puts above it.
```

Lines 59-67 · comment · before `AiResponseStyle.Apply()`

```
// Appends the brevity instruction when the player has AI
// Summarization on, and returns the guidance untouched when off.
// The tool reference is appended here rather than typed into
// extraSystemGuidance, because that field is serialized into
// TerrainPreview.unity - a code-only edit to its default would
// silently do nothing, and an Inspector edit is one forgotten
// step away from going stale. Appending keeps it tied to the
// build. It goes after the guidance so the answer-style rules
// stay dominant, and before the brevity directive Apply adds.
```

Lines 83-84 · comment · before `onUnavailable: message => onSuccess?.Invoke(message),`

```
// The adviser being switched off is a normal answer, not an error,
// so it is delivered through the success path and reads in-character.
```

Assets/Scripts/TerrainPreview/AIAdviser/FarmAdvisorToolReference.cs

28 comments · 28 comment lines removed · file had 99 lines

Lines 3-30 · doc comment · before `public static class FarmAdvisorToolReference`

```
/// <summary>
/// The list of things the player can actually do, appended to the adviser's
/// system instruction.
///
/// Without this the adviser answers "what can I use against this typhoon?"
/// from general agricultural knowledge, because nothing in its context says
/// what this game contains. It knows the weather, soil, crops and pest levels;
/// it does not know that a Windbreak Kit exists. So it recommends real
/// practices with no item behind them - and the player hunts the shop for
/// something that was never there.
///
/// Two entries here are actively counter-intuitive and are the reason the
/// effectiveness column matters: irrigation during a typhoon and a drainage
/// canal during a drought are both scored as HARMFUL by
/// <see cref="CropMaintenanceCatalog"/>, and both are exactly what a
/// general-knowledge answer would reach for.
///
/// Every line is read off the game's own tables - ScoreCropAction and
/// ScoreWorldAction for the effectiveness, the Allowed map for the per-crop
/// restrictions, and the mitigation lists on each pest rule for the
/// treatments. If those change, this has to change with them; nothing checks
/// it at build time.
///
/// The framing is deliberately a lookup table rather than a topic. An earlier
/// draft opened with "TOOLS THAT EXIST IN THIS GAME" and read as something to
/// talk about, which risks turning "are my crops okay?" into a shopping list.
/// The opening paragraph now names the question types it must stay out of.
/// </summary>
```

Assets/Scripts/TerrainPreview/AIAdviser/FarmAdvisorUIBuilder.cs

57 comments · 57 comment lines removed · file had 699 lines

Line 10 · doc comment · before `public enum AdvisorMood`

```
/// <summary>Which looping Antonio portrait is playing.</summary>
```

```

Line 13 · doc comment · before Hello,
/// <summary>Greeting – shown each time the chat panel is opened.</summary>

Line 15 · doc comment · before Thinking,
/// <summary>Waiting on the Gemini reply.</summary>

Line 17 · doc comment · before Teaching
/// <summary>Delivering an answer.</summary>

Lines 21–29 · doc comment · before public class FarmAdvisorUIBuilder : MonoBehaviour
/// <summary>
/// The AI-Adviser chat. Hidden behind a round corner button next to the pause
/// button; pressing that button toggles the panel open and closed.
///
/// Art comes from the shared <see cref="UIThemeSprites"/> asset and is entirely
/// optional – with no sprites set, this falls back to the original dark panel.
/// Antonio's three looping portrait videos are Inspector fields on this
/// component (VideoClip cannot live on the shared sprite asset).
/// </summary>

Lines 57–59 · comment · before private const float PanelWidth = 1400f;
// Matches the board art's 2048x1242 aspect. Everything below is laid out
// inside the plank area only – the rolled logs at each side and the
// nailed sign plank across the top are kept clear.

Line 71 · comment · before chatPanel.SetActive(false);
// Starts closed – the round button in the corner opens it.

Line 103 · trailing comment · on the line scaler.MatchWidthOrHeight = 1f; // landscape: scale by height
// landscape: scale by height

Line 107 · comment · before private void CreateToggleButton()
// ----- Open / close -----

Line 109 · doc comment · before private void CreateToggleButton()
/// <summary>The round button beside the pause button that opens the chat.</summary>

Line 113 · comment · before HudRegistry.RegisterIconButton(HudIconButton.SlotAdviserChat, go);
// Not created through HudIconButton.Create, so it registers itself.

Line 124 · comment · before rect.anchoredPosition = new Vector2(art != null ? 20f + size + 12f : 152f, -20f);
// Sits immediately right of the pause button (20 + size + 12 gap).

Line 172 · comment · before SetMood(AdvisorMood.Hello, force: true);
// Every fresh open greets the player again.

Line 187 · comment · before public void SetMood(AdvisorMood mood, bool force = false)
// ----- Portrait -----

Lines 189–192 · doc comment · before public void SetMood(AdvisorMood mood, bool force = false)
/// <summary>
/// Switches Antonio's looping portrait. Called by
/// <see cref="FarmAdvisorChatSystem"/> around each Gemini request.
/// </summary>

Line 244 · comment · before GameObject videoGo = new GameObject("PortraitVideo", typeof(RectTransform), typeof(RawImage));
// The video surface sits inside the frame's painted border.

Line 272 · comment · before portraitPlayer.audioOutputMode = VideoAudioOutputMode.None;
// The portraits are silent animations; keep them out of the audio mix.

Line 277 · comment · before private void BuildUI()
// ----- Panel -----

Line 292 · comment · before panelRect.anchorMin = panelRect.anchorMax = new Vector2(0.5f, 0.5f);
// Centred board, sized to the art's own aspect.

Line 316 · doc comment · before private void BuildThemedPanel(Transform parent)
/// <summary>Layout that matches the painted chat board.</summary>

Line 319 · comment · before Sprite labelArt = theme?.askQuestionsLabel;
// Hanging "Ask Questions!" sign across the top-left of the board.

Line 338 · comment · before BuildPortrait(parent, new Vector2(-355f, 75f), 340f);
// Antonio's portrait on the left of the plank area.

Lines 341–343 · comment · before RectTransform window = BuildChatWindow(parent, new Vector2(175f, 75f), new Vector2(720f, 340f));
// Transcript window starts exactly where the portrait frame ends
// (portrait centre -355 + half of 340 = -185) so the two edges meet
// without overlapping, and runs to the right-hand log.

Line 372 · comment · before rowRect.anchoredPosition = new Vector2(0f, -250f);
// Sits lower down the planks, clear of the transcript window above.

Line 424 · comment · before GameObject viewportGo = new GameObject()
// Viewport

```

Lines 436–437 · comment · before `float pad = windowSprite != null ? 52f : 0f;`
// Must clear the window sprite's 9-slice border, which renders at its
// native pixel size (~45) regardless of how the panel is scaled.

Line 449 · comment · before `GameObject contentGo = new GameObject()`
// Content

Line 478 · comment · before `GameObject textGo = new GameObject()`
// Chat Text

Line 508 · comment · before `scrollRect.viewport = viewportRect;`
// Connect scroll rect

Lines 534–538 · comment · before `questionInput.characterLimit = 400;`
// Comfortably more than any real farming question needs, and short
// enough that nobody can paste a wall of text designed to bury the
// adviser's rules under it. Convenience only, not the guard: this
// limit is inside the APK, so the server enforces its own ceiling
// and is the one that actually holds.

Lines 571–574 · comment · before `placeholderText.verticalOverflow = VerticalWrapMode.Overflow;`
// Same guard as the other typed-into boxes: Unity's default Truncate
// discards a line taller than its rect instead of clipping it, so a
// large text setting would leave the adviser's question box drawing
// nothing while still taking keystrokes.

Line 618 · doc comment · before `private void BuildInputArea(Transform parent)`
/// <summary>Bottom-left input row used by the unthemed fallback layout.</summary>

Line 634 · comment · before `public void AppendMessage(string speaker, string message)`
// ----- Chat plumbing -----

Line 676 · comment · before `private static Text CreateText(Transform parent, string name, int size, TextAnchor alignment)`
// ----- Small helpers -----

Assets/Scripts/TerrainPreview/Audio/FarmAudioController.cs

11 comments · 11 comment lines removed · file had 97 lines

Lines 5–11 · doc comment · before `public class FarmAudioController : MonoBehaviour`
/// <summary>
/// Farm-scene audio driver. Crossfades day/night background music at the
/// day/night boundary and crossfades weather ambience when the weather
/// changes, using <see cref="GameAudioManager"/>. Additive only – it reads
/// the existing clock and weather state and never changes gameplay.
/// Add this to a GameObject in the TerrainPreview scene and assign the clips.
/// </summary>

Line 32 · comment · before `if (WeatherSystem.Instance != null)`
// All Awakes have run by Start, so the weather singleton exists.

Lines 48–50 · comment · before `if (GameAudioManager.HasInstance)`
// This controller is the only thing that turns weather ambience on,
// so it must turn it back off when the farm scene unloads – otherwise
// it keeps looping forever on the persistent GameAudioManager.

Assets/Scripts/TerrainPreview/ClimateEvaluation/ClimateEvaluationModels.cs

2 comments · 2 comment lines removed · file had 69 lines

Line 28 · comment · before `public int quantity;`
// Shared task/evaluation metadata. Old saves deserialize with safe defaults.

Line 35 · comment · before `public string recordOrigin;`
// "Direct" for instrumented interactions, "Observer" for state-derived actions.

Assets/Scripts/TerrainPreview/ClimateEvaluation/ClimateEvaluationPopupBuilder.cs

35 comments · 35 comment lines removed · file had 419 lines

Line 53 · trailing comment · on the line `scaler.matchWidthOrHeight = 1f; // landscape: scale by height`
// landscape: scale by height

Line 81 · comment · before `startPopupText.color = themed ? new Color(0.20f, 0.12f, 0.04f, 1f) : Color.white;`
// Dark ink reads on the light plank; white on the plain fallback.

Lines 89–90 · comment · before `new Vector2(0f, themed ? 55f : 30f),`
// ClimateEventBoard.png's painted edge ends at panel-local y = -170.6;
// at the old 30 the button's base reached -160, right against it.

Lines 113–115 · comment · before `new Vector2(0f, themed ? 60f : 24f),`
// Centred in the bottom plank band: at the old 34 the button's base
// reached -326, past the plank edge at -314 and onto the outline.
// 60 leaves 14px of plank below it and 14px up to the scroll area.

Lines 132–134 · comment · before `rect.anchoredPosition = new Vector2(0f, -44f);`
// ClimateEvaluateBoard.png draws 1:1 at this panel's size, and its top
// plank edge sits at panel-local y = +313. At the old -20 the title's
// glyphs straddled that edge and spilled onto the painted outline.

Lines 155–156 · comment · before `rect.anchoredPosition = new Vector2(0f, -100f);`
// Follows the title down, staying clear of the first plank seam at
// panel-local y = +215 so no dark line cuts through the digits.

Lines 169–178 · comment · before `GameObject scrollView = new GameObject()`
// No Mask here: the Viewport below does the clipping, exactly as every
// other scrolling panel in the game builds it.
//
// A Mask with showMaskGraphic=false draws its graphic only to write the
// stencil buffer, and that draw is alpha-clipped. The themed board wants
// no backdrop of its own, so this Image sits at alpha 0 - every fragment
// is then discarded, the stencil is never written, and every child of the
// mask fails the stencil test. The evaluation text rendered as nothing at
// all while the title and score, being siblings of this object rather
// than children, kept showing.

Lines 191–196 · comment · before `scrollRectTransform.offsetMin = new Vector2(themedBoard ? 158f : 30f, themedBoard ? 140f : 100f);`
// The board's planks stop at panel-local x = -404 and +388 - not a
// symmetric pair, since the painted plank field sits about 8px left of
// the panel's own centre. The old +-130 put the text at -420 and +420,
// i.e. under both rolled logs, worst on the right. These insets leave
// 24px of bare plank on each side once the layout group's 12px padding
// is taken off. The bottom lifts to -220 to give the Okay! button room.

Line 201 · comment · before `scrollBg.color = themedBoard`
// The board already provides the panel look behind the text.

Line 308 · doc comment · before `private void CreateSign(Transform parent, Sprite art, Vector2 size, float offsetY)`
/// <summary>Hanging painted sign above a popup; no-op when no art is set.</summary>

Line 349 · comment · before `bg.sprite = art;`
// The word is painted into the art, so no Text child is added.

Lines 400–403 · comment · before `resultPopup.transform.SetAsLastSibling();`
// The weather/time HUD shares this canvas, so draw order is sibling
// order and whichever builder ran last wins. The HUD was painting over
// the board's top-left corner and the score line; claiming the last
// slot on open puts the popup in front wherever it happens to be built.

Assets/Scripts/TerrainPreview/ClimateEvaluation/ClimateEventTracker.cs

19 comments · 19 comment lines removed · file had 531 lines

Lines 82–87 · doc comment · before `public void RecordObservedAction(ClimateActionRecord record)`

/// <summary>
/// Records an action inferred from state changes. It uses a distinct
/// origin so FarmTaskActionHub can collapse only the observer copy of
/// an already instrumented interaction while preserving legitimate
/// repeated direct actions.
/// </summary>

Lines 110–113 · comment · before `if (previousDuplicate != null &&`
// Enrich the observer copy before task progress sees it. This is
// important when a 500 ml spray empties the pump and the observer can
// only infer a generic item name; the earlier direct record still knows
// the exact liquid that was used.

Lines 120–124 · comment · before `bool deferTaskRouting =`
// A direct sprayer record created by the existing inventory code does
// not contain the affected condition or the before/after severity. The
// state observer supplies that richer record a fraction of a second
// later. Do not send the low-detail copy to task progress, otherwise it
// can either be counted twice or hide the condition-aware observer copy.

Line 187 · trailing comment · on the line `float minimumDay = record.gameDay - 0.0025f; // about 3.6 game minutes`
// about 3.6 game minutes

Lines 210–212 · comment · before `if (IsTreatmentAction(previousAction) &&`
// Existing sprayer instrumentation and the state observer may use
// different action names/items for the same treatment. Pair them
// when one copy is low-detail and both target the same crop.

Assets/Scripts/TerrainPreview/ClimateEvaluation/ClimateResilienceGeminiClient.cs

13 comments · 13 comment lines removed · file had 61 lines

Lines 8–15 · doc comment · before `public class ClimateResilienceGeminiClient : MonoBehaviour`

/// <summary>
/// Grades how well the player handled a typhoon or extreme drought.
///
/// The Gemini key used to sit on this component and therefore shipped inside
/// the APK. The call now goes through the game's backend, which holds the key
/// and can switch the adviser off centrally. The evaluation prompt stays here

```
/// so it remains tunable in the Inspector.  
/// </summary>
```

Lines 40–41 · comment · before `List<string> parts = new List<string>`
// Guidance and payload are kept in a single block, exactly as this
// client composed them before the call moved server-side.

Lines 44–46 · comment · before `AiResponseStyle.Apply(extraSystemGuidance) +`
// Brevity is applied to the guidance rather than to the whole
// block, so it stays ahead of the payload and is never read as
// part of the JSON being evaluated.

Assets/Scripts/TerrainPreview/ClimateMaintenance/ClimateActionAutoObserver.cs

5 comments · 5 comment lines removed · file had 293 lines

Lines 8–12 · doc comment · before `public class ClimateActionAutoObserver : MonoBehaviour`
/// <summary>
/// Observes existing game systems that do not directly call ClimateEventTracker.
/// This lets the evaluation retain planting, crop removal, digging, inventory,
/// pest-trap, fruit-bag, and drainage actions performed during an active event.
/// </summary>

Assets/Scripts/TerrainPreview/ClimateMaintenance/ClimateMaintenanceInteractionSystem.cs

19 comments · 19 comment lines removed · file had 843 lines

Lines 63–67 · comment · before `if (hit.collider.GetComponentInParent<SeedlingTentInstance>() != null)`
// Prepared ground and the Seedling Tent belong to the farming system:
// a Mulch Bag laid on a raised bed, a transplant, or any tap on the
// tent. Handing them over here stops "must be used directly on a crop"
// from swallowing them. A structure kit aimed at prepared ground still
// builds on the terrain behind it, as it always did.

Lines 141–142 · comment · before `if (!IsWithinReach(crop.Transform.position))`
// Measured to the crop's root rather than the tap point, so tapping high
// on a tall plant still counts as standing beside it.

Lines 259–261 · comment · before `if (!IsWithinReach(position))`
// Structures are built on the ground the player tapped, so reach is
// measured to that point. Checked after the ownership test so "you do not
// own it" still takes priority, matching the farming system's ordering.

Lines 423–424 · comment · before `if (placementOffset.x != 0f || placementOffset.z != 0f)`
// Slide the model sideways in its OWN space, so the nudge stays on the
// same side of the object however the placement rotated it.

Lines 574–575 · comment · before `sampledPoint = nearestAnyHit.point;`
// Allows placement when a grass or surface collider
// is covering the generated TerrainCollider.

Lines 820–824 · doc comment · before `private bool IsWithinReach(Vector3 position)`
/// <summary>
/// Defers to the farming system so both interaction paths agree on how far the
/// player can reach, and there is only one distance to tune. Falls open when
/// the farming system is absent rather than blocking the player.
/// </summary>

Assets/Scripts/TerrainPreview/ClimateMaintenance/ClimateMaintenanceToastUI.cs

10 comments · 10 comment lines removed · file had 85 lines

Lines 29–30 · comment · before `rect.anchoredPosition = new Vector2(0f, 195f);`
// The hotbar occupies y 25..175 from the screen bottom, so at the old 125
// this toast opened underneath it. 195 clears its top edge by 20.

Lines 50–52 · comment · before `textRect.offsetMin = new Vector2(plank != null ? 80f : 18f, 10f);`
// Inset past the plank's rope loops so the message stays on the wood.
// MaintenanceToast.png is 1240x170 drawn at 820x110, which puts the ropes
// at UI x 50–63 and 765–777; at the old 60 the text began on the left one.

Lines 60–64 · comment · before `label.color = Color.white;`
// Dark ink reads on light wood; white on the plain fallback.
// White on both backgrounds. The plank art is mid-brown and the dark
// brown this used to use on it sank into the wood grain, which is a
// poor result for the label that carries every failure reason the
// maintenance system reports.

Assets/Scripts/TerrainPreview/ClimateMaintenance/ClimateMitigationWorldObject.cs

17 comments · 17 comment lines removed · file had 632 lines

Lines 24–28 · doc comment · before `public static ClimateMitigationWorldObject LastInspected { get; set; }`
/// <summary>
/// The structure whose readout was last shown, so the developer tools can
/// act on "the one you just clicked" rather than needing a picker of their
/// own. Mirrors how the crop editor follows the inspected crop.
/// </summary>

```
Line 31 · doc comment · before public void DevFillResource()  
/// <summary>Developer tools only: top the tank up without waiting for rain.</summary>
```

```
Lines 51-55 · doc comment · before private static readonly List<ClimateMitigationWorldObject> Active =  
/// <summary>  
/// Every structure currently standing, kept by the structures themselves.  
/// Crops ask "is there shade over me?" every frame, and searching the scene  
/// for that on every crop every frame would be far too slow on a phone.  
/// </summary>
```

```
Lines 77-80 · doc comment · before public static bool IsShadeOver(Vector3 position)  
/// <summary>  
/// Whether a Shade Net or a Greenhouse covers this point. Young cacao  
/// needs one of them overhead.  
/// </summary>
```

```
Lines 596-597 · comment · before string resource =  
// Each part on its own line. They used to be joined with nothing in  
// between, so the board read "GreenhouseRadius: 10.0 m".
```

Assets/Scripts/TerrainPreview/ClimateMaintenance/CropClimateMaintenanceState.cs

49 comments · 49 comment lines removed · file had 511 lines

```
Lines 18-26 · comment · before private const float MulchScaleFull = 1f;  
// One mulch prefab reused at three sizes. The pile reads correctly on a  
// full-grown crop but buries a sprout completely, swallowing the tap that  
// opens the crop information panel.  
//  
// These are MULTIPLIERS on the prefab's own scale, not absolute values.  
// The prefab root sits at 1 with the mesh scaled up inside it, so writing  
// an absolute number here would be multiplied by that mesh scale and  
// produce an enormous pile. Full size therefore means "exactly what the  
// prefab already was".
```

```
Lines 162-170 · doc comment · before public void AdoptPreparedGround(bool raisedBed, string bedPatchId, bool mulched, Terrain terrain)  
/// <summary>  
/// Takes over ground the player prepared before planting.  
///  
/// A raised bed built with the shovel is the same bed the Raised Bed Kit  
/// makes - the same terrain lift under the same patch id - so the crop  
/// records it exactly as the kit would: the drainage bonus, and a flag that  
/// stops a kit being installed on top of it. A bed mulched before planting  
/// counts as the crop's mulch.  
/// </summary>
```

```
Lines 184-186 · comment · before if (terrain != null && terrain.terrainData != null)  
// Measured the way ApplyRaisedBedAndSnapCrop measures it - the root's  
// height above the ground at the bed's peak - so a reload, which lifts  
// the bed again on bare terrain, puts the crop back at this height.
```

```
Lines 230-231 · comment · before GameObject visual = Instantiate(prefab, transform, false);  
// Spawn using the crop as the parent, but do not reuse the  
// prefab root's saved scene position.
```

```
Lines 275-280 · doc comment · before public static float GetMulchScaleMultiplier(FarmCropType cropType, PlantVisualStage stage)  
/// <summary>  
/// Mulch is a single prefab reused at five sizes rather than five separate  
/// prefabs, so the pile can follow the crop through its growth stages  
/// without ever being destroyed and respawned. Each size is chosen so the  
/// crop stays visible and tappable above the pile.  
/// </summary>
```

```
Lines 283-284 · comment · before if (stage == PlantVisualStage.Planted)  
// The planting material itself - a seedling, a runner, a sucker - is  
// smaller than any sprout model, so it gets the smallest pile of all.
```

```
Lines 288-290 · comment · before if (cropType == FarmCropType.Pineapple)  
// Pineapple reuses one model for its sprout and second stage, and that  
// model sits low enough that even the sprout size hides it. It stays  
// small through both and only reaches full size on the adult prefab.
```

```
Lines 298-300 · comment · before if (cropType == FarmCropType.Squash ||  
// Squash, strawberry and tomato stay low and spreading at every stage, so  
// the full-size pile buries them even once grown. Their sprouts are  
// barely taller than the pile itself and need smaller still.
```

```
Lines 325-326 · comment · before return growthVisuals != null  
// A crop with no stage controller never swaps prefabs either, so the  
// full-size pile remains correct for it.
```

```
Lines 342-343 · comment · before visual.transform.localPosition = Vector3.zero;  
// Re-centre before scaling so repeated calls cannot drift the pile off  
// the crop. The caller re-seats it on the terrain afterwards.
```

```
Lines 350-353 · doc comment · before private void RefreshMulchScale()  
/// <summary>  
/// Keeps an installed mulch pile matched to the crop stage, so a sprout
```

```
/// growing into its second-stage prefab grows its mulch with it.
/// </summary>
```

Lines 439–442 · comment · before `Vector3 position = TerrainModificationService.SnapToHeightmapVertexC`

```
// Move the crop onto the vertex that will become the mound's peak before
// raising anything. The heightmap can only lift vertices, so the peak
// cannot land between them - without this the crop sits on the slope of
// its own bed rather than on top.
```

Assets/Scripts/TerrainPreview/ClimateMaintenance/CropRuntimeAdapter.cs

5 comments · 5 comment lines removed · file had 145 lines

Lines 42–45 · doc comment · before `public string PlantingMaterial`

```
/// <summary>
/// The planting material the crop grew from, by item name. A crop planted
/// before materials were recorded reports what its old seed is called now.
/// </summary>
```

Line 60 · doc comment · before `public float NurseryDays => coconut != null ? coconut.nurseryDays : banana != null ? banana.nurseryDays : trop`
`/// <summary>Game days the crop spent in the Seedling Tent; 0 if it was planted directly.</summary>`

Assets/Scripts/TerrainPreview/ClimateMaintenance/TerrainModificationService.cs

34 comments · 34 comment lines removed · file had 180 lines

Lines 9–16 · comment · before `private static readonly ConditionalWeakTable<Terrain, HashSet<string>> AppliedPatchesByTerrain =`

```
// Keyed by the Terrain object reference itself, not GetInstanceID().
// Unity can recycle instance IDs within a session once the old Terrain
// GameObject is destroyed (e.g. on scene reload), which previously made
// this guard falsely think a patch was already applied to a brand-new
// terrain and silently skip re-carving it - the bug behind raised beds /
// drainage canals reverting after a farm save+reload. A ConditionalWeakTable
// tracks patches per actual Terrain instance and cleans itself up once a
// terrain is destroyed/GC'd, so there is no risk of ID collisions or leaks.
```

Lines 26–36 · doc comment · before `public static Vector3 SnapToHeightmapVertex(Terrain terrain, Vector3 worldPosition)`

```
/// <summary>
/// Returns the world X/Z of the heightmap vertex nearest <paramref name="worldPosition"/>,
/// leaving Y untouched.
///
/// A terrain can only raise its vertices, so a mound's peak always lands on
/// one - it cannot sit between them. On this terrain the vertices are about
/// 2.7 m apart (700 m over 256 steps) while a raised bed is only 3.2 m across,
/// so a crop left at an arbitrary position ends up on the side of its own bed.
/// Snapping the crop to the vertex that becomes the peak puts it on top; the
/// crop moves by at most half a vertex spacing, which is not noticeable.
/// </summary>
```

Lines 67–72 · doc comment · before `public static bool RemoveRaisedBed(Terrain terrain, Vector3 center, float radiusMeters, float heightMeters,`
`st`

```
/// <summary>
/// Lowers a raised bed that was lifted earlier with the same centre, size
/// and patch id, returning the ground to how it was. Used when the player
/// fills in a bed nothing was planted in. Does nothing for a patch that is
/// not currently applied, so it can never dig a pit.
/// </summary>
```

Lines 100–108 · comment · before `float metresPerSampleX = data.size.x / (resolution - 1);`

```
// Measure the falloff from the crop's true position, not from the rounded
// sample index.
//
// centerX/centerZ are whole heightmap samples, and on this terrain one
// sample spans about 2.7 m (700 m over 256 steps). Rounding the centre to
// the nearest sample could therefore shift the peak up to ~1.4 m away from
// the crop - close to half the bed's 3.2 m radius, which is why the crop
// ended up on the side of the mound instead of on top of it. Using the
// real world distance puts the peak exactly on the crop.
```

Assets/Scripts/TerrainPreview/CropInstances/BananaPlantInstance.cs

63 comments · 63 comment lines removed · file had 636 lines

Line 69 · doc comment · before `private const float TyphoonStressPerDay = 13f;`

```
/// <summary>Banana's own storm and drought ratings, on the tropical catalogue's 0-14 scale.</summary>
```

Lines 70–80 · comment · before `private const float TyphoonStressPerDay = 13f;`

```
// Banana is the most typhoon-prone crop on the farm, and 13 puts it above
// every catalogued crop - past mangosteen and strawberry at 12. That is
// not an arbitrary nudge: blow-down is the defining typhoon loss in
// Philippine banana growing, because the plant carries huge leaves on a
// water-filled pseudostem over shallow roots, so it catches the wind like
// a sail and snaps rather than bending.
//
// It used to be 6 - the same as coconut, which survives the same storm.
// A game about climate adaptation in Davao cannot tell a player those two
// choices carry equal risk, because banana is exactly the crop a Davao
// farmer stands to lose.
```

Lines 83–87 · doc comment · before `private const float DroughtStressPerDay = 9f;`

```
/// <summary>
/// Shallow roots and a heavy, constant water demand, so a dry spell bites
/// sooner than it does on the tree crops – but not as hard as on cacao or
/// mangosteen, which sit at 13 and 14.
/// </summary>
```

Line 90 · doc comment · before `private const float IdealMoistureMin = 0.60f;`

```
/// <summary>Bounds of the moisture band <see cref="GetWaterScore"/> treats as ideal.</summary>
```

Line 94 · doc comment · before `private const float WeatherStressFullScalePerDay = 40f;`

```
/// <summary>See TropicalCropPlantInstance for the reasoning behind both constants.</summary>
```

Lines 98–100 · comment · before `private float simulatedStress;`

```
// Baselines this simulation owns; the gap between these and the public
// fields is what the pest system, mitigation structures and player actions
// contributed since the previous tick.
```

Line 119 · trailing comment · on the line `nextProductionGameDay = plantedGameDay + 96f; // first fruiting around 96 days`

```
// first fruiting around 96 days
```

Lines 143–148 · doc comment · before `public void ApplyStartingAge(float ageDays)`

```
/// <summary>
/// Makes a freshly planted crop the age it already reached before the
/// field – a plantlet's days hardening in the Seedling Tent. The first
/// harvest moves by the same amount, so the schedule stays measured from
/// the plant's true age.
/// </summary>
```

Lines 181–182 · comment · before `if (!simulationBaselineReady)`

```
// Adopt whatever the fields currently hold on the first tick, covering
// a fresh planting, a restored farm, and the dev tools.
```

Line 194 · trailing comment · on the line `dryRate *= 1.20f; // banana dries faster`

```
// banana dries faster
```

Lines 236–239 · comment · before `float weatherStressPerDay = 0f;`

```
// Weather stress per game day. Not scaled by deltaGameDays – it feeds
// the stress target below, which the smoothing converges on. Adding it
// after the smoothing, as this used to, let the pull back to target
// erase it every tick.
```

Lines 253–258 · comment · before `directHealthPenalty += Mathf.Lerp(2f, 8f, TyphoonStressPerDay / 12f);`

```
// The same curve coconut and the eleven catalogued crops use.
// Banana used to take a flat -8 applied further down instead,
// which happened to equal what this produces at its new
// constant – but it was a separate number that ignored the
// constant entirely, so retuning the stress above would have
// silently left the health damage behind.
```

Lines 269–271 · comment · before `bool waterlogged =`

```
// Waterlogging against banana's own moisture ceiling. This used to come
// from a shared fixed 0.85 threshold inside the weather system, which
// fired while banana was still inside the band it treats as ideal.
```

Lines 312–313 · comment · before `simulatedHealth = Mathf.Clamp(`

```
// Smooth this simulation's own baselines toward their targets, fade
// what other systems contributed, then publish the sum.
```

Lines 334–335 · comment · before `stage = GetStage(GetAgeYears());`

```
// Re-read the stage after simulating, matching coconut and the tropical
// crops. Without this the transition was never reported for banana.
```

Lines 565–577 · doc comment · before `public void DevSetStressBaseline(float value)`

```
/// <summary>
/// Developer tools only: force this crop's stress to a value and move its
/// own baseline with it.
///
/// Writing <c>stress</c> alone would not hold. Each simulation step
/// recovers whatever another system wrote as
/// <c>externalStress = stress - simulatedStress</c> and then fades it with a
/// two-day time constant, so a directly assigned number decays back toward
/// whatever the soil dictates – which is right for a mulch bag and wrong
/// for a deliberate test setup. Moving the baseline too leaves no external
/// offset to fade, so the value stays until the simulation itself drifts
/// it.
/// </summary>
```

Line 624 · comment · before `fieldPlantedGameDay = -1f;`

```
// A forced stage has to show that stage, not the planting material.
```

Assets/Scripts/TerrainPreview/CropInstances/CoconutTreeInstance.cs

55 comments · 55 comment lines removed · file had 617 lines

Lines 63–70 · doc comment · before `private const float TyphoonStressPerDay = 4f;`

```
/// <summary>
/// Coconut's own storm and drought ratings, on the same 0–14 scale the
/// tropical crop catalogue uses. This simulation previously had no typhoon
```

```

/// or drought branch at all, which left coconut the one crop of the
/// thirteen a storm could not touch. The values sit at the resistant end
/// deliberately - a deep-rooted palm rides out weather that flattens
/// strawberry - but resistant is not immune.
/// </summary>

```

Lines 71-79 · comment · before `private const float TyphoonStressPerDay = 4f;`

```

// Coconut is the most storm-hardy crop on the farm, and these numbers say
// so: 4 is below every one of the eleven catalogued crops, the lowest of
// which is pineapple at 5. A palm bends with the wind and sheds it
// through its fronds instead of catching it, which is why coconut stands
// through storms that flatten everything around it - and it is the
// difference the game should be teaching about planting near the coast.
//
// It used to be 6, identical to banana, which put the most typhoon-proof
// crop and the most typhoon-prone one on the same footing.

```

Line 82 · doc comment · before `private const float DroughtStressPerDay = 5f;`

```

/// <summary>Deep roots reach water long after shallow-rooted crops wilt.</summary>

```

Line 85 · doc comment · before `private const float IdealMoistureMax = 0.80f;`

```

/// <summary>Upper end of the moisture band <see cref="GetWaterScore"/> treats as ideal.</summary>

```

Line 88 · doc comment · before `private const float WeatherStressFullScalePerDay = 40f;`

```

/// <summary>See TropicalCropPlantInstance for the reasoning behind both constants.</summary>

```

Lines 92-94 · comment · before `private float simulatedStress;`

```

// Baselines this simulation owns; the gap between these and the public
// fields is what the pest system, mitigation structures and player actions
// contributed since the previous tick.

```

Lines 140-145 · doc comment · before `public void ApplyStartingAge(float ageDays)`

```

/// <summary>
/// Makes a freshly planted crop the age it already reached before the
/// field - time in the Seedling Tent, or the head start of a bought
/// seedling. The first harvest moves by the same amount, so the schedule
/// stays measured from the plant's true age.
/// </summary>

```

Lines 179-180 · comment · before `if (!simulationBaselineReady)`

```

// Adopt whatever the fields currently hold on the first tick, covering
// a fresh planting, a restored farm, and the dev tools.

```

Lines 212-215 · comment · before `float weatherStressPerDay = 0f;`

```

// Weather stress per game day. Not scaled by deltaGameDays - it feeds
// the stress target below, which the smoothing converges on. Adding it
// after the smoothing, as this used to, let the pull back to target
// erase it every tick.

```

Lines 229-230 · comment · before `directHealthPenalty += Mathf.Lerp(2f, 8f, TyphoonStressPerDay / 12f);`

```

// Matches the curve the tropical crops use, so coconut sits on
// the same scale as the other twelve rather than outside it.

```

Lines 241-242 · comment · before `bool waterlogged =`

```

// Waterlogging against coconut's own moisture ceiling. This used to
// come from a shared fixed 0.85 threshold inside the weather system.

```

Lines 303-304 · comment · before `simulatedHealth = Mathf.Clamp(`

```

// Smooth this simulation's own baselines toward their targets, fade
// what other systems contributed, then publish the sum.

```

Lines 548-560 · doc comment · before `public void DevSetStressBaseline(float value)`

```

/// <summary>
/// Developer tools only: force this crop's stress to a value and move its
/// own baseline with it.
///
/// Writing <c>stress</c> alone would not hold. Each simulation step
/// recovers whatever another system wrote as
/// <c>externalStress = stress - simulatedStress</c> and then fades it with a
/// two-day time constant, so a directly assigned number decays back toward
/// whatever the soil dictates - which is right for a mulch bag and wrong
/// for a deliberate test setup. Moving the baseline too leaves no external
/// offset to fade, so the value stays until the simulation itself drifts
/// it.
/// </summary>

```

Line 607 · comment · before `fieldPlantedGameDay = -1f;`

```

// A forced stage has to show that stage, not the planting material.

```

Assets/Scripts/TerrainPreview/CropInstances/TropicalCropPlantInstance.cs

74 comments · 74 comment lines removed · file had 699 lines

Lines 62-67 · doc comment · before `private const float WeatherStressFullScalePerDay = 40f;`

```

/// <summary>
/// Converts "stress per game day" from the weather tables into a share of
/// the 0-1 stress target. At 40, the most typhoon-sensitive crop settles
/// near 48% weather stress and the most resistant near 30%, so the per-crop

```

```

/// difference is plainly visible without pinning every plant at 100.
/// </summary>

Lines 70-75 · doc comment · before private const float ExternalEffectTimeConstantDays = 2f;
/// <summary>
/// How long, in game days, an effect applied from outside this simulation
/// keeps its grip before fading. A steady 3.5/day pest settles around 7
/// points of health loss; a cured pest or a passed storm releases over
/// roughly the same span instead of marking the plant permanently.
/// </summary>

Lines 78-84 · doc comment · before private const float YoungCacaoSunStressPerDay = 6f;
/// <summary>
/// Extra stress per game day on young cacao standing in open sun, on the
/// same scale as the weather tables. The user's notes call shade critical
/// for young cacao, and the Shade Net Kit is the game's shade. At 6 it
/// settles about fifteen points of stress – plainly visible on the crop
/// board, not enough to kill a watered plant.
/// </summary>

Lines 87-92 · comment · before private float simulatedStress;
// The stress and health this simulation last produced on its own. health
// and stress are shared fields – the pest system, mitigation structures
// and player actions all write to them between ticks – so the gap between
// these baselines and the public fields is exactly what those other
// systems contributed. Tracking it separately is what lets the smoothing
// below converge on its target without erasing their work.

Lines 138-143 · doc comment · before public void ApplyStartingAge(float ageDays)
/// <summary>
/// Makes a freshly planted crop the age it already reached before the
/// field – days in the Seedling Tent, a grafted seedling's head start, or
/// a bought seedling's age. The first harvest moves by the same amount,
/// so the schedule stays measured from the plant's true age.
/// </summary>

Line 155 · doc comment · before public bool NeedsShade =>
/// <summary>Young cacao – before it starts to fruit – wants shade overhead.</summary>

Lines 189-191 · comment · before if (!simulationBaselineReady)
// Adopt whatever the fields currently hold the first time this runs.
// Covers a fresh planting, a farm restored from the backend, and any
// dev tool that sets the stats directly.

Lines 199-202 · comment · before float externalStress = stress - simulatedStress;
// Anything the public fields have drifted from the baselines was
// written by another system since the previous tick – pest damage,
// mitigation relief, watering, a harvest. Measure it now so the
// smoothing further down can preserve it instead of erasing it.

Lines 226-230 · comment · before moisture = Mathf.Clamp(
// Moisture is integrated before the weather stress is measured, so the
// waterlogging and rain-relief tests below read this tick's moisture.
// The coconut and banana simulations already worked this way; reading
// the pre-drying value here made the same storm judge identical soil
// differently depending on which crop was standing in it.

Lines 265-271 · comment · before float weatherStressPerDay = 0f;
// Stress per game day from the weather, deliberately NOT scaled by
// deltaGameDays: it feeds the stress target below and the smoothing
// converges on that target over time. Scaling it here and adding it
// after the smoothing – which is what this used to do – meant the pull
// back to target erased the whole contribution on the very next tick,
// so a typhoon moved a durian's stress by 0.0003 out of 100 instead of
// the 18/day its tables call for.

Lines 294-297 · comment · before bool waterlogged =
// Waterlogging is judged against this crop's own moisture ceiling
// rather than a shared constant, and now applies once instead of
// stacking with a second fixed-threshold check inside the weather
// system.

Lines 339-343 · comment · before simulatedHealth = Mathf.Clamp(
// Smooth this simulation's own baselines toward their targets, then
// let the contributions other systems made fade on their own clock,
// and publish the sum. Pest damage of 3.5/day used to be wiped out
// every tick by a 22/day pull toward target; it now holds the plant
// down for as long as the infestation lasts.

Lines 580-592 · doc comment · before public void DevSetStressBaseline(float value)
/// <summary>
/// Developer tools only: force this crop's stress to a value and move its
/// own baseline with it.
///
/// Writing <c>stress</c> alone would not hold. Each simulation step
/// recovers whatever another system wrote as
/// <c>externalStress = stress - simulatedStress</c> and then fades it with a
/// two-day time constant, so a directly assigned number decays back toward
/// whatever the soil dictates – which is right for a mulch bag and wrong
/// for a deliberate test setup. Moving the baseline too leaves no external

```

```

/// offset to fade, so the value stays until the simulation itself drifts
/// it.
/// </summary>

```

Line 669 · doc comment · before `fieldPlantedGameDay = -1f;`

```

// A forced stage has to show that stage, not the planting material.

```

Assets/Scripts/TerrainPreview/CropInstances/TropicalCropTypes.cs

11 comments · 11 comment lines removed · file had 665 lines

Lines 79–83 · doc comment · before `public static InventoryItemType GetSeedItem(TropicalCropKind crop)`

```

/// <summary>
/// The crop's usual planting material. Pineapple, mango and strawberry no
/// longer come from seed items – see PlantingMaterialCatalog for every
/// material a crop can be planted from.
/// </summary>

```

Lines 657–662 · comment · before `}`

```

// GetPestDamageMultiplier used to live here: a per-crop, per-pest damage
// table that nothing ever called. Per-crop pest tuning is owned by the
// PestDiseaseRule assets in DavaoPestDiseaseDatabase, which cover all
// thirteen crops and are what the simulation actually reads. Keeping a
// second, unused table invited the wrong answer to "where is corn's
// armyworm weakness applied?" – it is in the database, not in code.

```

Assets/Scripts/TerrainPreview/DevTools/DevCropVitals.cs

25 comments · 25 comment lines removed · file had 150 lines

Lines 5–27 · doc comment · before `public static class DevCropVitals`

```

/// <summary>
/// Reads and writes one crop's live readings, for the developer tools only.
///
/// Three unrelated classes grow crops – coconut, banana and everything else –
/// and none of them share a base type, so every caller that wants to touch a
/// crop generically has to branch across all three. This puts that branch in
/// one place.
///
/// Only five of the ten readings are editable, and the split is not arbitrary:
///
/// Drainage, Fertility and Soil Suitability are sampled from the soil once
/// in Initialize() and never resampled, so writing them holds for good.
///
/// Water and Stress are simulated every step, so writing them sets a
/// starting point rather than a permanent value – the simulation will drift
/// both back toward what the soil and weather dictate. Stress goes through
/// DevSetStressBaseline so that it at least does not spring straight back
/// from the external-effect channel.
///
/// Health, Avg Health, Tree Stage and Crop Name are left alone. Health is
/// computed from the other five, so forcing it would be overwritten within a
/// step or two and would only mislead.
/// </summary>

```

Line 33 · doc comment · before `public float stress;`

```

/// <summary>0–100.</summary>

```

Line 35 · doc comment · before `public float water;`

```

/// <summary>0–1, shown to the player as a percentage.</summary>

```

Assets/Scripts/TerrainPreview/DevTools/DevToolsUIBuilder.cs

252 comments · 252 comment lines removed · file had 2085 lines

Lines 16–19 · doc comment · before `private const int DevPanelSortingOrder = 30000;`

```

/// <summary>
/// Above every panel the game builds, below the on-screen typing bar,
/// which sits at 32000 and has to stay above everything.
/// </summary>

```

Lines 22–26 · doc comment · before `private static bool DesktopToolsAllowed => !Application.isMobilePlatform;`

```

/// <summary>
/// On a desktop or in the editor the panel simply exists, toggled with
/// F1. There is no account check because there is no account worth
/// checking: anyone running the editor already has the project.
/// </summary>

```

Lines 29–42 · doc comment · before `private bool mobileToolsGranted;`

```

/// <summary>
/// Set only once the server has confirmed both that this account is on
/// the admin list and that the deployment currently allows the tools on
/// a phone. Never decided here: the list and the switch both live on the
/// server, so a phone has to ask, and an ordinary player is told no.
///
/// Worth being honest about the limit of this. The panel's local actions
/// are simulated on the device, so a rebuilt APK could perform them
/// whatever the server says. What the gate buys is that the ordinary
/// build has no button at all. The actions that reach other people –

```

```

/// deleting an account, sending an event to another player - are checked
/// against the admin list on the server every time and do not depend on
/// this at all.
/// </summary>

Line 45 · doc comment · before private bool MobileLayout => !DesktopToolsAllowed;
/// <summary>True when the panel is being drawn for a touch screen.</summary>

Line 76 · comment · before private Text cropEditTitle;
// ---- crop info edit ----

Lines 85-90 · doc comment · before private readonly List<GameObject> cropEditBlock = new List<GameObject>();
/// <summary>
/// Every object the crop-editor section owns, so the whole block can be
/// hidden when no crop is selected. Collected as a range of the content's
/// children rather than tracked one by one, which keeps the flow-layout
/// helpers unchanged.
/// </summary>

Lines 103-106 · doc comment · before private float durationBlockHeight;
/// <summary>
/// How much vertical room the duration label and field occupy, so hiding
/// them can pull everything below back up by exactly that much.
/// </summary>

Lines 110-115 · doc comment · before private readonly System.Collections.Generic.List<(string type, string payload, string label)>
/// <summary>
/// Every option the picker offers: what to send, and how to name it on
/// screen. The wording is carried alongside rather than parsed back out
/// of the label, so a row can be reworded for readability without
/// changing what pressing it triggers.
/// </summary>

Line 124 · doc comment · before private string pendingDeleteEmail;
/// <summary>The address the open confirmation is about.</summary>

Line 127 · doc comment · before private bool deleteAccountBusy;
/// <summary>One admin request at a time; deleting is not something to fire twice.</summary>

Line 138 · comment · before if (HasWeatherPanelArt())
// These must also appear on Android.

Lines 158-160 · comment · before StartCoroutine(RequestMobileTools());
// A phone builds nothing yet. The panel appears only if the
// server says this account may have it, which takes a round trip
// and cannot be answered here.

Lines 170-171 · comment · before if (WeatherSystem.Instance != null)
// Weather changes do not tick the clock, so the panel needs its own
// signal to swap plates the moment a storm starts or clears.

Lines 189-196 · comment · before if (panelRoot != null &&
// The crop editor follows whatever crop is on the info panel, and the
// two can be opened in either order. Opening the dev panel refreshes
// it, but tapping a crop while the panel is already open cannot - the
// toggle never runs. Comparing the bound crop each frame covers both
// orders, and costs one reference comparison.
//
// Deliberately above the desktop guard: on a phone Update returns
// early, and the section has to work there too.

Lines 204-206 · comment · before if (!DesktopToolsAllowed)
// F1 belongs to the desktop build. A phone toggles with the on-screen
// button instead, so a keyboard plugged into a tablet still cannot
// open anything the server did not grant.

Lines 213-215 · comment · before ToggleDevPanel();
// Through the shared toggle rather than SetActive, so opening with
// the key does everything opening with the button does - it used
// to skip the crop-editor refresh entirely.

Line 220 · comment · before private IEnumerator RequestMobileTools()
// ----- tools on a phone

Lines 222-229 · doc comment · before private IEnumerator RequestMobileTools()
/// <summary>
/// Asks the server whether this account may open the tools here, and
/// builds them if so.
///
/// The wait exists because a saved login is restored asynchronously: on a
/// cold launch this runs before AuthSession has a token, and asking then
/// would get an unauthenticated refusal and never retry.
/// </summary>

Lines 248-249 · comment · before _ => { });
// Silent on purpose. Every ordinary player runs this, and a
// failed check should leave the game exactly as it was.

```

Lines 266-269 · doc comment · before private void BuildMobileToolsButton()

```
/// <summary>
/// The plain button that opens the panel on a phone, tucked under the
/// map on the right edge where the farm HUD leaves a gap.
/// </summary>
```

Lines 281-285 · comment · before const float moneyPlankTop = 14f;

```
// Below the small map, derived rather than eyeballed. The map hangs
// from the same top-right corner at -(14 + moneyPlankSize.y + 12),
// which is -176 with the stock 150-tall plank, and it is 300 tall -
// so its bottom edge is at -476. A first attempt at -430 put this
// button behind that last 46 units of it.
```

Lines 321-325 · doc comment · before private void BuildDevPanelCloseButton(RectTransform parent)

```
/// <summary>
/// The X in the panel's own corner. A phone has no F1, and although the
/// DEV button toggles, a panel that covers a third of the screen wants a
/// close where the eye already is.
/// </summary>
```

Lines 370-371 · comment · before if (MobileLayout && !mobileToolsGranted)

```
// Second guard, after the one that decided to build at all. Nothing
// else calls this on a phone, but the panel is worth two checks.
```

Lines 381-382 · comment · before RefreshCropEditSection();

```
// Picks up whatever crop was tapped since the panel was last open,
// and hides the block again if none was.
```

Line 414 · trailing comment · on the line scaler.MatchWidthOrHeight = 1f; // landscape: scale by height

```
// landscape: scale by height
```

Lines 429-436 · doc comment · before private void BuildWeatherPanel()

```
/// <summary>
/// The wooden weather panel: one Image whose sprite is swapped per weather
/// state, with the clock, date and temperature drawn on the plank.
///
/// All five sprites must share one canvas size with the plank in the same
/// place - the text positions below are fixed, so a sprite whose plank sits
/// elsewhere would make the text drift when the weather changes.
/// </summary>
```

Line 445 · comment · before RectTransform rect = go.GetComponent<RectTransform>();

```
// Top-left, tucked under the pause and AI-Adviser buttons.
```

Lines 458-459 · comment · before float w = panelSize.x;

```
// Offsets are expressed as fractions of the panel so tweaking
// weatherPanelSize keeps the text on the planks.
```

Lines 468-469 · comment · before clockRect.anchoredPosition = new Vector2(w * 0.11f, -h * 0.275f);

```
// Shifted left from 0.16: the clock sat right of its plank's centre
// because the sun disc on the left was not accounted for.
```

Lines 473-475 · comment · before weatherText.color = Color.white;

```
// White, unlike the clock above it. The clock sits on the pale top
// plank where dark brown reads well, but the date and temperature sit
// on the shaded lower plank, where the same brown all but disappeared.
```

Lines 481-482 · comment · before infoRect.anchoredPosition = new Vector2(w * 0.16f, -h * 0.70f);

```
// Dropped from 0.66 so the date and temperature sit low on their plank
// rather than riding its upper edge.
```

Lines 504-508 · doc comment · before private void RefreshWeatherPanelSprite()

```
/// <summary>
/// Clear weather follows the clock (sunny by day, night after dark). Rain,
/// typhoon and drought hold their own panel regardless of the hour, and only
/// hand back to sunny/night once the weather returns to clear.
/// </summary>
```

Lines 534-535 · comment · before if (chosen == null)

```
// Fall back to the sunny plate rather than blanking the HUD if a slot
// has not been filled in yet.
```

Lines 609-618 · comment · before Canvas overlay = panelRoot.AddComponent<Canvas>();

```
// Draw above every other panel on the shared canvas.
//
// Without this the panel is ordered by when it was created, and the
// farm HUD's map is built afterwards and lands on top of it - which
// is exactly what it did, hiding half the seed buttons behind the
// map. Its own sorting layer settles that no matter what is built
// later, so a new panel added next month cannot bury it again.
//
// Below MobileTextEntryOverlay's 32000: the on-screen typing bar has
// to stay above everything, this one included.
```

Lines 623-624 · comment · before panelRoot.AddComponent<GraphicRaycaster>();

```
// A nested canvas needs its own raycaster or nothing inside it can be
// clicked; the parent's does not reach through.
```

```

Lines 629-633 · comment · before panelRoot.transform.localScale = Vector3.one * 1.25f;
// Scaled as a whole rather than re-laid-out. Every row, field and
// button grows together, so a panel designed for a mouse becomes
// comfortable under a thumb without a second set of sizes to keep
// in step. 1.25 is the most that keeps a 760-tall board inside a
// 1080 canvas.

Lines 710-711 · comment · before foreach (InventoryItemType material in PlantingMaterialCatalog.AllMaterials)
// One button per planting material, from the catalog, so a material
// added later gets its button without touching this list.

Lines 801-803 · comment · before float beforeDuration = nextY;
// Height of the label plus the field plus their spacing, so hiding
// the pair can close the gap they leave behind rather than leaving a
// hole in the middle of the panel.

Lines 811-814 · comment · before CreateButton(contentRect, "Force Here (my farm)", OnForceEventHerePressed, 340f);
// Same picker, applied to this farm instead of someone else's. The
// four hard-coded buttons above only reach four of the forty-odd
// conditions in the database; this reaches all of them without a
// second list to keep in step.

Lines 821-823 · comment · before CreateLabel(contentRect, "Time to pass", 16, 26f);
// One field feeds both buttons: the number means days or hours
// depending on which is pressed, so there is no second box to keep
// in step and no way to send a value typed into the wrong one.

Line 845 · comment · before SetCropEditVisible(false);
// Nothing is selected when the panel is first built.

Line 849 · comment · before private void BuildTankSection()
// ----- water tank

Lines 851-859 · doc comment · before private void BuildTankSection()
/// <summary>
/// Fills the water tank the player last clicked.
///
/// A tank gathers 45 a day in rain and 85 in a typhoon against a capacity
/// of 500, so filling one honestly costs six game days of storm. That is
/// fine to watch once, when the collection itself is what is being tested,
/// and pure waiting every other time - the irrigation tests only need a
/// tank that already has water in it.
/// </summary>

Line 900 · comment · before private void BuildCropEditSection()
// ----- crop info edit

Lines 902-909 · doc comment · before private void BuildCropEditSection()
/// <summary>
/// Edits the readings of the crop currently shown on the info panel.
///
/// Built last on purpose. The panel lays its contents out in a single
/// downward flow, so a block anywhere else could only be hidden by
/// shifting everything below it; at the end, hiding it is just a shorter
/// content rect.
/// </summary>

Lines 964-969 · doc comment · before private void RefreshCropEditSection()
/// <summary>
/// Names the selected crop and fills the boxes with its current numbers,
/// so one value can be changed without retyping the rest - and so a pair
/// of plants can be matched by reading one and typing those figures into
/// the other.
/// </summary>

Lines 1011-1013 · comment · before DevCropVitals.Vitals next = new DevCropVitals.Vitals
// A box left empty or holding something unparseable keeps the crop's
// present value, so a slip of the keyboard cannot silently zero a
// reading the developer never meant to touch.

Lines 1043-1044 · comment · before SoilAwareTerrainGenerator terrain =
// Redraw the info panel so the board shows the new numbers rather than
// the ones it was opened with.

Line 1059 · comment · before private void CreateEventSelector(Transform parent, float width, float height)
// ----- send to another player

Lines 1061-1069 · doc comment · before private void CreateEventSelector(Transform parent, float width, float height)
/// <summary>
/// Fills the option list and shows the current pick on a button.
///
/// Not a Unity Dropdown. That control opens its list as a child of
/// itself, and this panel lives inside a masked ScrollRect, so the list
/// was clipped away by the mask with half an entry escaping over the
/// label above it. The picker below opens on the canvas root instead,
/// clear of the scroll view entirely.
/// </summary>

```

```

Lines 1085-1086 · comment · before foreach (PestDiseaseType pest in
// Read from the enum, so a condition added later shows up here
// without this list being touched.

Lines 1129-1131 · comment · before SetDurationVisible(
// Duration only means anything to a weather event. The server drops
// it for a pest command, so leaving the box on screen would invite a
// developer to set a number that quietly does nothing.

Lines 1138-1146 · doc comment · before private void SetDurationVisible(bool visible)
/// <summary>
/// Shows or hides the duration pair, closing the gap behind it.
///
/// This panel is laid out once with fixed positions rather than by a
/// layout group, so hiding a row does not reflow anything on its own -
/// everything below has to be moved by hand, and the scroll content
/// resized to match, or the panel is left with a hole and a stretch of
/// dead scroll at the bottom.
/// </summary>

Line 1159 · comment · before float shift = visible ? -durationBlockHeight : durationBlockHeight;
// Rows sit at negative y, so moving up means adding.

Lines 1174-1177 · doc comment · before private void ShowEventPicker()
/// <summary>
/// The option list, on the canvas root above the dev panel so no mask or
/// sibling order can hide it. Built once and reused.
/// </summary>

Line 1203 · comment · before Canvas overlay = eventPicker.GetComponent<Canvas>();
// Above the dev panel own 30000, below the typing bar 32000.

Lines 1239-1240 · comment · before int index = i;
// Captured per row. One shared variable would leave every button
// selecting whatever the loop finished on.

Lines 1319-1325 · doc comment · before private void OnForceEventHerePressed()
/// <summary>
/// Forces the picked pest on this farm.
///
/// Weather is deliberately refused: the farm already has Force Rain,
/// Typhoon and Drought buttons of its own, and those take the duration
/// field, which this path has no way to pass.
/// </summary>

Lines 1416-1424 · doc comment · before private void SendTimeSkip(string commandType, string unit)
/// <summary>
/// Sends a clock skip to the target player.
///
/// The server holds the real ceilings and refuses anything past them;
/// this only catches an empty or zero box so the obvious mistake does
/// not cost a round trip. Skipping time is not undoable on the receiving
/// farm - crops age, weather rerolls, objectives regenerate - so neither
/// side quietly rounds a number down.
/// </summary>

Line 1441 · doc comment · before private void SendCommand(AdminCommandRequestDto request)
/// <summary>Fills in the target and posts, with the shared guards.</summary>

Line 1487 · comment · before private void OnDeleteAccountPressed()
// ----- delete an account

Lines 1489-1492 · doc comment · before private void OnDeleteAccountPressed()
/// <summary>
/// Looks the address up first, so the confirmation can name a real
/// account rather than asking the developer to trust their own typing.
/// </summary>

Lines 1572-1576 · doc comment · before private void ShowDeletePrompt(AdminAccountLookupDto account)
/// <summary>
/// A plain dark box over the screen. Deliberately unstyled - no player
/// ever sees this, and dressing it in wooden planks would only make it
/// look like something they are meant to.
/// </summary>

Line 1693 · doc comment · before private Text CreateStatusLabel(Transform parent, float height)
/// <summary>A wrapping line for results, unlike CreateLabel's fixed row.</summary>

Lines 1722-1723 · comment · before if (weatherPanelImage != null)
// Themed panel: clock on the top plank, date and temperature below,
// and the plate itself swapped to match the weather.

Line 1780 · doc comment · before private void OnSeedlingsReadyPressed()
/// <summary>Every sown bag in the Seedling Tent skips to ready to transplant.</summary>

Lines 1823-1826 · doc comment · before private void OnReplayTutorialPressed()
/// <summary>
/// Runs the beginner guide again on this farm. Testing it otherwise means

```

```
/// creating a whole new account every time, since it only ever runs once.
/// </summary>
```

Line 1840 · doc comment · before `private void OnSkipTutorialPressed()`

```
/// <summary>Ends the guide immediately and hands back the full HUD.</summary>
```

Lines 1858-1859 · comment · before `Debug.LogWarning()`

```
// Saying "shown" when there is nothing to show sends you hunting
// for a rendering bug that does not exist.
```

Lines 1952-1955 · doc comment · before `private Text CreateLabel(Transform parent, string textValue, int fontSize, float height)`

```
/// <summary>
/// Returns the label so a caller can hide or reword it later. Most
/// callers ignore it.
/// </summary>
```

Assets/Scripts/TerrainPreview/DevTools/FpsCounterHud.cs

36 comments · 36 comment lines removed · file had 252 lines

Lines 7-30 · doc comment · before `public class FpsCounterHud : MonoBehaviour, IPointerClickHandler`

```
/// <summary>
/// Frame rate readout for the device performance tests, in a small box to the
/// left of the money plank. A developer tool, so it is drawn plainly in the same
/// dark style as the Dev Tools panel rather than with the game's wooden art.
///
/// One live number cannot answer what those tests ask. The requirement (NFR-01)
/// is an average of at least 30 frames per second over a continuous ten-minute
/// session, so the box shows the live rate, the average since measuring began,
/// the lowest one-second rate in that time - the stutters a player actually
/// notices - and how long it has been measuring. Tapping the box starts a new
/// measurement, so a run can begin from the same moment on every phone.
///
/// Measuring waits until the farm has finished loading, because loading-screen
/// frames say nothing about gameplay, and a phone coming back from the
/// background does not count the time it was away as one enormous frame. The
/// game caps itself at 60 frames per second (MobilePerformanceBootstrap), so no
/// phone reads higher than that.
///
/// The same figures are written to the device log once a minute, so a run can
/// also be read back with adb logcat rather than only from screenshots.
///
/// Switched on and off with Show Fps Counter on the Inventory UI Builder in the
/// TerrainPreview scene. It is a testing aid; switch it off for release builds.
/// </summary>
```

Line 37 · doc comment · before `private const float TargetFps = 30f;`

```
/// <summary>NFR-01's floor. The live rate turns red below it.</summary>
```

Lines 69-70 · comment · before `rect.anchoredPosition = moneyPlank != null`

```
// Level with the amount printed on the money plank, which sits a little
// below that plank's middle, and clear of the coins on its left end.
```

Line 76 · comment · before `Image image = go.GetComponent<Image>();`

```
// The Dev Tools panel's background.
```

Line 80 · comment · before `image.raycastTarget = true;`

```
// The box is the button that restarts the measurement.
```

Line 98 · comment · before `if (skipNextFrame)`

```
// The first frame back from the background spans the whole time away.
```

Lines 193-196 · doc comment · before `private static bool WorldReady()`

```
/// <summary>
/// Whether gameplay has started. No persistence manager means a scene opened
/// straight in the Editor, which has no farm load to wait for.
/// </summary>
```

Lines 244-245 · comment · before `text.horizontalOverflow = HorizontalWrapMode.Overflow;`

```
// One line each, kept on screen at any text size instead of being dropped
// when the Text Size setting grows the font past its row.
```

Assets/Scripts/TerrainPreview/DevTools/UiResponseTimerHud.cs

53 comments · 53 comment lines removed · file had 364 lines

Lines 11-39 · doc comment · before `[DefaultExecutionOrder(-32000)]`

```
/// <summary>
/// Times how long each farm screen takes to appear after the tap that opens it,
/// for the device performance tests (NFR-02: visible feedback within 2 seconds).
/// A developer tool, drawn plainly like the Dev Tools panel, under the FPS counter.
///
/// The clock starts at the frame in which the tap is processed and stops at the
/// end of the frame in which the screen is first drawn, so it includes whatever
/// work opening the screen does - building rows, laying out text - as well as
/// rendering it. It measures the screen appearing, not data arriving from the
/// server afterwards; the marketplace, farmer list and AI Adviser still fill in
/// once their requests return.
///
```

```

/// Nothing in the builders is changed. The timer watches each screen's own
/// open state - the panels registered with HudRegistry, the map's expanded
/// state, and the pause and settings roots - and credits a screen that opens to
/// the most recent tap. One tap is credited to one screen, and a screen that
/// opens more than 5 seconds after the last tap (a notification, say) is not
/// counted.
///
/// Planting, watering and harvesting are timed the same way, from the tap to
/// the end of the frame the result is drawn in. They are credited when the
/// farming system reports them, which is inside the tap's own frame, so the
/// crop information panel an action refreshes is not also timed as opened.
///
/// Each result is also written to the device log as a [UI] line. Tap the box to
/// list every screen timed so far with its last, average and slowest time.
/// Switched on and off with Show Ui Response Timer on the Inventory UI Builder in
/// the TerrainPreview scene.
/// </summary>

```

Line 46 · doc comment · before `private const double LimitMs = 2000.0;`

```

/// <summary>NFR-02's limit, in milliseconds.</summary>

```

Lines 88-91 · doc comment · before `public static UiResponseTimerHud Create(Transform canvas, RectTransform fpsCounter, RectTransform moneyPlank)`

```

/// <summary>
/// Builds the box under <paramref name="fpsCounter"/>, or where the FPS counter
/// would be when that is switched off.
/// </summary>

```

Lines 118-119 · comment · before `Image image = go.GetComponent<Image>();`

```

// The Dev Tools panel's background. The box is the button that shows
// and hides the full list.

```

Line 181 · comment · before `FarmingInteractionSystem.FarmActionApplied -= OnFarmActionApplied;`

```

// The event is static, so a destroyed box must not stay subscribed.

```

Lines 185-189 · doc comment · before `private void OnFarmActionApplied(string action)`

```

/// <summary>
/// Called inside the tap's frame, after this box's Update and before its
/// LateUpdate, so the tap is taken here - before the crop information panel
/// the action refreshed could be credited with it.
/// </summary>

```

Lines 201-202 · comment · before `frameStart = Time.realtimeSinceStartupAsDouble;`

```

// Runs before the EventSystem (see the execution order above), so this is
// the moment the frame that handles the tap began.

```

Line 227 · comment · before `lastTapTime = double.NaN;`

```

// One tap opens one screen; anything opening later is not its doing.

```

Lines 241-242 · comment · before `lastTapTime = double.NaN;`

```

// Tapping this box opens no screen, so the tap is not left waiting to be
// credited to something that opens a few seconds later.

```

Lines 309-314 · doc comment · before `private void ResolveRoots()`

```

/// <summary>
/// The pause and settings screens are not in HudRegistry and the map is built
/// by its own bootstrap, so they are looked up by name until they exist. Their
/// open state is taken as it is when found, so a screen found already open is
/// not mistaken for one that just opened.
/// </summary>

```

Assets/Scripts/TerrainPreview/Economy/PesoPrice.cs

40 comments · 40 comment lines removed · file had 94 lines

Lines 6-25 · doc comment · before `public static class PesoPrice`

```

/// <summary>
/// Prices in centavos, wallets in whole pesos.
///
/// The shop and the shipping bin use the Davao City government price list,
/// which has centavos in it - corn seed is P388.89, a coconut sells for
/// P16.89. The wallet, the save file, the marketplace and the server all
/// count whole pesos, and moving all of that to centavos would have meant
/// rewriting every saved farm. So prices are held exactly, in centavos, and
/// only the total of a purchase or a sale becomes pesos: rounded to the
/// nearest peso, with half a peso rounding up.
///
/// The server repeats this rule (PesoRounding in the backend) when it charges
/// a marketplace listing fee, and the two must round identically or the fee a
/// player is shown would differ from the fee they pay. That is why rounding
/// is done in whole-number arithmetic rather than with Math.Round, which in
/// C# rounds halves to the nearest even number while Java rounds them up.
///
/// Deliberately free of UnityEngine, so the rule can be compiled and checked
/// on its own against the server's copy.
/// </summary>

```

Line 28 · doc comment · before `private const float MaxInspectorPesos = 21474836f;`

```

/// <summary>The largest price that still fits in centavos.</summary>

```

Lines 31–34 · doc comment · before `public static int Centavos(decimal pesos)`

```
/// <summary>
/// A price written in pesos, as centavos. Pass a decimal literal (388.89m)
/// so nothing is lost on the way.
/// </summary>
```

Lines 43–48 · doc comment · before `public static int CentavosFromInspector(float pesos)`

```
/// <summary>
/// A price typed into the Inspector, as centavos. Inspector fields are
/// floats, and 16.89 cannot be held exactly in a float, so the value goes
/// through decimal – which keeps the seven significant digits a float
/// really has – before it is rounded to a whole centavo.
/// </summary>
```

Lines 60–64 · doc comment · before `public static int TotalPesos(int centavosEach, int count)`

```
/// <summary>
/// What <paramref name="count"/> items at <paramref name="centavosEach"/>
/// come to in whole pesos: the total is rounded to the nearest peso, halves
/// rounding up. The price of each one is never rounded on its own.
/// </summary>
```

Lines 75–78 · doc comment · before `public static string Label(int centavos)`

```
/// <summary>
/// A price for display: "P388.89", or "P15" when there are no centavos, so
/// whole-peso prices read the way they always have.
/// </summary>
```

Assets/Scripts/TerrainPreview/Economy/ShippingBinSpawner.cs

12 comments · 17 comment lines removed · file had 196 lines

Lines 19–21 · doc comment · before `public GameObject SpawnedBin => spawnedBin;`

```
/// <summary>
/// Allows FarmPersistenceManager to capture the shipping-bin transform.
/// </summary>
```

Lines 26–31 · block comment · before `if (!FarmLoadContext.IsRestoring)`

```
/*
 * For a new farm, create the bin normally near the player.
 *
 * For a returning farm, do not create it here. The saved
 * position will be restored by FarmPersistenceManager.
 */
```

Lines 38–41 · doc comment · before `public void SpawnBinNearPlayerNow()`

```
/// <summary>
/// Used when creating a new farm or loading an older save that
/// does not yet contain shipping-bin information.
/// </summary>
```

Lines 48–50 · doc comment · before `public GameObject RestoreBin()`

```
/// <summary>
/// Recreates the shipping bin using its exact saved transform.
/// </summary>
```

Line 79 · comment · before `if (spawnedBin.GetComponent<ShippingBinSeller>() == null)`

```
// Ensure the save system can identify the spawned shipping bin.
```

Assets/Scripts/TerrainPreview/Economy/ShopItemDescriptions.cs

32 comments · 32 comment lines removed · file had 510 lines

Lines 5–22 · doc comment · before `public static class ShopItemDescriptions`

```
/// <summary>
/// The shop's item descriptions.
///
/// Every fact here is taken from the simulation's own tables rather than
/// written from general knowledge, so a player who reads a description and
/// then plays sees the same numbers:
///
/// growth timing and yield – TropicalCropCatalog, CoconutTreeInstance,
///                           BananaPlantInstance
/// pests per crop          – DavaoPestDiseaseDatabase.asset
/// treatments per pest     – the mitigation list on each rule
/// allowed care actions    – CropMaintenanceCatalog
/// temperature comfort     – CropClimateRules
///
/// If any of those change, the matching lines here have to change with them.
/// Nothing reads this at runtime except the shop panel, so a stale line is
/// silent – it will not fail a build or throw.
/// </summary>
```

Lines 34–37 · doc comment · before `private static string DistrictAvailability(InventoryItemType item)`

```
/// <summary>
/// Where a seed can be had, built from DistrictCropPools rather than written
/// out per crop, so it cannot drift from what the shop actually allows.
/// </summary>
```

Lines 56–57 · comment · before `if (PlantingMaterialCatalog.TryGet(PlantingMaterialCatalog.UpgradeLegacy(item),`
`// Every planting material – and the old seed items, as the material`
`// that replaced them – reads its crop's text plus its own route.`

Line 66 · comment · before `case InventoryItemType.SprayerPump:`
`// ----- Pest and disease tools -----`

Line 175 · comment · before `case InventoryItemType.MulchBag:`
`// ----- Climate care, one crop at a time -----`

Line 225 · comment · before `case InventoryItemType.IrrigationSystemKit:`
`// ----- Farm-wide structures -----`

Lines 280–283 · doc comment · before `private static string MaterialDescription(PlantingMaterialInfo material)`
`/// <summary>`
`/// A planting material: its crop, then how this particular material gets`
`/// into the field, then the real-world timing the game compresses.`
`/// </summary>`

Line 291 · doc comment · before `private static string CropText(FarmCropType crop)`
`/// <summary>What the crop is like once it is growing, shared by all its materials.</summary>`

Assets/Scripts/TerrainPreview/Economy/ShopUIBuilder.cs

154 comments · 154 comment lines removed · file had 1047 lines

Lines 15–22 · comment · before `private static readonly Dictionary<InventoryItemType, int> seedPriceCentavos = new Dictionary<InventoryItemTyp`
`// Static so the daily-objective system can price an item without owning`
`// a shop panel. It is the only price list in the game, and it stays the`
`// only one: a task that asks the player to buy something has to work out`
`// the cost from the same numbers the shop will actually charge.`
`//`
`// Held in centavos. The seeds use the Davao City government prices, and`
`// corn has centavos in it; PesoPrice explains how a price in centavos`
`// becomes a charge in whole pesos.`

Lines 25–30 · comment · before `{ InventoryItemType.CacaoSeed, PesoPrice.Centavos(25m) },`
`// Planting materials. The seeds keep the Davao City government prices.`
`// The five materials that replaced the old seed items keep the price`
`// the old seed had (pineapple sucker = the old pineapple seed, and so`
`// on), so a player's money buys what it bought before. The banana`
`// plantlet, the grafted mango seedling and the ready squash seedling`
`// are new, and priced above the material they skip ahead of.`

Line 47 · comment · before `{ InventoryItemType.AphidTrap, PesoPrice.Centavos(35m) },`
`// Equipment / pest tools`

Line 59 · comment · before `{ InventoryItemType.MulchBag, PesoPrice.Centavos(25m) },`
`// Climate mitigation and crop maintenance`

Line 103 · trailing comment · on the line `scaler.matchWidthOrHeight = 1f; // landscape: scale by height`
`// landscape: scale by height`

Line 110 · comment · before `HudIconButton.Create(`
`// Part of the shared round-icon row along the HUD's top-left edge.`

Line 133 · comment · before `panelRect.anchorMin = panelRect.anchorMax = new Vector2(0.5f, 0.5f);`
`// Centred board layout: shelves on the left, preview on the right.`

Lines 168–170 · comment · before `signRect.sizeDelta = new Vector2(440f, 150f);`
`// ShopLabel.png is 1040x300 (3.47:1), so preserveAspect keeps it`
`// undistorted and this width sets the sign's overall size. Narrowed`
`// from 520 so the sign stops dominating the top of the board.`

Lines 172–173 · comment · before `signRect.anchoredPosition = new Vector2(0f, -16f);`
`// Lowered from +6 so the sign overlaps the board's top edge and`
`// reads as mounted on it rather than floating above it.`

Lines 192–197 · comment · before `statusText = CreateLabel(shopPanel.transform, "", 20,`
`// Starts blank rather than carrying a standing "Select an item to`
`// buy." prompt. The row still exists so buy results and errors`
`// ("Not enough money", "Inventory is full") have somewhere to show.`
`// Back on the plank at the position it held before the Check`
`// Description button existed; the shelf and that button were lifted`
`// to make room rather than pushing this line off the board.`

Line 215 · comment · before `private const int ShelfColumns = 4;`
`// ----- Themed shelf layout -----`

Line 231 · comment · before `private DescriptionBoard descriptionBoard;`
`// ----- Item description board -----`

Line 239 · doc comment · before `private const float ShelfCentreX = 160f + ShelfShiftX + 250f - PanelHalfWidth;`
`/// <summary>Centre of the 4x4 shelf, in the panel's own coordinates.</summary>`

Line 244 · doc comment · before `private static readonly InventoryItemType[] ShopStock =`
`/// <summary>Every item the shop sells, in the order they appear on the shelf.</summary>`

```

Line 247 · comment · before InventoryItemType.CacaoSeed, InventoryItemType.DurianSeed,
// Planting materials, grouped by crop in PlantingMaterialCatalog order.

Lines 298-302 · comment · before shelfGrid.sizeDelta = new Vector2(430f, 398f);
// Rows are spaced by sizeDelta.y / 4, so this has to match the painted
// compartments in ShopShelve.png. At the old 380 the spacing came out
// ~4px short per row, and the error accumulated downward until the
// bottom row straddled the shelf plank instead of sitting in the cell.
// 398 with a 3px drop matches the art across all four rows.

Line 306 · comment · before prevPageButton = CreateIconButton(theme?.shopPrevPageButton, "<<",
// Paging arrows sit either side of the shelf; hidden when everything fits.

Lines 317-321 · comment · before private const float PanelWidth = 1300f;
// ---- Board layout tuning ----
// ShopBoard.png is natively 1120x700; the extra width here stretches it.
// The shelf stays a fixed distance from the left edge and the preview a
// fixed distance from the right, so widening the board opens up the gap
// between them rather than moving either one.

Lines 326-331 · doc comment · before private const float ContentLift = 50f;
/// <summary>
/// Raises every control inside the board by this many pixels. The contents
/// were laid out bottom-heavy, leaving a band of bare wood along the top;
/// lifting them all by one shared value keeps their spacing intact while
/// centring the group. Increase to lift further, decrease to drop back.
/// </summary>

Lines 334-339 · doc comment · before private const float ShelfShiftX = 80f;
/// <summary>
/// Shifts the shelf and its two paging arrows right as one group. At the
/// shelf's original X the prev arrow sat partly on the board's left rolled
/// log; this moves the group onto the flat plank area while preserving the
/// arrows' alignment against the shelf edges.
/// </summary>

Lines 342-343 · comment · before private const float PreviewWidth = 300f;
// The preview frame and the Amount / Buy controls under it are all derived
// from these so the column stays aligned if the frame is moved or resized.

Line 349 · doc comment · before private const float PreviewCentreX =
/// <summary>Centre of the preview frame, relative to the panel's centre.</summary>

Lines 368-371 · comment · before frameImage.preserveAspect = false;
// ShopItemFrame.png carries no 9-slice border, so preserveAspect
// would lock it to its native 0.87:1 and ignore the wider box.
// Turning it off is what actually widens the frame; give the sprite
// a border in the Sprite Editor if the stretch ever reads too far.

Line 380 · comment · before previewNameText = CreateLabel(frame.transform, "", 19, Vector2.zero, 270f, 44f);
// Name plank across the top of the frame art.

Lines 386-387 · comment · before previewNameText.resizeTextForBestFit = true;
// Long names such as "Water Storage Tank Kit" overran the plank at a
// fixed size; best-fit shrinks only those that need it.

Line 405 · comment · before previewPriceText = CreateLabel(frame.transform, "", 20, Vector2.zero, 230f, 44f);
// Price plank across the bottom.

Lines 410-411 · comment · before priceRect.anchoredPosition = new Vector2(0f, 46f);
// Raised off the frame's bottom edge so the price sits on the plank art
// rather than below it.

Lines 415-417 · comment · before CreateLabel(shopPanel.transform, "Amount:", 22,
// Amount and Buy form one column directly beneath the preview frame.
// These labels anchor to the panel's top-centre, so X is the offset from
// centre and Y is the drop from the top edge.

Lines 423-431 · comment · before amountInput.contentType = InputField.ContentType.IntegerNumber;
// Digits only, and at most two of them.
//
// The field took any text before. Letters were not dangerous -
// ParseAmount falls back to 1 - but the fallback was silent, so a
// player who typed something odd got one seed and no explanation.
// Two characters also makes the 1-99 ceiling visible while typing,
// instead of quietly clamping a 500 down to 99 after the fact. On a
// phone IntegerNumber opens the number pad rather than the full
// keyboard, which is the same reason the sign-up date boxes use it.

Lines 435-436 · comment · before CreateIconButton(theme?.shopBuyButton, "BUY",
// Buy anchors to the panel's top-right, so its X is converted from the
// shared centre line to keep it under the same column.

Lines 444-449 · doc comment · before private static void ApplyPlankTextStyle(Text label)
/// <summary>
/// Styles a label that sits on the preview frame's wooden planks. The
/// previous dark brown was near-invisible against that art, so this uses a
/// light fill with a hard outline, which stays readable over both the plank

```

```

/// and the item icon behind it.
/// </summary>

Line 497 · doc comment · before private void RefreshShelf()
/// <summary>Draws one page of stock onto the 4x4 shelf.</summary>

Line 509 · comment · before bool paged = pageCount > 1;
// Arrows only appear when the stock actually needs more than one page.

Lines 553-555 · comment · before bool offered = DistrictCropPools.IsAvailableToPlayer(item);
// Selected stock is tinted; the shelf art already draws the cell itself.
// Seeds this farm's district does not grow stay on the shelf, greyed out,
// so they can still be picked and read about - they just cannot be bought.

Lines 586-588 · comment · before if (descriptionBoard != null && descriptionBoard.IsOpen)
// Opening is the Check Description button's job. Picking a different
// item while the board is already open refreshes it in place rather
// than making the player close and reopen it.

Lines 592-593 · comment · before RefreshShelf();
// No "Selected ..." line: the preview frame already names the item and
// shows its price, so the status row is left for buy results and errors.

Line 597 · doc comment · before private void BuildCheckDescriptionButton(UIThemeSprites theme)
/// <summary>The button under the shelf that opens the description board.</summary>

Line 622 · doc comment · before private void ShowDescriptionFor(InventoryItemType item)
/// <summary>Fills the shared board for one item and opens it.</summary>

Line 664 · doc comment · before private Sprite ItemIcon(InventoryItemType item)
/// <summary>Reuses the backpack's icon table so the shop shows the same art.</summary>

Lines 723-724 · comment · before float y = -10f;
// The same stock as the themed shelf, so both layouts sell the same
// things. This plain list only appears when the board art is missing.

Lines 778-779 · comment · before int totalCost = PesoPrice.TotalPesos(GetSeedPriceCentavos(seedType), amount);
// Priced in centavos, paid in whole pesos: it is the total that gets
// rounded, never the price of each one.

Lines 809-811 · comment · before int centavos = StockedPriceCentavos(seedType);
// P999 rather than free for anything unlisted. A missing entry must
// never make an item free; StockedPriceCentavos below is the query that
// is allowed to answer "not sold", and only because nothing buys with it.

Lines 816-821 · doc comment · before public static int StockedPriceCentavos(InventoryItemType item)
/// <summary>
/// What the shop charges for one of an item, in centavos, or 0 when it does
/// not stock it at all. Zero means the item cannot be bought at any price,
/// which is a different answer from "it is expensive" and has to stay
/// distinguishable.
/// </summary>

Lines 827-835 · doc comment · before public static int CostFor(InventoryItemType item, int quantity)
/// <summary>
/// What buying <paramref name="quantity"/> of an item costs, in whole pesos,
/// rounded exactly as the shop rounds it when it charges.
///
/// Read by the daily objectives to decide whether a task is affordable
/// before offering it, so a task is never judged affordable by a peso the
/// shop then asks for. Check StockedPriceCentavos first when "not sold"
/// matters - this returns 0 for that too.
/// </summary>

Lines 841-844 · doc comment · before public static bool SellsToPlayer(InventoryItemType item)
/// <summary>
/// Whether the shop will sell the item to this farm: it is stocked, and if it
/// is a seed, the farm's district grows it.
/// </summary>

Line 850 · doc comment · before private static readonly Color UnavailableTint = new Color(0.42f, 0.42f, 0.42f, 0.8f);
/// <summary>A dim grey that still lets the seed's art read, for stock this farm cannot buy.</summary>

Lines 874-877 · comment · before if (open)
// Siblings on a canvas draw in hierarchy order, and the weather/time
// panel is built by a different script, so creation order alone
// decides who covers whom. Re-parenting to last on open keeps the
// shop above the HUD regardless of which script ran first.

Lines 882-884 · comment · before if (open)
// The description board is deliberately not restored with the rest of
// the panel: reopening the shop should show the shelf, not whatever
// was last read. It comes back on the next selection.

Line 888 · comment · before if (open && shelfGrid != null)
// Redrawn on open so the shelf reflects the current page and selection.

```

Lines 892-897 · doc comment · before `private static void FitStatusText(Text text)`

```
/// <summary>
/// Lets the status row shrink a long message to fit instead of cutting it off.
/// On the themed board the row is one line tall, and a result such as a
/// refused seed or a large purchase runs past one line - the second line was
/// simply never drawn.
/// </summary>
```

Lines 973-981 · comment · before `placeholder.verticalOverflow = VerticalWrapMode.Overflow;`

```
// A 48-tall field leaves 36 for its text after padding. Playpen Sans at
// font 22 needs about 32 of that, so raising the text size ran out of
// room - and Unity's default Truncate drops the line rather than
// clipping it, which made the field look like it was refusing input
// when it was actually accepting it and drawing nothing.
//
// Only the vertical mode is touched: InputField drives horizontal
// overflow itself to scroll a single line, so overriding that would
// break long values.
```

Assets/Scripts/TerrainPreview/Farming/CropClimateRules.cs

3 comments · 3 comment lines removed · file had 257 lines

Line 170 · comment · before `if (temperatureC >= profile.idealMin &&`

```
// Temperature is inside the ideal range.
```

Line 177 · comment · before `if (temperatureC < profile.idealMin)`

```
// Temperature is too cold.
```

Line 189 · comment · before `return Mathf.Clamp01(`

```
// Temperature is too hot.
```

Assets/Scripts/TerrainPreview/Farming/CropFieldDescriptions.cs

15 comments · 15 comment lines removed · file had 102 lines

Lines 3-17 · doc comment · before `public static class CropFieldDescriptions`

```
/// <summary>
/// What each line of the crop readout means, shown by the Crop Information
/// Description button on that panel.
///
/// The readout was ten numbers with no explanation anywhere in the game - a
/// player could see "Stress: 31.5" and "Avg Health: 64.3" without being told
/// which of them costs them fruit, or why there are two health figures at all.
///
/// Facts here come from the simulation itself: the yield formulas in
/// TropicalCropCatalog, CoconutTreeInstance and BananaPlantInstance, and the
/// water scoring in each crop class. The ideal moisture band is deliberately
/// described rather than quoted, because it is per crop - coconut is happy
/// between 45 and 80 percent, banana wants 60 to 90, and the tropical crops
/// each carry their own range in TropicalCropCatalog.
/// </summary>
```

Assets/Scripts/TerrainPreview/Farming/CropNaming.cs

15 comments · 15 comment lines removed · file had 98 lines

Lines 8-11 · doc comment · before `public static class CropNaming`

```
/// <summary>
/// Friendly per-crop names like "corn_1", "cacao_2". These are a display/AI-facing
/// alias only - the machine <c>cropId</c> GUID and all matching logic are unchanged.
/// </summary>
```

Lines 14-18 · doc comment · before `public static string NextName(string cropTypeLabel)`

```
/// <summary>
/// Next friendly name for a crop type, e.g. "corn_1" then "corn_2". Numbering is
/// derived by scanning the live crops, so it keeps counting correctly after a
/// save/reload with no persistent counter to reset.
/// </summary>
```

Line 44 · doc comment · before `public static Dictionary<string, string> LiveIdToNameMap()`

```
/// <summary>{ cropId GUID -> friendly name } for every live crop.</summary>
```

Lines 61-65 · doc comment · before `public static string Humanize(string aiText)`

```
/// <summary>
/// Replaces any raw crop GUID inside AI-generated text with the crop's friendly
/// name, so the player never sees the machine id even if the model leaks one.
/// GUIDs are unique 32-char strings, so the replacement can't corrupt other text.
/// </summary>
```

Assets/Scripts/TerrainPreview/Farming/CropProtectionState.cs

8 comments · 8 comment lines removed · file had 165 lines

Lines 5-7 · doc comment · before `public class CropProtectionState : MonoBehaviour`

```
/// <summary>
/// Stores persistent non-spray protection applied to one crop.
/// </summary>
```

Lines 106–110 · doc comment · before `public void RestoreSaveData(bool fruitBag, bool drainageImprovement, GameObject fruitBagPrefab)`

```
/// <summary>
/// Brings back protection from a farm save. Unlike installing, this does
/// not raise drainage: the crop's saved drainage already includes the
/// kit's bonus, so adding it again would count it twice.
/// </summary>
```

Assets/Scripts/TerrainPreview/Farming/DigSpot.cs

19 comments · 19 comment lines removed · file had 149 lines

Lines 6–12 · doc comment · before `public enum PreparedPlotKind`

```
/// <summary>
/// What a patch of prepared ground currently is.
///
/// The shovel tills bare ground first; tapping the tilled patch again turns it
/// into the preparation the crop needs. Values are saved by name, so new ones
/// can go anywhere, but keep None first as the default.
/// </summary>
```

Lines 22–30 · doc comment · before `public class DigSpot : MonoBehaviour`

```
/// <summary>
/// A patch of prepared ground: tilled, then dug into a planting hole, built
/// into a raised bed or opened into a furrow. It carries the soil sampled when
/// it was dug, so the crop planted here gets the soil of this spot.
///
/// Kept under its old name so the DigSpot prefab and every system that already
/// looks for dug spots – the tutorial, the daily tasks, the task observer –
/// keep working.
/// </summary>
```

Line 59 · doc comment · before `[NonSerialized] public GameObject visual;`

```
/// <summary>The model currently shown for this preparation.</summary>
```

Line 62 · doc comment · before `[NonSerialized] public GameObject mulchVisual;`

```
/// <summary>The mulch laid over a raised bed.</summary>
```

Line 65 · doc comment · before `public bool IsPrepared =>`

```
/// <summary>Ready to take a crop: a hole, a bed or a furrow, not just tilled ground.</summary>
```

Assets/Scripts/TerrainPreview/Farming/FarmingInteractionSystem.cs

121 comments · 121 comment lines removed · file had 2370 lines

Lines 6–7 · comment · before `public partial class FarmingInteractionSystem : MonoBehaviour`

```
// Split across files: preparing ground, planting and transplanting live in
// FarmingInteractionSystem.Planting.cs.
```

Lines 86–87 · comment · before `public Vector3 drainageCanalPlacementOffset = new Vector3(0f, -0.25f, 0f);`

```
// Starting point for the canal: sunk slightly so it reads as dug into the
// ground rather than resting on it. Tune X/Z in the Inspector to slide it.
```

Lines 184–188 · comment · before `public float coconutSellPesos = 16.89f;`

```
// Shipping bin prices in pesos, from the Davao City government price list.
// Floats so centavos can be typed in (a coconut is 16.89); PesoPrice rounds
// each sale's total to whole pesos. Renamed from the old whole-peso fields
// on purpose: the old prices saved in the scene are left behind instead of
// quietly overriding these.
```

Line 258 · doc comment · before `private Transform playerReference;`

```
/// <summary>Cached player transform used to measure interaction reach.</summary>
```

Lines 261–267 · doc comment · before `public static event System.Action<string> FarmActionApplied;`

```
/// <summary>
/// Raised once a tap has planted a seed, watered a crop or cut a harvest,
/// with "Planting", "Watering" or "Harvesting". Nothing in the game needs
/// it: the screen timer listens so the device tests can time these actions
/// (NFR-02). It is raised inside the tap's own frame, after the action and
/// its crop information have been applied.
/// </summary>
```

Line 278 · comment · before `Debug.LogException(e);`

```
// A listener is a testing aid and must never undo the action.
```

Lines 303–305 · comment · before `if (TryHandlePlantingTap(hit, selected))`

```
// The Seedling Tent, a transplant in progress, and prepared ground.
// Prepared ground stays where it was dug, so every action on it is
// reach-checked there, or the player could plant from across the farm.
```

Lines 314–315 · comment · before `if (!IsWithinReach(clickedTrap.transform.position))`

```
// Cleaning removes the trap, so it needs reach. Inspecting it
// below does not.
```

Lines 402–404 · comment · before `if (IsWithinReach(clickedTree.transform.position))`

```
// Tending the crop needs reach; reading its status does not. Reach is
// measured to the trunk rather than the tap point, so tapping high in
// a mature canopy still counts as standing next to the tree.
```

```

Lines 527-528 · comment · before if (selected == InventoryItemType.AphidTrap)
// Traps are placed on the ground the player tapped, so reach is measured
// to that point.

```

```

Lines 609-612 · comment · before if (disease == null)
// Said out loud rather than swallowed. A crop only gains its pest
// component once something infects it, so using one of these on a
// healthy plant used to consume the tap and report nothing at all -
// indistinguishable from the item being broken.

```

```

Lines 702-705 · comment · before if (disease == null)
// Said out loud rather than swallowed. A crop only gains its pest
// component once something infects it, so using one of these on a
// healthy plant used to consume the tap and report nothing at all -
// indistinguishable from the item being broken.

```

```

Line 783 · comment · before return false;
// Returning false allows normal harvesting.

```

```

Lines 853-854 · comment · before return false;
// No active condition accepts sanitation.
// Allow normal Machete harvesting instead.

```

```

Lines 1036-1037 · comment · before if (componentWasAdded)
// Default values are only applied when the prefab
// does not already contain the component.

```

```

Lines 1180-1192 · doc comment · before public bool IsWithinReach(Vector3 position)
/// <summary>
/// True when <paramref name="position"/> is close enough to the player to act on.
///
/// Measured from the player rather than the camera so the limit still means
/// "within reach of where I am standing" when the view is detached, as it is
/// under the free camera. Distance is full 3D on purpose: standing on a ridge
/// should not let the player reach the valley floor below.
///
/// Callers pass the position of the thing being acted on - a crop's root, a
/// trap, the patch of ground - rather than the raycast hit point. Tapping the
/// canopy of a mature coconut hits several metres above its base, which would
/// otherwise read as out of reach while the player stands against the trunk.
/// </summary>

```

```

Lines 1198-1211 · comment · before Vector3 player = GetPlayerPosition();
// Measured on the ground plane, deliberately ignoring height.
//
// Several crops are planted with a vertical offset so their art sits
// correctly: squash, strawberry and pineapple are sunk 4.5m, corn,
// eggplant and tomato 2m, while pomelo and mangosteen are raised 2m.
// Those offsets move the transform this distance is measured to, so a
// 3D check spent most of the budget on height the player cannot close -
// a squash root 4.5m below a player standing ~1m above ground is
// already 5.5m away and could never be watered or maintained, however
// close they walked. Crops with no offset worked, which is why it
// failed on some plants and not others.
//
// The player walks on terrain and cannot fly, so horizontal separation
// is the only distance they are able to act on.

```

```

Lines 1219-1223 · doc comment · before private static bool IsCropActionItem(InventoryItemType item)
/// <summary>
/// Items that do something to a crop when tapped on it, as opposed to items
/// that only bring up its status. Used to decide whether an out-of-reach tap
/// deserves a "too far" message or should just show the crop's information.
/// </summary>

```

```

Lines 1233-1237 · doc comment · before public bool ReportOutOfReach()
/// <summary>
/// Reports that the target is out of reach. Returns true so the tap counts as
/// handled: the player gets told why nothing happened instead of the action
/// silently falling through to soil inspection.
/// </summary>

```

```

Lines 1244-1247 · doc comment · before private Vector3 GetPlayerPosition()
/// <summary>
/// The player's world position, falling back to the camera when no player
/// controller is present (the terrain preview scene runs without one).
/// </summary>

```

```

Lines 1277-1278 · comment · before if (soilAwareTerrain != null)
// Goes through the panel so the readout is revealed - it now starts
// hidden and only appears when there is something to report.

```

```

Lines 1388-1391 · comment · before Collider[] nearby = Physics.OverlapSphere(
// Triggers are ignored so the tap-target capsule added by
// EnsureCropTapTarget does not count as part of the plant here. Spacing is
// still measured against the crops' own mesh colliders, exactly as before
// that capsule existed, so planting distances are unchanged.

```

Lines 1866–1867 · comment · before `InventoryItemType savedMaterial =`
// What the crop was planted from decides the model it shows for its
// first days in the field. None for a crop saved before that existed.

Lines 2147–2156 · doc comment · before `private void SnapObjectBottomToTerrain(GameObject go, Terrain terrain, float yOffset,`
/// <summary>
/// Rests a prefab's lowest point on the terrain.
///
/// <paramref name="buryFraction"/> then sinks it by that share of its own
/// measured height - 0 leaves it sitting on the surface, 0.8 buries all but
/// the top fifth. It is expressed as a fraction rather than in metres because
/// these prefabs carry large scales (the termite station is 30x), so the same
/// metre value means something different for each one, whereas a fraction
/// reads the object's real size and works for any of them.
/// </summary>

Lines 2249–2263 · doc comment · before `private void EnsureCropTapTarget(GameObject root)`
/// <summary>
/// Gives a crop a finger-sized tap target that does not shrink with its stage.
///
/// Each stage prefab carries a collider auto-fitted to its mesh, and a seedling
/// mesh is tiny - the tomato sprout's box works out to roughly four centimetres
/// across once its scale is applied. That is far smaller than a fingertip, so
/// young plants were nearly impossible to tap on a phone even though the game
/// registered the hit correctly whenever it happened to land.
///
/// The capsule lives on the crop root rather than on the visual, so it survives
/// the prefab swap at every stage and a plant is equally tappable as a seedling
/// and as a full-grown tree. It is a trigger: its job is to be found by the
/// interaction ray, not to block the player walking - which the tiny mesh
/// colliders never did either, so movement is unchanged.
/// </summary>

Line 2274 · trailing comment · on the line `tapTarget.direction = 1; // Y axis, matching how a plant stands.`
// Y axis, matching how a plant stands.

Lines 2278–2282 · comment · before `Terrain terrain = TemporaryTerrainGenerator.ResolveActiveTerrain();`
// Stand the capsule on the soil, not on the crop root. The root is
// where the planting height puts it - several metres above the ground
// for most crops in the farm scene - and a capsule centred there
// floated over young plants. The growth visual controller keeps it on
// the ground from here on, including after a raised bed lifts it.

Assets/Scripts/TerrainPreview/Farming/FarmingInteractionSystem.Planting.cs

97 comments · 97 comment lines removed · file had 1084 lines

Lines 8–19 · doc comment · before `public partial class FarmingInteractionSystem`
/// <summary>
/// The planting half of the farming system: preparing ground, planting the
/// sixteen planting materials, and transplanting seedlings from the Seedling
/// Tent.
///
/// The route is decided by the planting material, not the crop
/// (<see cref="PlantingMaterialCatalog"/>). Every crop needs the same ground
/// steps whichever way it arrives: the shovel tills bare ground, and tapping
/// the tilled patch again turns it into a planting hole (tree crops, banana,
/// coconut), a raised bed (tomato, eggplant, squash, pineapple, strawberry) or
/// a furrow (corn). Strawberry also needs its bed covered with a Mulch Bag.
/// </summary>

Line 85 · comment · before `private bool TryHandlePlantingTap(RaycastHit hit, InventoryItemType selected)`
// ----- tap routing

Lines 87–90 · doc comment · before `private bool TryHandlePlantingTap(RaycastHit hit, InventoryItemType selected)`
/// <summary>
/// Taps that belong to planting: the Seedling Tent, a transplant in
/// progress, and prepared ground. Returns true when the tap was used.
/// </summary>

Lines 104–106 · comment · before `if (selected == InventoryItemType.Shovel && spot.plotKind == PreparedPlotKind.Tilled)`
// The shovel still turns tilled ground into a hole, bed or furrow
// while a seedling is being carried. Every tap used to go to the
// transplant, so the seedling had to be put back first.

Line 153 · doc comment · before `private bool ReportBareGroundPlanting(InventoryItemType selected)`
/// <summary>A planting material tapped on bare ground: say what to do instead.</summary>

Line 213 · comment · before `private bool TryTillGround(RaycastHit hit)`
// ----- tilling

Line 220 · comment · before `if (terrainCollider == null || hit.collider != terrainCollider)`
// The shovel only works the generated terrain.

Lines 233–234 · comment · before `if (!IsWithinReach(groundPosition))`
// The tap ray runs the length of the map, so without a reach limit the
// shovel works wherever the player happens to be looking.

```

Line 262 · comment · before GetSoilForPlanting(groundPosition, out SoilSample soil, out string district);
// Soil is sampled now, while the ground is being worked.

Line 295 · doc comment · before private float PlayerFacingYaw()
/// <summary>The direction the player is looking, flattened, so a furrow runs away from them.</summary>

Lines 302-305 · doc comment · before private DigSpot CreatePreparedGround(Vector3 groundPosition, float yaw, Vector3 terrainNormal)
/// <summary>
/// The prepared-ground object itself: an invisible tap target carrying the
/// soil record, with the model for its current preparation as a child.
/// </summary>

Lines 315-316 · comment · before BoxCollider target = root.AddComponent<BoxCollider>();
// A trigger, so the player walks over prepared ground instead of into
// it; taps still find it because the tap ray includes triggers.

Line 323 · doc comment · before private void ApplyPreparedGroundVisual(DigSpot spot, Terrain terrain)
/// <summary>Swaps in the model for the ground's current preparation and fits the tap target to it.</summary>

Lines 401-405 · doc comment · before private static void StripPlantingVisual(GameObject visual)
/// <summary>
/// Removes everything on a visual that would make it act like something
/// else: a dug-spot component (which would steal taps from the real one),
/// colliders and physics.
/// </summary>

Line 416 · comment · before private void ShowPrepareChoice(DigSpot spot)
// ----- hole / bed / furrow

Lines 422-423 · comment · before new FarmChoicePopup.Choice("Planting Hole",
// Short enough for two lines under each plank; tapping the finished
// ground lists every crop it takes.

Line 440 · comment · before if (spot == null || spot.plotKind != PreparedPlotKind.Tilled)
// The popup waits for the player, so the ground may be gone or changed.

Lines 467-469 · comment · before Vector3 peak = TerrainModificationService.SnapToHeightmapVertex(terrain, spot.transform.position);
// The heightmap can only lift its own vertices, so the bed's peak is
// the vertex nearest the tilled patch; the patch moves onto it (by
// at most half a vertex spacing) so it sits on top of its own bed.

Line 567 · comment · before private static string CheckGroundFor(DigSpot spot, PlantingMaterialInfo info, string name = null)
// ----- planting

Lines 569-573 · doc comment · before private static string CheckGroundFor(DigSpot spot, PlantingMaterialInfo info, string name = null)
/// <summary>
/// Why this material cannot go into this ground, or null when it can.
/// <paramref name="name"/> is what the player is holding, when that is not
/// the material itself - a seedling carried from the tent.
/// </summary>

Lines 670-675 · doc comment · before private GameObject PlantOnPreparedGround(
/// <summary>
/// Plants into prepared ground and returns the new crop, or null when it
/// could not be planted (a message has already been shown). The material is
/// only taken once the crop exists, and the crop is removed again if taking
/// it fails, so nothing is lost or duplicated either way.
/// </summary>

Line 714 · comment · before spot.Consume();
// The prepared ground becomes the crop's; its model goes with it.

Lines 730-738 · doc comment · before private bool HasOtherCropNearby(GameObject newCrop)
/// <summary>
/// A last spacing check between crop bases, measured on the ground plane.
///
/// Tilling already keeps new ground <see cref="minDistanceBetweenTrees"/>
/// away from every crop and every other patch, and every crop now starts on
/// such a patch. A raised bed then moves its patch onto the nearest
/// heightmap vertex - up to about two metres - so this allows for that
/// shift rather than refusing a bed the player was allowed to build.
/// </summary>

Lines 796-799 · doc comment · before private GameObject CreateCrop(
/// <summary>
/// Builds a crop of the material's kind at the planting position. Returns
/// null (with a warning) when the crop's models are not assigned.
/// </summary>

Lines 893-894 · comment · before return Enum.TryParse(crop.ToString(), out kind) &&
// The two enums share their names for every crop except coconut and
// banana, which have classes of their own.

Lines 900-903 · doc comment · before private PlantGrowthVisualSet CropVisualSet(FarmCropType crop, InventoryItemType material)
/// <summary>
/// A crop's full set of stage models, plus the planting-material model it
/// shows for its first days in the field.
/// </summary>

```

Lines 937-940 · doc comment · before `public GameObject GetPlantedMaterialPrefab(FarmCropType crop, InventoryItemType material)`

```
/// <summary>
/// The model a crop shows for its first days in the field: the seednut,
/// sucker or runner itself, or the seedling as it looked in its bag.
/// </summary>
```

Line 955 · doc comment · before `public GameObject GetSeedlingPrefab(FarmCropType crop)`

```
/// <summary>The seedling model shown in a Seedling Tent bag, per crop.</summary>
```

Lines 974-978 · doc comment · before `private static float GetPlantedBuryFraction(InventoryItemType material)`

```
/// <summary>
/// How much of each planting-material model goes under the soil. Every one
/// of these models includes its roots, which make up the bottom quarter to
/// two-fifths of it, so they are sunk until the roots are hidden.
/// </summary>
```

Lines 983-984 · comment · before `case InventoryItemType.CoconutSeednut: return 0.42f;`

```
// Roots and the lower half of the nut, leaving part of it showing
// as the notes describe.
```

Line 994 · doc comment · before `public const float SeedlingRootShare = 0.38f;`

```
/// <summary>Share of a seedling model's height taken up by its roots and seed.</summary>
```

Line 997 · doc comment · before `private void RestorePlantingDetails(GameObject root, CropSaveDto save)`

```
/// <summary>Restores a crop's planting-material look and protection after a farm load.</summary>
```

Line 1003 · comment · before `public List<PreparedPlotSaveDto> CapturePreparedGround()`

```
// ----- saving and loading
```

Lines 1011-1012 · comment · before `if (spot == null || spot.occupied || spot.plotKind == PreparedPlotKind.None)`

```
// Ground already planted is gone; ground being planted this very
// frame belongs to its crop.
```

Lines 1055-1056 · comment · before `if (terrain != null && terrain.terrainData != null)`

```
// The heightmap is rebuilt bare on every load, so the ground is
// sampled afresh rather than trusting the saved height.
```

Assets/Scripts/TerrainPreview/Farming/GrowthStageVisualController.cs

36 comments · 36 comment lines removed · file had 334 lines

Lines 11-14 · comment · before `Planted`

```
// Appended so saved or serialized stage numbers keep their meaning.
// The planting material itself - a seednut, a sucker, a runner, or the
// seedling that came out of the Seedling Tent - shown for the first few
// days after it goes into the ground.
```

Line 76 · doc comment · before `public const float PlantedVisualDays = 3f;`

```
/// <summary>How long, in game days, the planting material stays on show after planting.</summary>
```

Lines 113-115 · comment · before `if (Time.time >= nextGroundCheckTime)`

```
// The ground can move under a planted seedling - a raised bed
// is built under it, or it is restored before the terrain is
// lifted - so it is re-seated rather than pinned to an offset.
```

Lines 193-197 · doc comment · before `private bool ShowsPlantedMaterial(float fieldPlantedGameDay)`

```
/// <summary>
/// Whether the crop is still in its first days in the field, and has a
/// planting-material model to show for them. Crops planted before this
/// existed carry no field-planting day and go straight to their stages.
/// </summary>
```

Lines 232-235 · comment · before `SeatPlantedVisual();`

```
// The model keeps its collider on purpose: crop spacing is measured
// against the crops' solid colliders (the tap capsule is a trigger
// and ignored), so without one a crop just planted could have
// another planted right on top of it.
```

Lines 243-251 · doc comment · before `private void SeatPlantedVisual()`

```
/// <summary>
/// Rests the planted model on the soil under the crop.
///
/// The stage models are placed with hand-tuned height offsets, because
/// every crop root sits some way above or below the ground (the scene's
/// planting height plus a per-crop offset). The planting-material models
/// have no tuned offset, so they are measured and seated instead, then
/// sunk by the set's bury fraction.
/// </summary>
```

Lines 270-279 · doc comment · before `private void KeepTapTargetOnGround()`

```
/// <summary>
/// Keeps the crop's finger-sized tap target standing on the soil.
///
/// The target is a capsule on the crop root, and the root sits wherever
/// the planting height put it - several metres above the ground for most
/// crops in the farm scene. Centred on the root, the capsule floated over
/// young plants, so a seedling could only be tapped by aiming at thin air
/// above it. It is measured from the ground under the crop instead, and
```

```
/// re-measured if that ground moves.
/// </summary>
```

Assets/Scripts/TerrainPreview/FarmTasks/AIAdvisorTaskGeminiClient.cs

28 comments · 28 comment lines removed · file had 235 lines

Lines 9–16 · doc comment · before `public class AIAdvisorTaskGeminiClient : MonoBehaviour`

```
/// <summary>
/// Asks the adviser to invent a farm task, and later to grade it.
///
/// Both calls used to go to Gemini directly with a key held on this component,
/// which shipped inside the APK. They now go through the game's backend, which
/// holds the key and pins the JSON response format. The prompts stay here so
/// they remain tunable in the Inspector.
/// </summary>
```

Lines 37–39 · comment · before `"Refer to a crop only by its cropName, such as mangosteen_1. Never write its cropId " +`

```
// The grader was quoting both, which put a 32-character hex string in
// the middle of a sentence on a wooden sign. The name is the half the
// player can act on – it is what they see when they click the crop.
```

Lines 43–47 · doc comment · before `private const string PlantingRules =`

```
/// <summary>
/// How planting works now, added to every task request. Kept in code rather
/// than in the Inspector text above, because the Inspector copy in the scene
/// would otherwise keep the old wording.
/// </summary>
```

Lines 88–93 · comment · before `"\n\nField rules. dialogue: two to four short sentences of the adviser " +`

```
// The two fields are described by what they must contain rather
// than by quoting the sentences the board wraps around them. The
// wording used to be quoted, to explain what the game would add –
// and quoting it is what taught the model to write it: the task
// came back already carrying the opening and the reward line, the
// board added its own, and the player read both twice.
```

Lines 103–106 · comment · before `"Name a crop by its cropName exactly as given, such as mangosteen_1, " +`

```
// The friendly name is the point, not a leak: mangosteen_1 is what
// the player reads when they click that crop, so naming it is how
// they know which of several mangosteens the task means. The GUID
// is the thing to keep out of the text.
```

Lines 199–200 · comment · before `onUnavailable: message => onError?.Invoke(message),`

```
// These two features need parseable JSON, so an unavailable adviser
// cannot be shown as a task – it has to fail the request instead.
```

Assets/Scripts/TerrainPreview/FarmTasks/AIAdvisorTaskSystem.cs

89 comments · 89 comment lines removed · file had 1620 lines

Lines 235–239 · comment · before `verdict.reason = PolishVerdictText(verdict.reason);`

```
// The grader writes about a specific crop and, left alone, quotes its
// machine GUID alongside the name – "the 'mangosteen_1' tree (ID:
// 9f435ff9...)". The name is the useful half, since that is what the
// player reads when they click the crop; the GUID is noise that fills
// two lines of a wooden sign.
```

Lines 540–550 · doc comment · before `private static bool CanObtainSeedFor(string cropType, string itemType)`

```
/// <summary>
/// Whether the player can get planting material for a planting objective:
/// they hold some already, a seedling of that crop is waiting in the
/// Seedling Tent, or their district's shop sells it. Without this the
/// adviser could set "plant tomato" on a farm whose district does not grow
/// tomato, and the task would sit there with no way to finish it. An
/// objective that names neither a crop nor a material is always fine.
///
/// The adviser's tasks have no deadline, so a material still to be raised
/// in the tent counts here, unlike for the one-day daily tasks.
/// </summary>
```

Lines 595–596 · comment · before `objective.itemType =`

```
// An old seed name – from a saved task, or from the model still using
// one – becomes the material that replaced it.
```

Lines 1145–1149 · doc comment · before `private static bool SameCropMaterial(string requirement, string plantedItem)`

```
/// <summary>
/// A planting objective that names one material ("plant two banana
/// suckers") is met by planting any material of the same crop – a banana
/// plantlet grows the same banana. What the adviser measures is the crop.
/// </summary>
```

Line 1223 · comment · before `string[] allowedItems =`

```
// Pest traps and pesticides are not typhoon protection.
```

Line 1465 · comment · before `private static readonly System.Text.RegularExpressions.Regex EchoedOpening =`

```
// ----- tidying the model's text
```

Lines 1467-1475 · doc comment · before private static readonly System.Text.RegularExpressions.Regex EchoedOpening =

```
/// <summary>
/// The opening the board prints for itself - "Now... Do ", or either
/// word alone.
///
/// A bare leading "Do" is stripped too, and that is deliberate even
/// though it looks lossy: the board puts "Do " back in front, so a task
/// reading "Do not let the soil dry out" comes out as "Now... Do not let
/// the soil dry out" rather than "Now... Do Do not...".
/// </summary>
```

Line 1481 · doc comment · before private static readonly System.Text.RegularExpressions.Regex PesoAmount =

```
/// <summary>A peso figure, which is what makes a sentence a candidate reward promise.</summary>
```

Lines 1486-1489 · doc comment · before private static readonly System.Text.RegularExpressions.Regex PromiseWording =

```
/// <summary>
/// The promise half. Required alongside a peso figure so that a genuine
/// objective - "sell crops worth P2000" - is never mistaken for one.
/// </summary>
```

Lines 1495-1499 · comment · before private static string StripBoardWording(string taskText)

```
// Note for anyone tempted to "tidy" names like mangosteen_1 out of this
// text: do not. That IS the crop's player-facing name - CropNaming builds
// it, and it is what the player sees when they click the crop. Rewriting
// it as "Mangosteen 1" would leave the task naming something the player
// cannot find on their own farm.
```

Lines 1501-1514 · doc comment · before private static string StripBoardWording(string taskText)

```
/// <summary>
/// Removes the parts of the adviser's wording that the board prints for
/// itself.
///
/// The board wraps the task as "Now... Do &lt;task&gt;" and then adds its own
/// "I'll give you P&lt;reward&gt; if you finish this task." The prompt asks the
/// model to leave both to the game, and it mostly does - but a prompt is
/// a request, not a guarantee, and when the model included them anyway
/// the player read the opening twice and the reward twice.
///
/// Applied when the task is stored rather than when it is drawn, so the
/// grading request that quotes the task back later gets the tidied text
/// too.
/// </summary>
```

Lines 1523-1527 · comment · before if (text.Length > 0 && char.IsUpper(text[0]))

```
// Lower case, because the board reads "Now... Do " straight into this
// and the model writes the task as a sentence of its own. Without
// this the board says "Now... Do Apply mulch". Safe because the
// prompt requires the task to open with a verb, and a verb is never
// a proper noun.
```

Lines 1534-1538 · doc comment · before private static string StripRewardPromise(string text)

```
/// <summary>
/// Drops any sentence that both names a peso amount and promises it to
/// the player. Needing both halves is what keeps "Sell crops worth
/// P2000" - an actual objective - out of the net.
/// </summary>
```

Line 1559 · doc comment · before private static readonly System.Text.RegularExpressions.Regex IdParenthetical =

```
/// <summary>An "(ID: ...)" aside, which is only ever the machine GUID.</summary>
```

Line 1565 · doc comment · before private static readonly System.Text.RegularExpressions.Regex CropGuid =

```
/// <summary>A crop GUID, with or without its dashes.</summary>
```

Lines 1570-1580 · doc comment · before private static string PolishVerdictText(string text)

```
/// <summary>
/// Takes the machine id out of the grader's write-up, leaving the crop
/// name the player can actually act on.
///
/// The three steps are ordered deliberately. The "(ID: ...)" aside goes
/// first, because next to a name it says nothing and turning it into
/// "(ID: mangosteen_1)" would only be redundant. CropNaming.Humanize runs
/// second, so a GUID standing on its own in a sentence becomes the crop's
/// name rather than a hole. Only what is left after that - a GUID for a
/// crop that is no longer live, which nothing can name - is deleted.
/// </summary>
```

Line 1590 · comment · before polished = System.Text.RegularExpressions.Regex.Replace(polished, "[\t]{2,}", " ");

```
// Tidy the gaps the removals leave behind.
```

Lines 1597-1603 · doc comment · before private static string PolishDialogue(string dialogue)

```
/// <summary>
/// Everything the adviser says, tidied for the board.
///
/// CropNaming.Humanize stays in the chain: it swaps a raw crop GUID for
/// the crop's friendly name, which is exactly the name the player reads
/// when they click that crop. Nothing else touches those names.
/// </summary>
```

Assets/Scripts/TerrainPreview/FarmTasks/DailyTaskCatalog.cs

14 comments · 14 comment lines removed · file had 709 lines

Line 22 · trailing comment · on the line `AddRoutineTasks(values);` // 1-65

```
// 1-65
```

Line 23 · trailing comment · on the line `AddGeneralTasks(values);` // 66-80

```
// 66-80
```

Line 24 · trailing comment · on the line `AddDroughtTasks(values);` // 81-105

```
// 81-105
```

Line 25 · trailing comment · on the line `AddTyphoonTasks(values);` // 106-130

```
// 106-130
```

Line 26 · trailing comment · on the line `AddPestDiseaseTasks(values);` // 131-177

```
// 131-177
```

Line 27 · trailing comment · on the line `AddAdvancedTasks(values);` // 178-200

```
// 178-200
```

Line 28 · trailing comment · on the line `AddNurseryTasks(values);` // 201-204

```
// 201-204
```

Lines 30-31 · comment · before `if (values.Count != ExpectedCount)`

```
// Ids are positions in this list, and a saved task keeps its id, so  
// new templates only ever go on the end.
```

Lines 576-580 · doc comment · before `private static void AddNurseryTasks(List<DailyTaskTemplate> list)`

```
/// <summary>  
/// The planting system's own jobs: working ground, and the Seedling Tent.  
/// A seed sown today cannot be transplanted before the day ends, so sowing  
/// and transplanting are separate tasks rather than one "raise a seedling".  
/// </summary>
```

Assets/Scripts/TerrainPreview/FarmTasks/DailyTaskSystem.cs

54 comments · 54 comment lines removed · file had 2135 lines

Lines 31-38 · doc comment · before `public int DaySkipCount { get; private set; }`

```
/// <summary>  
/// How many times the player has turned in for the night this session.  
///  
/// A running count rather than a flag, because skipping immediately  
/// generates the next day's tasks and that generation resets  
/// <c>state.skipUsed</c> back to false. Anything polling the flag would  
/// usually look a frame too late and find it already cleared.  
/// </summary>
```

Lines 139-149 · comment · before `if (!TutorialState.Completed)`

```
// While the beginner guide is running, the day is pinned to the two  
// jobs the guide actually walks the player through: dig a spot, then  
// plant something. Antonio hands them the book and tells them the work  
// is already done, so the tasks have to be ones he definitely made them  
// do - a randomly drawn "plant a vegetable crop" or "water three crops"  
// leaves the reward uncollectable and the tour stuck on that step with  
// no way forward.  
//  
// Both kinds below match on the action alone, with no crop or item  
// filter, so nothing about which district the farm is in or which seeds  
// it was dealt can stop them completing.
```

Lines 456-457 · comment · before `return PlantingAvailability.CountFreeBags() >= Mathf.Max(1, template.targetAmount) &&`

```
// Room in the tent, and enough to sow held today - a task  
// cannot count on a purchase the player may not make.
```

Lines 1155-1156 · comment · before `task.itemType = PlantingMaterialCatalog.UpgradeLegacyList(task.itemType);`

```
// A task saved before the planting materials may still name an  
// old seed ("BananaSeed"); planting what replaced it has to count.
```

Lines 1318-1336 · doc comment · before `private static bool CanObtainRequiredItems(DailyTaskTemplate template)`

```
/// <summary>  
/// Whether the player can actually get hold of what the task needs today  
/// - not merely whether they own one of the thing.  
///  
/// The old check asked only "do you hold at least one", which was wrong  
/// in a way that only showed up on multi-step tasks. A task to mulch  
/// three crops passed on the strength of a single MulchBag, and the  
/// player then had to buy two more; at 3 x P130 for the dearer sprays  
/// that can cost more than the whole day's reward, so the objective was  
/// set at a price the day could not pay for.  
///  
/// So the quantity is now worked out properly and the shortfall is  
/// priced against the money on hand. Holding at least one is still  
/// required - that part is deliberately unchanged, because a task naming  
/// an item the player has never seen reads as a puzzle rather than a  
/// chore.  
///
```

```
/// Alternatives separated by "|" are satisfied if any one of them works.
/// </summary>
```

Lines 1357-1358 · comment · before `int needed = PlayerInventory.IsTool(item)`

```
// A tool is used rather than used up, so one covers the task
// however many crops it names. A consumable is spent per use.
```

Line 1367 · trailing comment · on the line `continue;` // Not sold to this farm, so the shortfall cannot be closed at any price.

```
// Not sold to this farm, so the shortfall cannot be closed at any price.
```

Line 1369 · comment · before `long cost = ShopUIBuilder.CostFor(item, needed - held);`

```
// Priced the way the shop will charge it, rounding included.
```

Lines 1420-1427 · doc comment · before `private static bool CanPlant()`

```
/// <summary>
/// Whether the player can plant this many of a crop in the field today:
/// any of its materials that goes straight in, or a ready seedling in the
/// Seedling Tent, with ground for it ready or preparable. The old check only
/// counted items ending in "Seed", which missed suckers, runners, seednuts
/// and every tent seedling, and counted nursery seeds that cannot reach the
/// field before the day - and its task - ends.
/// </summary>
```

Assets/Scripts/TerrainPreview/FarmTasks/FarmTaskActionHub.cs

28 comments · 28 comment lines removed · file had 410 lines

Lines 8-13 · doc comment · before `public static class FarmTaskActionHub`

```
/// <summary>
/// Routes successful farm actions to the Daily Task and AI Adviser Task systems.
/// It also removes only cross-source duplicates (for example, one direct action
/// record plus one state-observer record for the same mulch application).
/// Repeated actions from the same source are never collapsed.
/// </summary>
```

Lines 27-29 · comment · before `private const float CrossSourceDuplicateWindowSeconds = 2f;`

```
// The state observer runs on a short interval after the direct interaction.
// Two seconds is long enough to pair those records without suppressing two
// legitimate direct clicks, because same-origin records are never deduped.
```

Lines 60-63 · comment · before `return;`

```
// Keep the direct record when it already exists. If the observer
// arrived first, the later direct record is still ignored here;
// both contain the same completed action and task progress remains
// exactly one. The climate tracker uses the same hub decision.
```

Lines 84-91 · doc comment · before `public static event Action<ClimateActionRecord> ActionRecorded;`

```
/// <summary>
/// Raised once for every action that survives de-duplication above.
///
/// The beginner guide listens here rather than hooking each interaction
/// system separately: digging, planting, watering and mulching all already
/// arrive at this one point, and they arrive already de-duplicated, so a
/// single mulch application cannot satisfy a tutorial gate twice.
/// </summary>
```

Lines 150-152 · comment · before `if (!SameOrUnspecified()`

```
// A treatment can have its exact condition only in the observer record.
// Treat an empty condition as a wildcard, but never merge two different
// named conditions.
```

Lines 170-171 · comment · before `if (string.IsNullOrEmpty(first.cropId) &&`

```
// Placement actions without a crop target are paired by item and nearby
// position. This prevents two separately placed traps from being merged.
```

Lines 211-212 · comment · before `if (action.Contains("cleantrap"))`

```
// Specific placement/protection families must be checked before
// generic words such as "water" (WaterStorageTank) or "trap".
```

Assets/Scripts/TerrainPreview/FarmTasks/FarmTaskActionObserver.cs

19 comments · 19 comment lines removed · file had 1025 lines

Line 48 · doc comment · before `private struct GroundObservation`

```
/// <summary>What a patch of prepared ground was at the last scan.</summary>
```

Lines 223-225 · comment · before `Route(new ClimateActionRecord`

```
// The material it was planted from - a sucker, a runner, a
// transplanted seedling - so "plant a fruit crop" style tasks
// can tell them apart.
```

Lines 244-247 · comment · before `if (now.harvestAvailable <`

```
// Watering is recorded directly by FarmingInteractionSystem through
// ClimateEventTracker.RecordWaterAction. Do not infer watering
// from moisture changes because rain and irrigation can also
// increase moisture automatically.
```

Lines 467-473 · doc comment · before `private void ScanDigSpots()`

```
/// <summary>
/// Ground work, in the three steps the shovel takes it through: newly
/// tilled ground ("TillGround"), tilled ground turned into a planting
/// hole, raised bed or furrow ("DigPlantingSpot" - a spot a crop can go
/// into, which is what that action always meant), and a raised bed covered
/// with mulch ("MulchBed").
/// </summary>
```

Lines 936-939 · comment · before `if (!string.IsNullOrEmpty(lastKnownTreatment))`

```
// A 500 ml spray can empty the pump before the observer scans the
// crop. Keep the previously observed liquid so the condition-change
// record still identifies Neem, Bt, Copper, disinfectant, or
// insecticide correctly.
```

Assets/Scripts/TerrainPreview/FarmTasks/FarmTaskContextBuilder.cs

4 comments · 4 comment lines removed · file had 362 lines

Lines 306-309 · doc comment · before `public static List<InventoryItemType> CropNameToPlantingMaterials(string cropType)`

```
/// <summary>
/// Every planting material of a crop named in a task ("Banana" gives the
/// plantlet and the sucker). Empty for a name that is not a crop.
/// </summary>
```

Assets/Scripts/TerrainPreview/FarmTasks/FarmTaskModels.cs

4 comments · 4 comment lines removed · file had 254 lines

Lines 30-31 · comment · before `TillGround,`

```
// The planting system. Tasks are saved by kind name, but keep new kinds at
// the end anyway so the numbers of the old ones never move.
```

Line 160 · doc comment · before `[Serializable]`

```
/// <summary>One Seedling Tent bag, as the adviser sees it.</summary>
```

Line 171 · doc comment · before `[Serializable]`

```
/// <summary>A patch of tilled or prepared ground with nothing planted in it yet.</summary>
```

Assets/Scripts/TerrainPreview/FarmTasks/FarmTaskPopupUI.cs

5 comments · 5 comment lines removed · file had 342 lines

Line 114 · comment · before `if (theme?.farmTaskLabel != null)`

```
// The painted sign carries the wording, so the plain title is hidden.
```

Line 161 · comment · before `scrollRect.offsetMin = new Vector2(themedBoard ? 120f : 40f, themedBoard ? 125f : 105f);`

```
// Wider inset with the board art so the text clears the rolled logs.
```

Line 164 · comment · before `scrollGo.GetComponent<Image>().color = themedBoard`

```
// The board already provides the panel look behind the text.
```

Line 277 · comment · before `image.sprite = art;`

```
// The word is painted into the art, so no Text child is added.
```

Line 317 · trailing comment · on the line `scaler.matchWidthOrHeight = 1f; // landscape: scale by height`

```
// landscape: scale by height
```

Assets/Scripts/TerrainPreview/FarmTasks/FarmTaskRuntimeBootstrap.cs

3 comments · 3 comment lines removed · file had 75 lines

Lines 57-59 · comment · before `foreach (ClimateActionAutoObserver legacyObserver in`

```
// The new observer is a superset of the older climate-only
// observer. Disable the older observer to avoid duplicate action
// records during drought and typhoon events.
```

Assets/Scripts/TerrainPreview/FarmTasks/FarmTaskUIBuilder.cs

38 comments · 38 comment lines removed · file had 914 lines

Line 31 · doc comment · before `private static readonly Color PageInk =`

```
/// <summary>Ink colour for text printed on the book's paper pages.</summary>
```

Line 86 · doc comment · before `public void TogglePanel()`

```
/// <summary>Opened and closed by the round Farm Objectives button.</summary>
```

Line 101 · comment · before `if (aiTaskScroll != null)`

```
// Otherwise a message left scrolled down reopens part-read.
```

Line 121 · comment · before `HudIconButton.Create()`

```
// The round button that opens this panel; the panel itself starts hidden.
```

Line 286 · comment · before `panel.SetActive(false);`

```
// Opened by the round Farm Objectives button, not shown by default.
```

Lines 290–294 · doc comment · before `private void BuildBook(Canvas canvas)`

```
/// <summary>
/// The open-book layout: daily objectives on the left page, Antonio's task
/// on the right. Positions are fractions of the book so resizing
/// objectivesBookSize keeps everything on the paper.
/// </summary>
```

Lines 314–316 · comment · before `float leftX = -size.x * 0.216f;`

```
// Centre of each page. Pulled inward from 0.235 / 0.237: the paper's
// outer edges curve away, so at the old values the taped labels and the
// text columns overhung the left and right edges of the paper.
```

Lines 320–321 · comment · before `float pageW = size.x * 0.39f;`

```
// The taped labels span the full page width; body text uses a narrower
// column so letters keep a margin instead of running to the paper edge.
```

Lines 325–326 · comment · before `CreatePageLabel("DailyLabel", theme?.dailyObjectivesLabel, "Daily Objectives",`

```
// Labels also sit lower than before – at the old Y their upper corners
// ran off the top of the paper.
```

Lines 359–360 · comment · before `aiTaskText = CreatePageScrollText(`

```
// Antonio's message varies in length because it is AI-generated, so it
// gets a scrolling block rather than a fixed label.
```

Lines 425–433 · doc comment · before `private Text CreatePageScrollText(string name, int fontSize,`

```
/// <summary>
/// A page text block that scrolls only when it needs to.
///
/// The text is clipped to <paramref name="size"/> by a mask, so a long
/// adviser message no longer spills down the paper. Because the scroll
/// movement is clamped, a message that already fits cannot be dragged at
/// all and behaves exactly like the plain label it replaces. No scrollbar
/// is attached, so nothing is ever drawn over the book art.
/// </summary>
```

Line 447 · comment · before `GameObject viewportGo = new GameObject(`

```
// RectMask2D needs no Image of its own, so the paper stays visible.
```

Lines 490–491 · comment · before `scroll.movementType = ScrollRect.MovementType.Clamped;`

```
// Clamped keeps short messages completely still – there is no rubber
// -band travel to reveal that a scroll view is there at all.
```

Line 520 · comment · before `image.sprite = art;`

```
// The word is painted into the taped-label art, so no Text child.
```

Lines 645–647 · comment · before `SetButtonState(`

```
// Antonio cannot be called out to inspect the farm while he is
// standing in front of the player running the beginner guide. The
// button lights up the moment he leaves.
```

Lines 785–786 · comment · before `if (image.sprite != null)`

```
// Painted buttons keep their art and just darken when unavailable;
// the plain fallback swaps green for grey as before.
```

Line 889 · trailing comment · on the line `scaler.matchWidthOrHeight = 1f; // landscape: scale by height`

```
// landscape: scale by height
```

Assets/Scripts/TerrainPreview/Harvest/CollectibleBanana.cs

1 comment · 1 comment line removed · file had 10 lines

Line 7 · trailing comment · on the line `public int amount = 12; // how many bananas this bunch gives`

```
// how many bananas this bunch gives
```

Assets/Scripts/TerrainPreview/Inventory/InventoryItemType.cs

9 comments · 9 comment lines removed · file had 78 lines

Line 39 · comment · before `Shovel,`

```
// Always add new enum items at the end.
```

Line 48 · comment · before `MulchBag,`

```
// Climate mitigation and crop-maintenance items. Keep new values at the end.
```

Lines 61–67 · comment · before `BananaPlantlet,`

```
// Planting materials for the planting system. Keep new values at the end:
// saves store items by name, but the scene and Inspector store the number.
//
// BananaSeed, CoconutSeed, MangoSeed, PineappleSeed and StrawberrySeed above
// are no longer sold. They stay only so older saves and marketplace data
// still parse; PlantingMaterialCatalog turns them into the material that
// replaced them the moment they are loaded.
```

Assets/Scripts/TerrainPreview/Inventory/InventorySlotUI.cs

6 comments · 6 comment lines removed · file had 150 lines

Line 47 · comment · before `if (owner.IsTrashMode)`

```
// In trash mode a tap means "throw this away", not "equip this".
```

Lines 64–67 · comment · before `if (owner.IsTrashMode)`

```
// Dragging is off while trashing. Picking a stack up and dropping it  
// somewhere would only be a way to move an item the player has just  
// said they want rid of, and the drag ghost sitting under the  
// confirmation board looks like a bug.
```

Line 73 · comment · before `if ((slot == null || slot.IsEmpty) &&`

```
// Empty backpack slots should allow scrolling instead of dragging an item.
```

Assets/Scripts/TerrainPreview/Inventory/InventoryTrashUI.cs

48 comments · 48 comment lines removed · file had 391 lines

Lines 6–16 · doc comment · before `public class InventoryTrashUI : MonoBehaviour`

```
/// <summary>  
/// Throwing things away from the backpack.  
///  
/// Kept out of InventoryUIBuilder because it is a mode rather than a piece  
/// of furniture: the backpack behaves differently while it is on, and the  
/// two boards it puts up have nothing to do with laying out slots. The  
/// builder owns the button and asks this for the rest.  
///  
/// Built at runtime out of planks the game already ships, like every other  
/// panel here – there are no UI prefabs to edit.  
/// </summary>
```

Lines 19–23 · comment · before `private const float BoardWidth = 900f;`

```
// TutorialPromptBoard.png is 1800x600 and carries no 9-slice borders, so  
// it cannot be stretched to an arbitrary box without the wood grain  
// stretching with it. Both boards are therefore drawn at the plank's own  
// 3:1 shape, and the height is derived from the sprite rather than typed  
// here, so replacing the art cannot silently distort it.
```

Line 38 · doc comment · before `private int pendingSlot = -1;`

```
/// <summary>The slot being emptied, and how much of it, once chosen.</summary>
```

Line 43 · doc comment · before `public System.Action Discarded;`

```
/// <summary>Raised once a slot has actually been emptied, so the backpack can repaint.</summary>
```

Lines 57–60 · doc comment · before `public void Begin(int slotIndex)`

```
/// <summary>  
/// Starts the flow for one slot: the amount question first when there is  
/// more than one in the pile, otherwise straight to the confirmation.  
/// </summary>
```

Lines 70–71 · comment · before `if (PlayerInventory.IsTool(slot.itemType))`

```
// Tools are the "Owned" entries. They cannot be re-bought as a stack  
// and the farm stops working without them, so they are not trash.
```

Line 105 · comment · before `private void OnAmountEntered()`

```
// ----- steps
```

Lines 118–120 · comment · before `if (!int.TryParse(typed, out int amount))`

```
// Every rejection re-asks rather than guessing. Silently rounding a  
// mistyped number would throw away a different amount than the  
// player asked for, and that is not recoverable.
```

Lines 153–154 · comment · before `confirmQuestion.text = pendingAmount > 1`

```
// The count sits beside the name exactly when it means something –  
// "delete 3 Mulch Bag" reads oddly for a single one.
```

Lines 164–166 · comment · before `InventorySlotData slot = CurrentSlot();`

```
// Re-read the slot rather than trusting what was there when the board  
// opened. Nothing else should have touched it, but throwing items  
// away is the one action here that cannot be undone.
```

Line 183 · doc comment · before `private InventorySlotData CurrentSlot()`

```
/// <summary>The slot this flow started on, if it still holds what it did.</summary>
```

Line 202 · comment · before `private void Build()`

```
// ----- building
```

Lines 214–215 · comment · before `blocker.raycastTarget = true;`

```
// Swallows presses on the slots behind, so a second item cannot be  
// picked while the first is still being confirmed.
```

Line 247 · comment · before `amountInput.contentType = InputField.ContentType.IntegerNumber;`

```
// Digits only, and never wider than the largest stack in the game.
```

Lines 255–259 · comment · before `CreateBoardButton(amountBoard.transform, "TrashAmountBack",`

```
// A way out, which this board did not have. Enter was the only  
// button on it, so a slot tapped by mistake left the player having
```

```
// to name an amount and then refuse it on the next board - and
// pressing Enter on an empty box just re-asked, which reads as being
// stuck rather than as a choice.
```

Lines 310-311 · comment · before `text.color = new Color(0.16f, 0.09f, 0.03f, 1f);`

```
// Dark on the pale plank, which is the same choice the trade and
// logout prompts make on their own boards.
```

Lines 368-370 · comment · before `text.verticalOverflow = VerticalWrapMode.Overflow;`

```
// Never Truncate: Unity drops a whole line that does not fit rather
// than clipping it, so at the largest text setting the question would
// simply not be drawn.
```

Assets/Scripts/TerrainPreview/Inventory/InventoryUIBuilder.cs

78 comments · 78 comment lines removed · file had 1026 lines

Lines 113-116 · doc comment · before `public bool IsTrashMode => trashMode;`

```
/// <summary>
/// True while the backpack is in its trash state: slots are click-only,
/// dragging is off, and a tap asks whether to throw the pile away.
/// </summary>
```

Line 151 · trailing comment · on the line `scaler.matchWidthOrHeight = 1f; // landscape: scale by height`

```
// landscape: scale by height
```

Line 161 · comment · before `FpsCounterHud fpsCounter = showFpsCounter`

```
// Straight after the money plank, which it lines up with.
```

Lines 199-200 · comment · before `GameObject textGo = new GameObject("MoneyText", typeof(RectTransform), typeof(Text));`

```
// The coins hang off the plank's left end, so the amount is printed
// on the right-hand portion only.
```

Lines 209-210 · comment · before `textRect.anchoredPosition = new Vector2(theme.moneyPlankSize.x * 0.18f,`

```
// Nudged left and down from (0.22, 0) to sit on the plank's centre;
// the previous offset pushed it high and toward the right end.
```

Lines 222-226 · comment · before `moneyText.verticalOverflow = VerticalWrapMode.Overflow;`

```
// The plank leaves 45 units for the text (150 * 0.30) at font 30 -
// 1.5x, which Playpen Sans clears by almost nothing and loses once
// the text setting scales the font past it. Horizontal was already
// freed here for long amounts; vertical needs the same or the
// balance silently renders as nothing.
```

Line 260 · comment · before `Sprite barArt = theme?.hotbarBar;`

```
// Wooden bar sits behind every slot, so it is created first.
```

Lines 310-311 · comment · before `panelBg.sprite = board;`

```
// The grid frame carries its own corner art, so it is drawn Simple
// at the sprite's own aspect rather than 9-sliced.
```

Lines 324-325 · comment · before `GameObject gridGo = new GameObject("BackpackGrid", typeof(RectTransform));`

```
// Every slot is laid out directly on the panel - the backpack is not
// scrollable, all 36 are visible at once.
```

Lines 338-340 · comment · before `Vector2 gridSize = theme.backpackGridSize;`

```
// Spread the slots evenly across the board's painted cell area so
// each one lands in the middle of a cell. Cell pitch comes from the
// grid size, which is Inspector-tunable to match the art exactly.
```

Lines 438-445 · doc comment · before `private void BuildTrashButton(RectTransform parent)`

```
/// <summary>
/// The Trash Item disc, in the backpack's bottom-left corner.
///
/// It is the same wooden knob the Logout and Account buttons use, and it
/// swaps to the backpack's own X while trashing - so the one control both
/// enters the mode and leaves it, and the player is never left hunting for
/// the way back.
/// </summary>
```

Line 472 · comment · before `textRect.offsetMin = new Vector2(10f, 14f);`

```
// Inset so two short words sit on the disc rather than over its rim.
```

Line 506 · doc comment · before `private void ApplyTrashButtonLook()`

```
/// <summary>The disc wears the X while trashing, and its label the rest of the time.</summary>
```

Line 527 · comment · before `trashButtonLabel.text = trashMode`

```
// The X art has the letter painted in, so no word is written over it.
```

Line 534 · doc comment · before `public void HandleTrashSlotClicked(int slotIndex)`

```
/// <summary>A slot tapped while trashing. Empty slots and tools are ignored.</summary>
```

Lines 590-597 · comment · before `countText.verticalOverflow = VerticalWrapMode.Overflow;`

```
// These two rects are sized as a fraction of the slot, so they give the
// text a fixed 1.39x its own font size of vertical room. That was enough
// for the old font's ~1.15x line box but not for Playpen Sans, whose
// ascenders and descenders push it closer to 1.5x - and Unity's default
// Truncate does not clip a line that does not fit, it discards it, so
// the stack count silently disappeared instead of looking cramped.
```

```
// Overflow lets the glyph box exceed the rect, which for a centred
// number over an icon costs nothing visually.
```

```
Lines 614-615 · comment · before labelText.verticalOverflow = VerticalWrapMode.Overflow;
// Same reasoning as the count above: a slot-number that vanishes is
// worse than one that overhangs its box by a pixel.
```

```
Lines 646-647 · comment · before bg.color = new Color(1f, 1f, 1f, 0f);
// The bar art already draws this cell, so the button itself is just
// the basket icon sitting in the locked seventh slot.
```

```
Lines 697-709 · doc comment · before private void ApplyTutorialTrashGate()
/// <summary>
/// Hides Trash Item for as long as the beginner guide is running.
///
/// A player who empties the bag during the tour throws away the starting
/// seeds Antonio is about to ask them to plant, and the tour has no way to
/// hand them back - the run is simply stuck. Hiding the control is safer
/// than greying it out, because there is nothing useful to trash yet
/// anyway.
///
/// Re-checked every time the backpack opens rather than once when it is
/// built: the tour finishes in the middle of a session, and the button has
/// to come back on its own when it does.
/// </summary>
```

```
Lines 731-734 · comment · before if (!open && trashMode)
// Leaving the backpack always leaves the trash state with it, so
// reopening it is never a surprise: coming back to a bag that
// deletes what you tap, because of a mode set minutes ago, is how a
// player loses something they meant to keep.
```

```
Lines 775-777 · comment · before ApplyTutorialTrashGate();
// Also re-checked here, not only on open: Refresh runs on every
// inventory change, so if the tour finishes while the backpack is
// already open the button appears without waiting for a reopen.
```

```
Lines 788-789 · comment · before if (backpackButton != null && backpackButton.background != null &&
// The basket art carries its own look, so the open/closed tint is only
// applied to the plain fallback button.
```

```
Lines 844-846 · comment · before if (slotUI.iconImage != null)
// While trashing, the things that cannot be trashed are faded out.
// Tapping a tool does nothing, and silently doing nothing reads as a
// broken button - greying them says why before it is pressed.
```

```
Lines 864-865 · comment · before bool themed = theme != null &&
// With the wooden art in place the cells are painted into the bar and
// the grid board, so a slot only tints itself when it is selected.
```

```
Lines 889-892 · doc comment · before public Sprite GetSpriteFor(InventoryItemType item) => GetSprite(item);
/// <summary>
/// The icon for an item. Public so other screens (the trade panel) can draw
/// the same art without duplicating the whole sprite table.
/// </summary>
```

Assets/Scripts/TerrainPreview/Inventory/PlayerInventory.cs

61 comments · 61 comment lines removed · file had 769 lines

```
Lines 72-74 · comment · before TutorialState.ResetForNewFarm();
// A genuinely new farm. Clear any tutorial progress and seed draw
// left in these statics by a farm played earlier in this same app
// session, or the new farm would inherit the old one's seeds.
```

```
Lines 97-109 · doc comment · before private void BuildStartingInventory()
/// <summary>
/// What a brand-new farm begins with.
///
/// This used to hand out all thirteen seed types plus every tool and
/// mitigation kit in the game, which made the district choice cosmetic and
/// left the shop with nothing to sell. A farm now starts with three seed
/// kinds drawn from its own district (see <see cref="DistrictCropPools"/>)
/// and only the tools that cannot be bought back.
///
/// The machete, shovel and watering can are deliberately included: the shop
/// does not stock them, so a player without them could never harvest, dig or
/// water and the farm would be unrecoverable.
/// </summary>
```

```
Lines 117-121 · comment · before if (!TutorialState.Completed)
// When the beginner guide is about to run, it hands these over itself -
// the shovel and seeds when it teaches digging, the watering can when it
// teaches watering, the machete at the end. Filling the hotbar here as
// well would both spoil the reveal and give the player two of everything.
// The guide also covers the case where it is declined.
```

```
Lines 125-126 · comment · before slots[0].Set(InventoryItemType.Machete, 1);
// Hotbar: harvesting tool, the district's three planting materials,
// water, dig.
```

```
Lines 142-146 · comment · before slots[HotbarSlotCount].Set(InventoryItemType.FruitBag, 10);
// Backpack: bags for carrying a harvest home, and a little mulch in
// case a strawberry runner was dealt - it cannot be planted until its
// bed is mulched. Everything else - sprayers, liquids, traps,
// mitigation kits, other districts' planting materials - is bought
// from the shop.
```

```
Line 247 · comment · before case SprayerLiquidType.Insecticide:
// Keep the old general insecticide item usable.
```

```
Lines 336-337 · comment · before pumpSlot.liquidML = Mathf.Max(
// The spray was used, even if it was the wrong
// treatment for the crop's current condition.
```

```
Lines 448-449 · comment · before item = PlantingMaterialCatalog.UpgradeLegacy(item);
// An old seed item - from a trade, an older farm's starting draw or the
// developer tools - arrives as the material that replaced it.
```

```
Line 469 · comment · before for (int i = 0; i < slots.Length && remaining > 0; i++)
// Fill existing stacks first.
```

```
Line 481 · comment · before for (int i = 0; i < slots.Length && remaining > 0; i++)
// Then use empty slots.
```

```
Line 502 · comment · before if (selectedItem == item && IsHotbarSlot(selectedSlotIndex))
// Consume from selected hotbar slot first.
```

```
Lines 531-541 · doc comment · before public bool DiscardFromSlot(int slotIndex, int amount)
/// <summary>
/// Throws away part or all of one slot's stack.
///
/// Slot-specific on purpose, unlike ConsumeItem, which spends from
/// wherever it finds the item. Trashing is aimed at a particular pile the
/// player pointed at, and taking those from a different slot would empty
/// the wrong one in front of them.
///
/// Tools are refused. They are the "Owned" entries the player cannot
/// re-buy in a stack, and a farm with no shovel cannot be dug.
/// </summary>
```

```
Lines 555-557 · comment · before int take = Mathf.Min(amount, slot.amount);
// Clamped rather than refused: the amount arrives from a text box,
// and asking for more than the pile holds most likely means "all of
// it" rather than a mistake worth an error.
```

```
Line 577 · comment · before if (!to.IsEmpty && to.itemType == from.itemType && !IsTool(from.itemType))
// Merge same stackable item.
```

```
Lines 594-595 · comment · before InventorySlotData temporary =
// Move to empty slot or swap with occupied slot.
// Preserve all data, including loaded sprayer liquid.
```

```
Lines 648-652 · doc comment · before public int SellAll(InventoryItemType item, int centavosEach)
/// <summary>
/// Sells every one of an item the player holds and returns how many went.
/// The price is per item in centavos; the payout is the total rounded to
/// whole pesos, because the wallet counts whole pesos.
/// </summary>
```

```
Lines 720-722 · comment · before slots[i].itemType = PlantingMaterialCatalog.UpgradeLegacy(itemType);
// The five old seed items were replaced by planting materials
// (banana seed by banana sucker, and so on). A farm saved with
// them gets the replacement in the same slot and amount.
```

Assets/Scripts/TerrainPreview/Map/FarmMapFieldInput.cs

23 comments · 23 comment lines removed · file had 140 lines

```
Lines 7-19 · doc comment · before public class FarmMapFieldInput : MonoBehaviour,
```

```
/// <summary>
/// Pinch, scroll wheel and drag on the opened farm map, handed to
/// FarmMapUIBuilder to turn into zoom and panning.
///
/// It sits on the green field inside the frame and is enabled only while the
/// map is open. The small corner map is a button that opens it, and has to
/// stay one.
///
/// Pointer down, up and click are answered here too, and do nothing. Unity
/// otherwise passes them up to the nearest parent that wants them - the
/// frame's button - and a held button tints its art, so the whole board would
/// darken for as long as a finger was dragging the map.
/// </summary>
```

```
Line 24 · doc comment · before private const float WheelStep = 1.2f;
/// <summary>Zoom change for one notch of the wheel.</summary>
```

Lines 39–42 · comment · before `if (map == null || TouchesDown() >= 2 ||`
`// Two fingers are a pinch, handled in Update from both at once. Each`
`// finger also reports a drag of its own, and following those as well`
`// would move the map twice. The right mouse button turns the camera,`
`// so it is left to do that.`

Lines 58–59 · comment · before `map.ZoomAtScreenPoint(eventData.position, wheel > 0f ? WheelStep : 1f / WheelStep);`
`// One step per notch whatever size of delta is reported, which varies`
`// between mice, trackpads and platforms.`

Lines 74–76 · comment · before `if (pinching && lastPinchDistance > 1f)`
`// Measured against the previous frame rather than where the fingers`
`// started, so the map follows them exactly: spreading them twice as`
`// far apart doubles the zoom, and moving both carries the map along.`

Assets/Scripts/TerrainPreview/Map/FarmMapIcons.cs

32 comments · 32 comment lines removed · file had 212 lines

Lines 6–20 · doc comment · before `public static class FarmMapIcons`

```
/// <summary>
/// The pictures the farm map draws instead of dots: a crop's fruit, and a
/// placed mitigation item's kit.
///
/// None of the art is assigned twice. The inventory already holds every one of
/// these for the hotbar and backpack, so the map asks it by item type, and a
/// sprite swapped there changes on the map as well.
///
/// Each picture is shrunk once before the map uses it. The source files are
/// 1254 pixels square with no mipmaps and a map icon is about thirty, so drawn
/// straight from the original, every screen pixel lands on a different few
/// texels each time the small map scrolls or turns, and the icons shimmer.
/// Halving the picture over and over averages that detail away first. Where
/// that cannot be done, the original sprite is drawn as it is.
/// </summary>
```

Lines 23–26 · doc comment · before `private const int IconPixels = 128;`

```
/// <summary>
/// Longest side of the shrunk copy: about twice the largest an icon is
/// drawn, zoomed all the way in, so it stays sharp on a dense phone screen.
/// </summary>
```

Line 35 · doc comment · before `public static Sprite ForCrop(FarmCropType crop)`

```
/// <summary>The crop's fruit, or null when there is no picture for it.</summary>
```

Line 41 · doc comment · before `public static Sprite ForClimateObject(ClimateWorldMitigationType type)`

```
/// <summary>The kit a placed climate structure was built from.</summary>
```

Line 47 · doc comment · before `public static Sprite ForAreaTrap(PestDiseaseMitigation mitigation)`

```
/// <summary>A pheromone trap or a termite bait station.</summary>
```

Lines 63–64 · comment · before `if (Cache.TryGetValue(item, out Sprite cached) && cached != null)`

```
// A cached sprite reads as null once Unity has destroyed it - leaving
// Play Mode does that to anything made at runtime - and is made again.
```

Lines 68–70 · comment · before `if (inventory == null && Time.unscaledTime >= nextInventorySearchTime)`

```
// Throttled, because the map asks once per crop on every rescan and a
// scene without an inventory would otherwise search for one hundreds
// of times a second.
```

Line 106 · comment · before `int width = Mathf.Max(targetWidth, Mathf.CeilToInt(area.width * 0.5f));`

```
// The first pass reads only this sprite's part of its texture.
```

Lines 114–115 · comment · before `while (width > targetWidth || height > targetHeight)`

```
// Each halving samples midway between four texels, so every pixel
// is an even average of them rather than a pick of one.
```

Lines 157–158 · comment · before `RenderTexture texture = RenderTexture.GetTemporaryC`

```
// sRGB both ways, so a linear-colour project averages real brightness
// and a gamma one is left alone.
```

Assets/Scripts/TerrainPreview/Map/FarmMapUIBuilder.cs

174 comments · 174 comment lines removed · file had 1067 lines

Lines 11–39 · doc comment · before `public class FarmMapUIBuilder : MonoBehaviour`

```
/// <summary>
/// Top-down farm map on a wooden board.
///
/// The farm is 700m across, so a player who planted a durian on the far side
/// has no way to find it again. This shows where everything is: the player,
/// every crop, the shipping bin, and every placed mitigation object. Crops are
/// drawn as their fruit and placed items as their kit, so a durian reads
/// differently from a coconut and a greenhouse from a canal; the player and the
/// shipping bin stay plain dots.
///
/// Two forms, one panel. Small, it lives under the money plank, zoomed in on
/// the player and turning with them, so the way they face is always up and the
/// N, E, S and W balls round its edge show which way that is. Tapped, it grows
```

```

/// to the middle of the screen, north-up with the four directions on planks
/// round the frame, and fits the whole farm. Pinching or scrolling zooms it,
/// dragging moves it, and zooming all the way back out returns it to exactly
/// that whole-farm view. Closing it returns it to the corner.
///
/// NOTHING HERE IS SAVED, and nothing needs to be. Every marker is read live
/// from the scene, and the farm save already restores all of it - crops, the
/// shipping bin's exact transform, mitigation objects and traps. Load the same
/// farm on another phone and the map redraws itself identically, because the
/// world it is reading has itself been restored.
///
/// Built entirely in code like the rest of the game's UI, and it creates
/// itself in TerrainPreview, so there is no scene wiring and no prefab. It uses
/// the sprites in UITheme and the item pictures already on the inventory; with
/// none assigned it still works, drawn in plain colours.
/// </summary>

Line 45 · comment · before SceneManager.sceneLoaded -= OnSceneLoaded;
// -= then += so we never double-subscribe if the domain isn't reloaded.

Line 109 · doc comment · before private const float CompassBallMargin = 4f;
/// <summary>Gap kept between a compass ball and the edge of the green.</summary>

Line 112 · doc comment · before private const float CompassPlankWidth = 200f;
/// <summary>Width of the direction planks round the opened map.</summary>

Line 136 · doc comment · before private float zoom = 1f;
/// <summary>Zoom of the opened map. 1 fits the whole farm.</summary>

Line 139 · doc comment · before private Vector2 panMetres;
/// <summary>How far the opened map's view has been dragged from the middle of the farm, in metres.</summary>

Line 142 · doc comment · before public bool IsExpanded => expanded;
/// <summary>True while the map is open full-size. Read by the beginner guide.</summary>

Line 145 · doc comment · before public void Collapse()
/// <summary>Folds the map back to its corner size. Used when another panel opens.</summary>

Line 153 · doc comment · before private readonly List<MapMarker> markers = new List<MapMarker>();
/// <summary>Everything drawn on the map, rebuilt on the rescan tick.</summary>

Line 162 · doc comment · before public Sprite icon;
/// <summary>The picture drawn in place of a dot, or null for a plain dot.</summary>

Line 165 · doc comment · before public int order;
/// <summary>Sort key that keeps identical pictures together.</summary>

Line 205 · doc comment · before private static Terrain ResolveTerrain()
/// <summary>Mirrors MapBoundarySystem so both agree on which terrain is the farm.</summary>

Line 234 · comment · before private void RebuildMarkers()
// ----- data

Lines 236-242 · doc comment · before private void RebuildMarkers()
/// <summary>
/// Rescans the scene for everything the map draws.
///
/// Safe against distance culling: DistanceCullable only toggles renderers,
/// leaving the GameObjects active, so crops far from the player are still
/// returned here and still appear on the map.
/// </summary>

Lines 257-259 · comment · before foreach (ClimateMitigationWorldObject item in
// The six placeable climate prefabs all share this component. Mulch,
// trellises, stakes and compost are crop-attached or consumed, carry no
// world object of their own, and so correctly never appear here.

Lines 270-271 · comment · before foreach (AreaMitigationTrapInstance trap in
// Termite bait stations and pheromone traps: also standalone prefabs
// the player drops and then has to find again.

Lines 292-293 · comment · before SeedlingTentInstance tent = Object.FindFirstObjectByType<SeedlingTentInstance>();
// The Seedling Tent, shown with its own round button art so the player
// can find it again after wandering off to plant.

Lines 301-304 · comment · before markers.Sort(ByOrder);
// Grouped by kind. Every kind of picture is its own texture, and where
// two different pictures overlap the canvas has to split its batch
// between them; in scene order the kinds interleave, so a dense planting
// could cost a draw call per crop.

Lines 326-327 · comment · before if (player != null)
// Added last so it is the last sibling, and so draws over any crop or
// object the player happens to be standing on.

Line 339 · doc comment · before private static MapMarker IconOrDot(Transform target, Sprite icon, float iconSize,
/// <summary>The picture when there is one, and the old coloured dot when there is not.</summary>

```

Line 353 · comment · before private Vector2 FieldSize

```
// ----- drawing
```

Lines 367-372 · doc comment · before private float PixelsPerMetre()

```
/// <summary>
/// UI pixels per world metre. Uniform on both axes so the farm is never
/// stretched: expanded fits the whole terrain inside the shorter side of
/// the field and multiplies by the zoom, small works back from how many
/// metres it should show.
/// </summary>
```

Lines 392-396 · doc comment · before private Vector2 ViewCentreWorldXZ()

```
/// <summary>
/// The world point the middle of the field represents: the player on the
/// small map, and on the opened one the middle of the farm, moved by however
/// far the player has dragged it.
/// </summary>
```

Lines 405-409 · doc comment · before private float PlayerHeadingDegrees()

```
/// <summary>
/// Which way the player is looking, in degrees clockwise from north. The
/// camera only ever pitches underneath the player, so the player's own turn
/// is the camera's heading.
/// </summary>
```

Lines 421-422 · comment · before float heading = expanded ? 0f : PlayerHeadingDegrees() * Mathf.Deg2Rad;

```
// The small map turns so the way the player faces is up. The opened map
// stays north-up, which is what the planks round its frame say.
```

Lines 439-440 · comment · before float east = (world.x - centre.x) * pixelsPerMetre;

```
// Terrain +Z is north and UI +Y is up, so Z maps straight onto Y
// and, unturned, the map reads north-up.
```

Lines 444-445 · comment · before Vector2 point = new Vector2()

```
// Turned anticlockwise by the heading, which lands anything lying
// straight ahead of the player straight up the map.
```

Lines 452-453 · comment · before if (Mathf.Abs(point.x) > halfField.x + size ||

```
// RectMask2D clips anything straying over the edge, but skipping
// markers that are far outside keeps the pool small when zoomed in.
```

Lines 465-467 · comment · before image.sprite = marker.icon != null ? marker.icon : dotSprite;

```
// A picture keeps its own shape inside the square; a dot is the
// square. Icons are never turned with the small map, so the fruit
// always stands upright.
```

Lines 508-515 · doc comment · before private void LayoutCompassBalls(float cos, float sin, Vector2 halfField)

```
/// <summary>
/// Puts each ball where its direction meets the edge of the green.
///
/// North, east, south and west are turned by the same rotation as the
/// markers, and each ball travels out from the middle that way until it
/// would touch the edge. As the player turns, the balls run round the
/// inside of the field and never onto the wood.
/// </summary>
```

Line 539 · comment · before public void ZoomAtScreenPoint(Vector2 screenPoint, float factor)

```
// ----- zoom and pan
```

Lines 541-545 · doc comment · before public void ZoomAtScreenPoint(Vector2 screenPoint, float factor)

```
/// <summary>
/// Zooms the opened map by <paramref name="factor"/>, keeping the spot
/// under <paramref name="screenPoint"/> - the cursor, or the middle of a
/// pinch - where it is.
/// </summary>
```

Line 569 · doc comment · before public void PanByScreenDelta(Vector2 screenPosition, Vector2 screenDelta)

```
/// <summary> Moves the opened map by a finger or mouse movement given in screen pixels. </summary>
```

Lines 581-582 · comment · before panMetres -= (now - before) / PixelsPerMetre();

```
// The farm moves with the finger, so the middle of the view moves the
// other way.
```

Lines 587-596 · doc comment · before private void ClampPan()

```
/// <summary>
/// Keeps the view on the farm: the edge of what is on screen may travel as
/// far as the edge of the terrain and no further.
///
/// At a zoom of 1 the whole farm already fits, so there is no room at all
/// and the view is pinned to the middle. That is what brings a player who
/// zooms back out to exactly the view the map opened on, from wherever they
/// had dragged it: the room shrinks as they zoom out and reaches nothing
/// just as they arrive.
/// </summary>
```

Line 610 · doc comment · before private bool TryFieldPoint(Vector2 screenPoint, out Vector2 local)

```
/// <summary> A screen point as a position relative to the middle of the field, in the units the markers are laid out in. </summary>
```

```

Line 621 · comment · before private void SetExpanded(bool value)
// ----- modes

Line 627 · comment · before zoom = 1f;
// The opened map always starts on the whole farm, however it was left.

Line 642 · comment · before root.transform.SetAsLastSibling();
// Above whatever else is on screen while it is open.

Lines 662-663 · comment · before fieldImage.raycastTarget = expanded;
// Only the opened map takes drags and pinches. Small, the field lets
// taps fall through to the frame, which is the button that opens it.

Line 667 · comment · before LayoutMarkers();
// The zoom changed, so redraw before the next frame renders.

Lines 671-674 · doc comment · before private Vector2 SmallAnchoredPosition()
/// <summary>
/// Tucked under the money plank, using the plank's own themed height so
/// the two never overlap if that art is resized.
/// </summary>

Line 679 · comment · before const float plankTop = 14f;
// The plank sits at (-20, -14) with a top-right pivot.

Line 686 · comment · before private void Build()
// ----- building

Line 710 · doc comment · before private void BuildBlocker()
/// <summary>Dims the game and swallows taps meant for the map, not the joystick.</summary>

Lines 752-754 · comment · before frameButton.onClick.AddListener(() =>
// Guarded rather than made non-interactable while open: Button's
// disabled state tints its target graphic to a half-transparent grey,
// which would wash the whole board out the moment it was opened.

Lines 776-782 · comment · before fieldImage.raycastTarget = false;
// RectMask2D rather than Mask: it clips by rectangle and needs no
// graphic of its own, so the field colour can be anything - including
// fully transparent, if the frame art already paints its own field.
// A stencil Mask would clip every marker away at alpha 0.
//
// Not a raycast target until the map is opened; SetExpanded switches
// it on so the field can take drags.

Lines 798-802 · doc comment · before private void BuildCompassBalls()
/// <summary>
/// The N, E, S and W balls on the small map. Inside the field and after the
/// markers, so they draw over a crop at the edge and are clipped to the
/// green like everything else in it.
/// </summary>

Line 815 · comment · before string[] letters = { "N", "E", "S", "W" };
// In the order LayoutCompassBalls places them.

Lines 849-850 · comment · before text.horizontalOverflow = HorizontalWrapMode.Overflow;
// Overflow both ways. This font's line box is taller than the ball
// at the largest text setting, and Truncate would then draw nothing.

Lines 859-863 · doc comment · before private void BuildCompassPlanks()
/// <summary>
/// North, East, South and West on planks at the middle of each side of the
/// opened map's frame. The opened map is always north-up, so they never
/// move.
/// </summary>

Lines 875-876 · comment · before Vector2 inset = InsetFraction;
// Centred on the wood of the frame, half the border's thickness in from
// each edge, which is only ever drawn at the opened size.

Lines 892-894 · comment · before Vector2 size = UIPlank.SizeFor(art, CompassPlankWidth, CompassPlankWidth * 0.4f);
// The board art's own proportions, so the wood is never squashed.
// TradeRequestBoard has no 9-slice border, so "sliced" simply stretches
// it to this box.

Lines 905-906 · comment · before plank.GetComponent<Image>().raycastTarget = false;
// The side planks reach over the edge of the field, and a drag that
// starts on one should still move the map.

Line 930 · comment · before image.sprite = art;
// The word is painted into the art, so no Text child is added.

Lines 960-963 · doc comment · before private static Sprite CreateCircleSprite()
/// <summary>
/// One soft-edged white circle, tinted per dot. A single shared sprite lets
/// every dot batch into one draw call however many are on screen.
/// </summary>

```

Lines 983-984 · comment · before `float alpha = Mathf.Clamp01(radius - distance);`
// One pixel of feather, so the dot keeps a clean edge when it
// is drawn far smaller than this texture.

Lines 998-1002 · doc comment · before `private static Sprite CreateCompassBallSprite()`
/// <summary>
/// An orange ball with a dark rim, behind the small map's direction letters.
/// Mipmapped, unlike the dot: the rim is a few pixels wide once drawn, and
/// without smaller copies to sample from it would break up into specks.
/// </summary>

Line 1059 · trailing comment · on the line `scaler.matchWidthOrHeight = 1f; // landscape: scale by height`
// landscape: scale by height

Assets/Scripts/TerrainPreview/Map/MapBoundarySystem.cs

36 comments · 36 comment lines removed · file had 338 lines

Lines 8-18 · doc comment · before `public class MapBoundarySystem : MonoBehaviour`
/// <summary>
/// Builds an invisible collision wall around the whole terrain edge (so the
/// player can't walk off into the void) plus a soft transparent fog curtain
/// just outside each edge (to hide the void and the hard terrain edge).
///
/// Self-contained: it auto-creates itself in the TerrainPreview scene, waits
/// for the runtime-generated terrain, then sizes everything to it.
///
/// It deliberately does NOT touch RenderSettings.fog - that global fog is
/// owned by <see cref="WeatherVisualController"/> for weather effects.
/// </summary>

Lines 24-26 · comment · before `SceneManager.sceneLoaded -= OnSceneLoaded;`
// -= then += so we never double-subscribe if the domain isn't reloaded.
// sceneLoaded fires for the initial scene too, so this also covers
// pressing Play directly inside TerrainPreview in the editor.

Lines 94-98 · doc comment · before `private static float GetDaylightAmount01()`
/// <summary>
/// Mirrors GameTimeSystem.UpdateSun()'s day/night curve (0 = full night,
/// 1 = full midday) so the fog tint stays in sync with the existing
/// sun-intensity transition, without modifying GameTimeSystem itself.
/// </summary>

Line 171 · comment · before `float lengthX = (maxX - minX) + wallThickness * 2f;`
// Extend length so adjacent walls overlap at the corners (no gaps).

Line 175 · comment · before `CreateWall("BoundaryWall_North",`
// North / South run along X.

Line 184 · comment · before `CreateWall("BoundaryWall_East",`
// East / West run along Z.

Lines 218-219 · comment · before `if (fogMaterial == null)`
// No usable shader: skip the fog entirely rather than spawning panels
// with a null material, which would render as magenta boxes.

Line 223 · comment · before `float lengthX = (maxX - minX) + fogOutsideOffset * 2f;`
// Panels overlap the corners so the ring is seamless.

Line 227 · comment · before `CreateFogPanel("EdgeFog_North", fogMaterial,`
// North / South panels lie in the XY plane (identity rotation).

Line 236 · comment · before `Quaternion sideRotation = Quaternion.Euler(0f, 90f, 0f);`
// East / West panels rotate 90 deg so their width runs along Z.

Line 258 · comment · before `Collider panelCollider = panel.GetComponent<Collider>();`
// Visual only - no physics interference with placement/interaction rays.

Lines 273-275 · comment · before `if (fogMaterialAsset != null)`
// An asset assigned in the Inspector is preferred: Shader.Find only sees
// shaders that survived build stripping, and nothing references these by
// asset, so on a device build they can all come back null.

Lines 279-280 · comment · before `Shader shader = Shader.Find("Sprites/Default");`
// Sprites/Default is an unlit, transparent, double-sided (Cull Off)
// shader that renders reliably in URP and the built-in pipeline.

Line 296 · comment · before `Color initialTint = Color.Lerp(nightFogColor, dayFogColor, GetDaylightAmount01());`
// Initial color; Update() overrides this every frame based on time of day.

Line 321 · trailing comment · on the line `float v = y / (float)(height - 1); // 0 = bottom, 1 = top`
// 0 = bottom, 1 = top

Line 323 · comment · before `float alpha = v <= fogDensePortion`
// Fully dense from the bottom up to fogDensePortion, then fade to 0.

Assets/Scripts/TerrainPreview/Nursery/NurseryRuntimeBootstrap.cs

6 comments · 6 comment lines removed · file had 49 lines

Lines 6–10 · doc comment · before `public static class NurseryRuntimeBootstrap`

```
/// <summary>
/// Creates the Seedling Tent's runtime systems in the farm scene, the same
/// way the climate-maintenance and farm-task systems are created, so the
/// scene needs no extra objects. Safe to call any number of times.
/// </summary>
```

Line 40 · comment · before `if (Object.FindFirstObjectByType<NurserySystem>() == null)`

```
// The nursery before its panel: the panel subscribes to it on Start.
```

Assets/Scripts/TerrainPreview/Nursery/NurserySystem.cs

59 comments · 59 comment lines removed · file had 720 lines

Line 10 · doc comment · before `Empty,`

```
/// <summary>No soil in the bag yet.</summary>
```

Line 13 · doc comment · before `Filled,`

```
/// <summary>Filled with soil, waiting for something to be sown.</summary>
```

Line 16 · doc comment · before `Germinating,`

```
/// <summary>A seed germinating in the seed tray, before it is pricked into this bag.</summary>
```

Line 19 · doc comment · before `NeedsPricking,`

```
/// <summary>Germinated and waiting for the player to prick it into the bag.</summary>
```

Line 22 · doc comment · before `Growing,`

```
/// <summary>Growing (or hardening) toward transplant size.</summary>
```

Line 25 · doc comment · before `Ready`

```
/// <summary>Ready to transplant.</summary>
```

Line 29 · doc comment · before `[Serializable]`

```
/// <summary>One seedling bag on the Seedling Tent's racks.</summary>
```

Lines 50–66 · doc comment · before `public class NurserySystem : MonoBehaviour`

```
/// <summary>
/// The farm's Seedling Tent: eight seedling bags on two racks, where nursery
/// materials are raised before they go to the field.
///
/// A bag is filled with soil, then sown. Durian and pomelo seeds germinate in
/// a seed tray first and wait for the player to prick them into their bag,
/// as the user's notes describe. When a seedling is ready the player picks it
/// in the tent panel and walks it to prepared ground: the next tap on a
/// matching hole or bed transplants it (see FarmingInteractionSystem).
///
/// Time is the game clock's, measured from the day each bag was sown, so days
/// skipped from the objectives book or the developer tools count too.
///
/// Created at runtime by <see cref="NurseryRuntimeBootstrap"/>, like the other
/// farm systems, so the scene needs no wiring beyond the prefabs on the
/// farming system.
/// </summary>
```

Line 73 · doc comment · before `public event Action Changed;`

```
/// <summary>Raised whenever a bag, the transplant state or the tent changes.</summary>
```

Lines 115–116 · comment · before `if (!tentHandled && !FarmLoadContext.IsRestoring &&`

```
// A new farm gets its tent once the world exists. A loaded farm gets
// it from RestoreSaveData, which runs before the world is marked ready.
```

Lines 126–127 · comment · before `if (Time.unscaledTime < nextStatusCheck)`

```
// Growth is continuous, but the tent only needs redrawing when a bag
// moves from one state to the next, or its seedling visibly grows.
```

Line 156 · comment · before `public SeedlingBag GetBag(int slot)`

```
// ----- reading
```

Line 203 · doc comment · before `public float DaysUntilNextStep(int slot)`

```
/// <summary>Game days until the bag's next step: germination, or ready to transplant.</summary>
```

Line 221 · doc comment · before `public float GrowthProgress(int slot)`

```
/// <summary>How far the seedling is toward transplant size, 0 to 1, for the models and the panel.</summary>
```

Line 247 · doc comment · before `public float DaysInNursery(int slot)`

```
/// <summary>Days a seedling has spent in its bag, counted from sowing.</summary>
```

Line 265 · doc comment · before `public int CountReady(FarmCropType? crop = null)`

```
/// <summary>Ready seedlings of a crop, for the daily tasks and the adviser.</summary>
```

Line 280 · comment · before `public bool FillBag(int slot, out string message)`

```
// ----- actions
```

```

Line 395 · doc comment · before public bool EmptyBag(int slot, out string message)
/// <summary>Throws out whatever is in a bag, soil included. The material is lost.</summary>

Line 439 · doc comment · before public bool TryGetTransplant(out PlantingMaterialInfo info, out float daysInNursery)
/// <summary>What is being carried to the field, without taking it out of its bag yet.</summary>

Lines 452-455 · doc comment · before public void FinishTransplant()
/// <summary>
/// Called once the seedling is in the ground. The soil goes with the
/// seedling, so the bag is left empty and has to be refilled to use again.
/// </summary>

Line 466 · doc comment · before public int DevMakeAllReady()
/// <summary>Developer tools: every sown bag skips straight to ready.</summary>

Line 506 · comment · before private Vector3 TentPosition => tent != null ? tent.transform.position : Vector3.zero;
// ----- tent

Lines 527-530 · comment · before Vector3[] offsets =
// To the player's left, on the other side from the shipping bin, and
// far enough out that the two never touch. Other spots are tried in
// turn so a farm loaded from an older save does not put the tent on
// top of its crops.

Line 609 · comment · before public NurserySaveDto CaptureSaveData()
// ----- saving/loading

Lines 658-659 · comment · before string material = PlantingMaterialCatalog.UpgradeLegacyName(saved.material);
// Anything no longer sown in bags is dropped rather than left to
// sit in a bag that can never finish.

Lines 679-680 · comment · before tentHandled = false;
// A farm saved before the Seedling Tent existed gets one now, near
// wherever the player was standing.

Line 687 · comment · before public static void RecordNurseryAction(
// ----- records

Lines 689-694 · doc comment · before public static void RecordNurseryAction(
/// <summary>
/// Reports a nursery action to the daily tasks, the adviser's task and the
/// beginner guide. It deliberately skips the climate event tracker: sowing
/// a bag is not a response to a typhoon or a drought, and counting it there
/// would pad the event's evaluation with unrelated work.
/// </summary>

```

Assets/Scripts/TerrainPreview/Nursery/PlantingAvailability.cs

28 comments · 28 comment lines removed · file had 196 lines

```

Lines 8-17 · doc comment · before public static class PlantingAvailability
/// <summary>
/// What the player could plant in the field today, for the daily tasks and
/// the adviser's task check.
///
/// A daily task lasts one game day, so "plant a crop" can only be offered for
/// something that can reach the ground today: a material that goes straight
/// in, a bought ready seedling, or a seedling already ready in the Seedling
/// Tent. A seed that still has days to spend in a bag cannot finish the task,
/// and offering it would leave the day's reward out of reach.
/// </summary>

Lines 20-23 · doc comment · before public static int CountPlantableToday(Func<InventoryItemType, bool> allow = null)
/// <summary>
/// How many plantings could be made today among the materials the filter
/// allows (every material when it is null).
/// </summary>

Lines 26-27 · comment · before GroundReadiness ground = GroundReadiness.Read();
// The ground is read once per question, not once per material: the
// daily tasks ask this for every template when a new day is drawn.

Line 68 · doc comment · before public static int CountTentSeedlingsPlantableToday()
/// <summary>Ready seedlings in the Seedling Tent that have ground they can go into today.</summary>

Line 90 · doc comment · before public static int CountPlantableToday(FarmCropType crop)
/// <summary>Plantings of one crop possible today, from any of its materials.</summary>

Lines 97-102 · doc comment · before public static bool CanPrepareGroundFor(FarmCropType crop)
/// <summary>
/// Whether ground for this crop is ready or can be made: an open patch of
/// the right kind already prepared, or the shovel to prepare one. A
/// strawberry also needs its bed mulched, so without a mulched bed it needs
/// a Mulch Bag too.
/// </summary>

Line 108 · doc comment · before public static int CountNurseryMaterialsHeld(Func<InventoryItemType, bool> allow = null)
/// <summary>Held materials that can be sown in a seedling bag.</summary>

```

Line 125 · doc comment · before `public static int CountFreeBags()`
/// <summary>Seedling bags that are empty or filled but not sown – room to sow.</summary>

Line 135 · doc comment · before `private struct GroundReadiness`
/// <summary>The open prepared ground on the farm and the tools in hand, read once.</summary>

Line 186 · comment · before `if (needsMulch && bed && mulchBag) return true;`
// An unmulched bed is ready for a strawberry once mulched.

Assets/Scripts/TerrainPreview/Nursery/PlantingMaterialCatalog.cs

100 comments · 100 comment lines removed · file had 598 lines

Lines 8–11 · doc comment · before `public sealed class PlantingMaterialInfo`
/// <summary>
/// One planting material: what it grows into, how it reaches the field, and
/// the real-world timing the game compresses.
/// </summary>

Line 18 · doc comment · before `public bool SowInBag;`
/// <summary>Raised in a seedling bag in the Seedling Tent before it goes to the field.</summary>

Line 21 · doc comment · before `public bool PlantDirect;`
/// <summary>Goes straight into prepared ground in the field.</summary>

Line 24 · doc comment · before `public bool ReadySeedling;`
/// <summary>Bought already raised, so it skips the tent and is transplanted straight away.</summary>

Line 27 · doc comment · before `public PreparedPlotKind Plot;`
/// <summary>The ground this crop needs in the field.</summary>

Line 30 · doc comment · before `public bool NeedsMulchedBed;`
/// <summary>The raised bed must be mulched before this goes in.</summary>

Line 33 · doc comment · before `public float NurseryDays;`
/// <summary>Game days from sowing in a bag to ready for transplanting.</summary>

Lines 36–41 · doc comment · before `public float PrickAfterDays;`
/// <summary>
/// Above zero for seeds that germinate in a seed tray first and are then
/// pricked into their bag by hand. The bag waits for the player after this
/// many days, and the rest of <see cref="NurseryDays"/> starts from the
/// pricking.
/// </summary>

Lines 44–47 · doc comment · before `public float ExtraAgeDays;`
/// <summary>
/// Extra age given on top of the nursery time. Used for the grafted mango,
/// which the notes say flowers sooner than a seed-grown tree.
/// </summary>

Line 50 · doc comment · before `public float ReadySeedlingAgeDays;`
/// <summary>How old a bought ready seedling already is, in game days.</summary>

Line 53 · doc comment · before `public string RealWorld;`
/// <summary>The real figure from the user's crop notes, shown beside the game one.</summary>

Line 56 · doc comment · before `public bool FieldReady => PlantDirect || ReadySeedling;`
/// <summary>A material that can go into the field today, without the tent.</summary>

Lines 60–78 · doc comment · before `public static class PlantingMaterialCatalog`
/// <summary>
/// The sixteen planting materials and the route each one takes to the field.
///
/// The planting material, not the crop, decides the route. The same crop can
/// come from different material – a banana from a tissue-cultured plantlet or
/// a sword sucker, a mango from a grafted seedling or a liso seed – and squash
/// seed can go either way, or be bought as a ready seedling. Every real-world
/// figure below is taken from the user's own notes in the Handoff folder
/// (crop_materials.txt and crop_production_notes.txt).
///
/// Nursery waits are compressed into a few game days on purpose. A game day
/// lasts fifteen real minutes, so even a five-month cacao nursery would take
/// tens of hours of play; the real figure stays visible in the text instead.
///
/// The five old seed items (banana, coconut, mango, pineapple and strawberry
/// seed) are kept in the item list only so that older saves, trades and
/// listings still read. They are turned into the material that replaced them
/// wherever they are loaded.
/// </summary>

Lines 81–85 · doc comment · before `public const float MaxNurseryAgeMultiplier = 2f;`
/// <summary>
/// A seedling left in its bag long after it was ready only counts up to
/// this multiple of its nursery time, so a forgotten bag does not turn into
/// a tree that is already bearing.
/// </summary>

```

Line 101 · doc comment · before public static readonly InventoryItemType[] AllMaterials =
/// <summary>The order the shop shelf and the developer tools list them in, grouped by crop.</summary>

Line 243 · comment · before public static bool TryGet(InventoryItemType item, out PlantingMaterialInfo info)
// ----- lookups

Line 250 · doc comment · before public static bool TryGet(string itemName, out PlantingMaterialInfo info)
/// <summary>By saved name, upgrading an old seed name first.</summary>

Line 268 · doc comment · before public static bool IsFieldReadyMaterial(InventoryItemType item)
/// <summary>Can go into prepared ground today: a direct material or a bought seedling.</summary>

Line 274 · doc comment · before public static List<InventoryItemType> MaterialsFor(FarmCropType crop)
/// <summary>Every material that grows into this crop, in shelf order.</summary>

Lines 286-289 · doc comment · before public static InventoryItemType DefaultMaterialFor(FarmCropType crop)
/// <summary>
/// The material assumed for a crop that was planted before materials were
/// recorded - what its old seed would be called today.
/// </summary>

Line 311 · doc comment · before public static PreparedPlotKind PlotFor(FarmCropType crop)
/// <summary>The ground a crop needs in the field, whatever it was grown from.</summary>

Lines 360-365 · doc comment · before public static string SeedlingName(PlantingMaterialInfo info)
/// <summary>
/// What a nursery material is called once it is growing in its bag: a seed
/// becomes "Cacao Seedling", while a banana plantlet or a grafted mango
/// seedling keeps its own name. Tacking "seedling" onto the material's
/// name gave "Cacao Seed seedling".
/// </summary>

Line 376 · doc comment · before public static bool TryGetCropType(string cropName, out FarmCropType crop)
/// <summary>"Banana" to the crop; false for anything that is not a crop name.</summary>

Line 394 · comment · before public static bool IsLegacySeed(InventoryItemType item)
// ----- old seed items

Line 396 · doc comment · before public static bool IsLegacySeed(InventoryItemType item)
/// <summary>One of the five seed items the new materials replaced.</summary>

Line 404 · doc comment · before public static InventoryItemType UpgradeLegacy(InventoryItemType item)
/// <summary>The replacement for an old seed item, or the item itself.</summary>

Line 410 · doc comment · before public static string UpgradeLegacyName(string itemName)
/// <summary>The same, for an item saved by name. Unknown names are returned as they were.</summary>

Lines 421-424 · doc comment · before public static string UpgradeLegacyList(string alternatives)
/// <summary>
/// Upgrades every old seed name inside an "A|B|C" alternative list, as the
/// daily tasks and the adviser's objectives store them, keeping the rest.
/// </summary>

Line 442 · comment · before public static float StartingAgeDays(PlantingMaterialInfo info, float daysInNursery)
// ----- timing

Lines 444-449 · doc comment · before public static float StartingAgeDays(PlantingMaterialInfo info, float daysInNursery)
/// <summary>
/// Age, in game days, a crop starts at when this material goes into the
/// field. The user decided nursery days count toward the crop's age; a
/// seedling left in its bag long after it was ready only counts up to
/// <see cref="MaxNurseryAgeMultiplier"/> times its nursery time.
/// </summary>

Line 477 · comment · before public static string DescribeRoute(PlantingMaterialInfo info)
// ----- text

Lines 479-482 · doc comment · before public static string DescribeRoute(PlantingMaterialInfo info)
/// <summary>
/// The planting route in plain words, for the shop, the Seedling Tent and
/// the adviser.
/// </summary>

Lines 540-543 · doc comment · before public static string CropBoardLine(string materialName, float nurseryDays)
/// <summary>
/// "Planted From" line for the crop board: the material, and how long it
/// spent in the tent.
/// </summary>

Line 560 · doc comment · before public static string RealWorldLine(string materialName)
/// <summary>"In real farms: ..." for the crop board.</summary>

Lines 569-573 · doc comment · before public static class CropPlantingText
/// <summary>
/// The two planting lines every crop board shows, formatted to drop straight
/// into the three crop classes' readouts: nothing at all for a crop planted
/// before materials were recorded.
/// </summary>

```

Line 576 · doc comment · before `public static string PlantedFromLine(string materialName, float nurseryDays)`
/// <summary>"Planted From: ..." plus its line break, or nothing.</summary>

Lines 583–587 · doc comment · before `public static string RealWorldLine(string materialName, FarmCropType crop)`
/// <summary>
/// A line break plus "In real farms: ...". A crop planted before materials
/// were recorded uses the material its old seed became, so every crop on
/// the farm shows its real-world timing.
/// </summary>

Assets/Scripts/TerrainPreview/Nursery/SeedlingTentInstance.cs

24 comments · 24 comment lines removed · file had 238 lines

Lines 5–10 · doc comment · before `[DisallowMultipleComponent]`

```
/// <summary>
/// The Seedling Tent standing on the farm. It shows each seedling bag on its
/// rack – empty, filled, or with its seedling growing out of it – and opens
/// the tent panel when tapped. The bags' contents live in
/// <see cref="NurserySystem"/>; this only draws them.
/// </summary>
```

Lines 40–43 · doc comment · before `public void SeatOnTerrain(Terrain terrain)`

```
/// <summary>
/// Rests the tent on the lowest ground under its floor, so no corner hangs
/// in the air on a slope; the uphill side settles into the soil instead.
/// </summary>
```

Line 75 · doc comment · before `public void Refresh(NurserySystem nursery)`

```
/// <summary>Redraws every bag from the nursery's state.</summary>
```

Line 85 · doc comment · before `public void RefreshGrowth(NurserySystem nursery)`

```
/// <summary>Only resizes the growing seedlings; cheaper than a full redraw.</summary>
```

Lines 120–121 · comment · before `string showing = null;`

```
// A seed still in the seed tray is not in the bag yet, so nothing
// grows out of the bag until it has been pricked into it.
```

Lines 153–157 · doc comment · before `private void ScaleSeedling(NurserySystem nursery, int slot)`

```
/// <summary>
/// Grows the seedling with its progress and keeps its roots inside the bag:
/// the models include their roots, so each is sunk by the root share of its
/// current height below the soil line.
/// </summary>
```

Line 169 · doc comment · before `private static float MeasureHeight(GameObject model, Transform anchor)`

```
/// <summary>A model's height at its prefab scale, in its anchor's units.</summary>
```

Line 184 · doc comment · before `private float BagTop(int slot, Transform anchor)`

```
/// <summary>Height of the soil in the bag above its anchor, in the anchor's own units.</summary>
```

Line 199 · comment · before `float worldTop = bounds.max.y - bounds.size.y * 0.12f;`

```
// Just below the rim, where the soil surface sits.
```

Lines 209–210 · comment · before `string name = "BagAnchor_" + slot;`

```
// No anchors set up: lay the bags out in two rows in front of the tent,
// so the nursery still shows something rather than nothing.
```

Assets/Scripts/TerrainPreview/Nursery/SeedlingTentUIBuilder.cs

37 comments · 37 comment lines removed · file had 1018 lines

Lines 9–18 · doc comment · before `public class SeedlingTentUIBuilder : MonoBehaviour`

```
/// <summary>
/// The Seedling Tent panel: a round HUD button, a board with the tent's eight
/// seedling bags, and the actions for the bag picked – fill it with soil, sow,
/// prick out, transplant or empty it.
///
/// Built in code like every other farm panel, on the shop's board art, with
/// the painted bag sprites for the bags themselves. Transplanting closes the
/// panel and shows a banner along the top of the screen until the seedling is
/// in the ground or the player cancels.
/// </summary>
```

Line 70 · doc comment · before `public static void OpenPanel()`

```
/// <summary>Opens the panel, creating it first if the scene has not yet.</summary>
```

Lines 131–134 · comment · before `bool showHeading = !TutorialDialogueUI.ObjectiveShowing;`

```
// The tutorial's objective plank covers the top centre of the
// screen, where this heading sits. It is the same plank art, so
// while it is up it stands in for the heading rather than half
// hiding it.
```

Line 139 · comment · before `if (Time.unscaledTime >= nextLiveRefresh)`

```
// Days-left figures tick down while the panel is open.
```

Line 156 · comment · before `public void Open()`

```
// ----- open/close
```

Line 188 · comment · before private void EnsureEventSystem()

```
// ----- building
```

Lines 254-255 · comment · before Sprite plank = theme?.tutorialObjectiveBoard;

```
// Heading on a blank plank, as the account screens do it: there is no
// painted "Seedling Tent" sign, and this keeps it in the same style.
```

Lines 373-374 · comment · before detailText = CreateText(detailArea.transform, "DetailText", 18, TextAnchor.UpperCenter,

```
// Below the frame, which runs 10 to 260 down, and above the action
// buttons, whose tops reach about 380 down.
```

Line 524 · comment · before private void Redraw()

```
// ----- drawing
```

Line 582 · doc comment · before private static string BagContentName(SeedlingBagStatus status, PlantingMaterialInfo info)

```
/// <summary>The seed while it is still in the seed tray, the seedling after that.</summary>
```

Lines 612-614 · comment · before string actionKey = selectedSlot + ":" + status + ":" + sown;

```
// The panel redraws every second for the countdowns; the buttons are
// only rebuilt when they would change, so a tap is never lost to a
// button being replaced under the finger.
```

Lines 697-701 · doc comment · before private void CreateActionButton(string label, Sprite art, UnityEngine.Events.UnityAction action,

```
/// <summary>
/// A round painted button with its caption under it when the art exists,
/// otherwise a written plank - so the painted bag icons get used, and the
/// actions without art still read clearly.
/// </summary>
```

Line 753 · comment · before private void SelectSlot(int slot)

```
// ----- actions
```

Line 816 · comment · before int inRow = Mathf.Min(pickerColumns, held.Count - row * pickerColumns);

```
// Each row is centred, so one or two choices do not sit off to the left.
```

Lines 914-915 · comment · before if (FarmConfirmPopup.Instance != null)

```
// Throwing away a seedling cannot be undone, so it is asked first -
// on the shared Yes/No board, which closes this panel while it asks.
```

Line 949 · comment · before private void SetStatus(string message)

```
// ----- helpers
```

Assets/Scripts/TerrainPreview/Persistence/CropPersistenceMapper.cs

9 comments · 9 comment lines removed · file had 413 lines

Lines 336-339 · doc comment · before private static string RestoredMaterial(CropSaveDto save)

```
/// <summary>
/// The saved material, with an old seed name turned into the material that
/// replaced it. Empty for a crop saved before materials were recorded.
/// </summary>
```

Lines 357-361 · doc comment · before public static void RestoreProtection(GameObject cropObject, CropSaveDto save, GameObject fruitBagPrefab)

```
/// <summary>
/// Puts back fruit bag and drainage kit protection. The drainage bonus
/// itself is already inside the saved drainage figure, so it is not added
/// a second time; only the flag that stops a second kit returns.
/// </summary>
```

Assets/Scripts/TerrainPreview/Persistence/FarmPersistenceManager.cs

19 comments · 19 comment lines removed · file had 312 lines

Lines 11-13 · comment · before public const int CurrentSchemaVersion = 4;

```
// 4: the planting system - the Seedling Tent and its bags, prepared ground,
// and each crop's planting material and protection. Version-3 farms load
// unchanged; they simply start with an empty tent and no prepared ground.
```

Lines 61-66 · doc comment · before public IEnumerator SaveFarm(bool initialSave = false, bool allowDuringTutorial = false)

```
/// <param name="allowDuringTutorial">
/// Set by the marketplace and trade flows, which must sync the farm before
/// the server will validate an offer against it. Refusing those saves left
/// the trade session unprepared, and because the failure only surfaced as a
/// status string the player just saw items refusing to go into the box.
/// </param>
```

Lines 70-74 · comment · before if (TutorialState.IsRunning && !allowDuringTutorial)

```
// Step 14 has the player open the save panel to look at it, and
// Antonio says he will handle the saving himself before he goes - which
// he does, from Finish(), after clearing this flag. A save landing
// mid-tour would also write a half-finished tutorial as the farm's
// permanent state.
```

Lines 167-169 · comment · before TutorialState.Restore(snapshot.tutorial);

```
// Restored first: PlayerInventory and the tutorial director both read
// this while coming up, and a farm saved before the tutorial existed
// must be marked complete before either of them looks.
```

Lines 211–212 · comment · before `if (farmingSystem != null) farmingSystem.RestorePreparedGround(snapshot.preparedPlots);`
// Prepared ground after the crops: a crop on a raised bed lifts its own
// bed on load, and an unplanted bed lifts its ground here.

Assets/Scripts/TerrainPreview/Persistence/FarmSnapshotModels.cs

17 comments · 17 comment lines removed · file had 269 lines

Lines 22–23 · comment · before `public NurserySaveDto nursery = new NurserySaveDto();`
// Added with save version 4. A version-3 farm has neither, and loads
// with a fresh Seedling Tent and no prepared ground.

Line 28 · doc comment · before `[Serializable]`
/// <summary>The Seedling Tent: where it stands and what is in each bag.</summary>

Line 48 · doc comment · before `[Serializable]`
/// <summary>A patch of tilled or prepared ground that has not been planted yet.</summary>

Lines 64–70 · doc comment · before `[Serializable]`
/// <summary>
/// Beginner-guide progress and this farm's dealt seed kinds.
///
/// A farm saved before the tutorial existed has no tutorial block at all; the
/// restore path treats that as "already completed" so a returning player is
/// never ambushed by the tour.
/// </summary>

Lines 191–192 · comment · before `public string plantingMaterial;`
// Added with save version 4; older saves read as unknown material and a
// crop that has been in the field for a long time.

Lines 197–199 · comment · before `public bool hasFruitBag;`
// Fruit bag and drainage kit protection. These were never saved before,
// so a reload silently removed both and let a second drainage kit stack
// another +25% drainage on the same crop.

Line 233 · comment · before `public string climateMitigationType;`
// Climate mitigation fields. Old saves safely deserialize with default values.

Assets/Scripts/TerrainPreview/Persistence/SaveFarmButtonBuilder.cs

3 comments · 3 comment lines removed · file had 77 lines

Lines 35–36 · comment · before `if (saveText != null)`
// Null once the painted icon is in use – the label only exists on the
// plain fallback button, but the button must still lock while saving.

Line 50 · comment · before `saveButton = HudIconButton.Create(`
// Part of the shared round-icon row along the HUD's top-left edge.

Assets/Scripts/TerrainPreview/PestDisease/AreaMitigationTrapInstance.cs

5 comments · 5 comment lines removed · file had 142 lines

Lines 6–10 · doc comment · before `public class AreaMitigationTrapInstance : MonoBehaviour`
/// <summary>
/// Used by Pheromone/Bait Traps and Termite Bait Stations.
/// It continuously applies a database-controlled mitigation
/// to crops inside its radius.
/// </summary>

Assets/Scripts/TerrainPreview/PestDisease/PestDiseaseAffectedCrop.cs

24 comments · 24 comment lines removed · file had 973 lines

Lines 66–68 · comment · before `public FarmCropType CropType`
// =====
// CROP INFORMATION
// =====

Lines 270–272 · comment · before `public ActivePestDisease GetCondition(`
// =====
// ACTIVE CONDITION MANAGEMENT
// =====

Lines 510–512 · comment · before `public void SimulateDisease(float deltaGameDays)`
// =====
// CONDITION SIMULATION
// =====

Lines 687–690 · comment · before `public float aphidsPercent`
// =====
// OLD API COMPATIBILITY
// Keeps AphidTrap, sprayer and adviser compiling.
// =====

Lines 878-880 · comment · before public bool HasAnyPestOrDisease()

```
// =====  
// INSPECTION  
// =====
```

Lines 949-956 · doc comment · before public static string GetInspectionTextFor()

```
/// <summary>  
/// The pest block for a crop, whether or not it has ever been infected.  
///  
/// A crop only gains this component once something takes hold, so this  
/// used to return nothing at all for a healthy plant and the readout  
/// simply had no pest line. "None" is a useful answer; a missing line  
/// reads as the game forgetting to tell you.  
/// </summary>
```

Assets/Scripts/TerrainPreview/PestDisease/PestDiseasePopupUI.cs

8 comments · 8 comment lines removed · file had 204 lines

Line 76 · trailing comment · on the line scaler.MatchWidthOrHeight = 1f; // landscape: scale by height

```
// landscape: scale by height
```

Line 106 · comment · before if (theme?.pestDiseaseLabel != null)

```
// Hanging "Pest & Disease Event" sign above the plank.
```

Lines 140-141 · comment · before Sprite okArt = theme?.pestDiseaseOkayButton != null

```
// Falls back to the Climate Event's Okay art, since it is the same word  
// in the same role - one sprite can serve both popups.
```

Lines 146-148 · comment · before Button okButton = CreateButton(panel.transform, "Okay",

```
// Same board and same defect as the Climate Event popup: the painted edge  
// ends at panel-local y = -169.7, and at -125 the button's base sat at  
// -158, hard against it.
```

Line 174 · comment · before bg.sprite = art;

```
// The word is painted into the art, so no Text child is added.
```

Assets/Scripts/TerrainPreview/PestDisease/PestDiseaseSystem.cs

34 comments · 34 comment lines removed · file had 1018 lines

Lines 8-9 · comment · before [Serializable]

```
// These classes are kept so your existing  
// FarmAdvisorContextBuilder continues compiling.
```

Line 111 · comment · before if (WeatherSystem.Instance != null)

```
// Initialization is safe regardless of Unity Start order.
```

Lines 144-146 · comment · before public void RegisterCrop()

```
// =====  
// CROP REGISTRATION  
// =====
```

Lines 215-217 · comment · before private void EvaluateDailyRisk()

```
// =====  
// DAILY RISK  
// =====
```

Lines 224-226 · comment · before if (FarmGraceperiod.IsCalmWeatherDay)

```
// Days 1 and 2 belong to the new farmer, not to the pests. See  
// FarmGraceperiod for why this is keyed to the date rather than to  
// whether the beginner guide is running.
```

Lines 441-447 · comment · before Debug.Log()

```
// Month, stage and temperature are recorded alongside the  
// appearance so a log can be checked against the rule that  
// allowed it - host crop, peak months, stage window and  
// temperature range are the four hard gates, and without these  
// fields an appearance cannot be audited after the fact. All  
// four values are already in scope; nothing here is computed for  
// the log's sake.
```

Lines 665-666 · comment · before if (rule.type ==

```
// Squash Mosaic Virus becomes much more likely  
// when Aphids are already present.
```

Lines 678-679 · comment · before if (rule.type ==

```
// Fusarium ear/kernel rot becomes more likely  
// after insect damage.
```

Lines 763-765 · comment · before [Obsolete()

```
// =====  
// DEVELOPMENT TIME SKIPS  
// =====
```

Lines 805-808 · comment · before public void ForceEvent()

```
// =====  
// DEVELOPMENT / FORCE EVENTS
```

```
// Keeps existing DevTools buttons working.
// =====
```

Line 836 · comment · before `if (rule == null)`
// Only valid host crops are affected.

Lines 891–893 · comment · before `public AdvisorPestDiseaseData`
// =====
// EXISTING ADVISER COMPATIBILITY
// =====

Assets/Scripts/TerrainPreview/Player/MobileHudBuilder.cs

17 comments · 17 comment lines removed · file had 229 lines

Lines 10–16 · doc comment · before `public static MobileHudBuilder Instance { get; private set; }`

```
/// <summary>
/// The scene's one active joystick/jump HUD. FirstPersonTerrainController
/// reads MoveInput/jumpPressed from a specific instance found via the
/// Inspector or FindFirstObjectByType, so a second, unwired copy left in
/// the scene would silently eat touch input without moving the player -
/// this guard makes any duplicate destroy itself instead.
/// </summary>
```

Lines 21–28 · doc comment · before `public int JumpPressCount { get; private set; }`

```
/// <summary>
/// How many times jump has been pressed this session.
///
/// Watchers such as the beginner guide need to know a jump happened
/// without stealing it: ConsumeJumpPressed clears the flag, so anything
/// polling that would swallow the input before the player controller saw
/// it and the player would never leave the ground.
/// </summary>
```

Line 102 · trailing comment · on the line `scaler.matchWidthOrHeight = 1f; // landscape: scale by height`
// landscape: scale by height

Line 179 · comment · before `image.sprite = jumpArt;`
// The word is painted into the art, so no Text child is added.

Assets/Scripts/TerrainPreview/SocialMarketplace/EconomyRevisionMerge.cs

2 comments · 2 comment lines removed · file had 163 lines

Lines 57–58 · comment · before `foreach (InventoryItemType item in SocialMarketplaceCatalog.MergeItems)`

```
// MergeItems, not only TradableItems: an item missing from this loop
// never reaches the backpack, and the next save would erase it.
```

Assets/Scripts/TerrainPreview/SocialMarketplace/SocialMarketplaceApi.cs

27 comments · 27 comment lines removed · file had 269 lines

Lines 13–32 · doc comment · before `public IEnumerator Heartbeat(Action onSuccess = null, Action<string> onError = null,`

```
/// <summary>
/// Refreshes this device's claim on the account while the farm is open.
///
/// The device id is required, not optional: the server reads a heartbeat
/// with no device as coming from a build that predates device tracking and
/// clears the holder, which would hand the account to any other phone.
/// </summary>
/// <param name="onBusy">
/// Raised on 409, meaning another device has taken this account. Kept apart
/// from <paramref name="onError"/> because the two need opposite handling:
/// a dropped connection must be ignored so a tunnel does not end the game,
/// while a takeover has to end it.
/// </param>
/// <param name="onSignedOut">
/// Raised on 401/403, meaning this device's login is no longer accepted -
/// the account's password was changed elsewhere, which retires every
/// token issued before it. Separate from <paramref name="onError"/> again
/// because it is permanent: no amount of waiting makes this token work,
/// so unlike a dropped connection it has to end the session.
/// </param>
```

Line 165 · doc comment · before `public IEnumerator GetUnseenSales()`
/// <summary>Sales the player has not been shown a notification for yet.</summary>

Line 172 · doc comment · before `public IEnumerator AcknowledgeSale(string listingId,`
/// <summary>Marks one sale as shown so its popup is not repeated.</summary>

Lines 193–197 · doc comment · before `public IEnumerator GetTrade(string tradeId, Action<TradeViewDto> onSuccess, Action<string> onError)`

```
/// <summary>
/// One trade by id, in whatever state it ended. The active trade call only
/// reports PENDING and ACTIVE trades, so this is how a finished trade that
/// dropped out of it is told apart from a cancelled one.
/// </summary>
```

Assets/Scripts/TerrainPreview/SocialMarketplace/SocialMarketplaceCatalog.cs

25 comments · 25 comment lines removed · file had 202 lines

Lines 9–10 · comment · before `public static readonly InventoryItemType[] TradableItems =`

```
// Must match TRADABLE_ITEMS in the backend's EconomyJsonService, minus the
// five old seed items the server still accepts from older game versions.
```

Lines 37–38 · comment · before `InventoryItemType.MulchBag,`

```
// Weather/climate mitigation consumables and kits.
// PruningShears remains excluded because it is an owned/reusable tool.
```

Lines 52–58 · doc comment · before `public static readonly InventoryItemType[] MergeItems = BuildMergeItems();`

```
/// <summary>
/// Every item whose count is merged back from the server after a sale or a
/// trade: the tradable items, plus the five old seed items. A trade with a
/// player still on an older version can hand one of those over; merged in
/// here, it becomes its new material as the backpack loads it, instead of
/// being silently dropped and then overwritten by the next save.
/// </summary>
```

Lines 72–75 · comment · before `private static readonly Dictionary<InventoryItemType, int> BaseValues =`

```
// Per item, in centavos. Seeds mirror the shop's Davao City prices and
// produce mirrors the shipping bin; the rest keep the NPC shop prices.
// The server holds the same table (EconomyJsonService.buildBaseValues) and
// the two must agree, or the listing fee shown is not the fee charged.
```

Line 95 · comment · before `{ InventoryItemType.PineappleSeed, PesoPrice.Centavos(10m) },`

```
// The old seed items, still valued as the server values them.
```

Line 125 · comment · before `{ InventoryItemType.MulchBag, PesoPrice.Centavos(25m) },`

```
// Match the existing NPC shop prices.
```

Line 144 · doc comment · before `public static int GetBaseValueCentavos(InventoryItemType item)`

```
/// <summary>An item's marketplace base value, per item, in centavos.</summary>
```

Lines 150–154 · doc comment · before `public static int CalculateListingFee(InventoryItemType item, int quantity, int askingPrice)`

```
/// <summary>
/// The listing fee: the items' base value in whole pesos, plus the asking
/// price. The base value total is rounded exactly as the server rounds it,
/// so the fee shown here is the fee the server charges.
/// </summary>
```

Lines 164–165 · comment · before `if (PlantingMaterialCatalog.TryGet(item, out PlantingMaterialInfo material))`

```
// Planting materials carry their own names ("Grafted Mango Seedling"),
// which the camel-case split below would get wrong.
```

Assets/Scripts/TerrainPreview/SocialMarketplace/SocialMarketplaceController.cs

121 comments · 121 comment lines removed · file had 864 lines

Line 20 · doc comment · before `public event Action<MarketplaceSaleDto> SaleNotified;`

```
/// <summary>Raised when one of this player's listings has been bought.</summary>
```

Lines 61–63 · comment · before `if (!paused && isActiveAndEnabled)`

```
// Coming back from the background is the likeliest moment to discover a
// takeover: heartbeats stopped while paused, which is exactly what lets
// another device claim the account.
```

Line 70 · comment · before `if (isActiveAndEnabled)`

```
// The backend also treats a player as offline after 45 seconds without a heartbeat.
```

Lines 89–91 · comment · before `SessionTakeover.Raise(takenOver);`

```
// Another device holds the account. Leave the farm rather
// than play on beside it: this device can no longer save,
// so every minute spent here is a minute thrown away.
```

Lines 98–106 · comment · before `Debug.LogWarning("[Session] Heartbeat refused: " + signedOut);`

```
// The account's password was changed somewhere else, which
// ends every session it had. Same reasoning as above – this
// device cannot save any more – but permanent, so the saved
// login goes with it and the player lands on the sign-in form.
//
// The wording is ours, not the server's: a rejected token
// comes back as a bare 401 with the reason in a header the
// client does not read, so quoting it would put "HTTP/1.1
// 401 Unauthorized" on a wooden plank in front of a player.
```

Lines 134–135 · comment · before `yield return new WaitForSecondsRealtime(`

```
// A live trade needs quick updates; the rest of the time poll
// slowly so the radio is not woken every few seconds on mobile.
```

Lines 159–163 · doc comment · before `public IEnumerator RefreshUnseenSales()`

```
/// <summary>
/// Looks for listings of this player's that someone bought while they were
/// elsewhere, raises one notification each, and refreshes the wallet so the
/// sale money is visible immediately rather than after the next reload.
/// </summary>
```

```

Lines 179-180 · comment · before yield return Api.AcknowledgeSale(sale.listingId, () => { }, _ => { });
// Acknowledged only after the event is raised, so a failure here
// means the player sees it again rather than never seeing it.

```

```

Lines 184-186 · comment · before yield return MergeServerEconomy();
// Merged, not copied: the sale only added money on the server, and a
// straight copy of the server's backpack also undid everything the seller
// had harvested, bought or used since their last save.

```

```

Lines 190-203 · doc comment · before public IEnumerator RefreshAdminCommands()
/// <summary>
/// Applies anything a developer has sent this player - a weather event,
/// an outbreak, or money.
///
/// Applied here on the client rather than written into the stored farm on
/// the server, because while a farm is being played this game holds the
/// authoritative copy of it: a change made to the saved snapshot would be
/// overwritten by this client's next save. The weather and pest systems
/// are client-side simulations in any case, so there is nothing on the
/// server that could start a typhoon.
///
/// Errors are swallowed. The endpoint is a developer convenience and a
/// failed poll must never interrupt someone's game.
/// </summary>

```

```

Lines 215-219 · comment · before if (error != null)
// Logged rather than swallowed. This used to fail silently, which
// meant a command that never arrived left no trace anywhere - no way
// to tell a refused request from an empty queue from a build that
// predates this code. The poll still never interrupts play; it just
// says so in the console now.

```

```

Lines 246-248 · comment · before PlayerInventory.Instance.AddMoney(granted);
// AddMoney raises OnInventoryChanged, which is what the
// money plank listens to - so the wallet updates on screen
// without anything here having to touch the HUD.

```

```

Lines 294-296 · comment · before Debug.LogWarning("[Admin] Ignored unknown command " + command.commandType + ".");
// A newer server sending a command this build predates. Ignored
// rather than logged as an error: it is already marked delivered,
// so it will not come back and nothing is stuck.

```

```

Lines 308-312 · comment · before if (requestFailed)
// A poll that never reached the server is not the same as "there is no
// trade" - both used to leave 'trade' null. Since the trade screen now
// closes itself when the session disappears, treating a dropped packet
// as a vanished trade would eject a player mid-negotiation. Keep the
// last known state and try again on the next tick instead.

```

```

Lines 318-326 · comment · before if (trade == null && previous != null && IsOpen(previous))
// The player who confirms FIRST gets a response that is not yet
// COMPLETED, so ConfirmTrade's own refresh never runs for them - the
// server carries out the exchange when the OTHER player confirms. This
// poll is how they find out, but /api/trades/active only reports PENDING
// and ACTIVE trades, so the finished trade never came back here as
// COMPLETED: it simply stopped being listed. Their backpack stayed as it
// was, and every save they made was refused as a revision conflict,
// until reopening the trade screen happened to merge the server's copy
// in. So when an open trade drops out of the list, ask how it ended.

```

```

Lines 334-336 · comment · before if (completed || ended == null)
// An unknown ending - a server without that call, or a dropped
// request - is merged as well. The merge only applies what changed
// on the server, so after a cancelled trade it changes nothing.

```

```

Lines 342-344 · comment · before ActiveTrade = ended;
// Raised before the trade is cleared below, so the trade board
// shows the success message and closes itself, exactly as it
// does for the player who confirmed last.

```

```

Line 353 · comment · before bool justCompleted =
// For a server that does list a trade as COMPLETED here.

```

```

Lines 361-362 · comment · before if (justCompleted)
// Merged before the board is told, so it never reports an updated
// backpack that has not been updated yet.

```

```

Lines 375-381 · doc comment · before private bool serverEconomyMergePending;
/// <summary>
/// Set while the server holds a change to this player's farm - a finished
/// trade, a sold listing - that has not reached the backpack yet. Retried on
/// every poll until it does, because the change only announces itself once:
/// a trade that finished while the connection dropped would otherwise never
/// be brought in, and every save would keep being refused.
/// </summary>

```

```

Lines 384-392 · doc comment · before private IEnumerator MergeServerEconomy()
/// <summary>
/// Brings a change the server made to this player's saved farm into the farm
/// being played.

```

```

///
/// Merged rather than copied over. The server's copy is only as new as the
/// player's last save, so replacing the backpack with it threw away whatever
/// they had harvested, bought or used since. The merge adds only what the
/// server changed on top of what is in the backpack now.
/// </summary>

Line 415 · comment · before Debug.LogWarning("[Economy] Could not fetch the server farm to merge: " + refreshError);
// Nothing was replaced, so the next poll can simply try again.

Lines 431–434 · comment · before Debug.LogWarning("[Economy] " + mergeError +
// The farm revision has already moved on to the server's, so leaving
// the backpack as it is would let the next save write the old items
// back over the exchange. The server's copy wins instead, at the cost
// of anything unsaved in the backpack.

Lines 443–448 · doc comment · before private static bool HasTradeChanged(TradeViewDto a, TradeViewDto b)
/// <summary>
/// True when anything the trade UI displays differs, including the offer
/// contents. Comparing only status/agreed/confirmed meant a player editing
/// their money or items produced no event, so the other side's panel never
/// redrew until it was closed and reopened.
/// </summary>

Lines 777–786 · doc comment · before public IEnumerator UpdateTradeOfferLive(string tradeId, UpdateTradeOfferRequestDto request,
/// <summary>
/// Pushes an offer without first re-saving the whole farm.
///
/// The server validates offers against the SAVED farm, so a save has to
/// happen at least once – but only once, via <see cref="PrepareTradeSession"/>
/// when the trade opens. Inventory cannot change while the player is sitting
/// in the trade screen, so repeating a full farm save on every drag would
/// just make the UI crawl. The exchange itself is re-validated server-side
/// at confirm time regardless.
/// </summary>

Lines 794–797 · doc comment · before public IEnumerator PrepareTradeSession(Action onReady, Action<string> onError)
/// <summary>
/// Saves the farm once so the server's copy matches local inventory before
/// live offer edits begin.
/// </summary>

Lines 840–842 · comment · before if (result.status == "COMPLETED")
// The same merge the other player gets from the poll. A failure is not
// reported as a failed trade – the exchange has already happened on the
// server – it is retried on the next poll instead.

```

Assets/Scripts/TerrainPreview/SocialMarketplace/SocialMarketplaceDtos.cs

4 comments · 4 comment lines removed · file had 180 lines

```

Line 113 · doc comment · before [Serializable]
/// <summary>One "your listing sold" notification for the seller.</summary>

Lines 173–175 · comment · before public string requesterDistrictName;
// Each player's farm district, so a seed can be refused the moment it is
// dragged in. Null from a server older than the district rule; an empty
// string for a farm with no district recorded, which uses the fallback pool.

```

Assets/Scripts/TerrainPreview/SocialMarketplace/SocialMarketplaceUIBuilder.Core.cs

31 comments · 31 comment lines removed · file had 563 lines

```

Line 30 · comment · before private Image profileAddFriendImage;
// This button's art changes with the relationship, so its Image is kept.

Lines 44–45 · comment · before private string chatCursor;
// Incremental chat state. chatCursor is the sentAt of the newest message
// already on screen, so each poll only asks the backend for what is new.

Line 82 · comment · before private readonly Queue<MarketplaceSaleDto> pendingSaleNotifications =
// Two sales in one poll must not overwrite each other's popup.

Line 140 · comment · before searchPlayersButton = HudIconButton.Create(
// Part of the shared round-icon row along the HUD's top-left edge.

Lines 161–165 · doc comment · before private GameObject CreateNotificationBadge(Button host, out Text countText)
/// <summary>
/// A small red count bubble on the corner of a round icon button. The icon
/// has no text label to carry the old "[2 request]" suffix, so unread counts
/// would otherwise become invisible.
/// </summary>

Line 208 · comment · before long unread = controller.UnreadMessages;
// UnreadMessages is a long, so the running total is kept in long too.

Line 213 · comment · before if (searchBadgeRoot != null)
// Round icon: show the count as a badge bubble.

```

```
Line 221 · comment · before if (searchPlayersButtonText != null)
// Plain fallback button: keep the original inline suffix.
```

```
Lines 243-245 · comment · before searchPanel.transform.SetAsLastSibling();
// CreatePanel only orders these at build time; the hotbar is built by
// a different script and can end up later in the canvas, drawing over
// the bottom of the board. Re-parenting on open settles it.
```

```
Lines 263-264 · comment · before if (sellItemsButton != null)
// Selling is off limits until Antonio has finished showing the
// player around, the same as Save Farm and Call Antonio.
```

```
Lines 314-318 · doc comment · before private void CreatePanelTitle(Transform parent, string fallbackText, Sprite art,
/// <summary>
/// A hanging painted sign above a panel. Falls back to the plain centred
/// title text when no art is set, so callers can always use this instead of
/// building a Text themselves.
/// </summary>
```

```
Lines 386-388 · comment · before if (art == null)
// A painted button carries its word in the art, so no Text child is
// added - but callers still receive a label reference, so one is made
// and left blank rather than returning null and breaking their code.
```

```
Lines 443-447 · comment · before placeholderText.verticalOverflow = VerticalWrapMode.Overflow;
// CreateText leaves labels on Truncate so a long player or item name
// stays inside its row. That is right for a label and wrong for a box
// being typed into: with only 4 units of padding here, a large text
// setting can make the line taller than the field, and Truncate drops
// it - the player then types and sees nothing. These two opt out.
```

Assets/Scripts/TerrainPreview/SocialMarketplace/SocialMarketplaceUIBuilder.Marketplace.cs

47 comments · 47 comment lines removed · file had 448 lines

```
Lines 12-14 · comment · before marketplacePanel = CreatePanel("PlayerMarketplacePanel", new Vector2(1400f, 800f));
// Widened from 1180: the six-control top row needed about 980 units, but
// only ~920 sat between the logs, so the prev arrow and Close were pushed
// out onto them at either end.
```

```
Lines 17-18 · comment · before CreatePanelTitle(marketplacePanel.transform, "PLAYER MARKETPLACE",
// PlayerMarketplaceLabel.png is 1360x300 (4.53:1). At 680 it covered 58%
// of the board; this holds it near a third, like the other signs.
```

```
Line 22 · comment · before CreateButton(marketplacePanel.transform, "<", new Vector2(0f, 1f),
// Row of controls sits inside the board's left/right log inset.
```

```
Lines 58-60 · comment · before sellPanel = CreatePanel("SellItemsPanel", new Vector2(1000f, 680f), 0.97f);
// Widened and heightened from 800x640, where the plank was only +/-270
// wide: the quantity and price fields ran to exactly 270 and the buttons
// dropped to -280 against a plank floor of -250.
```

```
Lines 62-63 · comment · before CreatePanelTitle(sellPanel.transform, "SELL ITEMS", Theme?.sellItemsLabel, 360f, 150f, -12f);
// SellItemsLabelMarketplace.png is 1120x300 (3.73:1). At 520 it filled
// 65% of the old board; 360 keeps it near a third like the other signs.
```

```
Lines 66-68 · comment · before CreateButton(sellPanel.transform, "<", new Vector2(0f, 1f),
// Five evenly spaced rows fill the plank from just under the sign down to
// the buttons, so the band of bare wood at the top is used. Row N starts
// at -(80 + N*94); each row is 50 tall, leaving a 44 gap between them.
```

```
Lines 98-99 · comment · before sellFeeText = CreateText(sellPanel.transform, "Fee", 19, TextAnchor.MiddleCenter,
// Still present so errors and results have somewhere to appear, but the
// listing-fee formula block was dropped from the design.
```

```
Line 107 · comment · before CreateButton(sellPanel.transform, "Create Listing", new Vector2(0.5f, 0f),
// Raised from y=40, which put their lower edge past the plank floor.
```

```
Lines 122-126 · doc comment · before private void BuildSaleNotificationPanel()
/// <summary>
/// "DanV bought your items!" - shown once per sale, driven by the
/// controller's unseen-sales poll. Sales are queued so two purchases in the
/// same tick do not overwrite each other.
/// </summary>
```

```
Line 145 · doc comment · before private void OnSaleNotified(MarketplaceSaleDto sale)
/// <summary>Queues a sale so each notification is seen in turn.</summary>
```

```
Lines 193-196 · doc comment · before private static int StepMarketplaceFilter(int index, int direction)
/// <summary>
/// Moves through the filter list, where -1 is ALL ITEMS, passing over seeds
/// this farm's district does not grow - there is nothing to find under them.
/// </summary>
```

```
Lines 225-231 · comment · before if (TutorialState.IsRunning)
// Nothing from other players is shown until the tour is over.
//
// Selling was already blocked during the tutorial, so a board full of
// strangers' goods was a shop the player could only buy from - and
```

```
// buying there spends the starting money before Antonio has explained
// what it is for. Returning early also means no listings request is
// made at all, rather than fetching them and hiding them.
```

Lines 262–265 · comment · before listings.RemoveAll(listing =>

```
// Seeds this farm's district does not grow are left out before anything is
// drawn. The player's own listings stay, whatever they hold, because those
// still need their Cancel or Claim button. The server filters the same way;
// this also covers an older server that does not.
```

Line 289 · comment · before Sprite buttonArt = captured.mine

```
// Own listing -> Cancel art; someone else's -> Buy art.
```

Lines 313–316 · comment · before if (TutorialState.IsRunning)

```
// Listing an item writes the farm to the backend, and the beginner
// guide must be the only thing that saves during the tour - a save
// mid-tour would store a half-finished tutorial as this farm's
// permanent state. The button is greyed out too; this is the backstop.
```

Lines 351–352 · comment · before sellFeeText.text = "Listing fee: P" + fee;

```
// The formula breakdown was removed; the fee total is still shown so
// the player is not charged an unexplained amount.
```

Lines 433–434 · comment · before return PlantingMaterialCatalog.TryParseItem(itemType, out InventoryItemType item)

```
// Any known item gets its proper name - including the old seed items a
// player on an older version may still list.
```

Assets/Scripts/TerrainPreview/SocialMarketplace/SocialMarketplaceUIBuilder.PlayersChat.cs

81 comments · 81 comment lines removed · file had 613 lines

Line 10 · comment · before private const float ChatPollMinInterval = 2f;

```
// Chat polls fast while a conversation is active and backs off when idle.
```

Lines 14–18 · doc comment · before private const float SocialBoardLogInset = 150f;

```
/// <summary>
/// SocialBoard.png is 9-sliced with 130px rolled-log caps, so the flat plank
/// only begins this far in from either edge. Controls are inset past it
/// instead of being drawn over the log art.
/// </summary>
```

Lines 21–24 · doc comment · before private const float RowRopeInset = 105f;

```
/// <summary>
/// SocialLisRow.png paints a rope loop near each end. This keeps a row's
/// text and button between them rather than on top of them.
/// </summary>
```

Lines 27–33 · doc comment · before private const float ChatBoardLogInset = 210f;

```
/// <summary>
/// ChatBoard.png needs a deeper inset than SocialBoard.png despite both
/// declaring a 130px border. On SocialBoard the logs sit clear of the plank,
/// so 130 is honest. On ChatBoard the logs overlap the plank and their art
/// runs on past the border into the stretchable region - about x179 once the
/// board is stretched - so anything placed at 150 landed on the log.
/// </summary>
```

Lines 38–39 · comment · before searchPanel = CreatePanel("PlayerSearchPanel", new Vector2(1400f, 760f));

```
// Widened from 1050: at that width the input plus both buttons did not
// fit between the logs, which pushed Request Send onto the right log.
```

Lines 42–46 · comment · before CreatePanelTitle(searchPanel.transform, "SEARCH PLAYERS / FRIENDS",

```
// searchFriendLabel.png is 1400x300 (4.67:1); preserveAspect keeps it
// undistorted, so this width just sets the sign's overall size. Kept
// near a third of the board width, matching the shop sign's proportion.
// The negative offsetY sinks it into the board: a smaller sign overlaps
// the top edge less, so at the default +6 it floated free of the board.
```

Line 50 · comment · before playerSearchInput = CreateInput(searchPanel.transform, "Enter player display name...",

```
// Inset so the fields clear the board's rolled log edges.
```

Lines 72–74 · comment · before CreateButton(searchPanel.transform, "Close", new Vector2(1f, 0f),

```
// The Trade button that sat here is gone. A PENDING trade now raises the
// trade-request popup via OnActiveTradeChanged, which reaches the player
// wherever they are rather than only while this panel is open.
```

Lines 84–85 · comment · before profilePanel = CreatePanel("PlayerProfilePanel", new Vector2(1100f, 620f));

```
// Widened from 820: the three action buttons are 210 wide each, which at
// the old width left them overlapping rather than merely touching.
```

Lines 87–92 · comment · before CreatePanelTitle(profilePanel.transform, "PLAYER INFORMATION",

```
// PlayerInformationLabel.png is 1400x300 (4.67:1), the same very wide art
// as the search sign. At 620 on an 820 board it covered three quarters of
// the width, which is what read as stretched.
// Sunk further so the sign covers the board's 70px top log cap. The art
// is opaque over almost its whole canvas, so at a shallower offset its
// lower edge stopped short of the log and the sign read as floating.
```

Lines 100-105 · comment · before `profileAddFriendButton = CreateButton(profilePanel.transform, "Add Friend",`
`// Art for this one is swapped at runtime between Add Friend / Request`
`// Sent / Friends, so the sprite is applied in OpenPlayerProfile instead`
`// of being fixed here.`
`// Three 210-wide buttons across the plank, with a 65px gap between each`
`// and 40px clear of the log caps on either side. Raised from y=48, which`
`// had them straddling the board's bottom log.`

Lines 119-120 · comment · before `CreateButton(profilePanel.transform, "Back", new Vector2(1f, 1f),`
`// Tucked into the plank's top-right corner: 20px inside the right log cap`
`// (130) and 20px below the top cap (70), so it sits on wood, not on log.`

Lines 133-134 · comment · before `chatPanel = CreatePanel("PlayerChatPanel", new Vector2(1350f, 720f),`
`// Widened again so the deeper ChatBoardLogInset does not cost content`
`// width: 1350 - 2*210 leaves 930, slightly more than the previous 900.`

Line 138 · comment · before `chatTitleText = CreateText(chatPanel.transform, "Title", 28, TextAnchor.MiddleLeft,`
`// Name and status only, sitting inside the board's left inset.`

Lines 168-170 · comment · before `if (TutorialState.IsRunning)`
`// Other farmers stay hidden until the tour is over. Refused here`
`// rather than after the request so no search for a real display name`
`// ever leaves the device during the tutorial.`

Lines 328-331 · doc comment · before `private void ApplyFriendButtonState(string relationship)`
`/// <summary>`
`/// Swaps the friend button between its three painted states. With no art`
`/// set it falls back to changing the plain text label instead.`
`/// </summary>`

Lines 408-409 · comment · before `HideMainPanels();`
`// Hide the other panels first.`
`// HideMainPanels calls CloseChatPanel, which clears chatPlayer.`

Line 412 · comment · before `chatPlayer = profile;`
`// Assign the player only after HideMainPanels finishes.`

Line 415 · comment · before `chatTitleText.text =`
`// Name and status only.`

Line 422 · comment · before `chatCursor = null;`
`// Fresh conversation view: no cursor yet, nothing rendered yet, poll fast.`

Lines 460-462 · comment · before `string cursor = chatCursor;`
`// Only the very first fetch pulls the whole history. After that the`
`// cursor makes the backend return just the messages we do not have,`
`// which is normally an empty list.`

Line 481 · comment · before `chatPollInterval = Mathf.Min(chatPollInterval * 2f, ChatPollMaxInterval);`
`// Idle conversation: gradually slow the poll down to save battery.`

Line 486 · comment · before `chatPollInterval = ChatPollMinInterval;`
`// New activity, so go back to the responsive interval.`

Lines 495-496 · comment · before `chatStatusText.text = "";`
`// The status line only surfaces errors and the empty-conversation hint`
`// now; the offline-storage note was removed from the design.`

Lines 499-500 · comment · before `yield return controller.RefreshNotifications();`
`// Reading the conversation clears unread messages server-side, so only`
`// refresh the badge when something actually arrived - not every tick.`

Lines 514-516 · comment · before `VerticalLayoutGroup layout = row.GetComponent<VerticalLayoutGroup>();`
`// The bubble grows with its text instead of using a fixed row height,`
`// so a long message wraps to two or three lines and the plank stretches`
`// to match rather than clipping.`

Lines 522-524 · comment · before `layout.padding = bubbleArt != null`
`// ChatBubble.png is 9-sliced {90,40,90,40}. The rope loops live inside`
`// the 90px left/right caps, so the text has to start past 90 - the old`
`// 46 put it straight over them.`

Line 526 · trailing comment · on the line `? new RectOffset(105, 105, 32, 32) // clear the rope ends`
`// clear the rope ends`

Lines 534-537 · comment · before `element.minHeight = bubbleArt != null ? 132f : 60f;`
`// The sprite is 172 tall with 40px top and bottom caps, leaving a 92px`
`// stretchable band. At the old 86 that band was crushed to 6px, which is`
`// what made the plank look squashed. 132 keeps it at a healthy 52px, and`
`// because this is only a MINIMUM the bubble still grows for long text.`

Lines 594-595 · comment · before `chatPollInterval = ChatPollMinInterval;`
`// Let the normal incremental path pick the new message up, so it is`
`// rendered once and the cursor advances with the stored timestamp.`

Assets/Scripts/TerrainPreview/SocialMarketplace/SocialMarketplaceUIBuilder.Trade.cs

180 comments · 180 comment lines removed · file had 1181 lines

Line 10 · trailing comment · on the line `private const int OfferSlotCount = 4; // the 2x2 grid`
`// the 2x2 grid`

Line 33 · comment · before `private GameObject tradeRequestPopup;`
`// The trade-request popup shown to the receiving player.`

Line 44 · comment · before `private GameObject tradeSeedNoticePanel;`
`// The popup for offering a seed the other player's district does not grow.`

Lines 50–53 · doc comment · before `private const string SeedUnavailableMarker = "available to the other player";`
`/// <summary>`
`/// Part of the server's refusal wording, which is the notice word for word.`
`/// Matching on it lets a refusal from the server open the same popup.`
`/// </summary>`

Line 56 · comment · before `private const float TradeBoardInset = 150f;`
`// ----- build`

Lines 58–62 · doc comment · before `private const float TradeBoardInset = 150f;`
`/// <summary>`
`/// Board inset for the trade panel. It uses socialBoard, whose logs sit`
`/// clear of the plank, so the declared 130px border plus a 20px margin is`
`/// enough - unlike ChatBoard, which needs the deeper ChatBoardLogInset.`
`/// </summary>`

Lines 67–69 · comment · before `tradePanel = CreatePanel("PlayerTradePanel", new Vector2(1600f, 980f),`
`// Taller and wider than before: the old 1450x820 could not fit the`
`// inventory grid, the money row and the buttons without them stacking`
`// on top of each other, which is why everything was crushed downward.`

Lines 80–81 · comment · before `BuildOfferArea(isMine: true, anchoredY: -70f);`
`// Both offer areas start just under the board's top log cap so the`
`// column fills the space that used to sit empty above them.`

Lines 96–97 · comment · before `tradeStatusText = CreateText(tradePanel.transform, "Status", 18, TextAnchor.MiddleCenter,`
`// Raised into the gap between Cancel and Set Trade instead of sitting`
`// under both, where the hotbar clipped it.`

Line 102 · comment · before `tradeSummaryText = tradeStatusText;`
`// Unused by the new layout but kept so older code paths still compile.`

Lines 118–119 · comment · before `rect.anchoredPosition = new Vector2(TradeBoardInset, -70f);`
`// Raised to sit directly under the top log cap; it used to start 180`
`// down, which pushed its lower half over the Add Money plank.`

Lines 140–143 · comment · before `tradeInventoryContent.sizeDelta = new Vector2(580f, 440f);`
`// Measured against the painted compartments in TradeInventoryUI.png:`
`// six columns at ~96.7 and six rows at ~73.3, i.e. the region inside the`
`// sprite's 60px border, sitting ~9px above the rect centre. The old`
`// 620x490 centred gave the wrong pitch, so cells drifted off their boxes.`

Lines 148–152 · doc comment · before `private void BuildOfferArea(bool isMine, float anchoredY)`
`/// <summary>`
`/// One player's offer area: name plank, 2x2 item grid, money plank and the`
`/// log/lock state icon. The bottom copy mirrors the other player and is`
`/// display-only.`
`/// </summary>`

Line 173 · comment · before `GameObject offerGrid = new GameObject(prefix + "OfferGrid", typeof(RectTransform), typeof(Image));`
`// 2x2 grid.`

Lines 179–183 · comment · before `gridRect.sizeDelta = new Vector2(280f, 280f);`
`// TradeOffer*.png is 440x440 9-sliced {60,60,60,60}. Those caps are drawn`
`// at native size, so at the old 220 they ate most of the rect and the`
`// painted 2x2 boxes were tiny. At 280 the inner region is 160, putting`
`// the compartment centres at +/-40 - which is what the cell maths below`
`// now uses, instead of the old 96/55 that overshot the boxes entirely.`

Line 200 · comment · before `GameObject moneyPlank = new GameObject(prefix + "MoneyPlank", typeof(RectTransform), typeof(Image));`
`// Money plank.`

Lines 206–211 · comment · before `moneyRect.sizeDelta = new Vector2(270f, 66f);`
`// TradeMoney*.png is 460x112 with an asymmetric border {200,30,30,40}:`
`// the money-bag art fills the fixed 200px left cap. Sliced at 230x56 the`
`// horizontal middle collapsed to 30px while the caps stayed full size, so`
`// the bag swallowed the plank - and 56 was below the 70px of vertical`
`// border, squashing it further. Drawn Simple with preserveAspect instead,`
`// the whole sprite scales evenly and keeps its shape at any size.`

Lines 217–218 · comment · before `Text moneyText = CreateText(moneyPlank.transform, "Money", 22, TextAnchor.MiddleCenter,`
`// The bag occupies the left ~44% of the art, so the amount is placed over`
`// the plank to its right rather than centred across the whole sprite.`

Line 227 · comment · before `GameObject icon = new GameObject(prefix + "StateIcon", typeof(RectTransform), typeof(Image));`
`// Log / lock state icon.`

```

Lines 232-235 · comment · before iconRect.pivot = new Vector2(0.5f, 0.5f);
// Centre pivot: TradeSpinner rotates the transform, and rotation happens
// about the pivot. At the old (1,1) the log swung around its own top-right
// corner instead of turning on the spot. anchoredPosition is now the
// icon's centre rather than its corner.

Lines 238-240 · comment · before iconRect.anchoredPosition = new Vector2(-285f, anchoredY - 212f);
// Grid and icon are treated as one 410-wide group centred under the name
// and money planks above them, rather than being pushed to the outer
// edges with a wide gap between.

Line 260 · comment · before Image background = slot.GetComponent<Image>();
// Transparent but still raycastable, so it can receive drops.

Lines 305-307 · comment · before rect.anchorMin = rect.anchorMax = new Vector2(0f, 1f);
// Anchored to the panel's top-left like the inventory grid above it, so
// the two stack predictably instead of being measured from opposite
// edges - which is how they ended up overlapping.

Lines 310-315 · comment · before rect.sizeDelta = new Vector2(600f, 64f);
// TradeAddMoneyPlank.png is 1400x148 (9.46:1). Its declared 9-slice border
// is only 60, but the rope loops sit at x89-111 and x1304-1325 - outside
// the caps, in the stretchable middle. Slicing therefore squeezed the
// ropes along with the wood and moved them unpredictably. Drawn Simple
// with preserveAspect the whole plank scales evenly and the ropes land at
// a known place: at 600 wide (scale 0.4286) they occupy 38-48 and 559-568.

Lines 332-333 · comment · before CreateButton(tradePanel.transform, "Enter", new Vector2(0f, 1f),
// Enter sits beside the plank, not on it - inside the plank its right
// edge landed on the rope. Placed clear of the lower offer grid's column.

Lines 343-344 · comment · before CreatePanelTitle(tradeConfirmPanel.transform, "TRADING",
// Sunk into the board like the other signs; at the default +6 it sat too
// high and left a strip of the top log showing beneath it.

Lines 351-353 · comment · before tradeConfirmAcceptButton = CreateButton(tradeConfirmPanel.transform, "Accept",
// TradeConfirmBoard.png is 9-sliced {130,70,130,70}, so on an 880x660
// panel the plank floor is y=-260. At the old y=55 the buttons reached
// -275 and hung over the bottom log; 90 puts them 20 clear of it.

Line 365 · doc comment · before private void BuildTradeRequestPopup()
/// <summary>The incoming "X wants to trade with you!" popup.</summary>

Lines 370-372 · comment · before CreatePanelTitle(tradeRequestPopup.transform, "TRADE REQUEST",
// TradeRequestLabel.png is 920x240 (3.83:1). At 460 it spanned 60% of the
// 760-wide popup, which is what read as stretched; the negative offsetY
// keeps the smaller sign seated on the board.

Line 406 · comment · before private void OnActiveTradeChanged(TradeViewDto trade)
// ----- lifecycle

Line 417 · comment · before StopOfferPushLoop();
// The session is gone, so nothing may keep pushing offers at it.

Lines 425-432 · comment · before bool finishedNormally = previous != null && previous.status == "COMPLETED";
// /api/trades/active only reports PENDING and ACTIVE sessions,
// so the moment the other player cancels it answers null rather
// than a CANCELLED view - which meant the CANCELLED branch below
// never ran for them and they were left staring at a dead panel
// with no way out. Close it here instead.
//
// A trade that finished normally is the exception: its result is
// left on screen for the player to read and dismiss themselves.

Lines 436-438 · comment · before FinishAndCloseTrade();
// This is the first player to have accepted: their own
// request came back with the trade still active, and it is
// this poll that discovers the other side finished it.

Line 449 · comment · before bool amTarget = trade.targetId == CurrentUserId;
// A PENDING trade the current player did not start is an invitation.

Lines 469-474 · comment · before if ((trade.status == "CANCELLED" || trade.status == "DECLINED") &&
// A cancel/decline from either side leaves nothing to look at, so the
// screen closes for BOTH players - this handler runs on every device
// via the poll, not just the one that pressed the button. COMPLETED is
// left open on purpose: the presser needs to see the success message,
// and Cancel Trade below now closes it safely once there is nothing
// left to cancel.

Line 487 · comment · before if (tradeRequestShownForId == trade.id && tradeRequestPopup.activeSelf)
// Do not re-open the popup the player just dismissed for this trade.

Lines 509-511 · comment · before tradePanel.transform.SetAsLastSibling();
// The hotbar is built by a different script and can land later in the
// canvas hierarchy, drawing over the Cancel / Set Trade row. Re-parenting
// to last on open keeps the trade board above it.

```

```

Lines 523-526 · doc comment · before private IEnumerator PrepareThenStartLiveOffers()
/// <summary>
/// Saves the farm once so the server can validate live offer edits, then
/// starts the debounced push loop.
/// </summary>

Lines 540-545 · comment · before tradePrepareError = string.IsNullOrEmpty(error)
// Remembered so a later drag can explain itself instead of
// reporting success it did not achieve, and left un-prepared so the
// next drag retries. Without the retry a single failed sync killed
// trading for the whole session: the push loop never started, so
// drags only ever updated the local dictionary and the offer box
// stayed empty no matter what the player did.

Lines 577-581 · doc comment · before private IEnumerator OfferPushLoop()
/// <summary>
/// Pushes the local offer shortly after it changes. Debounced so a burst of
/// drags becomes one request instead of several, while still reaching the
/// other player quickly.
/// </summary>

Lines 625-628 · comment · before pendingTradeItems.Clear();
// The server refused a seed this device could not check - an
// older server that did not send the other player's district.
// Put the box back to what the server last accepted, so the
// seed is out of the offer here as well.

Line 640 · comment · before private TradeOfferDto MyOffer(TradeViewDto trade)
// ----- rendering

Line 706 · comment · before if (active && myAgreed && otherAgreed &&
// Both locked in: time for the final give/get confirmation.

Line 754 · comment · before TradeSpinner spinner = image.GetComponent<TradeSpinner>();
// Only the unlocked log spins; a padlock stays upright.

Line 770 · doc comment · before private void RenderTradeInventory()
/// <summary>Draws the player's own backpack as drag sources.</summary>

Lines 836-839 · doc comment · before private Sprite InventoryIcon(InventoryItemType item)
/// <summary>
/// Borrows the backpack's icon table. Cached because this is called once per
/// visible cell, and a scene-wide search per item would be wasteful.
/// </summary>

Line 848 · comment · before private void OnItemDroppedIntoOffer(InventoryItemType item)
// ----- interactions

Lines 855-857 · comment · before if (!IsSeedAvailableToPartner(item))
// A seed the other player's district does not grow never goes into the
// box - it would hand them a seed they have no other way to get. It stays
// in the backpack, which is where it was all along.

Line 874 · comment · before if (already == 0 && pendingTradeItems.Count >= OfferSlotCount)
// A new item needs a free cell; the 2x2 grid only holds four stacks.

Lines 886-888 · comment · before tradeStatusText.text = tradePrepareError ?? "Preparing trade...";
// Nothing is pushing offers yet, so saying "offer updated" would be
// a lie - the other player would never see it. Say what is actually
// wrong and try the sync again.

Lines 897-902 · doc comment · before private bool IsSeedAvailableToPartner(InventoryItemType item)
/// <summary>
/// Whether the other player's district grows the item. Anything that is not a
/// crop seed is always allowed. When the server did not say which district
/// the other player is in, this does not guess: the server still checks, and
/// its refusal comes back through PushOffer.
/// </summary>

Lines 930-933 · doc comment · before private void BuildTradeSeedNotice()
/// <summary>
/// Built the first time it is needed rather than with the rest of the trade
/// screen, so a player who never offers the wrong seed never pays for it.
/// </summary>

Line 978 · comment · before StartCoroutine(SetTradeSequence());
// Flush any queued edits before locking, so the agreed offer is current.

Lines 1009-1013 · comment · before bool alreadyAccepted = displayedTrade.requesterId == CurrentUserId
// Pressable again each time the board opens, so a declined-and-reopened
// confirmation is not stuck showing the previous attempt's state - but
// not once this player has accepted. The first player to accept gets the
// trade back still ACTIVE, which closes and reopens this board, and it
// used to reopen with Accept lit again while they waited on the other.

Lines 1028-1031 · comment · before if (tradeConfirmAcceptButton != null)
// Greyed the moment it is pressed. The first player to accept then
// waits on the second, and with the button still lit there was no way

```

```
// to tell a press that registered from one that did not - so players
// pressed it again while the first request was still in flight.
```

Lines 1043-1047 · comment · before if (value != null && value.status == "COMPLETED")

```
// The second player to accept gets COMPLETED back from their
// own request rather than from a later poll, and nothing here
// used to close the trade board for them - so they were left
// sitting on a finished trade while the other player's board
// had already closed.
```

Line 1053 · comment · before if (tradeConfirmAcceptButton != null)

```
// Pressable again, or a failed accept leaves the board dead.
```

Lines 1060-1068 · doc comment · before private void FinishAndCloseTrade()

```
/// <summary>
/// Shows the result for a moment, then closes the trade board.
///
/// Both players end up here - one from their own confirm response, the
/// other from the poll that finds the session gone - so both boards close
/// on their own. The board used to be left open for the player to dismiss
/// with Cancel, which technically worked but asked somebody to press
/// "cancel" on a trade that had already succeeded.
/// </summary>
```

Lines 1082-1083 · comment · before RenderTradeInventory();

```
// The backpack column still showed the items traded away while the
// result was on screen, for the player whose trade finished by poll.
```

Lines 1094-1095 · comment · before yield return new WaitForSecondsRealtime(2.5f);

```
// Long enough to read the line above, short enough that neither
// player is left waiting on the other's screen.
```

Lines 1139-1143 · comment · before if (displayedTrade.status != "ACTIVE" && displayedTrade.status != "PENDING")

```
// COMPLETED here means the presser is just dismissing the success
// screen - the trade already exists on the server as done, so hitting
// the cancel endpoint would only bounce back "this trade is already
// closed." This button doubles as the close action whenever there is
// nothing left to actually cancel.
```

Line 1170 · doc comment · before public class TradeSpinner : MonoBehaviour

```
/// <summary>Spins a RectTransform; used for the unlocked "log" trade state.</summary>
```

Assets/Scripts/TerrainPreview/SocialMarketplace/TradeDragSource.cs

15 comments · 15 comment lines removed · file had 113 lines

Lines 8-12 · doc comment · before public class TradeDragSource : MonoBehaviour, IBeginDragHandler, IDragHandler, IEndDragHandler

```
/// <summary>
/// An inventory entry inside the trade screen that can be dragged onto one of
/// the player's own trade slots. Dropping is handled by <see cref="TradeDropSlot"/>;
/// this side only carries the item identity and draws the drag ghost.
/// </summary>
```

Line 18 · doc comment · before public static InventoryItemType Dragging { get; private set; } = InventoryItemType.None;

```
/// <summary>The item currently being dragged, or None when nothing is.</summary>
```

Line 69 · comment · before Dragging = InventoryItemType.None;

```
// Cleared after the frame's drop handlers have run.
```

Lines 74-77 · doc comment · before public class TradeDropSlot : MonoBehaviour, IDropHandler, IPointerClickHandler

```
/// <summary>
/// One cell of a player's 2x2 trade area. Accepts a dragged inventory item,
/// and a plain click removes one unit again.
/// </summary>
```

Line 80 · doc comment · before public Action<InventoryItemType> onItemDropped;

```
/// <summary>Raised with the dropped item; the panel decides whether it fits.</summary>
```

Line 83 · doc comment · before public Action<InventoryItemType> onSlotClicked;

```
/// <summary>Raised when a filled slot is clicked, to remove one unit.</summary>
```

Line 86 · doc comment · before public InventoryItemType occupant = InventoryItemType.None;

```
/// <summary>What this cell currently shows, or None when empty.</summary>
```

Line 89 · doc comment · before public bool interactable = true;

```
/// <summary>False for the other player's slots, which are display-only.</summary>
```

Assets/Scripts/TerrainPreview/Soil/SoilGridsJsonParser.cs

6 comments · 6 comment lines removed · file had 86 lines

Line 54 · comment · before case "sand":

```
// SoilGrids sand/silt/clay often come in tenths of percent
```

Line 60 · comment · before case "phh2o":

```
// pH often comes in tenths
```

Line 64 · comment · before `case "soc":`

`// organic carbon: compress to a game-friendly range`

Line 68 · comment · before `case "cfvo":`

`// coarse fragments volume: tenths of percent works well`

Line 72 · comment · before `case "bdod":`

`// bulk density: keep it around your existing gameplay thresholds`

Line 76 · comment · before `case "nitrogen":`

`// nitrogen: compress to a readable gameplay range`

Assets/Scripts/TerrainPreview/Soil/SoilGridsService.cs

3 comments · 3 comment lines removed · file had 79 lines

Line 26 · comment · before `onDone?.Invoke(new SoilSample`

`// fallback fake sample so your prototype still works`

Lines 46–47 · comment · before `SoilSample sample = SoilGridsJsonParser.Parse(json);`

`// You should parse this with Newtonsoft Json package in Unity.`

`// Package Manager -> Add package -> Newtonsoft Json`

Assets/Scripts/TerrainPreview/Soil/SoilInfoPanelBuilder.cs

51 comments · 51 comment lines removed · file had 217 lines

Lines 6–20 · doc comment · before `public class SoilInfoPanelBuilder : MonoBehaviour`

```
/// <summary>
/// The crop inspection readout on the left of the farm HUD.
///
/// It stays hidden until the player actually inspects a crop - players can no
/// longer read raw soil chemistry off the bare ground, so there is nothing to
/// show until a plant is clicked. Art comes from the shared
/// <see cref="UIThemeSprites"/> asset and is optional.
///
/// The readout scrolls. It used to be a single truncating label sized for
/// eight lines, which meant that raising the text size silently dropped
/// whatever no longer fitted - fertility and soil suitability simply vanished
/// off the bottom, with nothing to tell the player they existed. It is now a
/// scroll view whose text overflows rather than truncates, so a longer readout
/// or a larger font makes it scrollable instead of making it incomplete.
/// </summary>
```

Lines 29–33 · doc comment · before `private const float PanelHeight = 310f;`

```
/// <summary>
/// Tall enough for most of the readout without scrolling, and short enough
/// that the panel plus its button still clear the bottom of the screen at
/// the largest UI scale, where the canvas is only 900 units high.
/// </summary>
```

Line 53 · trailing comment · on the line `scaler.matchWidthOrHeight = 1f; // landscape: scale by height`

`// landscape: scale by height`

Lines 67–68 · comment · before `pRect.sizeDelta = board != null`

`// The board is 9-sliced, so any aspect is fine - only the middle`
`// stretches, the logs stay correct.`

Lines 86–88 · comment · before `float padX = board != null ? 110f : 16f;`

`// Must clear the sprite's 9-slice border, which renders at its native`
`// pixel size (L/R 95, T/B 45) no matter how the panel is scaled - a`
`// smaller inset than that would run the text onto the rolled logs.`

Line 97 · comment · before `panelRoot.SetActive(false);`

`// Nothing to show until a crop is inspected.`

Lines 101–109 · doc comment · before `private void BuildScrollingReadout(Transform parent, float padX, float padY)`

```
/// <summary>
/// The readout, as scroll content rather than a fixed label.
///
/// The label IS the content: stretched horizontal anchors give it exactly
/// the viewport's width, and the size fitter reports its real height to the
/// scroll rect. Overflow rather than Truncate is the part that matters -
/// a truncating label drops whole lines once the font grows, and the
/// player has no way to know anything is missing.
/// </summary>
```

Lines 158–162 · doc comment · before `private void BuildFieldGuideButton(Transform parent, UIThemeSprites theme)`

```
/// <summary>
/// The button under the board that explains what the numbers mean. Built
/// through <see cref="DescriptionBoard"/> so it is the same plank the shop
/// and the district map use.
/// </summary>
```

Line 170 · comment · before `RectTransform rect = button.GetComponent<RectTransform>();`

`// Hangs just under the board's bottom edge, centred on it.`

Line 183 · doc comment · before `public void ShowInfo(string text)`

```
/// <summary>Fills the panel and reveals it.</summary>
```

Line 186 · comment · before `if (TutorialState.CropInspectionLocked)`

```
// The beginner guide keeps this shut until it teaches inspection.
```

Lines 196–197 · comment · before `if (scroll != null)`

```
// A new crop starts at the top rather than wherever the last readout  
// was left scrolled to.
```

Line 202 · doc comment · before `public void HideInfo()`

```
/// <summary>Hides the panel until the next crop inspection.</summary>
```

Lines 208–211 · comment · before `if (fieldGuide != null)`

```
// The field guide is parented to the canvas rather than the panel, so  
// that it can centre on the screen instead of on a panel pinned to the  
// left edge. That means it does not inherit the panel's hiding, and  
// has to be closed here or it would outlive the board it explains.
```

Assets/Scripts/TerrainPreview/Terrain/DistanceCullable.cs

13 comments · 13 comment lines removed · file had 189 lines

Lines 5–11 · doc comment · before `[DisallowMultipleComponent]`

```
/// <summary>  
/// Marks a spawned farm object for distance-based render culling.  
/// Only the object's renderers are toggled – the GameObject and every  
/// script on it stay active, so growth timers, mitigation effects,  
/// selling, collection, etc. keep running while the object is hidden.  
/// Attach one per spawned root via <see cref="Attach"/>.  
/// </summary>
```

Line 15 · doc comment · before `public static DistanceCullable Attach(GameObject go)`

```
/// <summary>Adds the component to <paramref name="go"/> if it does not already have one.</summary>
```

Line 90 · doc comment · before `public void Recache()`

```
/// <summary>Re-reads renderers/bounds after the object's visual hierarchy changes.</summary>
```

Lines 102–103 · comment · before `NeutralizeBillboardLOD();`

```
// Let this culler own visibility for trees; disable the billboard  
// sprite-swap so they fully disappear instead of turning into a flat sprite.
```

Lines 145–146 · comment · before `if (r == null || !r.gameObject.activeInHierarchy)`

```
// Skip inactive renderers (e.g. a dormant billboard child) so  
// their degenerate bounds don't inflate the cull radius.
```

Assets/Scripts/TerrainPreview/Terrain/DistanceCullingManager.cs

6 comments · 6 comment lines removed · file had 152 lines

Lines 6–11 · doc comment · before `public class DistanceCullingManager : MonoBehaviour`

```
/// <summary>  
/// Central, throttled distance-based renderer culler for farm content.  
/// Registered objects have only their renderers toggled (never their  
/// GameObject or scripts), so all gameplay logic keeps running while hidden.  
/// Lazily created the first time a <see cref="DistanceCullable"/> registers.  
/// </summary>
```

Assets/Scripts/TerrainPreview/Terrain/SoilAwareTerrainGenerator.cs

50 comments · 50 comment lines removed · file had 667 lines

Lines 49–52 · doc comment · before `public void ShowMessage(string text)`

```
/// <summary>  
/// Shows a farming feedback message (dug ground, planted seed, warnings)  
/// on the same panel used for crop inspection.  
/// </summary>
```

Lines 58–61 · doc comment · before `private void SetInspectionText(string text)`

```
/// <summary>  
/// Writes a crop inspection readout and reveals the panel. The panel is  
/// hidden the rest of the time – raw ground soil is no longer inspectable.  
/// </summary>
```

Line 189 · comment · before `if (FarmPersistenceManager.Instance != null)`

```
// The inspection panel stays hidden until the player clicks a crop.
```

Line 218 · comment · before `layer.metallic = 0f;`

```
// IMPORTANT: make terrain look like rough grass, not shiny plastic/water
```

Lines 223–228 · comment · before `TemporaryTerrainGenerator.ClearSmoothnessAlpha(layer.diffuseTexture as Texture2D);`

```
// TerrainLit reads gloss as albedo.alpha * _Smoothness0, and the engine  
// rewrites _Smoothness0 for us, so the line above is not on its own  
// enough to stop the ground mirroring the sky. Clearing the alpha makes  
// the product zero regardless. Skipped for a non-readable imported  
// texture, which cannot be edited at runtime – set its Alpha Source to  
// "None" in the importer instead.
```

Lines 518-525 · doc comment · before `public void InspectAtScreenPosition(Vector2 screenPosition)`

```
/// <summary>
/// Tapping bare ground no longer reports raw soil chemistry - only crops
/// can be inspected. This clears the panel so a previous crop's readout
/// does not linger after the player looks away.
///
/// Soil data itself is untouched: <see cref="TryGetSoilAtWorldPosition"/>
/// and the terrain/soil generation still use the full samples internally.
/// </summary>
```

Lines 551-559 · doc comment · before `public static GameObject LastInspectedCrop { get; private set; }`

```
/// <summary>
/// The crop whose readout is currently on the info panel.
///
/// The developer tools edit this one rather than asking the player to arm
/// a separate pick mode: tapping a crop to read it is already the gesture
/// for choosing it. Cleared when nothing has been inspected yet, and it
/// goes stale rather than dangling if the crop is destroyed - callers
/// check for null.
/// </summary>
```

Lines 598-603 · doc comment · before `public bool ShowFullCropInfo(GameObject cropObject)`

```
/// <summary>
/// The full readout for whatever crop this object belongs to.
///
/// Returns false when the object is not a crop, so a caller can fall back
/// rather than silently showing nothing.
/// </summary>
```

Lines 636-644 · doc comment · before `public void ShowPestDiseaseInfo(PestDiseaseAffectedCrop disease)`

```
/// <summary>
/// Shown after a mitigation item is used on a crop.
///
/// This used to print a shorter board of its own - crop, health, stress,
/// water and the pest block - so the player saw two different readouts for
/// the same plant depending on how they looked at it, and the mitigation
/// one hid drainage, fertility and suitability. It now shows exactly what
/// tapping the crop shows.
/// </summary>
```

Lines 653-654 · comment · before `SetInspectionText()`

```
// Not a crop we have a full readout for - better a short board than
// no feedback that the treatment landed.
```

Assets/Scripts/TerrainPreview/Terrain/TemporaryTerrainGenerator.cs

107 comments · 107 comment lines removed · file had 965 lines

Line 84 · trailing comment · on the line `From255(128, 255, 0), // < 200m, green`

```
// < 200m, green
```

Line 85 · trailing comment · on the line `From255(255, 255, 0), // 201 - 500m, yellow`

```
// 201 - 500m, yellow
```

Line 86 · trailing comment · on the line `From255(255, 166, 0), // 501 - 1000m, orange`

```
// 501 - 1000m, orange
```

Line 87 · trailing comment · on the line `From255(255, 80, 0), // 1001 - 1500m, orange-red`

```
// 1001 - 1500m, orange-red
```

Line 88 · trailing comment · on the line `From255(255, 0, 0) // > 1500m, red`

```
// > 1500m, red
```

Line 93 · trailing comment · on the line `From255(128, 255, 0), // 0 - 8%, green`

```
// 0 - 8%, green
```

Line 94 · trailing comment · on the line `From255(255, 255, 0), // 8 - 18%, yellow`

```
// 8 - 18%, yellow
```

Line 95 · trailing comment · on the line `From255(255, 180, 0), // 18 - 30%, yellow-orange`

```
// 18 - 30%, yellow-orange
```

Line 96 · trailing comment · on the line `From255(255, 85, 0), // 30 - 50%, orange-red`

```
// 30 - 50%, orange-red
```

Line 97 · trailing comment · on the line `From255(255, 0, 0) // > 50%, red`

```
// > 50%, red
```

Lines 113-114 · comment · before `Debug.LogError()`

```
// Named, because more than one of these can sit in a scene and a
// bare message gives no clue which one is short of a sprite.
```

Lines 182-183 · comment · before `float[,] slopeMap = new float[resolution, resolution];`

```
// Kept so the natural relief pass can tell lowland from mountainside;
// the slope reading was previously computed and thrown away.
```

Lines 221-222 · comment · before `float lowland = Mathf.Clamp01(1f - elevationRamp);`

```
// add natural lowland hills, fading out as the ground climbs
```

```

// out of the lowest class instead of stopping at a hard line

Lines 260-263 · comment · before reliefMeters *= Mathf.Lerp(0.5f, 1f, Mathf.Clamp01(slopeRamp - 1f));
// Gentle ground keeps half the relief, as it always has. Written
// as a ramp rather than an "is this class 0 or 1" test so the two
// halves meet smoothly; at every whole class this is the same
// 0.5 / 0.5 / 1 / 1 / 1 the step version gave.

Lines 333-334 · comment · before float flatness = 1f - Mathf.Clamp01(slopeMap[y, x]);
// Full strength on the flat, tapering to nothing on a real
// mountainside so the shaping above is left intact.

Lines 340-342 · comment · before float detail = Fbm(
// Centred on half the amplitude rather than added on top,
// so troughs cannot be clipped away at zero and the mean
// ground level stays where the shaping put it.

Lines 360-371 · doc comment · before private static float Fbm(float x, float y, int octaves, float persistence)
/// <summary>
/// Fractional Brownian motion: several octaves of Perlin noise stacked, each
/// half the amplitude and double the frequency of the last.
///
/// A single Perlin octave reads as smooth blobs, which is what made the old
/// lowland pass look artificial. Stacking octaves puts small undulations on
/// top of broad ones, which is how real ground behaves.
///
/// Returns 0..1, and is a pure function of its inputs - the same farm always
/// regenerates the same terrain, which is what lets the save file store only
/// the map area rather than the whole heightmap.
/// </summary>

Lines 504-516 · doc comment · before private float SampleLegendRampRobust(
/// <summary>
/// Where a pixel sits along a legend ramp, as a continuous position from 0
/// to the last legend index.
///
/// This replaced a version that snapped every pixel to whichever of the
/// five legend swatches was nearest. That was right for the old maps, which
/// were drawn as five flat bands, but the newer official maps are smooth
/// gradients - and snapping a gradient to five values turns a hillside into
/// five terraces with cliffs between them.
///
/// A colour that IS one of the swatches still returns that swatch index
/// exactly, so nothing about a flat-band map reads differently than before.
/// </summary>

Lines 575-583 · doc comment · before private float GetLegendRampPosition(Color pixel, Color[] legendColors, out float bestDistance)
/// <summary>
/// The closest point on the legend ramp to this colour, expressed as a
/// position along it. The ramp is treated as the straight lines joining
/// consecutive swatches in colour space, so a colour half way between the
/// yellow and the orange comes back as 1.5 rather than being rounded to one
/// of them. The distance to that closest point is handed back too, so the
/// caller can still reject pixels that are not on the ramp at all - roads,
/// labels, the white background.
/// </summary>

Lines 633-636 · doc comment · before private float LerpAcrossClasses(float rampPosition, float c0, float c1, float c2, float c3, float c4)
/// <summary>
/// Blends between the five per-class figures at a fractional ramp position.
/// At a whole number it returns that band figure unchanged.
/// </summary>

Lines 659-661 · comment · before private float GetElevationMeters(float elevationRamp)
// What each legend band stands for. The figures are unchanged; they are
// just read at a fractional position now, so a gradient map lands between
// two bands instead of being forced onto one of them.

Line 663 · comment · before private float GetElevationMeters(float elevationRamp)
//
//          < 200m   201-500   501-1000   1001-1500   > 1500m

Line 670 · comment · before private float GetSlopeStrength01(float slopeRamp)
//
//          0-8%    8-18%   18-30%   30-50%   > 50%

Lines 681-686 · doc comment · before private void WarnIfSourceMapsDisagree()
/// <summary>
/// The three source pictures are read at the same normalized coordinates,
/// so they only line up if they were exported over the same ground. Shape is
/// the one part of that this can check, and a mismatch there means the farm
/// is being built from the elevation of somewhere else entirely.
/// </summary>

Lines 721-728 · doc comment · before public static Terrain ResolveActiveTerrain()
/// <summary>
/// Robustly resolves the active generated terrain. A bare
/// FindFirstObjectByType<TemporaryTerrainGenerator>().targetTerrain lookup
/// is not reliable right after a scene load / farm restore (it can return
/// null even though the terrain was already generated), so this mirrors the
/// same defensive fallback chain already used for placing mitigation objects

```

```
/// in ClimateMaintenanceInteractionSystem.ResolveTargetTerrain().
/// </summary>
```

Lines 774–782 · doc comment · before `private void ApplyTerrainMaterial(Terrain terrain)`

```
/// <summary>
/// Forces the terrain onto an explicitly referenced URP material.
///
/// The terrain is created at runtime, so nothing in any scene references a
/// terrain material at build time and Unity's shader stripper drops the URP
/// terrain shader from the player build – which is why the terrain renders
/// magenta on a phone while looking correct in the Editor. Assigning an
/// asset in <see cref="terrainMaterial"/> creates that build-time reference.
/// </summary>
```

Lines 825–838 · doc comment · before `public static void ClearSmoothnessAlpha(Texture2D texture)`

```
/// <summary>
/// Clears a terrain texture's alpha channel so the ground renders matte.
///
/// URP's TerrainLit is compiled with _SMOOTHNESS_TEXTURE_ALBEDO_CHANNEL_A,
/// which means a layer's gloss is albedo.alpha * _Smoothness0 – the diffuse
/// texture's alpha channel IS the smoothness map. Setting the layer's
/// smoothness to 0 alone proved not to be enough, because _Smoothness0 is
/// marked "set by terrain engine" and is rewritten outside our control;
/// zeroing the alpha pins the product at 0 whatever that value becomes.
///
/// Only the alpha is touched. RGB – the colour the player actually sees –
/// is untouched, and the heightmap, collider and soil data never read this
/// texture at all, so gameplay is unaffected.
/// </summary>
```

Lines 854–855 · comment · before `ClearSmoothnessAlpha(croppedMap);`

```
// croppedMap is created fresh by CropTexture, so this does not touch the
// imported source sprite.
```

Assets/Scripts/TerrainPreview/Tutorial/TutorialDialogueUI.cs

137 comments · 137 comment lines removed · file had 676 lines

Line 10 · doc comment · before `public enum AntonioExpression`

```
/// <summary>Which portrait Antonio wears for a line.</summary>
```

Lines 19–32 · doc comment · before `public class TutorialDialogueUI : MonoBehaviour`

```
/// <summary>
/// The board Antonio talks from during the beginner guide: his portrait in a
/// frame on the left, his words on the plank beside it.
///
/// It shows one line at a time, typed out, and advances on a tap. While the
/// guide is waiting for the player to actually do something, the board stays
/// up with the instruction still readable and swaps the "tap to continue"
/// caret for a short objective hint – so nobody is ever left looking at an
/// empty screen wondering what the game wants.
///
/// Built in code like the rest of the game's UI. With no art assigned it
/// falls back to plain planks and still works, so the guide can be played and
/// tested before Antonio's portraits exist.
/// </summary>
```

Line 35 · doc comment · before `private const float TypeSpeed = 38f;`

```
/// <summary>Characters revealed per second while a line is typing.</summary>
```

Lines 38–47 · doc comment · before `private sealed class Portrait`

```
/// <summary>
/// One expression: its looping video, the texture it draws into, and the
/// image showing that texture.
///
/// Each expression gets its own player rather than one player swapping
/// clips, because Antonio changes expression on almost every line and a
/// clip swap has to Prepare() first – which shows a black frame each time.
/// Four players prepared once up front swap instantly; only the visible one
/// is ever playing, the rest sit paused.
/// </summary>
```

Line 61 · doc comment · before `private const int PortraitTextureSize = 512;`

```
/// <summary>Portrait render size. Square, and larger than it is ever drawn.</summary>
```

Line 83 · doc comment · before `public bool LineFinished => revealed >= fullLine.Length;`

```
/// <summary>True once the whole line is on screen and a tap would advance.</summary>
```

Lines 90–94 · doc comment · before `public static bool ObjectiveShowing =>`

```
/// <summary>
/// Whether the objective plank is on screen. It sits at the top centre,
/// over where a panel's own heading plank goes, so the Seedling Tent
/// steps its heading aside while the plank is up.
/// </summary>
```

Lines 108–109 · comment · before `TickPortraitWatchdog();`

```
// Runs even while the board is hidden or the line has finished typing:
// the surface can be lost at any moment, including between lines.
```

```

Lines 112-114 · comment · before if (objectiveRoot != null && objectiveRoot.activeSelf)
// Panels bring themselves to the front as they open, so the plank has
// to keep reclaiming it - otherwise the objectives book covers the very
// instruction telling the player what to do with it.

Lines 129-133 · doc comment · before public void ShowLine(string speaker, string line, AntonioExpression? expression, Action advanceCallback)
/// <summary>
/// Shows one line. <paramref name="speaker"/> null or empty makes it a
/// narration beat: italic, no name, and the portrait keeps whatever
/// expression it was already wearing so his face does not flicker.
/// </summary>

Lines 142-143 · comment · before if (currentExpression.HasValue &&
// Coming back from a gate the board was only faded, not rebuilt, but a
// pause can survive that - make sure he is moving again.

Lines 151-154 · comment · before EnsurePortraitsPrepared();
// Prepare on the first line rather than in Awake: nothing under an
// inactive root prepares, and the board is hidden until now. The intro
// runs fourteen lines before Antonio changes expression, which is far
// more than enough time for all four to finish decoding their opener.

Lines 164-165 · comment · before bodyText.fontStyle = isNarration ? FontStyle.BoldAndItalic : FontStyle.Bold;
// Bold black on both: the old browns were close enough to the plank's
// own wood tone that the words sank into the grain and were hard to read.

Lines 178-186 · doc comment · before public void ShowObjective(string hint)
/// <summary>
/// Hands the screen back to the player.
///
/// The dialogue board is most of the lower half of the screen, so leaving
/// it up while the player is meant to be digging or opening the shop hides
/// the very thing they are being asked to use. It steps aside, and a narrow
/// plank at the top carries the instruction instead. The board returns the
/// moment the job is done.
/// </summary>

Lines 191-192 · comment · before revealed = fullLine.Length;
// Snap the line fully open before stepping aside, so the board is not
// mid-typewriter when it comes back.

Lines 204-207 · doc comment · before public void SetObjectiveText(string hint)
/// <summary>
/// Rewrites the instruction on a plank that is already up, for an objective
/// that changes with what the player does. Leaves everything else alone.
/// </summary>

Lines 227-231 · doc comment · before private void SetBoardShown(bool shown)
/// <summary>
/// Fades the board rather than deactivating it: the portrait videos live
/// underneath, and a VideoPlayer on an inactive object stops. Alpha keeps
/// them running so Antonio is still mid-gesture when he comes back.
/// </summary>

Line 247 · comment · before if (waitingForGate)
// While a gate is open the board is only a sign, not a button.

Lines 253-254 · comment · before revealed = fullLine.Length;
// First tap completes the line rather than skipping it, so a fast
// tapper cannot blow through dialogue without seeing it.

Lines 281-282 · comment · before return;
// No clip assigned for this expression yet. Leave whatever is
// already showing rather than blanking the frame mid-conversation.

Lines 298-304 · doc comment · before private void TickPortraitWatchdog()
/// <summary>
/// Repairs the portrait textures after the graphics surface has been
/// recreated - which Android does when the app is paused, or when the soft
/// keyboard opens. The contents are discarded, the texture reports itself
/// as uncreated, and the RawImage draws nothing. This is the same failure
/// that turned the main menu white, guarded the same way.
/// </summary>

Lines 321-322 · comment · before portrait.Player.targetTexture = portrait.Texture;
// The old native handle died with the surface, so the player has to
// be pointed at the texture again or its frames go nowhere.

Line 330 · comment · before private void Build()
// ----- building

Lines 374-377 · doc comment · before private void BuildObjectivePlank()
/// <summary>
/// The narrow plank at the top that carries the current instruction while
/// the board is out of the way.
/// </summary>

Line 392 · comment · before rect.anchoredPosition = new Vector2(0f, -(20f + HudIconButton.Size + 16f));
// Below the round button row, which starts 20 down and is 90 tall.

```

Lines 427-433 · comment · before `objectiveText.verticalOverflow = VerticalWrapMode.Truncate;`

```
// Shrinks a long objective to fit the plank instead of cutting it off,
// the same fix as the shop's status row. The plank holds one line at
// this size, so "Buy anything, then close the shop" wrapped and its
// second line was never drawn - and the Text Size setting, which
// raises the font but not the plank, made more objectives wrap. Best
// fit never goes above the font size, and the text size setting scales
// both limits, so short objectives still grow with that setting.
```

Lines 438-441 · comment · before `objectiveText.color = Color.white;`

```
// White rather than the dark brown this used to be. The plank art is
// mid-brown, so brown-on-brown left the current objective - the one
// line telling the player what the tutorial is waiting for - the least
// readable text on screen.
```

Lines 453-454 · comment · before `GameObject go = new GameObject("Portrait", typeof(RectTransform), typeof(RectMask2D));`

```
// RectMask2D clips every portrait to the frame's painted opening, so a
// clip whose shape does not match can never spill over the woodwork.
```

Lines 458-459 · comment · before `portraitRect = go.GetComponent<RectTransform>();`

```
// Anchored to the board's top-left so the offset reads the way it is
// described in the theme: X runs right, Y runs down.
```

Lines 466-469 · comment · before `Debug.Log("[Tutorial] Portrait opening: size " + size + " at offset " + offset +`

```
// Printed because these two values live in the UITheme asset, and an
// asset that already holds numbers ignores any later change to the C#
// defaults. If Antonio sits wrong in his frame, this line says exactly
// what rectangle is being used so it can be compared with the art.
```

Lines 482-483 · comment · before `if (clip == null)`

```
// An expression with no clip assigned simply does not exist, and
// ApplyExpression leaves the previous one showing instead.
```

Lines 487-490 · comment · before `float clipAspect = clip.height > 0`

```
// The clip's own shape, so nothing is stretched. A square texture would
// squash a portrait-shaped video into a square and then stretch it back
// out across the opening, which is what made Antonio look wrong and
// pushed him past the frame.
```

Lines 505-507 · comment · before `Vector2 opening = portraitRect.sizeDelta;`

```
// Fitted inside the opening rather than filling it: the whole of
// Antonio stays visible, and because the videos are on black and the
// opening is black, the letterboxing is invisible.
```

Lines 530-534 · comment · before `GameObject playerGo = new GameObject("Player_" + expression, typeof(VideoPlayer));`

```
// Parented to the portrait holder, NOT to the RawImage above: that image
// is switched off whenever its expression is not showing, and a
// VideoPlayer on an inactive object will not prepare or play. Keeping
// the players on an always-active parent is what lets all four sit
// ready so swapping expressions is instant.
```

Lines 576-577 · comment · before `Vector2 portraitSize = Theme != null ? Theme.tutorialPortraitSize : new Vector2(240f, 240f);`

```
// The plank the words sit on starts to the right of the portrait frame
// and stops short of the board's painted edges.
```

Lines 587-591 · comment · before `speakerText.verticalOverflow = VerticalWrapMode.Overflow;`

```
// Font 26 in the 34-tall strip below gives only 1.31x the font size of
// vertical room, which is under Playpen Sans's line box - and Unity
// discards a line that does not fit rather than clipping it, so
// Antonio's name disappeared while his sentence stayed. The body text
// below already opts out for the same reason; the name needs it too.
```

Lines 615-617 · comment · before `hintText.verticalOverflow = VerticalWrapMode.Overflow;`

```
// 1.55x here, so "tap to continue" still fits today - but it is the
// prompt telling the player how to advance the tutorial at all, so it
// is not left one text-size notch away from vanishing.
```

Lines 641-642 · comment · before `text.raycastTarget = false;`

```
// Nothing here is a raycast target: the whole board is one button, and
// a label swallowing the tap would make the dialogue feel broken.
```

Line 661 · trailing comment · on the line `scaler.matchWidthOrHeight = 1f; // landscape: scale by height`

```
// landscape: scale by height
```

Assets/Scripts/TerrainPreview/Tutorial/TutorialDirector.cs

112 comments · 112 comment lines removed · file had 958 lines

Lines 10-23 · doc comment · before `public class TutorialDirector : MonoBehaviour`

```
/// <summary>
/// Runs Antonio's beginner guide: the steps that hand a new player the game
/// one piece at a time instead of all at once.
///
/// The whole tour is a flat list of beats. A beat is either something Antonio
/// says, something the game does silently (reveal a button, hand over an item),
/// or something the player has to do before the tour will continue. Walking one
/// list keeps the ordering readable next to the written script and means a step
/// can be moved by moving its lines, with no state machine to rewire.
///
```

```

/// It creates itself in TerrainPreview like the other farm systems, so there is
/// no scene wiring, and it does nothing at all on a farm that has already been
/// through it.
/// </summary>

Lines 37-39 · comment · before HudRegistry.ReleaseControl();
// Hand the HUD back to the game and forget the old farm's objects.
// Must not empty the registry outright - builders that register during
// Awake have already run by the time this fires.

Line 49 · comment · before private abstract class Beat { }
// ----- the beats

Line 60 · doc comment · before private sealed class DoBeat : Beat
/// <summary>Silent: reveals a button, hands over an item, flips a lock.</summary>

Line 72 · doc comment · before public bool LiveHint;
/// <summary>Re-read while the gate is open, for a hint that follows what the player does.</summary>

Line 87 · comment · before private void Awake()
// ----- lifecycle

Lines 113-117 · comment · before float waitedUntil = Time.unscaledTime + 60f;
// A farm being loaded carries its tutorial flag inside the save, and
// that save is not applied until the terrain has finished generating -
// which is far later than this. Deciding now would read a default
// "not completed" and run the whole tour again on a farm that finished
// it long ago, on top of the player's real crops.

Lines 124-125 · comment · before Debug.LogWarning("[Tutorial] Farm still loading after 60s; skipping the beginner guide.");
// The farm never finished loading. Whatever went wrong, replaying
// the tour over a half-restored farm would only make it worse.

Lines 133-135 · comment · before yield return null;
// Wait for the farm to finish building itself: the HUD registers its
// pieces during those builders' own Start, and hiding a button before
// it exists would do nothing.

Lines 141-144 · comment · before TutorialState.Offered = true;
// Hide the HUD before asking. If the player says yes the tour starts on
// a clean screen; if no, it all comes straight back. Asking first and
// hiding afterwards would flash the whole interface away in front of
// someone who just said they did not want a tutorial.

Lines 148-150 · comment · before CreateBlackout();
// A new farmer should not be looking at their land before the story
// says they have arrived. Black goes up first and only lifts once the
// player has answered.

Lines 156-159 · doc comment · before private void CreateBlackout()
/// <summary>
/// Full-screen black over the freshly generated farm, so the world is not
/// on show before Antonio has arrived to introduce it.
/// </summary>

Line 178 · comment · before blackout.raycastTarget = true;
// Swallows taps on anything underneath while it is up.

Line 184 · doc comment · before private IEnumerator FadeOutBlackout(float seconds)
/// <summary>Lifts the black over <paramref name="seconds"/>, then removes it.</summary>

Lines 203-206 · doc comment · before private void AskWhetherToRun()
/// <summary>
/// Offers the tour before it starts, using the same Yes/No plank as the
/// Save Farm confirmation so it looks like every other prompt in the game.
/// </summary>

Lines 211-212 · comment · before if (popup == null)
// No popup in the scene: run the guide rather than silently skipping
// it, since a new farm with no tour is the worse of the two failures.

Lines 226-227 · comment · before if (HudRegistry.TryGetPiece(HudPiece.ConfirmPopup, out GameObject popupGo))
// Asked on top of the blackout, which was created after the popup was
// built and therefore sits above it by default.

Lines 239-241 · comment · before yield return FadeOutBlackout(2.5f);
// The farm fades up on an empty screen, and only then does Antonio
// call out - the arrival reads as an arrival rather than as a dialogue
// box appearing over a world that was already there.

Lines 249-252 · doc comment · before private void DeclineTour()
/// <summary>
/// Player declined. Give them everything the tour would have handed over and
/// get out of the way - a farm with no shovel and no HUD is not a game.
/// </summary>

Lines 267-268 · comment · before Finish();
// Finish restores the whole HUD first, so the black lifts on a farm
// that is already fully dressed and ready to play.

```

Lines 289-294 · doc comment · before private void RefreshLiveHint()

```
/// <summary>
/// Rewrites a live gate's instruction twice a second, so a player who has
/// gone off the expected path - dug the wrong kind of plot, filled theirs
/// back in - is told how to get back instead of reading an instruction
/// that no longer fits.
/// </summary>
```

Line 304 · comment · before private void Advance()

```
// ----- the walk
```

Line 341 · comment · before HudRegistry.EndTutorialControl();

```
// Everything that was held back while Antonio was talking.
```

Line 352 · comment · before if (FarmPersistenceManager.Instance != null)

```
// SaveFarm is a coroutine, so it has to be started rather than called.
```

Line 357 · comment · before private void HideEverything()

```
// ----- helpers
```

Lines 361-364 · comment · before HudRegistry.BeginTutorialControl();

```
// Hands visibility to the guide. Anything the builders register after
// this point arrives hidden too, which is what the joystick and the
// jump button needed - they were being built after the old one-shot
// hide had already run, so they stayed on screen through the whole tour.
```

Lines 369-371 · comment · before ShowPanelsAsBuilt();

```
// Panels are not part of the reveal - their own buttons open them, and
// the guide only watches them. Leave them exactly as their builders
// left them so nothing about their behaviour changes.
```

Lines 429-430 · comment · before TutorialGates.FarmAction gate = new TutorialGates.FarmAction(actionType, hint, extra);

```
// Created up front so it is already listening when its step is reached -
// otherwise an action performed a moment early would be missed.
```

Line 436 · doc comment · before private void WaitForAction(string actionType, Func<string> liveHint,

```
/// <summary>The same gate, with an instruction that is re-read while it waits.</summary>
```

Lines 462-468 · doc comment · before private static string PlayerName

```
/// <summary>
/// What Antonio calls the player: the name they signed up with.
///
/// AuthSession keeps the whole profile on CurrentUser, not just the token,
/// so the display name is already here. Falls back to "neighbour" if the
/// profile is missing, which fits how he talks anyway.
/// </summary>
```

Lines 481-485 · doc comment · before private static void GrantCompletionKit()

```
/// <summary>
/// The tools the shop does not stock. Handed over at the end rather than at
/// the start so the hotbar stays empty while Antonio is filling it himself,
/// but before the player could ever need them - harvesting comes days later.
/// </summary>
```

Line 492 · comment · before private void BuildBeats()

```
// ----- the script
```

Lines 545-548 · doc comment · before private static PlantingMaterialInfo LessonMaterial()

```
/// <summary>
/// The starting material this lesson plants: the first one dealt that goes
/// straight into the ground. The deal always includes one.
/// </summary>
```

Line 557 · comment · before PlantingMaterialCatalog.TryGet(InventoryItemType.BananaSucker, out PlantingMaterialInfo fallback);

```
// Only a hand-edited or very old deal lacks one; bananas grow everywhere.
```

Line 562 · doc comment · before private static PlantingMaterialInfo TentLessonMaterial()

```
/// <summary>The starting material the Seedling Tent lesson sows, or null if none was dealt.</summary>
```

Line 599 · comment · before if (!lessonInDeal)

```
// Covered for a deal that somehow has nothing to plant straight away.
```

Lines 603-604 · comment · before if (lesson.NeedsMulchedBed)

```
// A strawberry runner goes in through mulch, and the mulch sacks
// only come out a few steps later.
```

Lines 622-624 · comment · before WaitForAction("DigPlantingSpot", () => GroundHint(lesson, choice, false),

```
// Only the kind of plot he asked for moves the lesson on. Any kind
// used to, and a player who picked another one was then told to
// plant into ground their material cannot go in.
```

Lines 644-648 · doc comment · before private static string GroundHint(PlantingMaterialInfo lesson, string choice, bool planting)

```
/// <summary>
/// The next thing to do toward the lesson's ground, read from the plots on
/// the farm as they are now. <paramref name="planting"/> is true once the
/// ground should be ready and the material is what is being waited on.
/// </summary>
```

Line 742 · comment · before Do(() => TutorialState.CropInspectionLocked = false);
// Inspection becomes available exactly when he introduces it.

Assets/Scripts/TerrainPreview/Tutorial/TutorialGates.cs

44 comments · 44 comment lines removed · file had 258 lines

Lines 6-15 · doc comment · before public static class TutorialGates

```
/// <summary>  
/// The conditions the beginner guide waits on, in one place.  
///  
/// Farm actions are read from <see cref="FarmTaskActionHub"/>, the same choke  
/// point the daily tasks use, so digging, planting, watering and mulching need  
/// no instrumentation of their own and arrive already de-duplicated. Panels are  
/// read from <see cref="HudRegistry"/> by watching whether their root object is  
/// active, which means "opened it and closed it again" is observable without  
/// any panel having to announce itself.  
/// </summary>
```

Line 18 · comment · before public sealed class MoveAndJump

```
// ----- movement
```

Lines 20-29 · doc comment · before public sealed class MoveAndJump

```
/// <summary>  
/// Walked a few metres, and jumped once if there is a jump button to press.  
///  
/// Distance is measured from the player's own transform rather than from  
/// the joystick, so it works whatever is driving the character. That  
/// matters because MobileHudBuilder does not build the on-screen controls  
/// unless the game is running on a phone: gating on the joystick meant this  
/// step could never be completed in the Editor, and the tour simply stopped  
/// there. Where there is no jump button, walking alone is enough.  
/// </summary>
```

Line 60 · comment · before Vector3 now = player.position;

```
// Flattened: falling down a slope is not walking.
```

Line 81 · comment · before public sealed class FarmAction : IDisposable

```
// ----- farm actions
```

Line 83 · doc comment · before public sealed class FarmAction : IDisposable

```
/// <summary>Waits for one recorded farm action of a given type.</summary>
```

Line 127 · comment · before public sealed class PanelOpenedThenClosed

```
// ----- panels
```

Lines 129-134 · doc comment · before public sealed class PanelOpenedThenClosed

```
/// <summary>  
/// Waits for a panel to be opened and then closed again.  
///  
/// Deliberately requires the open before the close: a panel that is already  
/// shut must not satisfy the gate the instant it is created.  
/// </summary>
```

Line 163 · doc comment · before public sealed class PanelOpened

```
/// <summary>Waits only for a panel to be opened, not closed again.</summary>
```

Line 183 · comment · before public sealed class BoughtSomethingAndClosedShop

```
// ----- money
```

Line 185 · doc comment · before public sealed class BoughtSomethingAndClosedShop

```
/// <summary>Waits for the player to spend money and shut the shop again.</summary>
```

Line 215 · comment · before public sealed class DailyRewardClaimed

```
// ----- objectives
```

Line 217 · doc comment · before public sealed class DailyRewardClaimed

```
/// <summary>Waits for the day's daily-task reward to be collected.</summary>
```

Lines 229-235 · doc comment · before public sealed class DaySkipped

```
/// <summary>  
/// Waits for the player to turn in for the night with Skip Next Day.  
///  
/// Counts from whatever the tally was when the gate opened rather than  
/// from zero, so a farm reloaded mid-tour cannot satisfy this with a skip  
/// that happened before Antonio asked for one.  
/// </summary>
```

Line 246 · comment · before if (daily == null)

```
// No task system means nothing to wait for; do not strand the tour.
```

Assets/Scripts/TerrainPreview/Tutorial/TutorialState.cs

47 comments · 47 comment lines removed · file had 138 lines

Lines 6-15 · doc comment · before public static class TutorialState

```
/// <summary>  
/// Everything the beginner guide needs to remember about a farm: whether the
```

```

/// tour has been taken, how far it got, and which three seed kinds this farm
/// was dealt.
///
/// Static and scene-independent, like <see cref="SelectedAreaState"/>, because
/// the district is chosen in one scene and used in another. It is written into
/// the farm snapshot so a player who closes the app mid-tutorial does not lose
/// their place, and so the seed draw can never change under them.
/// </summary>

```

Line 18 · doc comment · before `public static bool Completed;`

```

/// <summary>Player has finished the tour, or chose to skip it.</summary>

```

Line 21 · doc comment · before `public static bool Offered;`

```

/// <summary>Player has already been asked whether they want the tour.</summary>

```

Line 24 · doc comment · before `public static int CurrentStep = -1;`

```

/// <summary>Index into the director's step table; -1 before it starts.</summary>

```

Lines 27–31 · doc comment · before `public static readonly List<InventoryItemType> StartingSeeds =`

```

/// <summary>
/// The three planting materials this farm was dealt from its district:
/// first one that goes straight into the ground, then one for the Seedling
/// Tent, then one of either (see DistrictCropPools.RollStartingSeeds).
/// </summary>

```

Lines 35–42 · doc comment · before `public static bool CropInspectionLocked;`

```

/// <summary>
/// Suppresses the crop info readout until Antonio has introduced it.
///
/// The same panel reports "ground dug successfully" as reports a crop's
/// health, so without this it pops up during the digging lesson - three
/// steps before he explains what any of those numbers mean, and while the
/// player is meant to be looking at the soil.
/// </summary>

```

Lines 45–49 · doc comment · before `public static bool IsRunning => Offered && !Completed;`

```

/// <summary>
/// True while the tour is running. Systems that must stay out of the
/// player's way during it - Save Farm, Antonio's own tasks, Skip Day -
/// check this rather than each holding their own flag.
/// </summary>

```

Lines 52–55 · doc comment · before `public static void ResetForNewFarm()`

```

/// <summary>
/// Wipes back to "brand new farm". Called when a farm is created and by the
/// Dev Tools replay button; NOT called on load, which restores instead.
/// </summary>

```

Lines 65–69 · doc comment · before `public static void EnsureStartingSeedsRolled(string districtName)`

```

/// <summary>
/// Deals this farm's seed kinds if it has not been dealt yet. Idempotent on
/// purpose: it is safe to call from more than one place during start-up,
/// and a farm that already has seeds keeps them.
/// </summary>

```

Line 83 · comment · before `public static TutorialSaveDto Capture()`

```

// ----- persistence

```

Lines 104–106 · comment · before `Completed = true;`

```

// An older save from before the tutorial existed. That farm is
// already being played, so treat the tour as done rather than
// ambushing a returning player with it.

```

Lines 129–131 · comment · before `StartingSeeds.Add(PlantingMaterialCatalog.UpgradeLegacy(seed));`

```

// A farm dealt one of the old seed items is dealt its
// replacement material instead, so a guide resumed or declined
// later hands over something that can still be planted.

```

Assets/Scripts/TerrainPreview/UI/FarmChoicePopup.cs

19 comments · 19 comment lines removed · file had 294 lines

Lines 10–20 · doc comment · before `public class FarmChoicePopup : MonoBehaviour`

```

/// <summary>
/// A board with a question and up to three answers, plus a way out.
///
/// The Yes/No popup cannot ask "hole, bed or furrow?", which is what tapping
/// tilled ground with the shovel has to ask. This is the same board and the
/// same wooden planks as that popup, with the answers written on blank planks
/// the way the account screens do it, so it reads as part of the same set.
///
/// It creates itself the first time it is needed, like the other runtime
/// panels, so the scene needs no wiring.
/// </summary>

```

Lines 89–92 · doc comment · before `public void Show(string message, IList<Choice> choices)`

```

/// <summary>
/// Shows the question with one plank per choice. More than three choices

```

```
/// are ignored; the Back button always closes without choosing.  
/// </summary>
```

Lines 122–123 · comment · before `float x = startX + i * spacing;`

```
// Plank, then its hint (up to two lines), then clear space above  
// the Back button, whose top is 144 below the centre.
```

Lines 252–253 · comment · before `Sprite backArt = theme?.verifyBackButton;`

```
// The shared Back art rather than No: nothing here is being refused,  
// the player is only leaving without choosing.
```

Assets/Scripts/TerrainPreview/UI/FarmConfirmPopup.cs

17 comments · 17 comment lines removed · file had 250 lines

Lines 19–21 · comment · before `private GameObject titleSign;`

```
// This popup is shared by several confirmations (Save Farm, trap cleanup,  
// plant removal), so the hanging sign is set per call rather than baked in -  
// otherwise every prompt would be titled "Save Farm?".
```

Lines 42–50 · doc comment · before `public void Show(string message, Action onYes, Sprite titleArt = null, Action onNo = null)`

```
/// <summary>  
/// Shows the confirmation. Pass <paramref name="titleArt"/> to hang a  
/// painted sign above it; omit it for a plain untitled prompt.  
/// </summary>  
/// <param name="onNo">  
/// Optional. Most prompts treat No as "do nothing", which is still the  
/// default, but the beginner guide needs to know it was declined so it can  
/// hand the player a playable farm instead of simply going quiet.  
/// </param>
```

Line 125 · trailing comment · on the line `scaler.matchWidthOrHeight = 1f; // landscape: scale by height`

```
// landscape: scale by height
```

Line 156 · comment · before `titleSign = new GameObject("TitleSign", typeof(RectTransform), typeof(Image));`

```
// Built once, hidden by default; Show() supplies the sprite per call.
```

Lines 187–188 · comment · before `float buttonY = board != null ? -70f : -62f;`

```
// SaveFarmBoard.png's painted edge ends at panel-local y = -127.5, and at  
// the old -95 the 62-tall buttons reached -126 - sitting right on it.
```

Line 220 · comment · before `bg.sprite = art;`

```
// The word is painted into the art, so no Text child is added.
```

Assets/Scripts/TerrainPreview/UI/WorldLoadingScreen.cs

4 comments · 4 comment lines removed · file had 250 lines

Lines 30–31 · comment · before `if (isShowing)`

```
// This catches UI that gets created after the loading screen starts,  
// like InventoryUIBuilder and DevToolsUIBuilder.
```

Line 55 · trailing comment · on the line `scaler.matchWidthOrHeight = 1f; // landscape: scale by height`

```
// landscape: scale by height
```

Line 109 · comment · before `if (theme?.loadingLabel != null)`

```
// Hanging "Generating Farm" sign; falls back to plain title text.
```

Assets/Scripts/TerrainPreview/Weather/FarmGraceperiod.cs

24 comments · 24 comment lines removed · file had 46 lines

Lines 5–24 · doc comment · before `public static class FarmGraceperiod`

```
/// <summary>  
/// The quiet start every new farm gets: no rain, no climate events, no pest or  
/// disease outbreaks for the first two game days.  
///  
/// A new player is being taught to dig, plant and water. Dropping a typhoon on  
/// them mid-lesson would destroy the one crop they own and make Antonio look  
/// like a liar, since he is at that point still explaining that bad weather is  
/// something that happens *later*.  
///  
/// Deliberately keyed to the game date rather than to tutorial progress. Tying  
/// it to the tutorial would freeze the weather forever on a farm whose owner  
/// declined the tour or abandoned it half way; keying it to the date means a  
/// player who skips the guide still gets the same two calm days, and normal  
/// weather resumes on day 3 no matter what.  
///  
/// Climate events need no guard of their own. ClimateEventTracker only starts  
/// tracking when a weather event begins, so holding the weather at Clear  
/// suppresses climate events - and their end-of-event AI evaluation - as a  
/// consequence rather than as a second special case.  
/// </summary>
```

Line 27 · doc comment · before `public const float CalmDays = 2f;`

```
/// <summary>Game days protected from the start. Day 1 and day 2.</summary>
```

Line 30 · doc comment · before `public static bool IsCalmWeatherDay`
/// <summary>True while the farm is still inside its opening calm spell.</summary>

Lines 39–40 · comment · before `return time.TotalGameDays < CalmDays;`
// TotalGameDays counts up from 0 on day 1, so "still inside day 2"
// means strictly less than 2 whole days elapsed.

Assets/Scripts/TerrainPreview/Weather/GameTimeSystem.cs

15 comments · 15 comment lines removed · file had 404 lines

Line 62 · comment · before `TotalGameDays =`
// Day 1 begins at the requested time.

Lines 72–73 · comment · before `if (WeatherSystem.Instance != null)`
// Initialization is idempotent, so script Start order
// no longer affects the first pest/disease evaluation.

Lines 163–167 · comment · before `if (WeatherSystem.Instance != null)`
// Required deterministic order:
// 1. The clock is already on the new day.
// 2. Roll the new day's weather and temperature.
// 3. Evaluate pest/disease risk using that weather.
// 4. Notify any additional daily systems.

Lines 214–215 · comment · before `FarmTaskActionObserver.Instance?.ResetBaseline();`
// Do not misclassify changes caused by skipped simulation as
// direct player actions after the skip finishes.

Line 235 · comment · before `SimulateContinuousMitigations(days);`
// Continuous mitigation is applied before disease damage.

Lines 291–292 · comment · before `ClimateMitigationWorldObject[] climateMitigations =`
// Climate mitigation objects also have continuous effects and
// stored resources, so they must advance during task-based skips.

Lines 348–349 · comment · before `float sunAngle =`
// 6 AM = sunrise, 12 PM = highest sun,
// 6 PM = sunset.

Assets/Scripts/TerrainPreview/Weather/WeatherSystem.cs

21 comments · 21 comment lines removed · file had 589 lines

Lines 87–88 · comment · before `RefreshCurrentConditions();`
// Weather no longer detects game-day changes here.
// GameTimeSystem owns the new-day order.

Lines 277–280 · comment · before `if (FarmGraceperiod.IsCalmWeatherDay)`
// A brand-new farmer gets two calm days before the sky is allowed to
// do anything to them. Without this a typhoon can land during the
// beginner guide, wiping out the first crop Antonio just taught them
// to plant and contradicting him mid-sentence.

Lines 392–393 · comment · before `float warmth =`
// Daily low at around 2 AM.
// Daily high at around 2 PM.

Lines 479–489 · doc comment · before `public float GetEventStressPerDay()`
/// <summary>
/// Stress per game day caused by the active weather event alone.
///
/// Waterlogging used to be folded in here against a fixed 0.85 moisture
/// line. That double-counted against crops which also run their own
/// waterlogging check, and it punished high-moisture crops such as
/// mangosteen (ideal max 0.90) inside the band their own profile calls
/// ideal. Each crop now tests waterlogging against its own band; what is
/// left here is only the part that is identical for every plant standing
/// in the same storm.
/// </summary>

Lines 541–542 · comment · before `RollDailyTemperatureRange();`
// Immediately adjust temperature and humidity
// according to the forced weather.

Assets/Scripts/TerrainPreview/Weather/WeatherVisualController.cs

116 comments · 116 comment lines removed · file had 721 lines

Line 107 · doc comment · before `private const float StormSkyMaxRadius = 1200f;`
/// <summary>Farthest out the storm sky is drawn. It is also kept inside the camera's far plane.</summary>

Line 110 · doc comment · before `private const float LightningFade = 0.06f;`
/// <summary>Seconds for a flash of lightning to fall to about a third; three times this and it is all but gone.</summary>

Line 113 · doc comment · before `private const float FillChangeThreshold = 0.002f;`
/// <summary>Smallest change in the fill light, per colour channel, worth applying.</summary>

```

Line 136 · doc comment · before private Color appliedFill = new Color(-1f, -1f, -1f, 1f);
/// <summary>The fill light last applied, in linear colour.</summary>

Line 145 · doc comment · before private float currentStormSkyCover;
/// <summary>How much of the normal sky the storm sky hides: 0 none, 1 all of it, sun included.</summary>

Line 149 · doc comment · before private float nextLightningTime = -1f;
/// <summary>When the next flash is due, or below 0 when none is scheduled.</summary>

Lines 233-234 · comment · before if (stormSky != null)
// Centred on the camera itself rather than on the rain above it, so
// the dome's horizon sits level with the player's eye.

Lines 308-309 · comment · before targetStormSkyCover = 0f;
// The colour is left as it was, so a storm clears away in its own
// grey instead of changing shade on the way out.

Lines 361-377 · doc comment · before private void ApplyRainMotion(float fallSpeed, float windSpeed)
/// <summary>
/// Makes the drops fall, whatever the particle system in the scene is set to.
///
/// RainParticles emits from a flat 18 x 18 metre box eight metres over the
/// camera. A box emits along its own forward axis, and nothing turns this
/// one toward the ground - its rotation in the scene is zero - so every drop
/// left at 22 m/s level with the ground, heading north, and none ever came
/// down into view. Rotating the object would not fix it cleanly either: the
/// box is flat on its own Y, so turned to point down it would stand up as a
/// wall.
///
/// So the drops are launched with no speed at all and moved by a velocity
/// set in world space, which no rotation of the object can bend. They are
/// drawn stretched along that velocity, because a drop falling this fast
/// covers at least a third of a metre between frames, and drawn as a dot it
/// reads as a flickering speck rather than as rain.
/// </summary>

Lines 389-390 · comment · before velocity.x = new ParticleSystem.MinMaxCurve(windSpeed);
// All three as constants: Unity rejects the change if the axes are in
// different modes.

Lines 401-402 · comment · before rainRenderer.cameraVelocityScale = 0f;
// Streaks follow the drop's own speed only. Stretched by the camera's
// as well, every drop would smear sideways whenever the player walked.

Lines 417-419 · comment · before currentStormSkyCover = Mathf.MoveTowards(currentStormSkyCover, targetStormSkyCover,
// Linear rather than eased like the rest, so the fade actually finishes:
// the last few percent of an eased one would leave the sun glowing
// through the clouds for seconds after the storm had arrived.

Line 449 · comment · before private void BuildFillLight()
// ----- fill light

Lines 451-472 · doc comment · before private void BuildFillLight()
/// <summary>
/// A soft light for everything the sun does not reach.
///
/// The scene's environment light has its intensity at 0 in the Lighting
/// settings, so the sun was the only light there was: the ground under a
/// shade net or greenhouse went black at noon, and at night crops vanished.
/// Turning that setting up would light every night like day, because that
/// light is worked out once from the daytime sky and never dims. Setting the
/// scene's ambient light from code had no visible effect either - Unity kept
/// the zero.
///
/// So the light comes from a real light, which nothing overrides: directional,
/// shining straight down, casting no shadows, so it reaches under a shade net
/// as easily as open ground. Being directional it looks the same whether a
/// quality level lights extra lights per pixel or only at mesh corners, and it
/// costs about the same as a second sun without shadows.
///
/// It is made the scene's sun source first. URP takes the sun source as its
/// main light when one is set, and otherwise the brightest directional light -
/// which at night, with the sun turned almost to nothing, would be this, and
/// the sky would put its sun disc overhead in the middle of the night.
/// </summary>

Lines 486-487 · comment · before fillLight.useColorTemperature = false;
// The project turns colour temperature on for lights; left on, it would
// tint this away from the colour set below.

Lines 491-496 · doc comment · before private void ApplyFillLight(bool force)
/// <summary>
/// Sets the fill to follow the clock. In daylight it is the weather's ambient
/// colour at a fraction of its strength, enough that shade still reads as
/// shade; as the sun goes down it settles to the minimum and never goes below
/// it, so crops and sprouts stay visible in the dark.
/// </summary>

```

```

Line 510 · comment · before if (!force &&
// Only when it has visibly changed: the light shifts slowly with the clock.

Lines 526-528 · comment · before fillLight.color = new Color(fill.r / peak, fill.g / peak, fill.b / peak, 1f).gamma;
// The project lights in linear intensity, where a light gives its colour
// converted to linear times its intensity. So the brightest channel goes
// in the intensity and the hue, converted back, in the colour.

Line 533 · comment · before private void BuildStormSky()
// ----- storm sky

Lines 535-548 · doc comment · before private void BuildStormSky()
/// <summary>
/// The overcast sky for rain and typhoons: a dome around the camera that
/// fades in over the normal sky.
///
/// The sky is Unity's procedural skybox, and nothing on it turns it grey or
/// puts its sun out gradually. Its sun disc is drawn from the sun light, and
/// dimming that light would darken the whole farm with it. So rather than
/// change the sky, this covers it: fully faded in it hides the sun
/// completely, and fading out uncovers the sky and sun exactly as they were.
///
/// It never writes depth, and the terrain and crops in front of it hide it
/// as usual. The radius stays inside the camera's far plane; ground beyond
/// it is lost in the storm's fog anyway.
/// </summary>

Lines 559-561 · comment · before GameObject probe = GameObject.CreatePrimitive(PrimitiveType.Sphere);
// The mesh is borrowed from a throwaway primitive, destroyed at once: its
// collider, a ball around the player, must not exist for even one
// physics step.

Line 574 · comment · before stormSky.transform.localScale = Vector3.one * (radius * 2f);
// The primitive is one metre across.

Lines 601-603 · comment · before Color overhead = currentStormSkyColor * Mathf.Lerp(stormSkyNightBrightness, 1f, DayLight());
// Overhead darkens with the day, as a real sky does. The horizon takes
// the fog colour unchanged, because that is the colour the fogged ground
// meets it in, day or night.

Line 607 · comment · before float flash = LightningFlash() * lightningBrightness;
// Lightning lights the clouds overhead more than the haze on the horizon.

Line 618 · doc comment · before private float Daylight()
/// <summary>1 with the sun well up, falling to 0 just after it sets.</summary>

Lines 624-625 · comment · before float elevation = Vector3.Dot(-sunLight.transform.forward, Vector3.up);
// GameTimeSystem turns the light to point straight down at noon, so the
// sun itself lies back along the light's forward axis.

Line 630 · comment · before private void UpdateLightning()
// ----- lightning

Lines 632-635 · doc comment · before private void UpdateLightning()
/// <summary>
/// Schedules lightning while a typhoon has the sky. Nothing is drawn here;
/// ApplyStormSky reads the flash every frame.
/// </summary>

Lines 641-642 · comment · before if (!typhoon || currentStormSkyCover < 0.95f)
// Only once the storm has covered the sky, so a flash never lights a sky
// that is still blue, or one already clearing.

Lines 662-666 · doc comment · before private void StartLightningFlash()
/// <summary>
/// One strike: a single flash, or a quick flicker of two or three, the way
/// real lightning strikes again down the same path - each one after the
/// first a little dimmer.
/// </summary>

Lines 681-686 · doc comment · before private readonly System.Random lightningRandom = new System.Random();
/// <summary>
/// Lightning draws on a generator of its own. Unity's shared one is also used
/// by gameplay code, and every number a flash took from it would shift the
/// numbers that code drew next - never its odds, but a purely visual effect
/// should leave gameplay's randomness alone entirely.
/// </summary>

Line 694 · doc comment · before private float LightningFlash()
/// <summary>How bright the lightning is right now, from 0 to 1.</summary>

Line 709 · comment · before flash = Mathf.Max(flash, lightningPulsePeaks[i] * Mathf.Exp(-since / LightningFade));
// At its brightest the instant it strikes, then gone in a moment.

Line 713 · comment · before if (elapsed > lightningPulseDelays[lightningPulseCount - 1] + LightningFade * 8f)
// Every pulse has struck and died away.

```

Assets/TutorialInfo/Scripts/Editor/ReadmeEditor.cs

1 comment · 1 comment line removed · file had 243 lines

Line 218 · comment · before `m_LinkStyle.normal.textColor = new Color(0x00 / 255f, 0x78 / 255f, 0xDA / 255f, 1f);`
`// Match selection color which works nicely for both light and dark skins`

Part B — Backend server

Backend agridabao-api: Java sources, application.yml and the Dockerfile. Line numbers are from commit 46cfb6d.

Dockerfile

4 comments · 4 comment lines removed · file had 24 lines

Line 1 · comment · before FROM eclipse-temurin:21-jdk AS build

```
# Build stage: compile the Spring Boot jar with the Gradle wrapper.
```

Line 5 · comment · before COPY gradlew ./

```
# Copy the wrapper first so dependency resolution is cached separately from source.
```

Line 14 · comment · before FROM eclipse-temurin:21-jre

```
# Runtime stage: JRE only, no Gradle or source in the shipped image.
```

Line 20 · comment · before EXPOSE 8080

```
# Railway injects PORT; application.yml reads it via ${PORT:8080}.
```

src/main/java/com/agridabao/api/admin/AdminCommand.java

7 comments · 12 comment lines removed · file had 97 lines

Line 15 · doc comment · before enum AdminCommandType {

```
/** What a queued command asks the receiving game to do. */
```

Line 17 · doc comment · before GRANT_MONEY,

```
/** Add pesos to the player's wallet. Uses amount. */
```

Line 19 · doc comment · before FORCE_WEATHER,

```
/** Start a weather event. Uses payload for the type and durationDays. */
```

Line 21 · doc comment · before FORCE_PEST_DISEASE,

```
/** Start a pest or disease outbreak. Uses payload for the enum name. */
```

Line 23 · doc comment · before PASS_DAYS,

```
/** Push the farm clock forward whole days. Uses amount. */
```

Line 25 · doc comment · before PASS_HOURS

```
/** Push the farm clock forward whole hours. Uses amount. */
```

Lines 29–34 · doc comment · before @Entity

```
/**
 * One instruction waiting for one player.
 *
 * Rows are kept after delivery rather than deleted, so there is a record of
 * what was sent to whom during a test session.
 */
```

src/main/java/com/agridabao/api/admin/AdminController.java

25 comments · 99 comment lines removed · file had 435 lines

Lines 37–44 · doc comment · before @RestController

```
/**
 * Operations that act on somebody else's account, for the developer tools.
 *
 * There is no way for a player to delete their own account in this build, so
 * an address used once was used forever – which made testing sign-up painful
 * and left no way to clear a tester's data afterwards. This is that missing
 * lever, and it is deliberately kept off the player-facing surface.
 */
```

Lines 54–60 · doc comment · before @PostMapping("/capability")

```
/**
 * What the calling account is allowed to do with the developer tools.
 *
 * Deliberately not admin-gated: every signed-in player asks this, and a
 * non-admin gets an honest "no" rather than a refusal. It reveals only
 * whether the caller themselves is on the list, never who else is.
 */
```

Line 66 · doc comment · before @PostMapping("/accounts/lookup")

```
/** Answers whether an address is in use, so the tool can confirm before deleting. */
```

Line 79 · doc comment · before @PostMapping("/commands")

```
/** Queues an event or a payment for one player. Admin only. */
```

Lines 86–92 · doc comment · before @PostMapping("/commands/pending")

```
/**
 * Anything queued for the caller, marked delivered as it is handed over.
 *
 * Not admin-gated, because every player's game polls it for itself – the
 * query is scoped to the caller's own id, so there is nothing here to reach
```

```
* anyone else's commands with.  
*/
```

Lines 106-109 · doc comment · before record AdminCapabilityResponse(boolean admin, boolean mobileToolsEnabled) {

```
/**  
 * @param admin          whether this account is on the admin list  
 * @param mobileToolsEnabled whether the server currently allows the tools on a phone  
 */
```

Lines 123-128 · doc comment · before record AdminCommandRequest(@NotBlank @Email @Size(max = 320) String email,

```
/**  
 * @param commandType GRANT_MONEY, FORCE_WEATHER or FORCE_PEST_DISEASE  
 * @param payload      the weather or pest name, ignored for a money grant  
 * @param amount       pesos for a grant  
 * @param durationDays how long a weather event should run  
 */
```

Lines 150-157 · doc comment · before private final Set<String> adminEmails;

```
/**  
 * The addresses allowed to use this controller, from APP_ADMIN_EMAILS.  
 *  
 * Empty is not "everyone" but "nobody": with no list configured every call  
 * here is refused. That is the safe direction to fail in - forgetting to set  
 * the variable costs a developer one confusing error, while the opposite  
 * default would hand every player in the game a button that deletes anyone.  
 */
```

Line 160 · doc comment · before private static final int MAX_MONEY_GRANT = 1_000_000;

```
/** Ceilings that turn a slipped digit into a refusal rather than a ruined save. */
```

Lines 164-168 · doc comment · before private static final int MAX_PASS_DAYS = 30;

```
/**  
 * A game day is 900 real seconds, so thirty of them is well past any single  
 * test session and far enough to see a crop through its stages. Beyond that  
 * a slipped digit costs the tester their farm rather than their afternoon.  
 */
```

Line 178 · doc comment · before private final boolean mobileToolsEnabled;

```
/** Whether an admin may open the tools on a phone. See application.yml. */
```

Lines 211-213 · comment · before return new AdminCapabilityResponse(admin, admin && mobileToolsEnabled);

```
// The phone flag is reported only to an admin. Telling a stranger whether  
// the mobile tools are switched on says something about the deployment  
// that they have no reason to know.
```

Lines 233-248 · doc comment · before @Transactional

```
/**  
 * Removes an account and everything belonging to it, freeing the address  
 * for a fresh sign-up.  
 *  
 * Almost all of it happens in the database rather than here: farm saves,  
 * settings, presence, friendships, friend requests, chat, trades and  
 * listings are all declared ON DELETE CASCADE against app_user, so removing  
 * the one row takes the rest with it. A sold listing keeps its record with  
 * buyer_id set to null, which is deliberate - the other player's sale  
 * history should not disappear because their buyer left.  
 *  
 * The verification codes are the exception. That table is keyed on the  
 * address rather than the account and carries no foreign key, so nothing  
 * cascades into it and its rows have to be cleared by hand or they outlive  
 * the account they belonged to.  
 */
```

Lines 258-259 · comment · before if (target.getId().equals(callerId)) {

```
// Deleting the account you are signed in with would revoke the token  
// mid-request and leave the tool unable to explain what happened.
```

Lines 276-283 · doc comment · before @Transactional

```
/**  
 * Queues one command for a player who is currently in the game.  
 *  
 * The online check is the point of the whole feature, not a nicety. These  
 * are meant to land while a tester is watching - a typhoon queued for  
 * somebody who has closed the game would arrive whenever they next opened  
 * it, possibly days later and with nobody expecting it.  
 */
```

Lines 316-317 · comment · before duration = duration == null ? 1 : Math.clamp(duration, 1, MAX_WEATHER_DAYS);

```
// Clamped rather than refused: a typo of 300 days would leave a  
// tester stuck in a typhoon for the rest of the session.
```

Lines 348-355 · doc comment · before @Transactional

```
/**  
 * Hands the caller their own queued commands and stamps them delivered.  
 *  
 * Marked as delivered when handed over rather than when applied. The  
 * alternative would need the game to acknowledge each one, and a dropped  
 * acknowledgement would replay a typhoon on the next poll - a worse failure  
 * than one command occasionally being lost to a dropped response.  
 */
```

Lines 380-387 · doc comment · before `private static Integer requireCount(Integer amount, int ceiling, String unit) {`

```
/**
 * A whole positive count within its ceiling.
 *
 * Refused rather than clamped, unlike a weather duration. Skipping time is
 * not undoable on the receiving farm: crops age, weather rolls and daily
 * objectives regenerate, so quietly turning a mistyped 200 into 30 would
 * still hand the player a farm they did not expect.
 */
```

Lines 411-413 · comment · before `if (callerEmail == null || !adminEmails.contains(callerEmail)) {`

```
// Checked against the account rather than against the token's email
// claim: an address that has since been changed should not keep the
// powers it had, and the claim is only as fresh as the token holding it.
```

src/main/java/com/agridabao/api/ai/AiController.java

41 comments · 73 comment lines removed · file had 231 lines

Lines 22-28 · doc comment · before `@RestController`

```
/**
 * The game's single entry point to the AI adviser.
 *
 * Unity used to call Google directly, which meant every build shipped the API
 * key inside the APK. The phone now calls this instead: it sends the prompt and
 * its player JWT, and the key stays on the server.
 */
```

Line 38 · doc comment · before `@GetMapping("/status")`

```
/** Lets the game show the adviser as unavailable without making a full request. */
```

Line 51 · doc comment · before `enum AiFeature {`

```
/** Which part of the game is asking; decides the generation settings applied. */
```

Line 53 · doc comment · before `ADVISOR,`

```
/** Free-form chat with the farm adviser. */
```

Line 55 · doc comment · before `CLIMATE_EVALUATION,`

```
/** Post-event write-up of how the player handled a typhoon or drought. */
```

Line 57 · doc comment · before `TASK_GENERATE,`

```
/** Invents one multi-day task; must return strict JSON. */
```

Line 59 · doc comment · before `TASK_CHECK`

```
/** Grades an assigned task; must return strict JSON. */
```

Lines 70-74 · doc comment · before `record AiGenerateResponse(boolean available,`

```
/**
 * @param available    false when the adviser is switched off; the game shows
 *                    {@code message} instead of an answer
 * @param finishReason "STOP" when the model completed its own sentence
 */
```

Lines 88-92 · doc comment · before `static final String UNAVAILABLE_MESSAGE =`

```
/**
 * Shown whenever the adviser cannot answer - switched off, unconfigured, or
 * the upstream call failed. Deliberately in the adviser's own voice so the
 * game never surfaces a technical error to a player.
 */
```

Lines 122-125 · comment · before `String systemInstruction =`

```
// Both of these are applied here rather than trusted from the phone. The
// prompt the client sends lives inside the APK, so a modified build can
// send anything at all; whatever it sends, it arrives underneath the
// server's rules and its last part arrives quoted.
```

Lines 143-144 · comment · before `log.warn("AI request for {} failed: {}", request.feature(), ex.getMessage());`

```
// An upstream outage is not the player's problem: answer in-character
// rather than surfacing a 500 in the middle of the game.
```

Lines 150-154 · doc comment · before `private static final int MAX_QUESTION_CHARS = 1000;`

```
/**
 * A farming question does not need an essay. Anything longer is either a
 * mistake or an attempt to bury an instruction deep enough that the model
 * loses track of the rules above it.
 */
```

Lines 157-168 · doc comment · before `private static List<String> prepareParts(AiFeature feature, List<String> parts) {`

```
/**
 * Quotes the player's question so it cannot be mistaken for part of the
 * prompt around it.
 *
 * Only the last part is treated as the question: the adviser sends the game
 * context first and the typed question last. Everything before it is
 * generated by the game rather than typed by anyone, so it needs no fence -
 * and fencing it would only bury the real question further from the rules.
 *
 * Features other than the adviser take no player text at all, so their parts
```

```
* are passed through exactly as they arrive.
*/
```

Lines 195–198 · doc comment · before `private static GeminiClient.GenerationConfig configFor(AiFeature feature) {`

```
/**
 * Mirrors the settings the Unity clients used before the move, so the
 * adviser's behaviour is unchanged by relocating the call.
 */
```

Lines 201–202 · comment · before `case ADVISOR -> new GeminiClient.GenerationConfig(0.4, 0.9, 3000, false);`

```
// Chat: slightly creative, and a ceiling high enough that a farm with
// several crop groups does not get its answer cut off mid-sentence.
```

Lines 204–211 · comment · before `case CLIMATE_EVALUATION -> new GeminiClient.GenerationConfig(null, null, null, false);`

```
// No token ceiling, which is how this ran before the call moved here.
//
// Gemini 2.5 Flash is a thinking model and its reasoning tokens count
// against maxOutputTokens. The climate payload is by far the largest
// prompt the game sends – the guidance plus every before/after crop
// snapshot and every recorded action – so a 3000 cap could be spent
// entirely on thinking, returning MAX_TOKENS with an empty parts array
// and therefore no evaluation text at all.
```

Lines 213–225 · comment · before `case TASK_GENERATE, TASK_CHECK ->`

```
// Task generation and grading must return parseable JSON, so the
// response MIME type is pinned and the temperature kept low.
//
// No token ceiling, for the same reason the climate write-up above
// has none. Gemini 2.5 Flash thinks before it answers and those
// reasoning tokens are charged against maxOutputTokens, so on a
// large farm-context prompt a 3000 cap could be spent almost
// entirely on thinking – the reply came back as MAX_TOKENS with the
// JSON cut off mid-object, the client could not parse it, and the
// game quietly fell back to a canned local task that had never
// looked at the farm. The tell was that the failed replies were
// SHORTER than the successful ones, which is backwards for a limit
// that is supposed to stop long answers.
```

src/main/java/com/agridabao/api/ai/AiGuardrails.java

4 comments · 39 comment lines removed · file had 126 lines

Lines 3–18 · doc comment · before `final class AiGuardrails {`

```
/**
 * The rules the adviser is held to, enforced here rather than in the game.
 *
 * The prompt the phone sends is deliberately tunable in Unity's Inspector, which
 * is convenient and also means it ships inside the APK: anyone who unpacks a
 * build can replace it, and the server would forward whatever came back. A
 * guardrail written there is a request, not a rule. This one is prepended on the
 * server to whatever the client supplies, so the floor holds no matter what the
 * phone asks for – the client's own text is admitted afterwards as style
 * guidance and is explicitly ranked below these rules.
 *
 * Only the free-form adviser gets this treatment. The climate write-up and the
 * two task features are driven entirely by game state rather than by anything a
 * player types, so they have no injection surface to defend and their prompts
 * are passed through untouched.
 */
```

Lines 21–25 · doc comment · before `static final String REFUSAL =`

```
/**
 * What Antonio says when asked something outside farming. Kept in his own
 * voice: a refusal that reads as a system message breaks the character the
 * rest of the game maintains.
 */
```

Lines 84–89 · doc comment · before `static String systemInstructionFor(AiFeature feature, String clientInstruction) {`

```
/**
 * The system instruction actually sent upstream: the server's rules first,
 * then the client's wording preferences, clearly subordinate.
 *
 * @param clientInstruction the phone's own guidance, which may be null
 */
```

Lines 100–111 · doc comment · before `static String fencePlayerQuestion(String question) {`

```
/**
 * Marks a player's question off as quoted material.
 *
 * The question used to be pasted straight into a sentence, so text shaped
 * like the surrounding prompt was read as part of it – a player could close
 * the question and open something that looked like a fresh instruction. The
 * fence gives the model an unambiguous end, and rule 3 above tells it what
 * everything inside is worth.
 *
 * The fence token is stripped from the player's own text first, so it cannot
 * be closed early by typing it.
 */
```

src/main/java/com/agridabao/api/ai/GeminiClient.java

7 comments · 29 comment lines removed · file had 169 lines

Lines 20–28 · doc comment · before @Component

```
/**
 * Talks to Google's Generative Language API on the game's behalf.
 *
 * The key lives here rather than in the Unity client. A Unity build ships its
 * serialized Inspector values inside the APK, so a key held on a MonoBehaviour
 * can be recovered by unpacking the package – anyone who did so could spend
 * against the project's quota. Keeping it server-side means the phone never
 * holds the secret; it only holds a player JWT, which is revocable and scoped.
 */
```

Lines 53–60 · doc comment · before public Result generate(String systemInstruction, List<String> userParts, GenerationConfig config) {

```
/**
 * One generation call.
 *
 * @param systemInstruction optional system prompt, or null to send none
 * @param userParts         the user text blocks, in order
 * @param config            per-feature generation settings
 * @return the model's text plus the reason it stopped
 */
```

Lines 100–101 · comment · before .header("x-goog-api-key", apiKey)

```
// Header rather than ?key= so the secret never lands in a URL,
// where proxies and access logs would record it.
```

Lines 129–132 · doc comment · before private Result parse(String rawJson) {

```
/**
 * Joins every text part of the first candidate, mirroring what the Unity
 * clients did locally so multi-part answers are not silently truncated.
 */
```

Line 154 · doc comment · before public record GenerationConfig(Double temperature,

```
/** Per-feature generation settings; nulls fall back to Gemini's defaults. */
```

Lines 161–165 · doc comment · before public record Result(String text, String finishReason) {

```
/**
 * @param finishReason "STOP" means the model finished its own sentence.
 *                     Anything else – usually MAX_TOKENS – means the answer
 *                     was cut off, which the game shows as a retry prompt.
 */
```

src/main/java/com/agridabao/api/auth/AuthController.java

4 comments · 4 comment lines removed · file had 62 lines

Lines 41–44 · comment · before @PostMapping("/password/forgot/request")

```
// Password recovery, in three steps: prove the account exists and email a
// code, confirm the code for a short-lived ticket, then spend the ticket on
// a new password. Splitting the last two means the player types the new
// password on its own screen, after the code has already been accepted.
```

src/main/java/com/agridabao/api/auth/AuthService.java

17 comments · 38 comment lines removed · file had 251 lines

Lines 22–26 · doc comment · before private static final String BAD_CREDENTIALS = "Invalid email/display name or password.";

```
/**
 * One wording for "we do not know that account" and "that is the wrong
 * password", so the login endpoint cannot be used to find out which
 * addresses and display names are registered.
 */
```

Line 53 · doc comment · before @Transactional

```
/** Step 1 of sign-up: validate the form and email a code. No account is created yet. */
```

Line 82 · doc comment · before @Transactional(noRollbackFor = UnauthorizedException.class)

```
/** Step 2 of sign-up: confirm the code, then create the account and issue a token. */
```

Lines 94–97 · comment · before if (pending.displayName() != null

```
// Checked again here, not only at step 1: the code lives for ten minutes
// and someone else may have taken the name in between. Without this the
// unique index would reject the insert with a database error instead of
// something the player can act on.
```

Line 119 · doc comment · before @Transactional

```
/** Step 1 of login: validate credentials and email a 2FA code. No token is issued yet. */
```

Lines 133–136 · comment · before return codeResponse("We emailed a login code to " + email + ".", code, email);

```
// The full address is safe to return here: the password has already been
// checked, so whoever is asking has proven the account is theirs. The
// client needs it because a player who signed in with a display name has
// nothing else to confirm the code against.
```

Line 140 · doc comment · before @Transactional(noRollbackFor = UnauthorizedException.class)

```
/** Step 2 of login: confirm the 2FA code, then issue a token. */
```

Lines 151-158 · doc comment · before @Transactional

```
/**
 * Step 1 of recovery: email a code to the account named by an address or a
 * display name.
 *
 * The reply says which account was found, in masked form, but never the full
 * address - a player who has forgotten their email is told enough to
 * recognise it, and a stranger guessing display names learns nothing usable.
 */
```

Line 173 · doc comment · before @Transactional(noRollbackFor = UnauthorizedException.class)

```
/** Step 2 of recovery: confirm the code and hand back a ticket for step 3. */
```

Lines 187-191 · doc comment · before @Transactional

```
/**
 * Step 3 of recovery: set the new password and sign the player straight in,
 * so they land on the menu rather than back at a login form they would have
 * to fill in with the password they just typed twice.
 */
```

Lines 210-216 · doc comment · before private Optional<AppUser> findByIdentifier(String identifier) {

```
/**
 * The account behind whatever the player typed into the single login box.
 *
 * Address first, display name second. That order matters: it means an
 * address always reaches the account that registered it, so a display name
 * can never be used to shadow someone else's login.
 */
```

src/main/java/com/agridabao/api/auth/CodeRequestResponse.java

1 comment · 13 comment lines removed · file had 23 lines

Lines 3-15 · doc comment · before public record CodeRequestResponse{

```
/**
 * Response for the "request a code" step. {@code devCode} is populated only when
 * {@code app.verification.expose-code} is true (development/testing), so real
 * deployments never leak the code over the wire.
 *
 * {@code email} is the address the code actually went to, so the confirmation
 * step knows which account it is confirming even when the player signed in with
 * their display name. It is filled in only where the caller has already proven
 * they own the account - by password on login, or by typing the address itself
 * on sign-up. Password recovery leaves it null on purpose: anyone can name any
 * display name, and filling this in would turn that into a way to read a
 * stranger's email address.
 */
```

src/main/java/com/agridabao/api/auth/EmailVerification.java

1 comment · 5 comment lines removed · file had 107 lines

Lines 27-31 · doc comment · before @Column(name = "user_id")

```
/**
 * The account this code belongs to, for the purposes that already know it
 * (password reset, email change). Null for SIGNUP - there is no account
 * yet - and for LOGIN, where the address alone identifies the account.
 */
```

src/main/java/com/agridabao/api/auth/EmailVerificationRepository.java

4 comments · 28 comment lines removed · file had 62 lines

Lines 15-28 · doc comment · before @Lock(LockModeType.PESSIMISTIC_WRITE)

```
/**
 * The live codes for an address, newest first, with their rows locked until
 * the transaction ends. Needs a transaction.
 *
 * Checking a code reads its attempt count, compares the code and writes the
 * count back. Unlocked, guesses that arrive together all read the same count,
 * so more than the allowed number of them get compared. Locked, each guess
 * waits for the one before it to finish and then sees the count it left - or
 * finds the code already used or deleted.
 *
 * A list rather than findFirst so the lock is taken by a plain ordered select
 * with no row limit. There is normally only one live code per address and
 * purpose.
 */
```

Lines 35-39 · doc comment · before Optional<EmailVerification> findFirstByUserIdAndPurposeAndConsumedAtIsNullOrderByCreatedAtDesc()

```
/**
 * The live code for an account, used by the email change: the code is sent
 * to an address the account does not own yet, so it cannot be found by the
 * account's current address.
 */
```

Line 43 · doc comment · before @Lock(LockModeType.PESSIMISTIC_WRITE)

```
/** The email change's live codes, locked - see findLockedByEmail above. */
```

Lines 50-57 · doc comment · before void deleteByEmail(String email);

```
/**
 * Every code for an address, whatever it was for.
 *
 * Used when an account is deleted. This table is keyed on the address and
 * has no foreign key to app_user, so unlike every other table holding a
 * player's data, nothing cascades into it: without these two the codes
 * outlive the account they belonged to.
 */
```

src/main/java/com/agridabao/api/auth/ForgotPasswordRequest.java

1 comment · 1 comment line removed · file had 11 lines

Line 6 · doc comment · before public record ForgotPasswordRequest()

```
/** Step 1 of recovery: the email or display name of the account to recover. */
```

src/main/java/com/agridabao/api/auth/ForgotPasswordVerifyRequest.java

1 comment · 7 comment lines removed · file had 18 lines

Lines 6-12 · doc comment · before public record ForgotPasswordVerifyRequest()

```
/**
 * Step 2 of recovery.
 *
 * The identifier is sent again rather than the account's email, because a
 * player who started this from their display name has never been told the
 * address and the server deliberately keeps it that way.
 */
```

src/main/java/com/agridabao/api/auth/LoginRequest.java

1 comment · 8 comment lines removed · file had 30 lines

Lines 7-14 · doc comment · before public record LoginRequest()

```
/**
 * Sign-in credentials.
 *
 * The first field is whatever the player typed into the one box on the login
 * board - an email address or their display name - so it carries no {@code @Email}
 * rule. {@code email} is the older name for the same thing and is still accepted,
 * because a phone running a build from before this change sends only that.
 */
```

src/main/java/com/agridabao/api/auth/PasswordResetTicketResponse.java

1 comment · 4 comment lines removed · file had 12 lines

Lines 3-6 · doc comment · before public record PasswordResetTicketResponse()

```
/**
 * Handed out once the emailed reset code checks out. It is not an access token
 * and the API refuses it as one; its only use is the reset call that follows.
 */
```

src/main/java/com/agridabao/api/auth/ResetPasswordRequest.java

1 comment · 1 comment line removed · file had 13 lines

Line 6 · doc comment · before public record ResetPasswordRequest()

```
/** Step 3 of recovery: the new password, authorised by the ticket from step 2. */
```

src/main/java/com/agridabao/api/auth/VerificationPurpose.java

2 comments · 6 comment lines removed · file had 15 lines

Line 6 · doc comment · before PASSWORD_RESET,

```
/** Recovering an account whose password the player has forgotten. */
```

Lines 8-12 · doc comment · before EMAIL_CHANGE

```
/**
 * Moving an account to a new address. The code goes to the NEW address, so
 * the record's email column is the one being moved to and its user_id is
 * the account doing the moving.
 */
```

src/main/java/com/agridabao/api/auth/VerificationService.java

5 comments · 18 comment lines removed · file had 199 lines

Line 62 · doc comment · before @Transactional

```
/** {@code newEmail} is where the code is sent - the address being moved to. */
```

Line 84 · doc comment · before `@Transactional(noRollbackFor = {UnauthorizedException.class})`

```
/** Returns the address the account is moving to, once its code checks out. */
```

Line 95 · comment · before `if (isUserScoped(purpose)) {`

```
// Only one live code per subject+purpose: replace any earlier ones.
```

Lines 156–163 · doc comment · before `private Optional<EmailVerification> findLive(String email, UUID userId, VerificationPurpose purpose) {`

```
/**
 * An email change is looked up by account, every other purpose by address.
 *
 * The address is the wrong key for a change: the code is sent to one the
 * account does not own yet, and a player who mistypes it and tries again
 * would otherwise leave the first attempt live and be able to confirm the
 * typo'd address afterwards.
 */
```

Lines 170–176 · doc comment · before `private Optional<EmailVerification> lockLive(String email, UUID userId, VerificationPurpose purpose) {`

```
/**
 * {@link #findLive}, but holding the code's row until the transaction ends.
 *
 * Every check of a code goes through here, so checks of the same code run one
 * at a time and the attempt limit still holds when guesses are sent together.
 * The consume methods are transactional, which the lock needs.
 */
```

src/main/java/com/agridabao/api/config/SecurityConfig.java

7 comments · 30 comment lines removed · file had 177 lines

Lines 56–65 · doc comment · before `private static JwtDecoder accessTokenDecoder(SecretKey secretKey,`

```
/**
 * The decoder the Authorization header is checked with.
 *
 * Deliberately not the {@link JwtDecoder} bean below. The password-reset
 * ticket is signed with the same key as an access token, so a plain decoder
 * would happily accept one as a bearer token and hand a half-finished reset
 * the run of the API. This one additionally insists the token says it is an
 * access token; JwtService keeps the unrestricted decoder so it can still
 * read the ticket on the reset endpoint itself.
 */
```

Lines 83–84 · comment · before `if (purpose == null || JwtService.PURPOSE_ACCESS.equals(purpose)) {`

```
// Absent counts as an access token: every token issued before the claim
// existed lacks it, and they stay valid for their full 30 days.
```

Lines 93–107 · doc comment · before `private static OAuth2TokenValidatorResult requireCurrentTokenVersion(`

```
/**
 * Refuses a token that was issued before the account's last password change.
 *
 * This is the one thing a stateless token cannot do on its own. Everything
 * else about a token is settled by its signature and its expiry, but "has
 * this session been revoked since" can only be answered by the account, so
 * this costs one lookup by primary key per authenticated request. That is
 * the price of being able to end a session at all, and it is worth paying:
 * without it, changing a password locks an intruder out of signing in again
 * but leaves the session they already have running for up to thirty days.
 *
 * The repository is resolved lazily. The decoder is built while the security
 * filter chain is being assembled, which is earlier than the JPA layer is
 * ready, and asking for the bean then would fail.
 */
```

Lines 119–121 · comment · before `return OAuth2TokenValidatorResult.success();`

```
// Nothing to check against. Refusing every request would take the
// whole API down over a startup ordering problem, which is a far
// worse failure than one session outliving a password change.
```

src/main/java/com/agridabao/api/farm/DistrictSeedPools.java

8 comments · 26 comment lines removed · file had 117 lines

Lines 11–29 · doc comment · before `public final class DistrictSeedPools {`

```
/**
 * Which crops a farm in each Davao district grows, and so which planting
 * materials it is allowed to obtain.
 *
 * <p>A farm can only get the planting materials of crops its own district
 * grows – every material of such a crop, so a Calinan farm can have both the
 * banana plantlet and the banana sucker. The game enforces that in the shop,
 * but a marketplace purchase and a trade are carried out here, so this is where
 * they are refused – a modified or older client cannot talk its way around it.
 *
 * <p>This must stay identical to {@code DistrictCropPools} in the game client,
 * fallback included. A farm with no district recorded, or one this table does not
 * know, uses the fallback pool on both sides.
 *
 * <p>Only planting material is restricted. Produce, tools and kits can be bought
 * and traded by anyone. The five seed items the planting materials replaced
 * (banana, coconut, mango, pineapple and strawberry seed) are still limited by
```

```
* their crop, because an older version of the game can still list and trade them.
*/
```

```
Line 35 · doc comment · before private static final Map<String, String> CROP_OF_MATERIAL = Map.ofEntries(
/** Each planting material, and the crop it grows into. */
```

```
Line 53 · comment · before entry("BananaSeed", "Banana"),
// Retired seed items, still sent by older versions of the game.
```

```
Line 74 · doc comment · before private static final Set<String> FALLBACK = Set.of("Coconut", "Banana", "Cacao", "Corn");
/** For a farm whose district is missing or unrecognised – the same four as the game. */
```

```
Line 77 · doc comment · before public static boolean isRestricted(String itemType) {
/** Whether the item is planting material, and so subject to district pools at all. */
```

```
Line 82 · doc comment · before public static boolean isAvailableIn(String itemType, String districtName) {
/** Whether a farm in {code districtName} may obtain the item. */
```

```
Line 90 · doc comment · before public static Set<String> cropsFor(String districtName) {
/** The crops a farm in {code districtName} grows, or the fallback pool. */
```

```
Line 99 · doc comment · before public static Set<String> poolFor(String districtName) {
/** Every planting material a farm in {code districtName} may obtain. */
```

src/main/java/com/agridabao/api/farm/EconomyJsonService.java

56 comments · 75 comment lines removed · file had 379 lines

```
Lines 25–30 · doc comment · before private static final Set<String> TRADABLE_ITEMS = Set.of(
/**
* Must match {code SocialMarketplaceCatalog.TradableItems} in the game. The
* five retired seed items (banana, coconut, mango, pineapple and strawberry
* seed) stay tradable here only so older versions of the game keep working;
* the current game turns any it receives into the material that replaced it.
*/
```

```
Lines 205–209 · doc comment · before public String districtOf(UUID userId) {
/**
* The district a player's saved farm is in, or an empty string when they have
* no saved farm or it does not record one – which {link DistrictSeedPools}
* treats as its fallback pool, exactly as the game does.
*/
```

```
Line 217 · doc comment · before public boolean isItemAvailableInDistrict(String itemType, String districtName) {
/** Whether a farm in this district may obtain the item. Only crop seeds are limited. */
```

```
Lines 238–241 · doc comment · before public int baseValueCentavos(String itemType) {
/**
* An item's marketplace base value, per item, in centavos. Turn a total into
* whole pesos with {link PesoRounding#toWholePesos}.
*/
```

```
Line 247 · comment · before return weatherMitigationBaseValue * 100;
// That table is still in whole pesos.
```

```
Line 309 · doc comment · before private static int maxStack(String itemType) {
/** Must match {code PlayerInventory.GetMaxStack} in the game. */
```

```
Lines 314–315 · comment · before return "AphidTrap".equals(itemType) ? 99 : 999;
// The game caps a stack of aphid traps at 99. A bigger stack written here
// was loaded into one slot anyway, past the game's own limit.
```

```
Lines 319–326 · doc comment · before private static Map<String, Integer> buildBaseValues() {
/**
* Marketplace base values, per item, in centavos.
*
* <p>Seeds follow the Davao City government seed prices and produce follows
* the shipping bin, both of which have centavos in them; the tools keep the
* NPC shop prices. Must match {code SocialMarketplaceCatalog.BaseValues} in
* the game client, or the listing fee a player is shown is not the one charged.
*/
```

```
Line 329 · trailing comment · on the line values.put("PineappleSeed", 1000); // P10
// P10
```

```
Line 330 · trailing comment · on the line values.put("BananaSeed", 1500); // P15
// P15
```

```
Line 331 · trailing comment · on the line values.put("CacaoSeed", 2500); // P25
// P25
```

```
Line 332 · trailing comment · on the line values.put("CoconutSeed", 1500); // P15
// P15
```

```
Line 333 · trailing comment · on the line values.put("PomeloSeed", 5000); // P50
// P50
```

Line 334 · trailing comment · on the line `values.put("MangoSeed", 3000); // P30`
`// P30`

Line 335 · trailing comment · on the line `values.put("MangosteenSeed", 7500); // P75`
`// P75`

Line 336 · trailing comment · on the line `values.put("DurianSeed", 6000); // P60`
`// P60`

Line 337 · trailing comment · on the line `values.put("CornSeed", 38889); // P388.89`
`// P388.89`

Line 338 · trailing comment · on the line `values.put("EggplantSeed", 820000); // P8200`
`// P8200`

Line 339 · trailing comment · on the line `values.put("SquashSeed", 300000); // P3000`
`// P3000`

Line 340 · trailing comment · on the line `values.put("StrawberrySeed", 300); // P3`
`// P3`

Line 341 · trailing comment · on the line `values.put("TomatoSeed", 950000); // P9500`
`// P9500`

Lines 342–344 · comment · before `values.put("BananaPlantlet", 3000); // P30`
`// Planting materials. The five that replaced the retired seeds above keep`
`// that seed's price; the plantlet, the grafted seedling and the ready`
`// squash seedling are priced above the material they skip ahead of.`

Line 345 · trailing comment · on the line `values.put("BananaPlantlet", 3000); // P30`
`// P30`

Line 346 · trailing comment · on the line `values.put("BananaSucker", 1500); // P15`
`// P15`

Line 347 · trailing comment · on the line `values.put("MangoGraftedSeedling", 8000); // P80`
`// P80`

Line 348 · trailing comment · on the line `values.put("MangoLiso", 3000); // P30`
`// P30`

Line 349 · trailing comment · on the line `values.put("CoconutSeednut", 1500); // P15`
`// P15`

Line 350 · trailing comment · on the line `values.put("PineappleSucker", 1000); // P10`
`// P10`

Line 351 · trailing comment · on the line `values.put("StrawberryRunner", 300); // P3`
`// P3`

Line 352 · trailing comment · on the line `values.put("SquashSeedling", 330000); // P3300`
`// P3300`

Line 353 · trailing comment · on the line `values.put("Coconut", 1689); // P16.89`
`// P16.89`

Line 354 · trailing comment · on the line `values.put("Banana", 5100); // P51`
`// P51`

Line 355 · trailing comment · on the line `values.put("Durian", 15000); // P150`
`// P150`

Line 356 · trailing comment · on the line `values.put("Pomelo", 20000); // P200`
`// P200`

Line 357 · trailing comment · on the line `values.put("Cacao", 27527); // P275.27`
`// P275.27`

Line 358 · trailing comment · on the line `values.put("Pineapple", 5000); // P50`
`// P50`

Line 359 · trailing comment · on the line `values.put("Mangosteen", 5000); // P50`
`// P50`

Line 360 · trailing comment · on the line `values.put("Mango", 11800); // P118`
`// P118`

Line 361 · trailing comment · on the line `values.put("Corn", 1500); // P15`
`// P15`

Line 362 · trailing comment · on the line `values.put("Eggplant", 6200); // P62`
`// P62`

Line 363 · trailing comment · on the line `values.put("Squash", 4200); // P42`
`// P42`

Line 364 · trailing comment · on the line `values.put("Strawberry", 35000); // P350`
`// P350`

Line 365 · trailing comment · on the line `values.put("Tomato", 15600); // P156`
`// P156`

Line 366 · trailing comment · on the line `values.put("AphidTrap", 3500); // P35`
`// P35`

Line 367 · trailing comment · on the line `values.put("InsecticideLiter", 9000); // P90`
`// P90`

Line 368 · trailing comment · on the line `values.put("DisinfectantLiter", 8000); // P80`
`// P80`

Line 369 · trailing comment · on the line `values.put("NeemSoapLiter", 10000); // P100`
`// P100`

Line 370 · trailing comment · on the line `values.put("BtBioInsecticideLiter", 13000); // P130`
`// P130`

Line 371 · trailing comment · on the line `values.put("CopperFungicideLiter", 12000); // P120`
`// P120`

Line 372 · trailing comment · on the line `values.put("PheromoneTrap", 9000); // P90`
`// P90`

Line 373 · trailing comment · on the line `values.put("FruitBag", 2500); // P25`
`// P25`

Line 374 · trailing comment · on the line `values.put("DrainageKit", 15000); // P150`
`// P150`

Line 375 · trailing comment · on the line `values.put("TermiteBaitStation", 11000); // P110`
`// P110`

src/main/java/com/agridabao/api/farm/PesoRounding.java

2 comments · 18 comment lines removed · file had 30 lines

Lines 3–19 · doc comment · before `public final class PesoRounding {`

```
/**
 * Centavo totals to whole pesos.
 *
 * <p>Seed and produce values follow the Davao City government price list, which
 * has centavos in it (a coconut is P16.89), so marketplace base values are held
 * in centavos. Wallets stay in whole pesos, so a total is rounded once, to the
 * nearest peso, with halves rounding up.
 *
 * <p>The game client repeats this exact rule in {@code PesoPrice.TotalPesos}
 * when it shows a player the listing fee. If the two rounded differently the fee
 * shown would not be the fee charged, which is why this is whole-number
 * arithmetic rather than {@code Math.round}: C#'s default rounding sends halves
 * to the nearest even number, and matching that from here would be fragile.
 *
 * <p>Kept in a class of its own with no dependencies, so it can be compiled and
 * checked against the client's copy without starting the application.
 */
```

Line 25 · doc comment · before `public static long toWholePesos(long centavos) {`

```
/** A centavo total as whole pesos, to the nearest peso, halves rounding up. */
```

src/main/java/com/agridabao/api/farm/WeatherMitigationTradableItems.java

1 comment · 6 comment lines removed · file had 38 lines

Lines 3–8 · doc comment · before `final class WeatherMitigationTradableItems {`

```
/**
 * Additional marketplace/trade item rules for climate and weather mitigation.
 *
 * This class is deliberately package-private. EconomyJsonService calls it before
 * applying its original item whitelist and base-value rules.
 */
```

src/main/java/com/agridabao/api/mail/MailService.java

14 comments · 20 comment lines removed · file had 188 lines

Lines 20–25 · comment · before `private static final String HEADER_IMAGE = "email/header.jpg";`

```
// JPEG rather than PNG: the banner is a photographic image with no
// transparency, and the PNG it replaced was 5.2 MB – 2752px wide for a
// banner drawn at 500. Every send base64-encoded that to roughly 7 MB, so
// MAX_INLINE_IMAGE_BYTES below dropped it and the emails went out with a
// broken image. At 1000px and quality 85 it is 137 KB and still 2x the
// width it is displayed at.
```

Lines 28–31 · doc comment · before `private static final long MAX_INLINE_IMAGE_BYTES = 400_000L;`

```
/**
 * Gmail clips messages larger than ~102 KB and big inline images make every
 * send slow, so an oversized header is dropped rather than shipped.
 */
```

Line 75 · doc comment · before `public void sendEmailChangeCode(String newEmail, String code) {`
/** Sent to the address being moved TO, which is the one being proven. */

Line 96 · comment · before `if (resend.isEnabled()) {`
// Preferred on cloud hosts: HTTPS instead of a blocked SMTP port.

Lines 113–114 · comment · before `MimeMessageHelper helper = new MimeMessageHelper(`
// RELATED (not MIXED) since the header image is an inline resource, not a
// real attachment; MIXED would nest it in a way some clients list as one.

Lines 164–165 · comment · before `resend.send(from, fromName, to, subject, html, imageBase64, "header.jpg", "header");`
// The filename has to match the real format: some mail clients decide how
// to decode an attachment from its extension, not its bytes.

Lines 170–173 · doc comment · before `private boolean isConfigured() {`
/**
 * True when any transport can actually deliver: the Resend API key, or a
 * complete SMTP username/password pair.
 */

src/main/java/com/agridabao/api/mail/ResendMailClient.java

3 comments · 12 comment lines removed · file had 109 lines

Lines 18–24 · doc comment · before `@Component`

/**
 * Sends mail through Resend's HTTPS API instead of SMTP.
 *
 * Cloud hosts (Railway, Render, Fly) block outbound SMTP ports to prevent spam,
 * so JavaMailSender times out there. This client talks to port 443, which is
 * never blocked, and is selected automatically whenever an API key is present.
 */

Lines 45–48 · doc comment · before `public void send(String fromAddress,`
/**
 * @param inlineImage optional base64 payload for the template's cid:header
 * image; pass null to send without it.
 */

Line 70 · comment · before `image.put("content_id", inlineImageContentId);`
// Matches src="cid:header" in the HTML templates.

src/main/java/com/agridabao/api/marketplace/MarketplaceController.java

16 comments · 19 comment lines removed · file had 508 lines

Line 98 · doc comment · before `@GetMapping("/sales/unseen")`
/** Sales this player has not been notified about yet. */

Line 104 · doc comment · before `@PostMapping("/sales/{listingId}/acknowledge")`
/** Marks one sale notification as shown so it is not repeated. */

Line 147 · doc comment · before `record MarketplaceSaleNotificationResponse(`
/** One "your listing sold" notification, shown to the seller once. */

Line 194 · doc comment · before `@Column(name = "sale_acknowledged", nullable = false)`
/** False until the seller has been shown the "your item sold" notification. */

Line 230 · comment · before `this.saleAcknowledged = false;`
// The seller has not seen this yet; the notification endpoint clears it.

Line 248 · doc comment · before `List<MarketplaceListing> findBySellerIdAndStatusAndSaleAcknowledgedFalseOrderBySoldAtAsc(`
/** Sales this seller has not been shown yet, oldest first. */

Lines 283–285 · comment · before `long itemTotal = PesoRounding.toWholePesos((long) baseCentavos * quantity);`
// Base values are held in centavos but the fee comes out of a whole-peso
// wallet, so the items' total is rounded once, here - the same way the game
// rounds it when it shows the player the fee.

Line 289 · comment · before `int baseWholePesos = (int) PesoRounding.toWholePesos(baseCentavos);`
// Whole pesos, for anything still reading this field as pesos.

Lines 314–316 · comment · before `String viewerDistrict = economy.districtOf(currentUserId);`
// A seed the viewer's district does not grow is not shown to them - they could
// not buy it anyway. Their own listings always show, so they can still cancel
// or claim them whatever they hold.

Lines 443–446 · doc comment · before `@Transactional(readOnly = true)`
/**
 * Sales the seller has not seen yet. Read-only: the client acknowledges them
 * separately, so a dropped response does not lose the notification.
 */

Lines 458–459 · comment · before `listing.getAskingPrice(),`
// The seller receives the asking price; the listing fee was
// already paid up front when the listing was created.

src/main/java/com/agridabao/api/security/JwtService.java

5 comments · 27 comment lines removed · file had 134 lines

Lines 22–27 · doc comment · before `public static final String PURPOSE_CLAIM = "purpose";`

```
/**
 * Names what a token may be used for. Access tokens issued before this
 * claim existed carry nothing, which {@link #PURPOSE_ACCESS} stands in for -
 * see AccessTokenPurposeValidator, which is what actually keeps a
 * single-purpose ticket out of the ordinary Authorization header.
 */
```

Lines 32–38 · doc comment · before `public static final String VERSION_CLAIM = "ver";`

```
/**
 * The account's token version at the moment this token was issued. The
 * server compares it against the account on every request, so a password
 * change - which increments the account's - retires every token issued
 * before it. Absent means zero: tokens minted before this claim existed keep
 * working until their account's first password change.
 */
```

Lines 41–45 · doc comment · before `private static final Duration RESET_TICKET_DURATION = Duration.ofMinutes(15);`

```
/**
 * How long the player has to choose a new password after their emailed code
 * checks out. Short on purpose: the ticket is proof of a passed check, so it
 * is worth as much as the code was, and it only has to survive one screen.
 */
```

Lines 80–87 · doc comment · before `public String issueResetTicket(AppUser user) {`

```
/**
 * Proof that this player answered the emailed reset code, handed to them so
 * the next screen can set a new password without asking for the code again.
 *
 * It is signed with the same key as an access token, so the resource server
 * would otherwise accept it as one; the purpose claim above is what stops
 * that, and it is the reason this ticket is safe to hand out at all.
 */
```

Line 102 · doc comment · before `public UUID readResetTicket(String token) {`

```
/** The account a reset ticket belongs to, or a 401 if it is not one. */
```

src/main/java/com/agridabao/api/settings/PlayerSettings.java

3 comments · 3 comment lines removed · file had 86 lines

Line 30 · doc comment · before `@Column(name = "ui_scale", nullable = false)`

```
/** Interface size multiplier; 1 is the size the game has always drawn at. */
```

Line 34 · doc comment · before `@Column(name = "text_scale", nullable = false)`

```
/** Text size multiplier, applied on top of {@link #uiScale}. */
```

Line 38 · doc comment · before `@Column(name = "ai_summarization", nullable = false)`

```
/** When true the adviser and climate evaluation answer in a couple of sentences. */
```

src/main/java/com/agridabao/api/settings/PlayerSettingsRequest.java

1 comment · 8 comment lines removed · file had 21 lines

Lines 3–10 · doc comment · before `public record PlayerSettingsRequest(`

```
/**
 * @param uiScale interface size multiplier. A client built before this field
 * existed omits it, and Jackson leaves an absent float at 0;
 * the service reads any non-positive value as "not supplied"
 * and keeps the stored value rather than clamping it to the
 * minimum, which would shrink the player's UI behind their back.
 * @param textScale text size multiplier, on top of {@code uiScale}. Same rule.
 */
```

src/main/java/com/agridabao/api/settings/PlayerSettingsService.java

9 comments · 9 comment lines removed · file had 94 lines

Line 16 · doc comment · before `static final float DEFAULT_UI_SCALE = 1f;`

```
/** 1 is the size the game has always drawn at, so this is "unchanged". */
```

Line 20 · doc comment · before `static final boolean DEFAULT_AI_SUMMARIZATION = false;`

```
/** Off is the adviser's existing full-length answer. */
```

Lines 26–28 · comment · before `private static final float UI_SCALE_MIN = 0.80f;`

```
// Mirrors GameSettings on the Unity side. The interface ceiling is bounded by
// the tallest panel still fitting once the canvas reference resolution shrinks;
// the text ceiling is lower because an oversized label wraps inside its rect.
```

Lines 60–63 · comment · before `float currentUiScale = settings != null ? settings.getUiScale() : DEFAULT_UI_SCALE;`

```
// A client that predates the scale fields sends neither, and Jackson leaves
// an absent float at 0. Clamping that would silently rewrite the player's
// interface to the minimum, so a non-positive value means "leave it alone":
// keep what is stored, or the default for a brand-new row.
```

src/main/java/com/agridabao/api/social/PresenceController.java

9 comments · 30 comment lines removed · file had 208 lines

Lines 37–43 · doc comment · before `@PostMapping("/heartbeat")`

```
/**
 * Claims or refreshes this account's play session for the calling device.
 *
 * Answers 409 when a different device is already playing on this account,
 * which is what stops one account being played on two phones at once. The
 * body is optional so a client built before device tracking keeps working.
 */
```

Line 79 · doc comment · before `@Column(name = "device_id", length = 128)`

```
/** The device holding this session; null on rows written before device tracking. */
```

Line 99 · doc comment · before `void claim(String newDeviceId, Instant now) {`

```
/** Hands the session to a new device and restarts its clock. */
```

Lines 114–118 · doc comment · before `private static final String IN_USE_MESSAGE =`

```
/**
 * Shown to the second device. It names the support address deliberately: to
 * the player this either means they left the game running somewhere else, or
 * somebody else has their password, and the second case needs a way out.
 */
```

Lines 130–138 · doc comment · before `@Transactional`

```
/**
 * Refreshes the session for {@code deviceId}, taking it over when nobody
 * else holds it.
 *
 * A device may claim when the row is missing, when the previous holder has
 * gone quiet for longer than {@link #ONLINE_WINDOW}, when it already holds
 * the row, or when the row predates device tracking and names no holder.
 * Anything else means a second device is genuinely playing right now.
 */
```

Lines 162–165 · doc comment · before `@Transactional`

```
/**
 * Releases the session. A device that never held it cannot end someone
 * else's, so a blocked second phone quitting does not free the first.
 */
```

Line 190 · comment · before `if (holder == null || !presence.getLastSeenAt().isAfter(now.minus(ONLINE_WINDOW))) {`

```
// Nobody named, or the holder has gone quiet: free to take.
```

Lines 195–196 · comment · before `if (deviceId == null || deviceId.isBlank()) {`

```
// A caller that cannot identify itself is never treated as an intruder,
// so an older build keeps behaving exactly as it did.
```

src/main/java/com/agridabao/api/trade/TradeController.java

8 comments · 11 comment lines removed · file had 675 lines

Lines 75–77 · comment · before `@GetMapping("/{tradeId}")`

```
// One trade in whatever state it ended. /active lists only open trades, so a
// trade the other player completes just disappears from there; this is how
// the game finds out it completed and brings the exchange into the backpack.
```

Lines 411–412 · comment · before `UUID recipient = actor.equals(trade.getRequesterId()) ? trade.getTargetId() : trade.getRequesterId();`

```
// A seed only reaches a farm whose district grows it. Refused here, as the
// offer is made, so the player who dragged it in is the one who hears why.
```

Line 482 · comment · before `requireSeedsAvailableIn(economy.district(targetSnapshot), requesterOffer, SEED_NOT_AVAILABLE_IN_TRADE);`

```
// Checked once more against the locked farms, right before anything moves.
```

Lines 497–500 · doc comment · before `static final String SEED_NOT_AVAILABLE_TO_PARTNER =`

```
/**
 * Said to the player making the offer, and word for word the game's popup, so
 * the game can recognise a refusal from here and open that popup.
 */
```

Line 504 · doc comment · before `static final String SEED_NOT_AVAILABLE_IN_TRADE =`

```
/** Said when both offers are already in and one of them has such a seed. */
```

src/main/java/com/agridabao/api/user/AccountFields.java

10 comments · 22 comment lines removed · file had 80 lines

Lines 7–11 · doc comment · before `public final class AccountFields {`

```
/**
 * The rules an email address and a display name have to follow, in one place
 * because sign-up, sign-in, password recovery and the Account panel all apply
 * the same ones and had no business each having their own idea of them.
 */
```

Line 13 · doc comment · before `public static final int DISPLAY_NAME_MIN = 3;`

```
/** Long enough to be typed on purpose, short enough for the plank it is drawn on. */
```

```
Line 24 · doc comment · before public static String normalizeDisplayName(String value) {  
    /** Blank becomes null, because a display name is optional and "" is not a name. */
```

```
Lines 32–36 · doc comment · before public static void validateDisplayName(String displayName) {  
    /**  
     * Checks a display name a player chose. Applied only to names being set from  
     * now on: accounts created before display names had to be unique keep  
     * whatever they have until their owner changes it.  
     */
```

```
Lines 48–50 · comment · before if (displayName.indexOf('@') >= 0) {  
    // Sign-in tries the typed text as an address first and as a display name  
    // second, so a name shaped like an address would be shadowed by whoever  
    // owns that address and could never be signed in with.
```

```
Lines 56–60 · doc comment · before public static String mask(String email) {  
    /**  
     * Enough of an address for its owner to recognise it, and not enough for  
     * anyone else to use. Shown when a player asks to reset a password from a  
     * display name, where they have proven nothing yet.  
     */
```

```
Lines 75–76 · comment · before return head + "****" + domain;  
    // A fixed run of stars rather than one per hidden character, so the  
    // length of the address is not given away either.
```

src/main/java/com/agridabao/api/user/AccountService.java

8 comments · 34 comment lines removed · file had 184 lines

```
Lines 20–26 · doc comment · before @Service  
    /**  
     * The three things a signed-in player can change about their account.  
     *  
     * Kept apart from AuthService because everything here already knows who is  
     * asking: the caller arrives with a valid token, so none of it has to guard  
     * against account enumeration the way sign-in and recovery do.  
     */
```

```
Lines 57–63 · doc comment · before @Transactional  
    /**  
     * Step 1 of changing an address: email a code to the NEW one.  
     *  
     * Sending to the new address rather than the current one is the whole point  
     * - it proves the player can actually receive mail there, so a typo cannot  
     * strand an account at an address nobody reads.  
     */
```

```
Lines 87–94 · doc comment · before @Transactional(noRollbackFor = UnauthorizedException.class)  
    /**  
     * Step 2: move the account across.  
     *  
     * A fresh token comes back with it. The old one keeps working - it names the  
     * account by id, not by address - but it still carries the previous address  
     * in its claims, and handing back a matching pair keeps the copy saved on the  
     * phone from disagreeing with the account.  
     */
```

```
Lines 100–101 · comment · before if (userRepository.existsByEmail(newEmail)) {  
    // Rechecked after the code is accepted: the code is good for ten minutes  
    // and somebody else may have registered that address in the meantime.
```

```
Lines 122–129 · doc comment · before @Transactional  
    /**  
     * Changes the password and, with it, ends every other session on the  
     * account - see AppUser.changePasswordHash.  
     *  
     * A fresh token comes back because the change retires the one the caller  
     * used to make it. Without that they would be signed out by their own  
     * password change, which is the opposite of what pressing the button means.  
     */
```

```
Lines 167–168 · comment · before if (userRepository.existsByDisplayNameIgnoreCaseAndIdNot(displayName, userId)) {  
    // Excluding this account so a player can restyle their own name -  
    // "juan" to "Juan" - without colliding with themselves.
```

src/main/java/com/agridabao/api/user/AppUser.java

5 comments · 24 comment lines removed · file had 107 lines

```
Lines 30–36 · doc comment · before @Column(name = "token_version", nullable = false)  
    /**  
     * Bumped whenever every existing session for this account must stop working.  
     * Each access token carries the value current when it was issued, and the  
     * server refuses any token whose value no longer matches - see  
     * SecurityConfig. Zero is the value for accounts that predate the column,  
     * and matches a token carrying no version at all.  
     */
```

Lines 69–71 · comment · before `public void changeEmail(String newEmail) {`
// Only the three fields a player can edit from the Account panel are
// mutable. Each stamps updatedAt itself so no caller can change an account
// without leaving a trace of when it happened.

Lines 78–91 · doc comment · before `public void changePasswordHash(String newPasswordHash) {`
/**
 * Sets a new password and ends every session the account already had.
 *
 * The two happen together on purpose. Someone changing their password
 * because another person is using their account expects that person to be
 * shut out, and a password change that leaves the intruder signed in for
 * another month does not do what the player thinks it does. Whoever makes
 * the change is handed a fresh token in the same response, so they stay
 * signed in and only the other sessions fall away.
 *
 * Changing an email deliberately does not do this: it is not a response to
 * losing control of the account, and signing the player out of their other
 * phone would be a surprise rather than a protection.
 */

src/main/java/com/agridabao/api/user/AppUserRepository.java

1 comment · 7 comment lines removed · file had 33 lines

Lines 14–20 · doc comment · before `Optional<AppUser> findFirstByDisplayNameIgnoreCase(String displayName);`
/**
 * Sign-in and password recovery accept a display name in place of an email.
 *
 * findFirst rather than findBy: a unique index guarantees at most one match,
 * but a duplicate slipping past it should cost one player an odd lookup, not
 * throw NonUniqueResultException and break sign-in for everyone.
 */

src/main/java/com/agridabao/api/user/ChangeEmailRequest.java

1 comment · 1 comment line removed · file had 12 lines

Line 7 · doc comment · before `public record ChangeEmailRequest(`
/** Step 1 of moving an account to a new address. */

src/main/java/com/agridabao/api/user/ChangePasswordRequest.java

1 comment · 1 comment line removed · file had 13 lines

Line 6 · doc comment · before `public record ChangePasswordRequest(`
/** Changing a password from inside the game, where the current one is known. */

src/main/java/com/agridabao/api/user/ConfirmEmailChangeRequest.java

1 comment · 7 comment lines removed · file had 17 lines

Lines 6–12 · doc comment · before `public record ConfirmEmailChangeRequest(`
/**
 * Step 2 of moving an account to a new address.
 *
 * Only the code is sent. Which address is being moved to is read from the
 * pending record on the server, so a client cannot confirm one address with a
 * code that was emailed to another.
 */

src/main/resources/application.yml

47 comments · 47 comment lines removed · file had 132 lines

Lines 2–3 · comment · before `port: ${PORT:8080}`
Cloud hosts (Railway, Render, Fly) inject the port to bind as \$PORT. Binding a
fixed 8080 there makes the health check fail and the deploy gets killed.

Lines 23–24 · comment · before `connectiontimeout: ${MAIL_CONNECT_TIMEOUT_MS:10000}`
Without these a blocked SMTP port hangs the request for minutes
instead of failing fast. 10s is plenty for a reachable server.

Lines 55–58 · comment · before `mail:`
Spring auto-registers a mail health check whenever spring-boot-starter-mail
is present. Cloud hosts block outbound SMTP, so that check fails there and
drags the whole /actuator/health to DOWN even though the API is serving
fine. Mail reachability is not a liveness condition for this service.

Lines 64–66 · comment · before `enabled: ${APP_AI_ENABLED:true}`
Master switch for the in-game AI adviser. Set APP_AI_ENABLED=false to stop
the server calling Gemini at all – the game then shows "Antonio is busy"
instead of an answer. Useful for capping spend or during a demo.

Lines 70–72 · comment · before `api-key: ${GEMINI_API_KEY:}`
The key now lives here rather than in the Unity build. A Unity APK ships
its Inspector values inside the package, so a key held on a MonoBehaviour

```
# can be extracted by unpacking it; server-side it never leaves the host.
```

Lines 79–85 · comment · before `access-token-minutes: ${APP_ACCESS_TOKEN_MINUTES:43200}`

```
# 30 days. The phone keeps its token so a returning player goes straight to
# the menu, and a 24-hour token would have sent anyone away for a day back to
# the login screen without them ever having logged out.
#
# The cost is that a token lifted off a device stays valid for that long and
# nothing revokes it early - pressing Logout only clears the phone's copy.
# Overridable per environment through APP_ACCESS_TOKEN_MINUTES.
```

Lines 93–95 · comment · before `api-key: ${RESEND_API_KEY:}`

```
# When set, mail goes out over Resend's HTTPS API instead of SMTP. Required
# on cloud hosts, which block outbound SMTP ports. Leave empty locally to
# keep using Gmail SMTP.
```

Lines 99–103 · comment · before `emails: ${APP_ADMIN_EMAILS:}`

```
# Comma-separated addresses allowed to use /api/admin, which can delete any
# account and everything belonging to it. Empty means nobody, not everybody:
# with this unset the endpoints refuse every caller. That is the safe way to
# fail - forgetting to set it costs one confusing error, while the opposite
# default would give every player in the game a way to delete anyone.
```

Lines 106–116 · comment · before `mobile-tools: ${APP_ADMIN_MOBILE_TOOLS:false}`

```
# Whether an admin account may open the developer tools on a phone. Off by
# default and separate from the list above, so the tools can be granted to
# a laptop without also arming them on every device that account signs in
# on. Turn it on for a test session, off again afterwards.
#
# Worth being clear about what this does and does not buy: the tools' local
# actions - money, seeds, skipping days - are simulated on the phone, so a
# rebuilt APK could do them whatever this says. What it protects is the
# ordinary build, where the button simply is not there. The destructive
# actions - deleting an account, sending events to another player - are
# gated on the email list server-side and are unaffected by this.
```

Lines 120–122 · comment · before `code-length: ${APP_VERIFICATION_CODE_LENGTH:6}`

```
# Six digits, the length most emailed codes use: beta testers found nine
# hard to carry from the email into the game. A shorter code leans harder on
# max-attempts and ttl-minutes below to stay out of reach of guessing.
```

Lines 127–130 · comment · before `expose-code: ${APP_VERIFICATION_EXPOSE_CODE:false}`

```
# Dev only: when true the code is logged to the console AND returned in the
# request-step response so testing works without a real inbox.
# Defaults to FALSE so a deployed server never hands out login codes to anyone
# who calls the endpoint. Set APP_VERIFICATION_EXPOSE_CODE=true locally only.
```

Files that had no comments

Listed so the archive accounts for every file that was scanned.

Assets/Editor/ClimateMaintenance/ClimateMaintenanceAssignmentValidator.cs
Assets/Editor/ClimateMaintenance/ClimateMaintenancePrefabGenerator.cs
Assets/Editor/PestDiseaseDatabaseSeeder.cs
Assets/Scripts/AreaSelection/Selection/SelectedAreaState.cs
Assets/Scripts/Shared/Backend/BackendRuntimeBootstrap.cs
Assets/Scripts/Shared/Settings/MobileOrientationLock.cs
Assets/Scripts/Shared/UI/HoldButton.cs
Assets/Scripts/TerrainPreview/ClimateMaintenance/ClimateMaintenanceRuntimeBootstrap.cs
Assets/Scripts/TerrainPreview/ClimateMaintenance/ClimateMaintenanceTypes.cs
Assets/Scripts/TerrainPreview/ClimateMaintenance/CropMaintenanceCatalog.cs
Assets/Scripts/TerrainPreview/ClimateMaintenance/DrainageCanalInstance.cs
Assets/Scripts/TerrainPreview/ClimateMaintenance/GreenhouseInstance.cs
Assets/Scripts/TerrainPreview/ClimateMaintenance/IrrigationSystemInstance.cs
Assets/Scripts/TerrainPreview/ClimateMaintenance/ShadeNetInstance.cs
Assets/Scripts/TerrainPreview/ClimateMaintenance/WaterStorageTankInstance.cs
Assets/Scripts/TerrainPreview/ClimateMaintenance/WindbreakInstance.cs
Assets/Scripts/TerrainPreview/CropInstances/CacaoPlantInstance.cs
Assets/Scripts/TerrainPreview/CropInstances/CornPlantInstance.cs
Assets/Scripts/TerrainPreview/CropInstances/DurianPlantInstance.cs
Assets/Scripts/TerrainPreview/CropInstances/EggplantPlantInstance.cs
Assets/Scripts/TerrainPreview/CropInstances/MangoPlantInstance.cs
Assets/Scripts/TerrainPreview/CropInstances/MangosteenPlantInstance.cs
Assets/Scripts/TerrainPreview/CropInstances/PineapplePlantInstance.cs
Assets/Scripts/TerrainPreview/CropInstances/PomeloPlantInstance.cs
Assets/Scripts/TerrainPreview/CropInstances/SquashPlantInstance.cs
Assets/Scripts/TerrainPreview/CropInstances/StrawberryPlantInstance.cs
Assets/Scripts/TerrainPreview/CropInstances/TomatoPlantInstance.cs
Assets/Scripts/TerrainPreview/Economy/ShippingBinSeller.cs
Assets/Scripts/TerrainPreview/Harvest/CollectibleCoconut.cs
Assets/Scripts/TerrainPreview/Harvest/CollectibleCacao.cs
Assets/Scripts/TerrainPreview/Harvest/CollectibleCorn.cs
Assets/Scripts/TerrainPreview/Harvest/CollectibleDurian.cs
Assets/Scripts/TerrainPreview/Harvest/CollectibleEggplant.cs
Assets/Scripts/TerrainPreview/Harvest/CollectibleMangosteen.cs
Assets/Scripts/TerrainPreview/Harvest/CollectiblePineapple.cs
Assets/Scripts/TerrainPreview/Harvest/CollectiblePomelo.cs
Assets/Scripts/TerrainPreview/Harvest/CollectibleSquash.cs
Assets/Scripts/TerrainPreview/Harvest/CollectibleStrawberry.cs
Assets/Scripts/TerrainPreview/Harvest/CollectibleTomato.cs
Assets/Scripts/TerrainPreview/Harvest/CollectibleTropicalCrop.cs
Assets/Scripts/TerrainPreview/Inventory/SprayerLiquidType.cs
Assets/Scripts/TerrainPreview/PestDisease/ActivePestDisease.cs
Assets/Scripts/TerrainPreview/PestDisease/AphidTrapInstance.cs
Assets/Scripts/TerrainPreview/PestDisease/PestDiseaseDatabase.cs
Assets/Scripts/TerrainPreview/PestDisease/PestDiseaseRule.cs
Assets/Scripts/TerrainPreview/PestDisease/PestDiseaseType.cs
Assets/Scripts/TerrainPreview/Player/FirstPersonTerrainController.cs
Assets/Scripts/TerrainPreview/Player/FreeCamController.cs
Assets/Scripts/TerrainPreview/Player/VirtualJoystick.cs
Assets/Scripts/TerrainPreview/SocialMarketplace/SocialMarketplaceRuntimeBootstrap.cs
Assets/Scripts/TerrainPreview/Soil/SoilGameplayRules.cs
Assets/Scripts/TerrainPreview/Soil/SoilSample.cs
Assets/Scripts/TerrainPreview/Terrain/GrassPatchTerrainSpawner.cs
Assets/Scripts/TerrainPreview/Terrain/TreeBillboardLOD.cs
Assets/TutorialInfo/Scripts/Readme.cs
src/main/java/com/agridabao/api/AgridabaoApiApplication.java
src/main/java/com/agridabao/api/auth/AuthResponse.java
src/main/java/com/agridabao/api/auth/RegisterRequest.java
src/main/java/com/agridabao/api/auth/UserResponse.java
src/main/java/com/agridabao/api/auth/VerifyCodeRequest.java
src/main/java/com/agridabao/api/chat/ChatController.java
src/main/java/com/agridabao/api/config/SchedulingConfig.java
src/main/java/com/agridabao/api/error/ApiError.java
src/main/java/com/agridabao/api/error/ApiExceptionHandler.java
src/main/java/com/agridabao/api/error/BadRequestException.java
src/main/java/com/agridabao/api/error/ConflictException.java
src/main/java/com/agridabao/api/error/ForbiddenException.java
src/main/java/com/agridabao/api/error/NotFoundException.java
src/main/java/com/agridabao/api/error/UnauthorizedException.java
src/main/java/com/agridabao/api/farm/FarmController.java
src/main/java/com/agridabao/api/farm/FarmSave.java
src/main/java/com/agridabao/api/farm/FarmSaveRepository.java
src/main/java/com/agridabao/api/farm/FarmSaveRequest.java
src/main/java/com/agridabao/api/farm/FarmSaveResponse.java
src/main/java/com/agridabao/api/farm/FarmService.java
src/main/java/com/agridabao/api/settings/PlayerSettingsController.java
src/main/java/com/agridabao/api/settings/PlayerSettingsRepository.java
src/main/java/com/agridabao/api/settings/PlayerSettingsResponse.java
src/main/java/com/agridabao/api/social/FriendshipAccessService.java
src/main/java/com/agridabao/api/social/FriendshipId.java
src/main/java/com/agridabao/api/social/PresenceAccessService.java
src/main/java/com/agridabao/api/social/SocialController.java
src/main/java/com/agridabao/api/user/ChangeDisplayNameRequest.java
src/main/java/com/agridabao/api/user/UserController.java
src/test/java/com/agridabao/api/AgridabaoApiApplicationTests.java